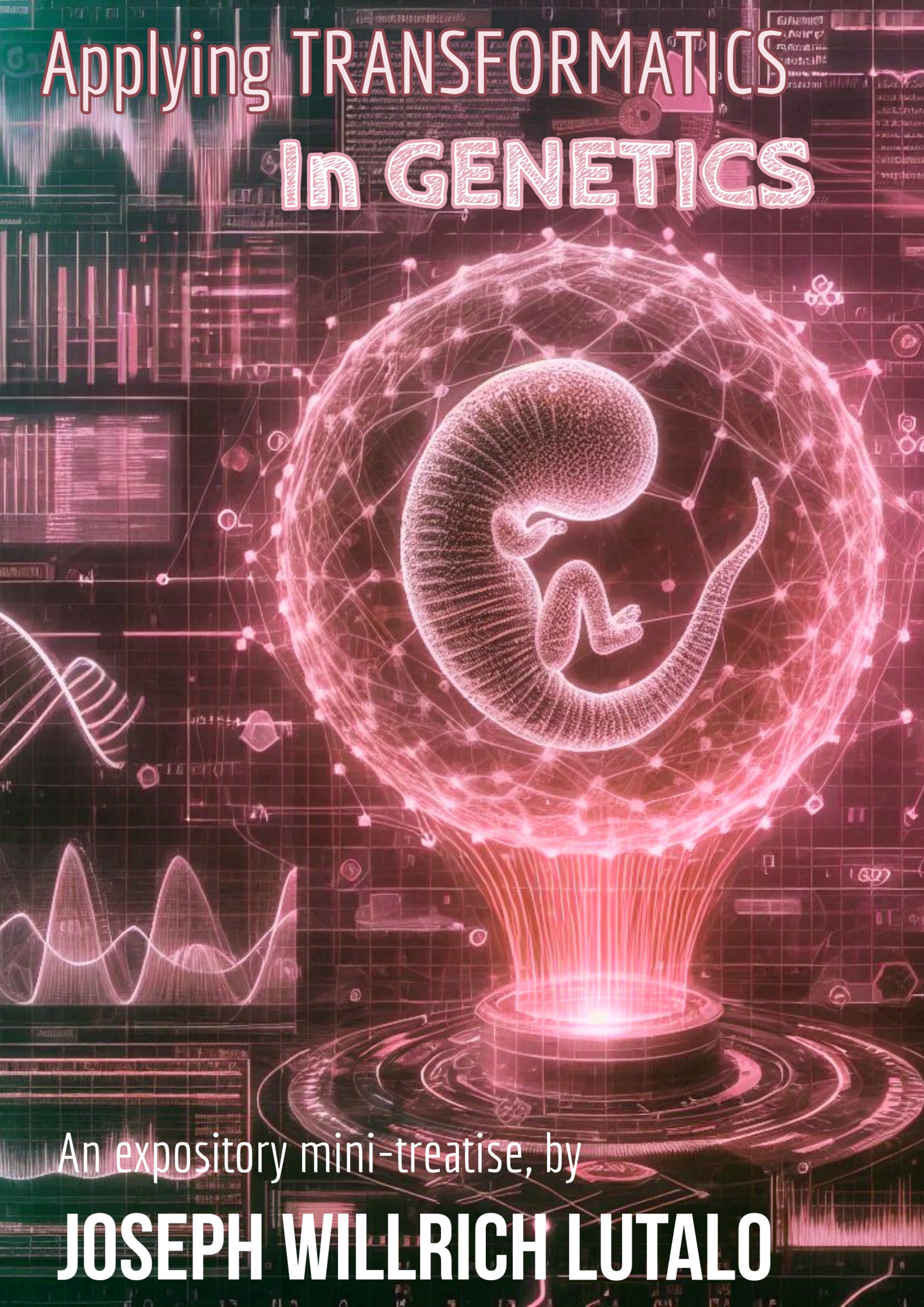


Applying TRANSFORMATICS In GENETICS



An expository mini-treatise, by

JOSEPH WILLRICH LUTALO

Applying
TRANSFORMATICS in
GENETICS

IN BRIEF

A treatise on the mathematics, philosophy and practice of
Transformatics for modern Geneticists.

ALSO BY JOSEPH WILLRICH LUTALO

- **FICTION** -

Shrines of the Free Men

Club 69

Embaga Ku Rina Island

Ensi N'Amaguru

Rock 'N' Draw

- **ACADEMIA** -

Concerning a Transformative Power in Certain Symbols, Letters and Words

*TEA TAZ - Transforming Executable Alphabet A: to Z: COMMAND SPACE
SPECIFICATION*

The Theory of Sequence Transformers & their Statistics

Novus Modernus Grimoire Lumtauto Magia

APPLYING TRANSFORMATICS IN GENETICS

A TREATISE

BY

JOSEPH WILLRICH LUTALO C.M.R.W.

I*POW • Internet

First published on the Internet in 2025 by

I*POW

(International Internet Portfolio of Writers)

<https://t.me/ipowriters>

Copyright © Joseph Willrich Lutalo 2026

The right of Joseph Willrich Lutalo to be identified as the author of this work has been asserted by him in accordance with the Copyright, Designs and Patents Act 1988.

A catalogue record for this book shall likewise become available from the British, as well as the Uganda National Library.

ISBN :

All rights reserved. No part of this publication may be reproduced, transmitted, or stored in a retrieval system, in any form or by any means, without permission in writing from I*POW.

—Λ—

I*POW 1st Edition on 29th August, 2025

I*POW 1st Revision on 6th September, 2025

I*POW 2nd Edition-DRAFT on 22nd January, 2026

Applying **TRANSFORMATICS** in GENETICS

Willrich J. Lutalo¹
joewillrich@gmail.com, jwl@nuchwezi.com

February 24, 2026

¹Currently a volunteering & Independent Principal Investigator at Nuchwezi Research — <https://nuchwezi.com>

Dedication

The friendly, 20th century philosophical entertainer, **Alan Watts**, such a fatherly, enlightening voice I keep hearing in my thoughts every once in a while; for I enjoy listening to his rants, once said *“You are the universe experiencing itself.”* I always wished to be a father to someone... a real boy or a girl out there... a soul... my own genetic code alive and existing independently of me. To experience and enjoy the expressions of my own genetic code remains among my greatest dreams. But, in case I don't see my own child in this lifetime, at least, this message should inspire those that read it, who come after, to know and trust that when we express ourselves, we in a way express those that brought us into the world. Personally, I would cherish it! I surely dedicate this work to you, my father and mother. I can't express enough how grateful I am, that you enabled me to be... You gave me a distinctive identity and a voice. **May this work help further your expressions in this world... and beyond.**

Contents

Preface	xiii
List of Abbreviations	xv
1 Introduction	3
1.0.1 How to Navigate the Book	7
2 Sequence Symbol Sets and Sequence Transforms Applied to Genetic Code	11
3 Sequence Abstraction using Symbol Sets: From Flat-Structure Sequences to Higher-Order Sequences	17
4 Sequence Analysis Using Symbol Sets	19
4.1 Testing Sequences for Membership in a Base	19
4.2 Symbol Sets of n-GRAMS and Testing Sequences for n-GRAM-based Properties	21
4.3 Concerning Genetic Pathologies based on Faulty Gene-Programs	23
5 Sequence Analysis Using Complement Sets	25
5.1 Significance of Sequence Complements, $\neg\Theta$, and Complement Symbol Sets, $\neg\psi_\beta$	28
5.2 Potential Applications of Sequence Complements or Inverses	30
6 Sequence Analysis Using The Anagram Distance and the Modal Sequence Statistic	35
6.1 Six Sequences and Four Scenarios	35
6.2 A First Analysis	38
6.3 SCENARIO A — the anagram distance measure	43
6.4 SCENARIO B — the sequence characteristic [statistic]	46
6.5 SCENARIO C	50
6.6 SCENARIO D — the population characteristic measure	51
6.6.1 Measuring Proximity to a Set of Sequences, Ω^n	51
6.6.2 Measuring Proximity between a Q_k and Members of a Sequence Population, Ω^n	56
6.7 Concerning Optimal Symbolic Storage of na-Sequences in Databases - Computing the Gödel Number for a Sequence Θ_n based on its Characteristic $\tilde{h}(\tilde{\Theta}_n)$	58
7 The Mathematics of Genome Sequencing: Sequence Alignment, The Modal Sequence, The Sequencing Machine and the Identifier Genome Sequence Law	63
7.1 Relevant Background	63
7.2 3 Core Problems to Solve	65
7.2.1 PROBLEM G1: Computing Maximum Common Subsequence: $\Theta_1 \diamond \Theta_2$	66
7.2.2 PROBLEM G2: Computing Overlap Resultant Sequences: $\Theta_1 * \Theta_2$	71
7.2.3 PROBLEM G3: Computing the Genome Sequence (GS): $(\Theta_1 \diamond \Theta_2)^*$	72
7.3 The Genome Sequencer	74
7.4 The Identity Genome Sequence (IGS) Law	76
8 Random Sequence Generators (RSGs)	79
8.1 The First Lu-Shuffle Algorithm (FLSA)	80
8.1.1 A Mathematical and Computational Specification of the FLSA Algorithm	80
8.1.2 Analysis and Examples of Applying FLSA	81
8.1.3 The Source Code for FLSA	84
8.2 The Second Lu-Shuffle Algorithm (SLSA)	86
8.2.1 The Mathematical and Computational Specification of the SLSA Algorithm	87

8.2.2	Analysis and Examples of Applying SLSA	88
8.2.3	The Source Code for SLSA	90
8.3	Generating Realistic Random na-Sequences	92
8.3.1	How to Generate a Random Nucleotide & The Random Symbol Generator	92
8.3.2	The USYMBOLSET Function	94
8.3.3	The PROTRACT Function	95
8.3.4	The Random Number Generator, RNG Function	95
8.3.5	The SHUFFLE Function	98
8.3.6	The SELECT Function	99
8.3.7	The RSG Function	99
9	Gene Expression in Living Organisms Leveraging Genetic Code (DNA $\rightarrow$ mRNA $\rightarrow$ Protein $\rightarrow$ Organism)	111
9.1	The Protein Manufacturing Algorithm	117
9.2	Gene Expression in Living Things and for Bio-Automata via Ribosomes	121
10	Transformatics and the Lu-Genome Expression System in Bio-Automata	123
10.1	The Ozin Genetic Code Cipher	126
10.1.1	Ω -to-OZIN Genetic Code Transcription Rules	129
10.1.2	Examples of Ω -Genetic Code Transcription	130
10.2	Genetic Code Sequences Processed as Modal Sequence Statistics	131
10.2.1	The Platonic Form Cipher and Position-Sensitive Genetic Code Translation	133
10.2.2	ψ_{Ω} -to- ψ_{pf} : PLATONIC Form Genetic Code Translation Rules	134
10.2.3	Examples of Complete Lu-Genome Expressions	138
10.3	The Extended Lu-Genome Expression System (x-LGES)	144
10.3.1	ψ_{Ω} -to- $(\psi_{oz}-\psi_{pf}-\psi_{oz})$: The x-LGES Algorithm	145
10.4	Actual DNA Sequences Expressed in the Extended Lu-Genome Expression System	147
10.4.1	Converting Between Nucleic Acid Code and Numeric Codes	148
10.4.2	Revisiting Numeric Sequences	150
10.4.3	x-LGES applied to named DNA Sequences	152
11	Conclusion	157
11.1	Future Directions	162
Appendix A	Other Functions of DNA Beyond Protein Synthesis	167
Appendix B	Origins of the OZIN Cipher	169
Appendix C	Computing Platonic-Form Encoding Keys (PFEKs), $\boxed{\psi_{\Omega}}_m$, for a Particular na-Sequence Θ and some m	175
C.0.1	Example of Resolving PFA Components, $\boxed{\omega_i^*}$	177
Appendix D	TRANSFORMATICS 101 - explained:	
	A rigorous, succinct introduction to all the essential foundations and proposed theory of Transformatics	181
D.1	Foundations of a New Mathematics of Sequences	181
D.1.1	Result 1: The Empty Sequence[1]	181
D.1.2	Result 2: A Symbol Set[2]	181
D.1.3	Result 3: A Sequence[1]	182
D.1.4	Result 4: Sequence Cardinality[3][2]	182
D.1.5	Result 5: A Sequence Symbol Set[4][2]	182
D.1.6	Result 6: A Sequence Transformation[5][1]	182
D.1.7	Result 7: A Sequence Transformer[6][5]	182
D.1.8	Result 8: A Sequence Filter[5]	182
D.1.9	Result 9: A Sequence Generator[5][7]	182
D.1.10	Result 10: A Sequence Sampler[8]	182
D.2	Proposed Abstractions Leveraging Transformatics	184
D.2.1	Proposal 1: The o-SSI Sequence from any Sequence[4]	185
D.2.2	Proposal 2: The Anagram Distance Between Any Two Sequences[3]	186
D.2.3	Proposal 3: The Transformer Compression Ratio of any Sequence Transformation[5]	187
D.2.4	Proposal 4: The Piecemeal Compression Ratio of Any Sequence Transformation[5]	188

D.2.5	Proposal 5: The Modal Sequence Statistic of Any Sequence[5]	189
D.2.6	Proposal 6: The Characteristic of Any Sequence[8]	191
D.2.7	Proposal 7: The Population Characteristic of Any Collection of Sequences[8]	195
D.2.8	Proposal 8: A Super Sequence from One or More Sequences[8]	196
D.2.9	Proposal 9: Programs as Chains of Sequence Transformers (Transformer-Chains)[8][9]	199
D.2.10	Proposal 10: Entropy of a Sequence[1]	202
D.3	Conclusion	204
Appendix E	Situating TRANSFORMATICS (as a Theory or Field) within Mathematics:	207
E.1	A Working Definition and Clarification on Fundamental Nomenclature	207
E.2	Comparison with Other Fields	216
E.2.1	Relation to Statistics	216
E.2.2	Relation to Linear Algebra	217
E.2.3	Relation to Calculus	224
E.2.4	Relation to Several Other Branches of [Applied] Mathematics	233
E.2.5	Transformatics as a new Algebra of Sequences	243
E.3	A Final Definition and Justification of a Field	244
Appendix F	Future Directions and Explorations of TRANSFORMATICS in both PHYSICS and GENETICS?	245
	Final Dedication	261
	About the Author	264
	Colophon	273

List of Figures

1.1	In the course of writing this work, an experiment to grow pineapples from the pineapple fruit tops of pineapples normally bought from nearby marketplace (Garuga/Bugabo-Bukaaya) yielded results! Here we see the first successful harvest of a pineapple from our home garden (at Nuchwezi Research). But also, this case inspired part of the discussion in Chapter 10 about how in real genome expression systems, the organism expressed from genetic code somewhat still includes the genetic code in the manifested organism as some kind of affix — a prefix (in leaves for example) or suffix (roots), from which the original organism (or the same exact genome) might be later reproduced!	4
1.2	A basic diagram of the chemical structure of DNA base-pairs, sourced from [10]	6
6.1	Some proposal to use Gödel’s idea of abstracting formulae such as formal symbolic sequences using distinct natural numbers.	58
8.1	A Tabulation Trace of Shuffling the Base-10 n-SSI Sequence with increasing values of k , the Initial Partitioning Index for Algorithm 3 , the First Lu-Shuffle Algorithm .	82
8.2	A Tabulation Trace of Shuffling basic base-case scenarios to help highlight how FLSA works.	82
8.3	A Tabulation Trace of Shuffling the na-Sequence Θ_{2na} from Equation 10.23 with Algorithm 3 (FLSA) .	83
8.4	A Tabulation Trace of Shuffling the na-Sequence Θ_{palna} from Equation 10.24 with Algorithm 3 (FLSA) .	84
8.5	The First Lu-Shuffle Algorithm implemented using Python	85
8.6	A clean version of the Python3 implementation of the FLSA	86
8.7	A Tabulation Trace of Shuffling the Base-10 n-SSI Sequence with increasing values of k , using Algorithm 4 (SLSA) , the Second Lu-Shuffle Algorithm .	89
8.8	A Tabulation Trace of Shuffling basic base-case scenarios to help highlight how SLSA works and how it relates to FLSA .	89
8.9	A Tabulation Trace of Shuffling the na-Sequence Θ_{2na} from Equation 10.23 with Algorithm 4 (SLSA) .	90
8.10	A Tabulation Trace of Shuffling the na-Sequence Θ_{palna} from Equation 10.24 with Algorithm 4 (SLSA) .	90
8.11	The Second Lu-Shuffle Algorithm implemented using Python	91
8.12	The USYMBOLSET (Ω) unspecific symbol set generator implemented using Python	94
8.13	The PROTRACT (Θ, n) sequence generator-transformer implemented using Python	95
8.14	The RNG (ll, ul) random number generator implemented using Python	96
8.15	A text dump from testing our RNG (ll, ul) that only utilizes System Time as entropy source, and uses our First Lu-Shuffle Algorithm to generate True Random Numbers	97
8.16	The SHUFFLE (Θ) sequence randomizer implemented using Python	98
8.17	The SELECT (n, Θ) sequence sampler function implemented using Python	99
8.18	The RSG (n, Θ) random sequence generator function implemented using Python	100
8.19	A Random DNA Sequence of exactly 200 random codons as generated using our RSG (n, Θ) random sequence generator algorithm.	108
9.1	An illustration of the gene translation process in a cell via protein-factories known as ribosomes (Illustration credit: GeeksForGeeks.org[11])	112
9.2	Flow-Chart summarizing the Ribosome-based Protein-Synthesis Process in a Living Cell	118
9.3	The RIBOSOME State Machine	119
10.1	A blackboard snapshot from the author’s Blackboard Adventures (https://t.me/bclectures) series on Telegram, on 20th AUG, 2025 , depicting two LGES example genome expressions, one for $\hat{\Theta} = \langle 218 \rangle$, the other for the PFA segment configuration $4_{pf} \otimes 6_{oz}$	123

10.2	The OZIN Cipher[12], a mapping from Decimal Symbols (our base- Ω) to Symbols for Intermediate Genetic Code.	127
10.3	The equivalent OZIN expression of the hypothetical Euler Virus genome sequence.	128
10.4	The equivalent OZIN expression of the HiFinelle genome sequence.	128
10.5	The equivalent OZIN expression of the Ω_3 genome sequence with multiple STOP-codon elements in the original genetic code, including some in the middle of the code.	129
10.6	The palindromic sequence Ω_{pal} transcribed into base-OZ.	131
10.7	The PLATONIC Cipher[13] a mapping from Decimal Symbols (our base- Ω) to Spatial-Geometric Forms (ψ_{pf}).	133
10.8	An example of a single amino-acid or rather, the PFA component of a FGE equivalent to one amino-acid segment expressed using the LGES system. This particular expression corresponding to a rendering of the configuration $4_{pf} \otimes 6_{oz} = 4_{pf} + 4 \times 6_{oz}$	137
10.9	The equivalent LGES complete genome expression of the Ω_3 genome sequence (from Equation 10.4)	139
10.10	The equivalent LGES complete genome expression of the Ω_{veuler} (Euler Virus) genome sequence (from Equation 10.16)	140
10.11	An alternative rendering of the same sequence as Ω_{veuler} originally depicted as the LGES genome expression in Figure 10.10	141
10.12	The x-LGES form rendering of the Ω_{veuler} , the Euler Virus , now with HEAD, BODY and ROOT segments.	146
10.13	An example of an actual organism from nature — an Earthworm, for which, like the x-LGES rendering of the sequence $\Omega_{HiFinelle}$, results in a genome expression for a creature that almost looks the same at the HEAD and ROOT. Image sourced from Wikimedia Commons[14].	152
10.14	The x-LGES form rendering of the sequence Θ_{mlla} , depicting its complete FGE with HEAD, BODY and ROOT segments.	155
B.1	An excerpt from architectural design notes by the author, from the early days of his home-based research laboratory. In this particular sketch note, we also get to see an idea similar to how gene expressions are actually a chain of glyphs that, as in the LGES protocols, are depicted as structures chained to each other along a <i>backbone structure</i> symbolically depicted as a line (curved lines in this case). Copyrights due to author and Nuchwezi Research Labs.	170
B.2	The OZIN Cipher in use. An excerpt from the Church of Dance Eternal OGF	172
B.3	The OZIN Cipher in use. An excerpt from author's research notes (circa 2019) on developing a cryptographic system leveraging a dial mechanism. Copyrights due to author and Nuchwezi Research Labs.	173
C.1	An example of a single PFA component equivalent to the configuration $8_{pf} \otimes 6_{oz}$	179
D.1	TEA EXAMPLE: SC-Computer : transforms any input sequence into its sequence characteristic!	194
E.1	Comparison of Several Fundamental Mathematical Structures related to Sequences in Transformatics	208
F.1	The surreal painting, "Arrival of the Anunnaki", by Joseph W. Lutalo, as part of their 2022 The Zermelo Frankael Dream Collection that was first exhibited at Ndugu Art Gallery then in Entebbe Town.	248

List of Tables

5.1	An example of exploring symbol and sequence complements in sequence transformations	26
6.1	A First Analysis of the 5 na-Sequences from Section 6.1	42
6.2	A Tabular Analysis of Θ_1 Vs Θ_4 so as to compute their Anagram Distance	46
9.1	Amino-Acid Codes and Names in Θ_4 , a DNA-encoded gene	116
9.2	Amino-Acid Codes and Names in Θ_4^* , a mRNA encoded gene	116
C.1	Platonic Form Encoding Keys (PFEK) mapped to corresponding values of $m = \mathcal{U}(\overset{\rightharpoonup}{\Theta})$ using SLSA	176
C.2	Platonic Form Encoding Keys (PFEK) mapped to corresponding values of $m = \mathcal{U}(\overset{\rightharpoonup}{\Theta})$ using FLSA	177

Preface

Jung restored the colour in science, and made me cherish the mind in new ways...

And that in exercising our will we might become aware of the divine in us and thus make glory manifest. God is not asleep in us!

— J. Willrich Lutalo, *Shrines of The Free Men*, 2018[15]

We are living in a world where man affects and influences nature so much, especially with his *unnatural* inventions in the form of animated and inanimate intelligence, implements and reality augmentations. But also, the world around man is constantly changing, and sometimes, in ways other than man is naturally comfortable with. That said, there is this idea that *a life unexplored isn't worth living*¹, and come what may, with every passing day, and as seasons and times change, it seems like the compulsion for humans to have to leave their traditional home environments and explore life elsewhere — say, in foreign countries, in/under the sea, *inside* the earth or away from it, say like in outer-space, etc. seems so unavoidable — in fact, it feels to me like it is inculcated into our very natures, so that, thinking about a future in the same exact space, with the same exact people, environment and circumstances as we wake up to every day just doesn't make much sense². That we must prepare ourselves — “we”, meaning *humans*, to face an uncertain future, but also, to lessen such uncertainty to manageable levels where possible or necessary, is much of what justifies our investments in and explorations of science and knowledge in general.

For those who trust *divinely inspired* wisdom³, the sacred scriptures (in **Genesis 1:28**) empower us thus:

¹I trust, it is attributed to ancient philosopher and student of **Socrates**, Aristocles son of Ariston aka. ‘Plato’.

²I have read a sane dose of **Daniel Defoe**’s classic, *The Life and Adventures of Robinson Crusoe*, a relic from 1719, and among things Defoe seemed to argue against, was the somewhat strong urge in some humans, to want to go “abroad upon Adventures, to rise by Enterprise, and make themselves famous in Undertakings of a Nature out of the common Road”.

³I keep an indispensable copy of **The New Jerusalem Bible**, and wouldn't wish to do much without consulting it first!

God blessed them, saying to them, *Be fruitful, multiply, fill the earth and subdue it. Be masters of the fish of the sea, the birds of heaven and all the living creatures that move on earth.*

Despite not having been funded by any one or any particular organization, the author, passionate about his pursuit of a **Doctorate in Philosophy** — of Computer Science or Mathematics, from the **University of Oxford**[16][17] or any other reputable institution⁴, and especially to secure a teaching or faculty position, somewhere⁵, embarked on this project, especially to further his earlier work on Transformatics[5] as well as to bring to better audiences, the now stable advances in his original, commercially-viable **TEA programming language**[9], by exploring new ground, proposing ideas and applications, in a domain potentially more in-demand and with far-reaching consequences for not just humanity, but all of nature, now, tomorrow, and in the deep, far-future.

⁴Refer to **ORCID**: <https://orcid.org/0000-0002-0002-4657>

⁵Because, in all fairness, knowledge must not only be sought or furthered, but must also be shared.

List of ACRONYMS

ABBR.	Definition
ADM	Anagram Distance Measure
COS	Consensus Overlapping Sequence
DNA	Deoxyribonucleic Acid
FGE	Full Genome Expression
FLSA	First Lu-Shuffle Algorithm
GXS	Genome Expression Suffix
IFA	Intermediate Form Affix
I*POW	International Portfolio of Writers
IGS	Identity Genome Sequence
JSON	JavaScript Object Notation
LGES	Lu-Genome Expression System
LNS	Lu-Number System
MCS	Maximum Common Subsequence
<i>m</i> RNA	<i>messenger</i> - RNA
MSS	Model Sequence Statistic
<i>n</i> -SSI	<i>natural</i> -Symbol Set Identity
<i>o</i> -SSI	<i>orthogonal</i> -Symbol Set Identity
ORS	Overlap Resultant Sequence
PFA	Platonic Form Aspect
RNA	Ribonucleic Acid
RNG	Random Number Generator
RPMSS	Representative Population MSS
RSG	Random Sequence Generator
SLSA	Second Lu-Shuffle Algorithm
TEA	Transforming Executable Alphabet
<i>t</i> RNA	<i>transfer</i> - RNA
WSL	Windows Subsystem for Linux
x-LGES	Extended LGES
YAML	Yet Another Markup Language

Abstract

It started out as just a paper but is now a treatise. **Applying TRANSFORMATICS in GENETICS** brings to surface the importance of leveraging the mathematical theory recently named “Transformatics”, that deals with the study, processing and analysis of especially sequences of symbols and their alteration via symbolic machines known as sequence transformers. In earlier and recent work by the author, it has been demonstrated to be a credible and powerful theory in designing, specifying and explaining the properties of automata operating on sequences to produce other sequences. In this particular work, we take that body of knowledge, as well as what we know about the important science of how biological life-forms are defined, transformed and expressed in nature based on genetic code sequences known as DNA (deoxyribonucleic acid), and use that to treat of old and new problems in the field of GENETICS, developing many new, and previously unrepresented solutions based on the theories, calculus and mechanics of sequence transformers. We have used transformatics to develop, but also distill many useful general results in the form of theorems as well as 3 laws concerning na-Sequences. We have demonstrated how sequence transformers can be leveraged to solve specific practical problems in genetics via numerous well-structured sequence transformations. We have invented and presented several new algorithms that would allow the concepts and solutions we developed in transformatics theory for various general problems to be readily turned into computable solutions using any programming language, for scenarios in genetics, but especially via leveraging TEA and Python. We have demonstrated how to leverage statistical analysis to make quantifiable differences and similarities between two or more na-Sequences (including entire populations of them) readily discernible. Leveraging transformatics, we have developed a new theory and systematic framework, the Lu-Genome Expression System, with which, via explorations by eye and hand, but also via the use of computers, the matter of how individual DNA sequences, and generally, any sequence that can be transformed into a numeric/decimal sequence, can be used to explore the natural concept of gene and genome expression, in both a visual-spatial, but also mathematical-symbolic way. We have tackled the fundamentally critical problem of genome sequencing, and have managed to not only cast it rigorously using the language and formalisms of transformatics, but have also managed to define an abstract machine that can take a set of DNA sequences potentially out of alignment, and then derive from them, via systematic and meaningful alignment procedures, a final, representative consensus genome sequence. Though this work is significantly theoretical and mathematical, we anticipate that the discussions, notes, formulations, solutions and overall expository treatment of the subject that it brings forth, shall help inspire actual domain experts and other researchers interested in genetics, genetic engineering, and related sciences, to pick up and apply our transformatics theory and ideas in both theory and practice.

Keywords: Mathematical Statistics, Linear Algebra, Transformatics, Artificial Intelligence, Sequence Processors, Sequence Analysis, DNA, Genetic Code, Genetic Analysis, Abstract Machines, Genome Sequencing, Genome Expression, Gene Programs, Transformers.

Chapter 1

Introduction



Figure 1.1: In the course of writing this work, an experiment to grow pineapples from the pineapple fruit tops of pineapples normally bought from nearby marketplace (**Garuga/Bugabo-Bukaaya**) yielded results! Here we see the first successful harvest of a pineapple from our home garden (at **Nuchwezi Research**). But also, this case inspired part of the discussion in **Chapter 10** about how in real genome expression systems, the organism expressed from genetic code somewhat still includes the genetic code in the manifested organism as some kind of affix — a prefix (in leaves for example) or suffix (roots), from which the original organism (or the same exact genome) might be later reproduced!

Sex seems to have been invented around two billion years ago. Before then, new varieties of organisms could arise only from the accumulation of random mutations --- the selection of changes, letter by letter, in the genetic instructions. Evolution must have been agonizingly slow. With the invention of sex, two organisms could exchange whole paragraphs, pages and books of their DNA code, producing new varieties ready for the sieve of selection. Organisms are selected to engage in sex --- the ones that find it uninteresting quickly become extinct. And this is true not only for the microbes of two billion years ago. We humans have a palpable devotion to exchanging segments of DNA today.

— Carl Sagan, *COSMOS*, 1981[18]

The material basis of heredity is DNA, a ladder-like molecule which carries a message in the form of a ‘four-letter’ code, the letters being four chemical bases, each of which may occupy any rung in the ladder.

— The Oxford Companion to the Mind[19]

In reality, we find that living, real organisms are influenced by genetics, their environment, bits of randomness and sometimes emergent behaviors that might not be readily captured by strict rules such as the genetic code of life. However, away from all possibilities, and focusing on what can be said of life expression via the genetic code known as DNA (the *deoxyribonucleic acid*) and RNA (*ribonucleic acid*), especially when applied to the vast spectrum of natural organisms on earth — from basic **prokaryotes** (single-celled organisms) such as the simple bacteria that actually have no nucleus[20], to **eukaryotes**; all the way from basic one-cell kinds such as amoeba[21] all the way to sophisticated creatures such as oak trees, vultures, dolphins and human beings! Talking of which, it might be important to set clear that though **viruses** are a kind of life-form[22], and yet, they are neither prokaryotes nor eukaryotes — especially because, fundamentally, they are merely some genetic code (DNA or RNA) “enclosed in a protein coat, lacking cell membranes or organelles”[22] — more technically they are **acellular entities**.

So, if we bring on-board ideas from **transformatics**[5][23] — a new mathematical theory¹ dealing with sequences of symbols or those of named structures and their processing as well as analysis; for example, if we take the transformatics theory concepts such as comparing sequences using new measures such as the Anagram Distance Measure (ADM), or perhaps leveraging the sequence summary measure known as the modal sequence statistic (MSS), and these then being applied to sequences such as DNA, we find that, independently of, and without needing to first consult or worry about mainstream genetic code analysis or genetic engineering theory and mechanics, that we can say many useful things about such genetic code sequences and that we might be able to break new ground or solve some otherwise still intractable problems concerning sequences of genetic code in general.

¹Also, refer to the **appendix** — particularly **Chapter D**, which reproduces verbatim, the “Transformatics 101” manuscript — a *mini-thesis* — that serves as the authoritative introduction to and baseline justification for this new mathematical theory.

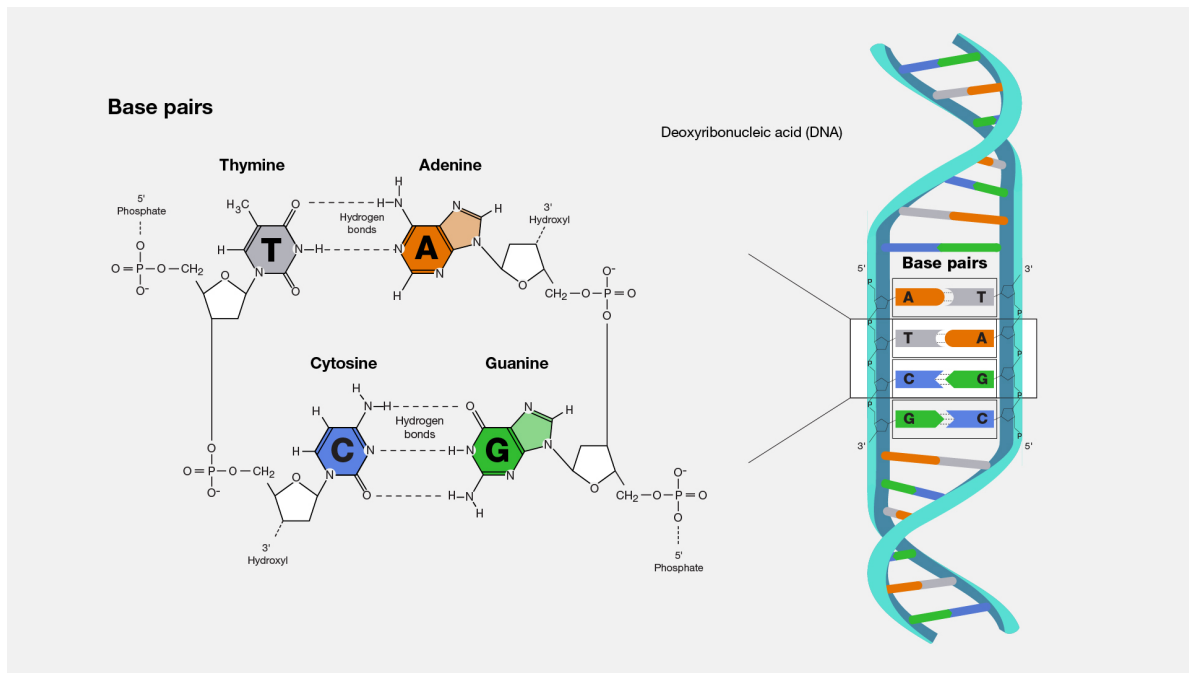


Figure 1.2: A basic diagram of the chemical structure of DNA base-pairs, sourced from [10]

For example, by borrowing a useful DNA-as-library metaphor from Venville et al[24], we would come to appreciate that by looking at *DNA as a library of books*, we have a model such as:

1. **Library:** DNA (as a whole) — the **Genome**, as the complete collection of genomic data about an organism.
2. **Bookshelves: Chromosomes** as organized storage of genes, and that they (chromosomes) are long, coiled-up strands of pairs of genes (essentially, the chromosome is a combination of two strands of genes, with one called the *template* and the other a *complement*, and that when they come together to form the “ladder” structure, at each step, the two genes forming a step are paired such that $A \leftrightarrow T$ and $C \leftrightarrow G$ [25] — also see **Figure 1.2**). So, for example, humans have 46 chromosomes in total in their “DNA library” (23 chromosome pairs, consisting of 22 autosomal pairs plus one pair of sex chromosomes). We know that during reproduction, the [full] genome (46 chromosomes in humans) splits up by half in either parent (via *meiosis*, which somewhat shuffles the complete genome and then halves it[26]), so that only one-half of the otherwise well-paired template-complement set that is the chromosome strand from each parent (as either sperm or ovum) goes to contribute to the final chromosome collection-set of the offspring[27][19]. Whereas, after fertilization or during normal cell division, the entire chromosome set (with well-paired strands) is duplicated/replicated wholesomely and losslessly so that the second/new cell thus created has an exact copy of the same chromosome set (entire genome) as was in the source cell[28].
3. **Books:** The **Genes** which make up a chromosome are the books in our genome library. Each gene is essentially a collection of “words” that are a sequence of one or more codons (see below), and which taken together, contain enough information/instructions to specify how to produce a specific protein[19][24].
4. **Words:** And then, at the next, lower abstraction level than genes, we have **Codons**. These are like “words” in each book, and each codon **only** codes for a single **amino acid**. Basically, a codon is a combination of any 3 of the “letters” of genetic code, for which, there are exactly four for DNA (A, C, G, T) and four for RNA (A, C, G, U). We also know that there are at most, 64 possible unique combinations of the four letters into triplets/3-grams/the codons[19].

5. **Letters:** Finally, at the most basic level of our genetic code/DNA-library/genome, we have “letters” that are technically known as **nucleotides**. Essentially, each nucleotide contains a **single nucleobase**, with only one difference between DNA and RNA as such; for DNA: $\{A, C, G, T\}$, and for RNA: $\{A, C, G, U\}$ — we have already come across the names of the four letters for the DNA, however, concerning that extra/new one, ‘U’, specific to RNA, its name is “Uracil”.

So, with that introduction clearing up much of the basic genetics nomenclature and concepts that I shall use in the rest of this work, we then dive into the meat of our undertaking as laid out in the next section.

1.0.1 How to Navigate the Book

This Book is Navigable:

To ease and leverage meaningful cross-referencing across the chapters — including extension materials in the appendix chapters, there is substantial employment of **emboldened**, typed, named, numbered and interactive^a references across the book. For consistency and ease of navigation, note that where they occur in the text, such shall appear and behave as either this link to **List of Acronymns** or **Transformer 26**.

Finally, note that the book does offer an alternative way to locate and navigate the text based on keywords and terms of importance — there is an **Index** at the end of the book, with which one can readily jump to sections in the book where a term or topic of interest is first introduced or is being discussed substantially. Concerning these, terms in the index aren’t necessarily highlighted in the text, however, as with the interactive references mentioned above, for those reading the electronic version of this book, the index offers an interactive mechanism to quickly jump to sections where the term is being applied.

^aEvery such link, when encountered in the electronic or digital edition of this book, is clickable, and when clicked, shall automatically jump to the location in the book where that reference points. For paper-printed or non-interactive reading experiences, the type (in form of a guiding concept name: **Definition**, **Transformer**, **Theorem**, **Chapter**,...) and a reference number (**1.2**, **B.3**, **8**,...) can be combined to determine where it is in the book the referenced item is.

The necessary foundation concepts and nomenclature have been well introduced in **Chapter 1** — the previous section.

In **Chapter 2** I shall formally define the DNA and RNA symbol sets, ψ_{DNA} and ψ_{RNA} . We shall look at the matter of the difference between the ordering of terms in these mathematically-oriented sets, relative to the common-place **A-T-C-G** ordering typical in biological and especially

genetics literature. I shall define the more universal ψ_{na} that would help apply logic to both DNA and RNA sequences, and shall generalize all such sequences as **na-Sequences**.

In **Chapter 3** we shall look at the idea of using n-grams such as the meaningful na-Sequence nucleobase 3-grams (technically known as **codons**) to abstract away various nuances of the flat-structure DNA or RNA sequence whose length might sometimes shoot into the millions or billions for realistic complete genome sequences. In **Chapter 9** we shall further these ideas when looking at how to process na-Sequences at n-gram level such as in naming or identifying subsequences. We shall also see that such abstractions might help manage complexity and simplify formalisms or algorithms dealing with na-Sequences such as in the formalization of protein synthesis for natural or artificial automata via codon-processing ribosomes.

In **Chapter 4** we shall look at how we might leverage the basic concept of a symbol set to analyze arbitrary na-Sequences. We for example shall look at how to automatically generate special ordered symbol sets corresponding to individual na-Sequences under various bases — DNA, RNA, na- or even sometimes lexically meaningful $AZ(\psi_{az})$. We shall see how to define special symbol sets based on n-gram abstractions of na-Sequences, such as might help in logic with special na-Subsequences such as START and STOP codons.

In **Chapter 5** we then shall consider the interesting matter of **complement sequences** and **complement symbol sets**. We shall look at how to generate complements given a starting sequence or symbol set. We shall look into the interesting fact that complementing ψ_{DNA} helps us arrive at the common **A-T-C-G** ordering of terms such as in ψ_{DNA} . And though we might not use it immediately, we shall see that we might also talk of orthogonal symbol sets and orthogonal symbol set identities (o-SSI[4]) in relation to na-Sequences and their complements.

In **Chapter 6** we shall dive into how to leverage the **ADM** and **MSS** to perform non-trivial sequence analyses. We shall utilize four scenarios starting with basic cases between two individual sequences of potentially unequal lengths, all the way to analyzing entire collections or populations based on their representative na-Sequences. We shall look at how to leverage the **TEA**[29][9] text-oriented, sequence-transformer chaining paradigm GPL to conduct some of the essential analyses proposed, readily via the command-line but also via any web browser on any major operating system, and especially with DNA/RNA sequences of any size stored in basic text files or merely held in strings. We shall see that using the concept of the **modal sequence statistic**, whether applied to individual na-Sequences, collections of them or just sub-sequences of longer DNA or RNA strands, can help us leverage statistical summary information about a sequence that might otherwise not be easy to glean or leverage from trivial inspection of a sequence. In this chapter too, particularly in **Section 6.6.1**, we shall develop an algorithm that might be utilized in applying artificial statistical intelligence to the matter of sequence classification problems relating to labeled datasets. The closely related **Section 6.6.2** shall consider the matter of identifying the **closest-sequence** from a collection of na-Sequences when given a particular query na-Sequence. Finally, **Section 6.7** shall consider the important matter of **optimal database storage of na-Sequences** as is typically found in real-world systems such as those that do whole-genome sequencing or in gene-banks, etc.

In **Chapter 7** we shall get into some deep mathematics around the critical problem of genome sequencing. We shall tackle the fundamental problem in genetics engineering by breaking it down into 3 major problems. **Problem G1** dealing with the matter of how to compute the longest common subsequence given any two sequences. **Problem G2** dealing with how, as happens during conventional genome sequencing, we have sections of two or more sequences that are not exactly the same, but which must be reduced to some **consensus sequence** that is most true to all the input/available disparate identifying sequences. And then in **Problem G3**, we shall consider how to arrive at the final best consensus sequence that also unifies all the disparate input sequences originally provided, into a single sequence that is then the consensus genome sequence of the entity under analysis.

We shall then turn our attention to the matter of how genetic code is translated into actual living organisms in **Chapter 9**, focusing at the molecular level, and shall tackle the matter of the

algorithm by which the most basic synthesis happens in a cell.

And then shall wrap up our core explorations in **Chapter 10**, from where we shall look at what results from actual processing of genome sequences or rather DNA code; the manifestation of some living thing, an organism or some aspect of it as specified in the underlying genetic code. We shall especially deal with minimal entities such as viruses and simple organelles. Most importantly though, we shall explore **genome expression** formally and conceptually so, with the assistance of a thought-experiment. A hypothetical genome-expression system — which we shall refer to as the **Lu-Genome Expression System** (LGES) shall be presented, and all our explorations shall be based on it. **Section 10.1** shall deal with a visual encoding method that leverages a digit-cipher named **OZIN**[30], and which is to be used to express storage-level genetic code (in decimal/numeric form), in its **intermediate form** — equivalent to DNA-to-mRNA transcription in living organisms. We shall finally turn attention to the final expression step — equivalent to mRNA-to-Protein/Function translation in nature, in **Section 10.2**, and shall get to see yet another innovation in the form of a genetic-code-to-spatial-form encoding based on a cipher we shall call the **Platonic Form Cipher** (**Section 10.2.1**). The essential algorithm for how to perform genome expression all the way from storage code into the final organism is depicted in **Algorithm 6**, and we shall close with some examples of complete, though hypothetical, fully-expressed genomes in **Section 10.2.3**.

The overall conclusion of the book is presented in **Chapter 11**, where we shall especially find properly distilled **KPIs** — metrics concerning what the major contributions of this work are — in literature, in mathematics, in genetics, in computer science, in philosophy, the humanities and arts. That chapter too, in **Section 11.1**, also clearly outlines the major problems, concerns, issues and areas of future exploration and research that this book proposes or could not immediately resolve in the present edition, and which are then left as **future directions** or **future work** for the reader, the student, the author, as well as any other interested or concerned researcher.

Finally, there are a few extra materials that have been set aside in the appendix, and these either augment or support earlier chapters in the book. In particular, we shall find:

1. **Chapter A** that covers 6 alternative functions that DNA plays in a living organism apart from protein synthesis which we mostly focus on in much of this book.
2. **Chapter B** that offers insights into where and how the interesting **OZIN Cipher** that we use in the **LGES** came to be.
3. **Chapter C** that covers the matter of how to compute **PFA**s for particular na-Sequences.
4. **Chapter D** that presents the formal distillation of all the essential foundational results (10) and proposed general [theoretical] applications (10) of transformatics, these being presented in this book for the first time, as a verbatim reprint of the material originally presented in the “**Transformatics 101 explained**” mini-thesis[23].
5. **Chapter E**, comprising of material and theorizing that comes a little later than the material originally in [23], and which attempts, for the first time ever, to rigorously situate the newly proposed sub-field and theory of transformatics, in the grand spectrum of most of mathematics as well as applied mathematics.
6. **Chapter F** that deviates from mere genetics, and demonstrates how Transformatics might be applied in any other mathematical science, with a focus on Physics and just one interesting problem - **teleportation** and **time travel**.

Chapter 2

Sequence Symbol Sets and Sequence Transforms Applied to Genetic Code

NOTE:

On **Potentially Similar Mathematics**: *Frobenoids*?

Frobenioids[31], as introduced by **Prof. Shinichi Mochizuki**, offer a category-theoretic abstraction of arithmetic structures such as divisors and line bundles, encoding transformations via morphisms that resemble symbolic rewritings under algebraic constraints[32]. In spirit, they parallel the logic of sequence transformers (from transformatics[5]), which operate by mapping one symbolic sequence to another through rule-based transformations constrained by set-theoretic, statistical or general mathematical logic. However, even though both systems might serve as structured environments for encoding and navigating symbolic changes across mathematical objects such as general sequences and sets, I just wanted to bring this up here, so students and researchers interested in our transformatics mathematics, and who wish to take the discussions and ideas I present in this work to even more advanced levels or in unconventional directions, to readily pick a leaf and comfortably proceed to do so^a.

^aAlso refer to **Chapter E** for more material — even more detailed — concerning contrasting transformatics to several other existing fields of mathematics and [applied] mathematical theories.

For the purposes of appreciating transformatics from a genetics engineering or general genetics research perspective, it shall be important to note that, like in the original transformatics paper[5], beginning by appreciating that whatever formalisms and mechanics we might develop or talk about concerning genetic code or rather DNA, had better begin with a realization that we can model DNA as merely an ordered set¹ — essentially, “a sequence” of symbols.

In the introduction (see **Chapter 1**), I have already called out both the names and symbols assigned to the most basic units of any genetic code; essentially, the *nucleobases* (or rather, nucleotides). For purposes of simplifying our mathematical logic later on, we shall here neatly define what roles these units play in the grand scheme concerning DNA and RNA code. Basically, we shall want to define the symbol sets for any DNA sequence and the symbol set for any RNA sequence.

¹This terminology to be used with precaution as we later clarify in **Section E.1**, since for the case of collections of symbols such as DNA, not only does order matter, but also, and unlike standard sets, duplication of elements or members is allowed — thus the preference for the more technically correct term; “sequence” — and unlike what some might find elsewhere, there is no need to stress that sequences are ordered sets by repeating ourselves when we say “ordered sequences” instead of just “sequence”!

Definition 1 (The DNA Symbol Set, ψ_{DNA}). *For any possible sequence of standard deoxyribonucleic acid (DNA) for any possible living organism, the distinct nucleic acid base units are known as nucleotides[33], and these are essentially and exactly only four^a;*

- Adenine(A)
- Cytosine(C)
- Guanine(G)
- Thymine(T)

And these are mapped to their representative, distinct single-letter symbols as shown. Thus, any possible DNA sequence must always consist of only one or more of those elements and nothing else. Thus, we might sum this up, using the symbol set concept[2] as applied to sequences[5] as such:

$$\psi(DNA) = \{A, T, C, G\} \quad (2.1)$$

Equation 2.1 helps to appreciate the extra non-intuitive fact that the special ordering of DNA base symbols in the order A-T-C-G is what is conventionally accepted[34][33] or commonly found in most genetics literature^b.

However, and especially because, for transformatics, we wish to work with an **ordered symbol set**[4] and not just any possible symbol set so that we can apply mathematical logic that respects the ordering of terms in any sequence[5], we shall then assume a convention similar to how we might derive an ordered symbol set for a sequence of numbers in some base (the concept $\psi_\beta(\Theta)$ — see **Definition 5** in [4]), and given we are using Latin-Alphabet symbols (from ψ_{az} [5]) for ψ_{DNA} , we might as well better define the **Lexically Ordered DNA Symbol Set**, ψ_{DNA} as such:

$$\psi_{DNA} = \psi_{az}(\psi(DNA)) = \langle A, C, G, T \rangle \quad (2.2)$$

*For all practical purposes unless where we merely wish to emphasize adherence to the tradition of ordering the nucleotides by their pairing order, we shall essentially imply $\psi_{az}(\psi(DNA))$ or rather ψ_{DNA} when we talk of the **DNA Symbol Set**.*

^aActually, or rather, in general, for nucleic acid sequences, the bases are four for either DNA or RNA, but, there are also conventions that extend this set to 17 or more to cater for cases like where there might be ambiguity about what the exact nucleotide in a particular position might be[33] — possibly during automated inference or sequencing of DNA sequences.

^bIt shall be important to bring it out at this point, that, especially for non-domain experts — people not trained in or normally practicing in genetics or related fields, that the common ordering of the nucleotides in the A-T-C-G ordering might seem unconventional or peculiar! For example, one might wonder, why are they not listed in their alphabetical order? So, for purposes of settings things clear for everyone, the author established[32], concerning this matter, that: “The order A, T, C, G isn’t alphabetical, and yet it’s the most commonly used sequence when referring to DNA bases.” We further learn that, the order A-T-G-C reflects a mix of historical usage and bio-chemical structure; **base-pairing logic** that the DNA’s double-helix is stabilized by “complementary base pairing” in which A pairs with T (via 2 hydrogen bonds), and C pairs with G (via 3 hydrogen bonds), so that listing A with its partner T and then C with G emphasizes this pairing symmetry[32]. Further, we learn that early molecular biology texts and sequencing protocols (especially post-Watson & Crick, 1953) adopted this order to reflect the “functional relationships” between bases. It became entrenched in educational materials, sequencing software, and databases. And lastly, that in visual and structural conventions — such as in diagrams (see **Figure 1.2**) and models, A–T and C–G are often shown side-by-side. Listing them in this order reinforces the **duality** of the DNA ladder’s rungs[32]. Lastly, that though there is no single documented moment when this convention begun, that the A-T-C-G order likely solidified in the **1970s - 1980s** during the rise of **Sanger sequencing** (developed in the 1970s), GenBank and EMBL databases, and overall in textbooks and molecular biology curricula.

Definition 2 (The **RNA Symbol Set**, ψ_{RNA}). *For any possible sequence of standard ribonucleic acid (**RNA**) for any possible living organism, the distinct nucleic acid base units are known as nucleotides, and these are essentially and exactly only four;*

- Adenine(A)
- Cytosine(C)
- Guanine(G)
- Uracil(U)

And these are mapped to their representative, distinct single-letter symbols as shown. Thus, any possible RNA sequence must always consist of only one or more of those elements and nothing else. Thus, we might sum this up, using the symbol set concept as applied to sequences as such:

$$\psi(RNA) = \{A, U, C, G\} \quad (2.3)$$

Equation 2.3 helps to appreciate the traditional ordering of RNA base symbols in the order A-U-C-G reminiscent of the natural base-pairing order of DNA after it is translated into RNA (U merely replacing T)[35]. And as with DNA, we shall want to have a proper, meaningful **ordered symbol set** for the RNA base symbols that we can later use in mathematical logic. Thus we shall equivalently define one for any standard RNA sequences as:

$$\psi_{RNA} = \psi_{az}(\psi(RNA)) = \langle A, C, G, U \rangle \quad (2.4)$$

*And for all practical purposes in this work as well as after, we shall essentially imply $\psi_{az}(\psi(RNA))$ or rather ψ_{RNA} when we talk of the **RNA Symbol Set** or the **Lexically Ordered RNA Symbol Set**.*

Now that we have ψ_{DNA} and ψ_{RNA} well defined, we might immediately apply them to their finest use here: defining and supporting the special symbol set, ψ_{na} , that would or could allow for several kinds of nucleic acid sequences — such as DNA for code stored in chromosomes and mitochondria, various kinds of RNA (*mRNA*, *tRNA*,...) and even synthetic/artificial/conceptual and also **random nucleic acid sequences**, etc. using a single universal nucleic acid symbol set. Thus we define ψ_{na} below:

Definition 3 (The **Universal Nucleic Acid Symbol Set**, ψ_{na}). *The **Universal Nucleic Acid Symbol Set**, ψ_{na} , is the set combining the ordered symbol sets for DNA and RNA sequences, and thus is comprised of the symbolic elements $\{A, C, G, T, U\}$.*

We shall appreciate how relevant and important **Definition 3** is, by realizing that any nucleic acid sequence, Θ , obeys the following law:

Law 1 (1 Universal Nucleic Acid Identifier Symbol Set: ψ_{na}). *The universal symbol set ψ_{na} is spanned by any natural nucleic acid sequence Θ .*

Proof. $\psi_{na} = \psi_{DNA} \cup \psi_{RNA} = \langle A, C, G, T, U \rangle \quad \wedge \quad \forall \omega \in \Theta, \omega \in \psi_{DNA} \vee \omega \in \psi_{RNA}. \quad \square$

As with many kinds of sequences dealt with in transformatics, the possession of a particular symbol set, such as ψ_{na} , allows for the practical use of such a set to implement logic in systems that can operate on signal based on symbolic expressions of well ordered elements in sequences or sub-sequences. For this matter then, we shall likewise want to make formal, the concept of a **na-Sequence**

Definition 4 (The **na-Sequences** — (D/R)-NA Sequences: $\Theta_{na} : \mathbb{N} \times \psi_{na}$). *Any sequence of either DNA or RNA nucleotides, or both — basically a sequence of nucleic acids, expressible as some sequence of symbols from ψ_{na} is a **na-Sequence** and its symbol set is potentially a superset of both ψ_{DNA} and ψ_{RNA} . Equivalently:*

$$\forall \Theta_{na} = \langle a_i, \rangle, \quad a_i \in \psi_{na} \implies a_i \in \psi_{DNA} \vee a_i \in \psi_{RNA} \quad (2.5)$$

If it is not immediately clear what the significance of **Defition 1** is or why it's important to unambiguously define ψ_{DNA} as well as the other basic formal symbol sets we have just formalized, then perhaps the mathematical discussions in later sections like **Chapter 4** and **Chapter 10** might help make this more obvious.

So, with those very essential definitions out of the way, we can begin to think of how we might appreciate and apply concepts from transformatics in genetics, in a clearer, straight forward manner.

For starters, we can certainly say that irrespective of how long or from where a particular DNA sequence, Θ_{DNA} , originated from, once any sequence of data (e.g unstructured or unprocessed raw genetic code sequence dump), a string (say a proprietary encoding of some DNA code), a name (say of a particular species of interest and whose actual/representative genome sequence is know) or even a number (an id of a genome sequence in some standard genome database, etc.) is mapped to standard DNA sequence code, we shall know that any such transformation or mapping obeys the following theorem:

Theorem 1 (ψ_{DNA} as the alphabet underlying any DNA sequence). *For any sequence of DNA, Θ_{DNA} , derived from some source sequence Θ , irrespective of whether that source is itself a DNA sequence or not, we know that such an encoding or mapping, if it results in a valid DNA sequence $\Theta_{DNA} : \mathbb{N} \times \psi_{DNA}$, obeys the following general transformation:*

Transformation 1. $\Theta \rightarrow \Theta_{DNA};$

$$\forall a_{i \in [1, \mathcal{L}(\Theta_{DNA})]} \in \Theta_{DNA} \quad \exists \rho \in \psi_{DNA} : a_i = \rho, \quad \forall i \in \mathbb{N}$$

Proof. Assume Θ_{DNA} contains some symbol α such that $\alpha \notin \psi_{DNA}$, it would contradict **Definition 1** which clearly dictates the legitimate membership of any DNA sequence. $\square$

And definitely, the equivalent truth for RNA sequences would likewise follow from **Definition 2**. So, we can then correctly tell that a sequence such as $\Theta_{insulin}$, which is the genetic code sequence defined as:

$$\Theta_{insulin} = \begin{array}{l} \text{ATGGCCCTGTGGATGCGCCTCCTGCCCCCTGCTGGCGCTGCTGGCCCTCTGGGGACC} \\ \text{CCAGCCGCAGCCTTTGTGAACCAACACCTGTGCGGCTCACACCTGGTGAAGCTCTCTAC} \\ \text{CTAGTGTGCGGGGAACGAGGCTTCTTCTACACACCCAAGACCCGCCGGGAGGCAGAGGAC} \\ \text{CTGCAGGTGGGGCAGGTGGAGCTGGGCGGGGGCCCTGGTGCAGGCAGCCTGCAGCCCTTG} \\ \text{GCCCTGGAGGGGTCCCTGCAGAAGCGTGGCATTGTGGAACAATGCTGTACCAGCATCTGC} \\ \text{TCCCTCTACCAGCTGGAGAACTACTGCAACTAG} \end{array} \quad (2.6)$$

And which sequence², despite having been obtained from an authority — the **Leiden Open Variation Database (LOVD)**, which is based on NCBI's RefSeq data[36], might need to be verified by hand or with a critical eye, and that's where a law such as **Theorem 1** would come in handy. Also, note that, unlike common sequence notations we might see in this work or that we have encountered before, we wrote $\Theta_{insulin}$ in a manner somewhat more convenient for the kind of verbose sequence that any actual na-Sequence code generally is. The notation; without opening or closing brackets around the sequence terms and neither commas betwixt them — reminiscent of the *String [Chart] Sequence* notation we developed recently (see **Transformation 15** in [5]).

We might for example start to wonder, how might we go about determining if some two DNA sequences, Θ_1 and Θ_2 are the same or perhaps if they share parts of each other, and by how much? We might wonder if they are the same sequence merely shuffled/anagrammatized — since we saw that this can occur during natural processes such as meiosis[26]. In case they are different, we might want to for example quantify the relative frequency of their distinct members, e.g mapping ψ_{DNA} or ψ_{RNA} to a sequence of relative frequencies, etc. These are all things we shall soon look into and know very well how to do or talk about, using genetics terms and the mathematical language and techniques from transformatics in the rest of this book.

² $\Theta_{insulin}$ as depicted here, is the short coding DNA sequence (CDS) for the human **INS** gene — the part that gets transcribed into mRNA and translated by ribosomes into the insulin protein[32]

Chapter 3

Sequence Abstraction using Symbol Sets: From Flat-Structure Sequences to Higher-Order Sequences

Especially for nucleic acid sequences, but also for any other kind of sequence Θ , there might be legitimate situations in which looking at or dealing with the flat-structure sequence — such as for the actually modest¹ $\Theta_{insulin}$ (refer to **Equation 2.6**), might be cumbersome, ineffective or unnecessary. And so, we are going to briefly look at ways that we might re-write verbose flat-sequences in more terse or higher-order **abstract ways** so as to focus on details that the flat-structure perhaps doesn't clearly express. This for example is naturally relevant in the processing of say nucleic acid sequences — since such is useful in scenarios where the processing of any such sequence is done in *non-unity n-tuples* or **n-grams** — such as with codons (nucleotide 3-grams) or as gene programs (self-contained protein synthesis or function-gene-program sequences composed using codons)², etc.

So, assuming we have some sequence of n symbols Θ_1 defined as such:

$$\Theta_1 = \langle a_1, a_2, \dots, a_n \rangle \quad (3.1)$$

If we wish to re-write the same sequence such that every k terms are grouped in the same subsequence, we can then re-write Θ_1 differently as such:

$$\Theta_1 \equiv \Theta_2 = \Theta_2(n, k) = \langle a_{11}, a_{12}, a_{13}, a_{1k}, a_{21}, a_{22}, \dots, a_{(i)(k)}, \dots, a_{(\frac{n}{k})(k-1)}, a_{(\frac{n}{k})(k)} \rangle \quad (3.2)$$

Of course, in **Equation 3.2** we have somewhat wanted to extrapolate well and illustratively, what would happen in an informative case such as when $k = 4$ and n is not only significantly

¹‘Modest’ because, genomes and realistic genome sequences can typically span millions of genes or billions of nucleotides. We for example know that for a typical human cell’s DNA there are ≈ 3.2 billion base pairs[37], and yet we know each base pair consists of two nucleotides, one for each strand of the double-helix (so that’s ≈ 6.4 billion), plus a few more ($\approx 16,569$ base pairs) from mitochondrial DNA[38]

²For purposes of re-using this relevant concept, we shall here define:

Definition 5 (A gene program). *In the domain of NA-processors, a **gene program**, $\mathbb{G}_p$ is any sequence of na-Sequence symbols, such that the sequence expresses or contains some sufficient instructions in form of genetic code, that specify what to do, and nothing else — this might for example be what protein or material to produce, or what DNA-driven function to perform — the protein is basically an effect, based on just the contents of the sequence — and nothing more, such that for any such sequence, its basic signature is $\mathbb{G}_p : \mathbb{N} \times \psi_{na} : p = \mathfrak{u}(\mathbb{G}_p)$. $\square$*

We shall find ways to leverage this definition to express formalisms about how living things and their properties or traits are driven by such computable, affectant and “well-ordered” sequences in the future. For example, a philosophy that all existence is but a composition of effects resulting from the processing of a basic genetic code:

Transformation 2. $\mathbb{G}_p \xrightarrow{O_{process}} \prod Effect_{i \in \mathbb{N}} \xrightarrow{O_{compose}} Existence$

larger than k , but is also its perfect multiple. Otherwise, we might more concisely write that same sequence generalization as:

$$\Theta(n, k) = \langle \langle a_1, a_2, a_3, a_4 \rangle_1, \langle a_5, a_6, \dots, a_8 \rangle_2, \dots, \langle a_{(n-k)}, \dots, a_{(n-k+k-1)}, a_{(n-k+k)} \rangle_{\frac{n}{k}} \rangle \quad (3.3)$$

So, this expression in **Equation 3.3** is our first proper appreciation of what higher-order sequences might be like when they are an abstraction of normal flat-structure sequences. We see for example here, that the $k = 4$ **bundling-parameter** means we shall process the original flat sequence³ k items at a time, and so, each subsequence in $\Theta_2(n, 4)$ then contains exactly (or at most) 4 items — we are assuming 4 is a factor of n to keep things simple. And so, we expect that, in total, we should have $\frac{n}{k}$ subsequences after applying the bundling transform:

Transformer 1 (The k -GRAM Generator). $\Theta = \langle a_1, a_2, \dots, a_n \rangle \xrightarrow{O_{bundle-k}(\cdot)} \Theta^* ;$
 $\Theta^* = \langle \langle a_1, a_2, \dots, a_k \rangle_1, \langle a_{k+1}, \dots, a_{2k} \rangle_2, \dots, \langle a_{(n-k+1)}, \dots, a_{(n-k+k-1)}, a_{(n-k+k)} \rangle_{\frac{n}{k}} \rangle$
 $\forall a_i \in \Theta \quad \exists a_{ij} \in \Theta^* : a_i = a_{ij} \quad \wedge \quad j \in [1, k]$
 For some $n, k \in \mathbb{N}$, and that $\mathcal{U}(\Theta^*) = \frac{n}{k} = \text{number of } k\text{-gram subsequences generated from } \Theta. \quad \square$

Thus, we see that the sequence depicted in **Equation 3.3** with $k = 4$ could as well be produced by the generator defined in **Transformer 1**. But not just that, if we applied that transformer to the DNA sequence $\Theta_{insulin}$ first presented in **Chapter 2** — see **Equation 2.6**, we can then produce a proper standard codon/3-gram mapping of that DNA sequence that would look something like:

$$\Theta_{insulin} \xrightarrow{O_{bundle-3}(\cdot)} \langle \langle \text{ATG} \rangle, \langle \text{GCC} \rangle, \langle \text{CTG} \rangle, \langle \text{TGG} \rangle, \langle \text{ATG} \rangle, \langle \text{CGC} \rangle, \langle \text{CTC} \rangle, \dots, \langle \text{ACC} \rangle \dots, \langle \text{TAC} \rangle, \langle \text{TGC} \rangle, \langle \text{AAC} \rangle, \langle \text{TAG} \rangle \rangle \quad (3.4)$$

And the way we have bundled up the nucleotides in **Definition 13**, **Equation 9.12** is exactly how a natural sequence processor such as a Ribosome (see details in **Transformer 14**) would process it so as to produce say the corresponding protein.

Of course, though we might not delve into it here or at the moment, we know that by chaining several sequence transformers — such that one operates on the output of the previous one to produce the next sequence (see **Transformation 2** as an example), we can arrive at even higher abstraction sequences that might render certain sequence processing programs easier to write, analyze or define. For example, we know that, much as a Ribosome Processor operates on a sequence of codons, and yet, any legitimate protein synthesis and genetic code sequence processing program will require some sort of necessary delimiters (such as the START and STOP codons — refer to the Protein Synthesis Flow Chart in **Figure 9.2**), and so that, we might (if we could), abstract away codons and instead generate from a flat-structure DNA or perhaps mRNA sequence, a higher-order n -gram sequence of genes or **gene-program sequences**⁴.

³Note that we might also refer to a “flat sequence” as a *first-order* sequence.

⁴We have defined gene programs in **Definition 5**.

Chapter 4

Sequence Analysis Using Symbol Sets

We now turn our attention to the matter of using ideas from transformatics to help analyze or understand genetic code sequences. For starters, we shall want to consider the matter of how to tell how far apart or dissimilar any two nucleic acid sequences might be. Of course, consequently, that would also translate into meaningful ways to tell how far apart not just low-level genetic-creations such as tissue, various cell-types, proteins, etc. might be based on the genetic code that specifies them, but also how different entire organisms — made up of all these things, and within the same species or not, might be¹.

Because we can learn much about an organism based on its characteristic genome, and since that can be mapped to a flat-structure sequence of symbols such as we have already encountered in earlier sections, it might start to make sense to leverage the similarity/distance measures we have already developed so as to apply them to quantifying the *formal distance* between say actually different species.

In an earlier paper[3], we have introduced and also demonstrated how a sequence-analysis measure called the **Anagram Distance Measure**, $\tilde{A}(\Theta, \Theta^*)$, might be used to quantify how far apart some two sequences Θ and Θ^* , that for the simplest scenarios better be of the same length, can be compared based on their common membership and the relative lexical ordering of the members in either sequence.

In the basic analysis we shall use to illustrate or develop our analytic concepts, we shall deal with hypothetical nucleic acid sequences mostly — especially DNA sequences, and their lengths shall be kept short to keep our analyses simple. Moreover, their membership or composition — in terms of nucleotides, might have nothing to do with actual/natural na-Sequences out there in the wild. The concepts and ideas we shall develop though, despite how we are going to develop them, should still be readily applicable to any or most nucleic acid sequences if not any kind of sequences in general.

4.1 Testing Sequences for Membership in a Base

First, assume we have the following three sequences

1. $\Theta_1 = CATGGGACTGCC$
2. $\Theta_2 = ATAATAAGAGGGATCTGA$
3. $\Theta_3 = AUAGGGAGAAUC$

We can start by attempting to answer the question: **Which of these sequences are DNA and which are RNA?**

So, to answer that question properly — *provably* so, we can start by reducing each of those sequences to their respective [ordered] sequence symbol sets. In fact, if we first relax the assumption

¹So that then, we perhaps can also talk of *genome analysis*.

that they are either DNA or RNA sequences, and merely look at them as though they were any kind of sequence (for which we don't know the base or base-symbol set yet), then we can merely use the definition of an **Unspecific Symbol Set of Θ in Any Base** — see **Definition 4** in [4]. The algorithm for how to do that is simple and is well presented in that definition — we basically create another sequence, in which we insert each distinct symbol from Θ that we encounter while processing the sequence from Left-to-Right². Thus, we might construct the relevant transformer for this as such:

Transformer 2 (Sequence Unspecified Symbol Set Generator).

$$\Theta \xrightarrow{O_{suss-gen}(\cdot)} \Theta^* \quad ; \quad \mathcal{L}(\Theta^*) \leq \mathcal{L}(\Theta) \quad \wedge \quad \Theta^* = \hat{\psi}(\Theta)$$

And thus, applying that transformer³ to the three sequences we have, we shall obtain the following results:

$$\textbf{Transformation 3. } \Theta_1 \xrightarrow{O_{suss-gen}(\cdot)} \langle C, A, T, G \rangle = \hat{\psi}(\Theta_1)$$

$$\textbf{Transformation 4. } \Theta_2 \xrightarrow{O_{suss-gen}(\cdot)} \langle A, T, G, C \rangle = \hat{\psi}(\Theta_2)$$

$$\textbf{Transformation 5. } \Theta_3 \xrightarrow{O_{suss-gen}(\cdot)} \langle A, U, G, C \rangle = \hat{\psi}(\Theta_3)$$

At this juncture, we then can more closely inspect the original/source sequences via these special resultant sequences — each of them a kind of symbol set, and then judge which of ψ_{DNA} or ψ_{RNA} they are best associated with. To proceed in a rigorous manner, we might also want to compute the **Natural Symbol Set of Θ in some Base- β** — see **Definition 5** in [4]. A suitable transformer to compute such a symbol-set (for which, unlike the unspecific symbol set, orders the terms in the resultant sequence by their natural order of occurrence in the base's ordered symbol set) would be as such:

Transformer 3 (Sequence β -Natural Symbol Set Generator).

$$\Theta \xrightarrow{O_{snss-gen-\beta}(\cdot)} \Theta^* \quad ; \quad \mathcal{L}(\Theta^*) \leq \mathcal{L}(\Theta) \quad \wedge \quad \Theta^* = \psi_{\beta}(\Theta)$$

And, since we already have an idea what the symbol sets for each of the three sequences are — from which we can guess and also disqualify some candidate bases — e.g, since $\hat{\psi}(\Theta_3) \setminus \psi_{DNA} = \{U\}$, then Θ_3 can't be a DNA sequence. However, to complete our analysis, note that:

$$\textbf{Transformation 6. } \Theta_1 \xrightarrow{O_{snss-gen-DNA}(\cdot)} \langle A, C, G, T \rangle = \psi_{DNA}(\Theta_1)$$

$$\textbf{Transformation 7. } \Theta_2 \xrightarrow{O_{snss-gen-DNA}(\cdot)} \langle A, C, G, T \rangle = \psi_{DNA}(\Theta_2)$$

$$\textbf{Transformation 8. } \Theta_3 \xrightarrow{O_{snss-gen-RNA}(\cdot)} \langle A, C, G, U \rangle = \psi_{RNA}(\Theta_3)$$

So, we can safely conclude that:

²Such that the leftmost term is the first to be processed.

³Please, be careful with **Transformer 2** — in particular, note that, the special notation employed for the unspecific symbol set, $\hat{\psi}(\Theta)$, shouldn't be confused with that of the somewhat related, though entirely different notation and notion of an unspecific symbol-set's modal sequence, $\hat{\psi}^>(\Theta)$ or the modal sequence statistic of a sequence's symbol set, $\psi^>(\Theta)$. Refer to **Chapter D** for more on these other concepts.

1. Since $\psi_{DNA}(\Theta_1) = \psi_{DNA}$, then Θ_1 is a DNA sequence.
2. Since $\psi_{DNA}(\Theta_2) = \psi_{DNA}$, then Θ_2 is a DNA sequence.
3. Since $\psi_{RNA}(\Theta_3) = \psi_{RNA}$, then Θ_3 is a RNA sequence.
4. Even if we forced it, note that, $\psi_{RNA}(\Theta_1) = \langle A, C, G, T \rangle \neq \psi_{RNA}$, or rather that $\hat{\psi}(\Theta_1) \setminus \psi_{RNA} = \{T\}$ so Θ_1 can't be an RNA sequence.

Talking of which, in case we had some variation of Θ_1 that has no instances of T in it, such as the sequence $\Theta_4 = CAAGGGACAGCC$, then we shall find that:

Transformation 9. $\Theta_4 \xrightarrow{O_{snss-gen}(\cdot)} \langle C, A, G \rangle$

and that:

Transformation 10. $\Theta_4 \xrightarrow{O_{snss-gen-DNA}(\cdot)} \langle A, C, G \rangle \subset \psi_{DNA}$

But also that:

Transformation 11. $\Theta_4 \xrightarrow{O_{snss-gen-RNA}(\cdot)} \langle A, C, G \rangle \subset \psi_{RNA}$

And thus, we have the peculiar case in which a nucleic acid sequence can both be a legitimate DNA or RNA sequence!

4.2 Symbol Sets of n-GRAMS and Testing Sequences for n-GRAM-based Properties

Next, we are going to consider the matter of whether some sequence conforms to some desired sequence characteristic or not, based on some aspect of either its symbol set or its composition despite the symbol set. In particular, we shall consider the matter of processing na-Sequences at 3-gram level as codons, and consider the special matter of START and STOP codons in relation to protein synthesis gene programs⁴ for example.

First, given the START-codon symbol set, $\psi_{na-START}$, a sequence of all the “start” kind codons whether of DNA or RNA type, is defined as such from all we currently know:

Definition 6 (The **Nucleic Acid START-codons**). *For any known organism, prokaryote or eukaryote, the only legitimate and known codons of the START-kind are any of the codons or START 3-grams in the unordered set $\psi_{na-START}$ — the **nucleic acid START-codon symbol set**.*

$$\psi_{na-START} = \{ATG, AUG, GTG, GUG, TTG, UUG\} \quad (4.1)$$

So that, if say a nucleic acid program for producing some protein via some gene program Θ_{na} were to be processed by an ideal ribosome (see **Transformer 14**), the ribosome would never produce anything — or, equivalently, there is no guarantee that any protein would be produced no matter what or any instructions the gene program contains, as long as it doesn't contain any one of the members of $\psi_{na-START}$.

⁴Note that **gene programs** were formally introduced in **Definition 5**.

One interesting consequence of that definition is the following law:

Law 2 (Non-Coding Gene Programs^a). *If any gene-program Θ_{na} is processed by a ribosome, and yet it has the property:*

$$\psi_{DNA}(\Theta_{na}) \cap \psi_{na-START} = \emptyset \quad \vee \quad \psi_{RNA}(\Theta_{na}) \cap \psi_{na-START} = \emptyset$$

It implies that Θ_{na} shall never be transcribed by the ribosome^b, and thus shall never result in any new protein or new amino-acid product within the containing cell system^c.

^aAlso known as **Introns** in some contexts.

^bEquivalently, that it shall never trigger the start of a transcription process.

^cRefer to ribosome definition for details: **Definition 13**.

So, that, if say a [potentially infinitely long] nucleic acid program for producing some protein via some gene program Θ_{na} were to be processed by an ideal ribosome processor (see **Definition 13**), the ribosome would **never start production** — or, equivalently, there is no guarantee that any products — neither amino-acid, nor completed/new protein, shall be synthesized by the ribosome if the gene program it processed didn't contain the necessary START-codon anywhere within it⁵.

Of course, though it might not be entirely unlikely that such *weird* non-coding gene program sequences exist — and there might be nothing wrong with them being natural too, however, we shall want to learn more about this from the domain experts in the future⁶.

That said, another, closely related case is that of the consequences of the STOP-codons or rather, the STOP-codon symbol set, $\psi_{na-STOP}$ defined as such:

Definition 7 (The Nucleic Acid STOP-codons). *For any known organism, prokaryote or eukaryote, the only legitimate and known codons of the STOP-kind are any of the codons or STOP 3-grams in the unordered set $\psi_{na-STOP}$ — the **nucleic acid STOP-codon symbol set**.*

$$\psi_{na-STOP} = \{TAA, UAA, TAG, UAG, TGA, UGA\} \quad (4.2)$$

Among important consequences of **Definition 7**, is that, if say a *non-STOPPING* nucleic acid program for producing some protein via some gene program Θ_{na} were to ever be processed by an ideal ribosome (see **Definition 13**), the ribosome might [start to] produce something — some amino-acids for example, but **would never return** — equivalently, there is no guarantee that any protein would be released by the protein manufacturing process/ribosome, no matter what or how many amino-acid productions and translate instructions the gene program contains and which have been executed by the ribosome. As long as the gene-program doesn't contain any one of the members of $\psi_{na-STOP}$ that is.

It is not immediately clear what the plausibility of such an *evil gene-program* existing out there in nature might be, but given normal genes sometimes mutate[19], it can't be ruled out that such awkward gene programs might exist. Though we won't dive into that here, who knows... from a computer security perspective, such a program might cause system errors or faults such as a System-Out-Of-Resources problem given the ribosome might attempt to produce an infinite length amino-acid chain⁷, or perhaps that the cell's protein manufacturing closure might run out

⁵These are examples of *genetic pathologies* that one might uncover via careful sequence analysis.

⁶**FUTURE WORK:** We for example might wish to know: do such malicious (because they might merely waste the body's processing resources or even space) gene programs naturally exist? Are they the result of some disease/genetic anomaly or are they merely accidents when they occur? Could they have some useful properties or consequences too? Etc.

⁷**FUTURE WORK:** It might be worth exploring if in fact nature or for particular natural, biological systems, there is some hard limit that would rule-out the possibility of a ribosome attempting to manufacture an infinite-length amino-acid chain for example. Or are such pathologies [im-]possible? What are they, and are they the result of such gene-program anomalies as we talk of here or not?

of space, or that the processor might end up hanging since the factory never is able to reach any of WAIT, DETACH or RESET states — see **The Ribosome State Machine** in **Figure 9.3** but also refer to the Protein Synthesis Process in **Figure 9.2** to clearly, logically appreciate these kinds of problems and corner-case scenarios.

4.3 Concerning Genetic Pathologies based on Faulty Gene-Programs

With the two major problematic gene-program scenarios we have explored in the preceding section, note that, in case a normally correct gene-program such as what we saw in $\psi_{insulin}$ (see **Equation 2.6**) is altered — perhaps by a natural mutation, or perhaps by a virus, or even in a totally malicious feat of gene-manipulation medicine or bio-engineering/bio-hacking, and such that a genetic code transformation occurs within the body’s system that results in a nucleic-acid sequence satisfying any of the above queer cases, who knows, but it might as well be the root case of some difficult flaw in the organism — could be an incurable and fatal [albeit hereditary] disease (especially if the faulty gene-program is inculcated into the basic genetic code at storage or during reproduction). Such might also be fatal, even when nonhereditary, in case it impacts some crucial or normal life support systems, and that the problem thus manifested has no workarounds nor simple way to be remedied, and that such might be the kinds of **hard problems for geneticists**, that might require not just medical doctors, biologists or perhaps pure-genetists to tackle, but also people like computer hackers, information-security experts and software debuggers⁸ to tackle and resolve them well — essentially, the kinds of problems that only careful and thoughtful na-sequence analysis and processing (such as with transformatics) might solve.

⁸It shouldn’t come as a surprise, that with a stronger marriage of computing theory and biology — or rather medicine in this case, that certain problems in nature — not just with humans or animals, but also with plants for example, and especially if they are either inheritable — meaning genetic, or if they might somehow be remedied via clever hacking or debugging and modifying of the organism via *special* [perhaps, *artificially introduced*] gene programs, might call for the skills of and expertise of software engineers and program debuggers from hardcore computer science and not just the medical or biological sciences! A great background work on this subject by the author might come in handy — refer to [39]

Chapter 5

Sequence Analysis Using Complement Sets

The idea of complements when applied to na-Sequences was first brought up in **Chapter 1**, and we get to see it depicted in an exemplary typical diagrammatic depiction of the DNA structure in **Figure 1.2**. However, apart from having talked about the fact that nucleic acids typically or naturally occur together in base-pairs depicting pairwise complements, we might wish to explore how to leverage transformatics to study this idea at a more general level, as well as from several different and overall useful perspectives. For example, we might wish to explore the matter of **na-Sequence** (as well as *any sequence*) complements — how they come up, how different kinds of sequence complements could be arrived at algorithmically, how they could be exploited,... and thus this section.

Assume we start by exploring simple binary sequences — especially because bits help generalize many ideas in theory and practice. We can for example start by noticing that, like in the case of the interesting **Lu-Number System**[1] — in which any kind of *basic information* can be abstracted by mapping it to either signal ($>$) or anti-signal ($<$), and that *complex information* can then be generated from that using transforms on basic LNS sequences or expressions, etc. We can start by looking at what could be said of the transforms on the binary sequence symbol set, ψ_{bin} — or better, ψ_{01} . In our formulations, we shall denote the complement of a symbol ρ by $\neg\rho$, and for a sequence Θ , with $\neg\Theta$.

Sequence Name	Sequence	Formula	Note
Θ1	0 1	ψ_{01}	Binary Symbol Set as a basic Sequence
Θ2	0 1 0 1	$\psi_{01} \cdot \psi_{01}$	Sequence Concatenation/Multiplication
Θ3	0 1 0 0 1	$\psi_{01}(1 + \psi_{01})$	Sequence as Linear Combination of other Sequences: case of memberwise addition of the above two sequences
Θ4	1 0 1 0	$\neg\Theta2 = \neg\psi_{01} \cdot \neg\psi_{01}$	Basic Complement Transforms
Θ5	1 0 1 1 0	$\neg\Theta3 = \neg(\psi_{01}(1 + \psi_{01})) = \neg(01001)$	Complement of a Formula

Table 5.1: An example of exploring symbol and sequence complements in sequence transformations

So, we can begin by noting that, if some sequence Θ spans some symbol set ψ_β — such as how all the sequences in **Table 5.1** span ψ_{01} , then we can say that the complement sequence, $\neg\Theta$, spans the complement symbol set $\neg\psi_\beta$. But not just that. You can readily tell from studying that table, that actually, if sequence Θ spans ψ_β , then it also spans $\neg\psi_\beta$ — meaning, its members essentially are one of the distinct elements from either symbol set even though the two sets might not necessarily have the same ordering of their members¹.

Further, note that, if ψ_β is the symbol set of some sequence Θ , and that the **complement symbol set** it corresponds to is $\neg\psi_\beta$, then we can comfortably say that both Θ and $\neg\Theta$ span $\neg\psi_\beta$ as well. This is true, because, for any sequence Θ , its complement, $\neg\Theta$ is the sequence of the memberwise complements of Θ . That is to say:

$$\psi_\beta = \neg(\neg\psi_\beta) \quad (5.1)$$

and

$$\Theta = \neg(\neg\Theta) \implies \neg\Theta = \neg(\Theta) \quad (5.2)$$

¹However, things might not necessarily be that simple for starters. Take the case of the last 3 rows in **Table 5.1**. How do we know how to open the brackets in the case of evaluating $\Theta3$ for example? And then for $\Theta4$, is it correct to apply the complement to each of the terms in the formula for $\Theta2$ or not? And then the last case — of $\Theta5$! Note that in this last case, things might start to get even more complicated; for example, it is not clear if it might be correct to first evaluate the formula before applying the complement operator to the items in the sequence — in which case we have $\neg(01001) = 10110$ (or even just 110 in case we treat these sequences as numbers); or if it is okay to apply the complement to the formula's items before evaluation — such as with $\neg(\psi_{01}(1 + \psi_{01}))$ that might result in a different 10010 — essentially, it starts to become obvious that we need to resolve how this complement operator works, and what properties it has when applied to either terms, sequences of terms or combinations of sequences of terms! Somehow, the beginnings of a special sequence algebra or perhaps a *sequence complement algebra*...

So that, if

$$\psi_\beta = \langle \prod_{i=1}^n a_i \rangle \implies \neg\psi_\beta = \langle \prod_{i=1}^n a_{n-(i-1)} \rangle \quad (5.3)$$

Note that **Equation 5.3** casts complementing some sequence $\Theta\langle\theta_1, \theta_2, \dots, \theta_n\rangle$ as $\neg\Theta\langle\theta_n, \theta_{n-1}, \dots, \theta_1\rangle$. And for any $\Theta : \mathbb{N} \times \psi_\beta$ we then know that:

$$\neg\Theta = \prod_{i=1}^n \neg(a_i) \quad (5.4)$$

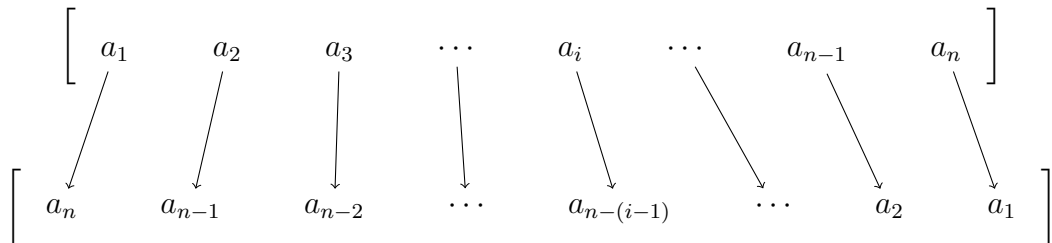
for $n = \mathcal{L}(\Theta)$ and that $\forall a_i \in \psi_\beta$,

$$\neg(a_i) = \rho_i \in \neg\psi_\beta \quad : i = I(a_i, \psi_\beta) = I(\rho_i, \neg\psi_\beta) : 1 \leq i \leq (n = \mathcal{L}(\psi_\beta) = \mathcal{L}(\neg\psi_\beta)) \quad (5.5)$$

So, one way to appreciate **Equation 5.5**, is by realizing that by knowing the members and their order in ψ_β , we can then systematically determine the members and ordering of $\neg\psi_\beta$ via **Equation 5.3**, and so that, for some a_i in ψ_β , its complement in $\neg\psi_\beta$ is the term at position index $n - i$ in ψ_β . Thus, as we saw in **Table 5.1**, for $a_0 = 0$ in ψ_{01} , the complement, $\neg a_0 = a_{\mathcal{L}(\psi_{01})-0} = a_2 = 1$. This mechanics can be extended or applied to sequences of arbitrary length and composition as necessary. For purposes of future use, we shall generalize this with a useful theorem as such:

Theorem 2 (Complement Sequences). *For any sequence $\Theta_n\langle\prod_{i=1}^n a_i\rangle$, its equivalent **complement sequence**, simply denoted $\neg\Theta_n$, is the sequence with the signature $\Omega_n\langle\prod_{i=1}^n \rho_i\rangle$ such that $\rho_i = a_{n-i} \quad \forall a_i \in \Theta_n$. We shall also write Ω_n as $\neg\Theta_n$ where necessary.*

Proof. By **Equation 5.3**, we see that if $\Theta_n = \langle a_1, a_2, a_3, \dots, a_{n-1}, a_n \rangle$, then we can derive its complement $\neg\Theta_n$ via the mapping of its members to corresponding members in its exact lateral inverse as such:



□

And since we are dealing with sequence transformations in this work, we might as well define the necessary transformer:

Transformer 4 (The Complement Transformer).

$$\Theta \xrightarrow{O_{complement}(\Theta)} \Theta^*; \quad \Theta^* = \neg\Theta$$

5.1 Significance of Sequence Complements, $\neg\Theta$, and Complement Symbol Sets, $\neg\psi_\beta$

First, we shall momentarily return to binary sequences.

We note that, for some binary sequence $\Theta : \mathbb{N} \times \psi_{01}$, the transform

$$\textbf{Transformation 12. } \Theta \xrightarrow{O_{complement}(\cdot)} \Theta^*; \Theta^* = \neg\Theta$$

which applies **Transformer 4**, produces a sequence of the same length as Θ but with all bits swapped/replaced by their complements, which, as per usual nomenclature in computer science for example, means the resultant sequence is produced from the source via a **bitwise NOT operation**, and which, for binary sequences, is as simple as merely flipping the bits individually.

Thus, if our source sequence was

$$\Theta = \langle 01011110 \rangle \tag{5.6}$$

Then

$$\textbf{Transformation 13. } \Theta \xrightarrow{O_{complement}(\Theta)} \langle 10100001 \rangle$$

And then, returning to nucleic acid sequences, we note that if we for example have some na-Sequence such as

$$\Theta = \langle ATCGCGTAT \rangle \tag{5.7}$$

That we wish to treat as a DNA-sequence, then its *na-Complement* sequence would be derivable from ψ_{DNA} as such:

Since $\psi_{DNA} = \langle A, C, G, T \rangle$, then by **Equation 5.3**, we know that:

$$\neg(\psi_{DNA}) = \neg\psi_{DNA} = \langle T, G, C, A \rangle \tag{5.8}$$

So that, $\forall \rho \in \psi_{DNA}$, the equivalent complement is via the mapping:

$$\begin{bmatrix} A & C & G & T \\ \downarrow & \downarrow & \downarrow & \downarrow \\ T & G & C & A \end{bmatrix}$$

Which is also the consequence of **Theorem 2**. Thus, for the sequence in **Equation 5.7**, the corresponding complement DNA sequence is:

$$\neg\Theta = \langle TAGCGCATA \rangle \quad (5.9)$$

This ideal can be further appreciated by noticing that the traditional DNA sequence base symbol set (which we shall denote by $\dot{\psi}_{DNA}$), is actually different from ψ_{DNA} , and is defined as:

$$\dot{\psi}_{DNA} = \langle A, T, C, G \rangle \quad (5.10)$$

We should come to appreciate this special symbol set, or even justify it — for logical and mathematical analyses, by acknowledging that it depicts not just a sequence of distinct symbols from ψ_{DNA} , but also their natural order based on both order of occurrence in ψ_{DNA} **and** their equivalent **symbol complements** in $\neg\dot{\psi}_{DNA}$! It is such a terrific discovery/realization. And before we proceed, we note that the $\dot{\psi}_{DNA} \rightarrow \neg\dot{\psi}_{DNA}$ mapping for pairwise-ordered DNA sequences is as such:

$$\left[\begin{array}{cccc} A & T & C & G \\ \downarrow & \downarrow & \downarrow & \downarrow \\ G & C & T & A \end{array} \right]$$

Because, from **Equation 5.10**, we can tell that:

$$\neg\dot{\psi}_{DNA} = \langle G, C, T, A \rangle \quad (5.11)$$

Perhaps, and worth checking out, even if momentarily, what would happen when the idea of complement symbols sets and complement sequences is applied to the more commonplace base-10 sequences (aka “usual numbers”²)?

First, note that since ψ_{10} — our choice of symbol for the decimal symbol set, is expressed as such:

$$\psi_{10} = \langle 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \rangle \quad (5.12)$$

Then, by **Theorem 2**, its complement is:

²And this shall not be a waste of space and time, since, as we shall find out in **Chapter 10**, base-10 symbols could underlie a very straight-forward system for implementing or theoretically exploring genome expression, a very important topic in all genetics!

$$\neg\psi_{10} = \langle 9, 8, 7, 6, 5, 4, 3, 2, 1, 0 \rangle \quad (5.13)$$

So that, for any base-10 digit, the corresponding **complement digit** is as in the mapping below:

$$\begin{array}{cccccccccc} \left[\begin{array}{c} 0 \\ \downarrow \\ 9 \end{array} & \begin{array}{c} 1 \\ \downarrow \\ 8 \end{array} & \begin{array}{c} 2 \\ \downarrow \\ 7 \end{array} & \begin{array}{c} 3 \\ \downarrow \\ 6 \end{array} & \begin{array}{c} 4 \\ \downarrow \\ 5 \end{array} & \begin{array}{c} 5 \\ \downarrow \\ 4 \end{array} & \begin{array}{c} 6 \\ \downarrow \\ 3 \end{array} & \begin{array}{c} 7 \\ \downarrow \\ 2 \end{array} & \begin{array}{c} 8 \\ \downarrow \\ 1 \end{array} & \begin{array}{c} 9 \\ \downarrow \\ 0 \end{array} \right] \end{array}$$

Such a peculiar cipher! Because, usual/familiar number sequences such as the **Hi-Fi o-SSI**[3], defined as such:

$$\Theta_{HiFi} = 8649137520 \quad (5.14)$$

Has the complement:

$$\neg\Theta_{HiFi} = 1350862479 \quad (5.15)$$

Which, given how that special sequence was derived/discovered(see **Section 5.1** in [4] or **Equation 19** in the same paper), it is interesting to note that its complement under ψ_{10} is also an *orthogonal symbol set identity*! For those who have studied the o-SSI paper[4], this can really be exciting for the study of symbol sets and sequences in general — yes, even for nucleic acid sequences! But, that said, of course, it shouldn't come as a surprise, since, in simple terms, a sequence's complement is just one very special kind of anagram among the set of all potential anagrams it can have, and so, before we leave the base-10 case, we can note that if some sequence $\Theta_{10} : \mathbb{N} \times \psi_{10}$ is a base-10 o-SSI, then its complement is also a base-10 o-SSI, and in fact, in general — even for non-numeric symbol sets such as ψ_{na}, ψ_{DNA} or special ψ_{RNA} , that:

Theorem 3 (Complement of an o-SSI is an o-SSI). *If some sequence $\Theta : \mathbb{N} \times \psi_{\beta}$ is an orthogonal symbol set identity for the base β , then its complement, $\neg\Theta$ is also an orthogonal symbol set identity for β .*

We shall see how **Theorem 3** might be applied, when dealing with algorithmic anagrammatization of sequences such as ψ_{10} in **Chapter 8**.

5.2 Potential Applications of Sequence Complements or Inverses

First, in genetics and analysis of genetic code sequences, note that, for especially na-Sequences in say the *non-native*(non-storage) form, such as the processed/derivative RNA-sequences, it might sometimes be useful to determine if some two sequences are the same, equivalent or closely related,

even though their apparent ordering of members, or even member composition looks unlike, **as long as their cardinalities are the same**³.

We should note, and interestingly so, that the complement of any sequence Θ , which is $\neg\Theta$, is just an anagram of Θ , and that in general, we can comfortably say:

Theorem 4 (Equivalence of Sequences under the Complement Transform). *If two sequences Θ and Ω that are **not equal**, do have the same cardinality and similar distribution of members, then they are essentially equivalent under the sequence complement transform and some projections of their form.*

Proof. The proofs are several:

1. Since $\mathcal{U}(\Theta) = \mathcal{U}(\Omega)$ and $\forall \alpha \in \Theta \quad \exists \rho \in \Omega : \alpha = \rho$ or equivalently, that $\forall a_i \in \Theta, \mathcal{U}(a_i \in \Theta) = \mathcal{U}(a_i \in \Omega)$, then there exists some anagram of Θ , denoted Θ^* such that $\Theta^* \xrightarrow{O_{complement}(\cdot)} \Omega$, which is possible by **Theorem 2**.
2. Because sequence complements are anagrams of each other, then if the two sequences are complements of each other, we expect that by the definition of the complement transformer (see **Transformer 4**), $\Omega = \neg\Theta$ or that $\Theta = \neg\Omega$.
3. We might also proceed by use of **modal sequences** (see **Definition 1** in [5]): Given the conditions on the two sequences, we then expect that if Θ is a sequence complement of Ω or vice-versa, then, computing their corresponding modal sequence statistic should result in the following result: $\overset{\sim}{\Theta} = \neg(\overset{\sim}{\neg\Omega})$ or equivalently, $\overset{\sim}{\Omega} = \neg(\overset{\sim}{\neg\Theta})$

□

Note that, for RNA, since $\psi_{RNA} = \langle A, C, G, U \rangle$ (**Equation 2.4**), then:

$$\neg\psi_{RNA} = \langle U, G, C, A \rangle \quad (5.16)$$

And this is what we'd expect via the complement mapping:

$$\begin{array}{cccc} \left[\begin{array}{c} A \\ \downarrow \\ U \end{array} & \begin{array}{c} C \\ \downarrow \\ G \end{array} & \begin{array}{c} G \\ \downarrow \\ C \end{array} & \begin{array}{c} U \\ \downarrow \\ A \end{array} \right] \end{array}$$

But, since the traditional pairwise ordered symbol set for RNA, $\dot{\psi}_{RNA}$ is

$$\dot{\psi}_{RNA} = \langle A, U, C, G \rangle \quad (5.17)$$

Then its corresponding complement, $\neg\dot{\psi}_{RNA}$, is

³This requirement to have their sizes the same, is a natural consequence of how a sequence and its anagrams relate — see [3]. However, once the basic concepts are well understood, this condition may be relaxed or merely be worked around. Also, for cases where we are concerned with sequence analysis and comparison where lengths either don't matter or are potentially different, refer to the kinds of analyses we conduct in the next chapter — see **Section 6.1** for example.

$$\neg\psi_{RNA} = \langle G, C, U, A \rangle \quad (5.18)$$

And so we have the associated complement mapping as

$$\left[\begin{array}{cccc} A & U & C & G \\ \downarrow & \downarrow & \downarrow & \downarrow \\ G & C & U & A \end{array} \right]$$

So, that, for some RNA-sequence such as Θ_{met} :

$$\Theta_{met} = \langle A, U, G \rangle \quad (5.19)$$

Which is the famous START-codon **Methionine** (see **Table 9.2**), has as its corresponding nucleic acid sequence complement under ψ_{RNA} :

Transformation 14. $\Theta_{met} \xrightarrow{O_{complement}(\Theta, \psi_{RNA})} \langle G, C, A \rangle$

*That codon isn't the same as the source (ATG/AUG/Methionine), but there's some debate concerning what it actually is called — **Table 1** in [19] names codon GCA as **Arginine**, while Wikipedia[40] names it **Alanine**.*

While — **and important to note**, its complement under ψ_{RNA} is [indeed] different!

Transformation 15. $\Theta_{met} \xrightarrow{O_{complement}(\Theta, \psi_{RNA})} \langle U, A, C \rangle$

and we know that codon UAC is Tyrosine[40]⁴.

It should perhaps be immediately noticeable that the ability to derive new/different/special sequences such as complement codons from ordinary na-Sequences might have some useful if not exciting applications as well as problems it awakens for the geneticists, molecular biologists and researchers of gene-sequencing⁵ and analysis. For example, apart from acknowledging that na-Sequence's complements denote potentially legitimate na-helix strand partners for any two sequences (in the form Θ and $\neg\Theta$), it would be further rewarding, to determine what other important things we might learn about how important sequences such as na-Sequences — especially relative to how they are utilized in nature, behave or alter their usual function, if replaced by their complements either partially or wholly. Definitely, true for any sequences in general... it must be exhilarating to discover what other things we can learn with or from them, via these kinds of sequence transforms.

⁴Concerning this particular codon, TAC/UAC, we find that though the Wikipedia reference tables assign it the name Tyrosine in both the DNA and RNA encodings, and yet, the physical reference we have at hand[19] (perhaps now out-of-date), assigns the name “Methionine” to TAC — Table 1 in that reference book did not list RNA codes, but we still would expect to use TAC to look-up the correct name for UAC.

⁵For those that do not know: Gene sequencing is the process of determining the exact order of nucleotides — A, T, C and G — in a segment of DNA that makes up a gene. It is like reading the biological “code” that instructs cells how to build proteins and regulate functions[32].

More **FUTURE WORK** for GENETICS Researchers:

It is already interesting to note that, under complement transforms (which are special kinds of anagrams), the natural/universal protein synthesis START-codon AUG/ATG is equivalent to GCA (**Transformation 14**) or UAC (**Transformation 15**), but, **are those other codons also START-codons?** That said, is there a possibility that some natural process or genetic code translation or transformation might naturally produce ATG or AUG from any of those corresponding [symbolic] complement sequences? What would this teach us about natural/biological processors? What of the possibilities of using synthetic/artificial processors to render such complement operations useful? And is it actually possible to *transmute* one set of nucleic acid sequences into another, with entirely different functions and identity via leveraging na-complements or complement transforms?

Chapter 6

Sequence Analysis Using The Anagram Distance and the Modal Sequence Statistic

We are going to talk about the matter of comparing or analyzing two or more na-Sequences using especially statistical measures that have been set down by transformatics theory for especially sequences of the kind we are interested in when exploring work in genetics.

6.1 Six Sequences and Four Scenarios

In particular, we shall be looking at na-Sequence analysis through lens of measures we have formalized earlier on¹, and shall break down our analysis into four representative scenarios:

1. **Scenario A:** Given some two particular instances of na-Sequences, $\Theta 1_n$ and $\Theta 2_n$, **not necessarily the same** ($(\Theta 1_n = \Theta 2_n) \vee (\Theta 1_n \neq \Theta 2_n)$), but which are of the same length, $\mathfrak{L}(\Theta 1_n) = \mathfrak{L}(\Theta 2_n) = n$, and with similar symbol sets², $(\hat{\psi}_{na}(\Theta 1_n) \cong \hat{\psi}_{na}(\Theta 2_n)) \wedge (\mathfrak{L}(\psi_{na}(\Theta 1_n)) = \mathfrak{L}(\psi_{na}(\Theta 2_n)) \leq n)$, **how to quantify how similar or dissimilar** they are?
2. **Scenario B:** Given some two particular instances of na-Sequences, $\Theta 1_n$ and $\Theta 2_k$, still not necessary the same, and of different lengths, $(\mathfrak{L}(\Theta 1_n) = n) \neq (\mathfrak{L}(\Theta 2_k) = k)$, but with **similar symbol sets**, $(\hat{\psi}_{na}(\Theta 1_n) \cong \hat{\psi}_{na}(\Theta 2_k)) \wedge (\mathfrak{L}(\psi_{na}(\Theta 1_n)) = \mathfrak{L}(\psi_{na}(\Theta 2_k)) \leq \text{Min}(n, k))$, how to quantify how similar or dissimilar they are?
3. **Scenario C:** Given some two **potentially different**, particular instances of na-Sequences, $\Theta 1_n$ and $\Theta 2_k$, of **different lengths**, n and k , and **with different symbol sets**, $((\hat{\psi}_{na}(\Theta 1_n) \neq \hat{\psi}_{na}(\Theta 2_k)) \vee (\mathfrak{L}(\psi_{na}(\Theta 1_n)) \neq \mathfrak{L}(\psi_{na}(\Theta 2_k))))$, how to quantify how similar or dissimilar they are?
4. **Scenario D:** Finally, how, when given a particular na-Sequence, Q_k , and two collections/samples of other sequences $(\Theta 1, \Theta 2, \Theta 3, \dots, \Theta m)$ and $(\Omega 1, \Omega 2, \Omega 3, \dots, \Omega n)$ of potentially dissimilar lengths, m and n respectively, and that belong to **labeled or classified populations** — POPA and POPB respectively, to determine which of the two populations the query sequence, Q_k , best-belongs

¹For those not yet familiar with the existing mathematical theories and results of transformatics, **Chapter D** in the appendix does provide all the necessary background context relevant for mastering all the new [applied transformatics] material we are exploring in this chapter.

²**NOTE** that in this case we are expressing this problem aspect using specific symbol sets — such as $\hat{\psi}_{na}(\Theta 1_n)$ — and not say unspecific symbol sets $(\psi(\Theta 1_n))$, nor implicit symbol sets $(\psi(\Theta 1_n))$, and neither natural symbol sets $(\psi_{na}(\Theta 1_n))$, because, we do not wish to assume that the two sequences have the same exact ordering of distinct elements from the specified base symbol set — ψ_{na} — a fact which for example the natural symbol set can't show/preserve, and yet, because we know the base, we do not want to only deal with base-less symbol sets such would be the case with the unspecific and implicit symbol set. For details concerning these different symbol sets, refer to the clarifying discussion offered in **Question 4** of [41].

to (so we classify it under the associated label), as well as determine which of the **particular** sample's member sequences — from that chosen collection it is closest to.

Note that, in conducting these analyses, we shall especially exploit the sequence-specific measures, $\tilde{A}(\cdot)$, the Anagram Distance Measure[3][5], and $\tilde{\Theta}$, the Modal Sequence Statistic[5], and that we shall use the following representative, even though somewhat hypothetical (for the first four), na-Sequences:

$$\Theta_{1_{DNA}} = \langle TACGGGCATTCC \rangle \quad (6.1)$$

$$\Theta_{2_{RNA}} = \langle AUGAUGCGCGGGCAT AUC \rangle \quad (6.2)$$

$$\Theta_{3_{DNA}} = \langle ATAATGGGGGCATTGA \rangle \quad (6.3)$$

$$\Theta_{4_{DNA}} = \langle TACTCCGGGCAT \rangle \quad (6.4)$$

$$\begin{aligned} \Theta_{insulin} = & \text{ATGGCCCTGTGGATGCGCCTCCTGCCCCTGCTGGCGCTGCTGGCCCTCTGGGGACC} \\ & \text{CCAGCCGCAGCCTTTGTGAACCAACACCTGTGCGGCTCACACCTGGTGAAGCTCTCTAC} \\ & \text{CTAGTGTGCGGGGAACGAGGCTTCTTCTACACACCCAAGACCCGCCGGGAGGCAGAGGAC} \\ & \text{CTGCAGGTGGGGCAGGTGGAGCTGGGCGGGGGCCCTGGTGCAGGCAGCCTGCAGCCCTTG} \\ & \text{GCCCTGGAGGGGTCCCTGCAGAAGCGTGGCATTGTGGAACAATGCTGTACCAGCATCTGC} \\ & \text{TCCCTCTACCAGCTGGAGAACTACTGCAACTAG} \quad (6.5) \end{aligned}$$

[illegible]

The first four sequences are merely hypothetical, and intentionally small/short, with $\Theta 1_{DNA}$, $\Theta 3_{DNA}$ and $\Theta 4_{DNA}$ being DNA-sequences, while $\Theta 2_{RNA}$ is just some RNA-sequence. $\Theta_{insulin}$ is a DNA-sequence we have already encountered in **Chapter 2** (see **Equation 2.6**), while the most verbose sequence we shall encounter in this work is named Θ_{rbcl} and is a real-life, biologically relevant RNA-sequence from a well-known plant protein: **RbcL**³, the large subunit of **RuBisCO** — the enzyme responsible for carbon fixation in photosynthesis. It is one of the most abundant and essential proteins in plants[32].

6.2 A First Analysis

NOTE:

TEA programming language INSTALLATION and Basic USAGE Instructions

TEA as a language is designed to be usable from almost anywhere there is a computer capable of running the TEA language interpreter — currently implemented for WEB (browser environments) using JavaScript, and for all other environments via Python3. However, to really get started using TEA, especially when one has a modern computer, a smartphone or some device capable of accessing the Internet and rendering web pages, one then need not install anything or download anything apart from merely visiting the following link:

TEA WEB IDE: <https://tea.nuchwezi.com>

So that, by visiting that page, one can readily access a usable basic and standard interface with which they can not only write, browse, read and run existing TEA programs — especially those sample/example applications that are made available as part of the growing list of “Standard TEA programs” — but that they can also share the TEA program source code either using links, or copy-pasted codes. Using the WEB IDE is the simplest, best method for trying out TEA that is also beginner-friendly. Also, other than accessing the TEA programs and runtime, the IDE offers access to a TEA debugger for help with analyzing or troubleshooting problematic TEA programs, access to the TEA documentation, links to the TEA community, lecture notes and videos, etc.

For those that need to use TEA from any other environment or platform other than the web — such as in non-interactive server-side programs, in command-line scripts, in stand-alone desktop programs, etc. The following installation method is what is currently most recommended:

1. To INSTALL TTTT and the TEA language [*plus some standard TEA programs packaged together with TTTT; ZHA and AMC*⁴.] on your system, run the following command in your terminal:

UNIX/Linux TEA installation command

```
1 curl -Ls https://bit.ly/installtea | bash
```

Once TEA is installed via the above command-line method, one can then readily view/browse the official documentation that comes along with that package via: `man tttt`

³ Θ_{rbcl} is actually just a sample mRNA Sequence (partial, from *Arabidopsis thaliana* RbcL gene), and is excerpted from a paper that includes the *Arabidopsis thaliana* chloroplast genome, which contains the rbcL gene encoding the large subunit of RuBisCO[42].

⁴Users of TEA via the WEB IDE automatically get access to all (30+) standard TEA programs/example applications, however, for users of TEA via the Debian package tttt, only a few of these applications are currently included/accessible via the default installation method — **ZHA**; the “Zee Hacker Assistant” and **AMC**; Advanced Mathematics Computer

Before we dive into the matter of solving the analytic problems under the four scenarios introduced in **Section 6.1** above, we shall want to begin with a background analysis that shall give us some high-level insights about any of the sequences in our problem set. In particular, we are going to start by analyzing each of those sequences, so as to know the following basic facts:

1. What is the sequence's symbol set: $\psi(\Theta) = ?$
2. Which of the three nucleic acid symbol sets does the sequence belong to — ψ_{DNA} , ψ_{RNA} or ψ_{na}
3. What is the relative frequency of each sequence's symbol-set members? $\forall \rho \in \psi(\Theta), \varrho(\rho \in \Theta) = ?$
4. How long is the sequence as a flat-structure sequence: $\varrho(\Theta) = ?$
5. How many codons does the sequence contain? $\Theta \rightarrow \Theta^* : \mathbb{N} \times \psi_{na}^3, \varrho(\Theta^*) = ?$

For purposes of helping others replicate the analyses and results we shall obtain, we recommend that the following analysis method be used:

We shall use TEA⁵[29][9] — a text-processing-oriented general-purpose computer programming language — an executable software language by the author, via its **tttt** Linux package and utility, to implement most of the analyses we wish to conduct via the command-line.

1. **To Process a Sequence via the Command-Line?** For example, copy-and-paste sequence Θ_1 into a file named `theta1_dna.txt`
2. **To compute the sequence symbol set?** For example, for Θ_1 , run the following command against the sequence file to display the unique symbols in the sequence, in their natural order of first-occurrence within the sequence:

```
cat theta1_dna.txt | tttt -c "b:"  
TACG
```

3. **To compute the modal sequence statistic?** First, note that the **modal sequence statistic** is well introduced, defined and how to use it demonstrated in [5]⁶. That said, for example, for Θ_1 , run the following command against the sequence file to display the unique symbols in the sequence, in their order of most frequent first, but also their natural order of first-occurrence within the sequence:

⁵Instructions for how to install and use TEA/Transforming Executable Alphabet general-purpose text-processing oriented computer programming language are offered via the project's GitHub: https://github.com/mcnemesis/cli_tttt — but also check the associated note presented at the start of this section — Otherwise, also note that one can readily try TEA as well as run any of the TEA programs we have included in this chapter and anywhere in the book, without installing any special software, merely by visiting the online, browser-based **TEA WEB IDE** via <https://tea.nuchwezi.com>

⁶See **Definition 1** in [5], but also refer to **Proposal 5** in **Chapter D**.

```
cat theta1_dna.txt | tttt -c "u!":""
CTGA
```

Note that, the actual command/TEA-code for this is just `u!:`, however, because of the way the Linux command-line treats the ‘!’ symbol as a special character — even inside strings, we must cleverly use it in a manner such as shown; we write “u!” “:” to avoid the “*unrecognized history modifier*” error if that code is written directly on the terminal. This shall apply to all the other cases where we must deal with the TEA command qualifier ‘!’ [29].

The other workaround is to **first turn off the BASH history expansion modifier** — this, so that we can just write clean TEA code such as `u!:` without errors, directly on the Linux terminal interface. So, to disable history expansion for the *active/current session* in your terminal, just run the command:

```
set +H
```

And after that, you can just write clean TEA code for the above task as such:

```
cat theta1_dna.txt | tttt -c u!:
CGTA
```

4. **To compute the distribution of symbols within a sequence?** For example, to compute how many times the symbol ‘T’ occurs in $\Theta 1$, we can merely run the code:

```
cat theta1_dna.txt | tttt -c "d!:T|g:|v:|v!:"
3
```

And for the symbol ‘C’, we just modify that code to count for ‘C’ as such:

```
cat theta1_dna.txt | tttt -c "d!:C|g:|v:|v!:"
4
```

But if we wish to do the same for all distinct symbols within the sequence $\Theta 1$ — **and especially without having to explicitly list which symbols we are counting within the TEA code** — meaning, no *hard-coding* of the elements from the sequence’s specific symbol set — and equivalently, merely making the program symbol-set-independent — or rather “applicable

to any sequence file”, we could have used “u!” as in the previous task, since that TEA command counts how many times each unique symbol occurs in a sequence so as to compute the modal sequence, however, currently (with TEA version **1.3.6**[9]), the actual frequencies aren’t returned with the output of “u!”, and so, we shall want to use a work-around⁷ leveraging the above method for computing the frequency of each symbol in the input sequence. We can use the following non-trivial TEA program for that then:

EXAMPLE: Computing Sequence Characteristic using TEA on command line

```

1  cat theta1_dna.txt | tttt -c "v:vSEQ|v:vANA:{}|u!:||v:vMS|l
    :lMS|y:vMS|d!:^.|v:vSY|y:vMS|d:~.|v:vMS|y:vSEQ|d*!:vSY|
    g:||v:||v!:|v:vSYN|g*:{}:vSY:vSYN|x*:vANA|v:vANA|y:vMS|f
    :~$:lFIN|j:lMS|l:lFIN|y:vANA"
2  C4T3G3A2

```

The essential result from running that program against $\Theta 1$ is the string **C4T3G3A2**, which tells us that ‘C’ occurs 4 times, ‘T’ 3 times, ... , and then ‘A’ only 2 times — the symbols in the sequence, and their frequencies, sorted in descending order of most frequent (mode) first, otherwise by their natural order of first occurrence (when two symbols have same frequency).

5. **To compute the cardinality of a sequence?** For example, for $\Theta 2$, run the following command against the sequence file to display the total number of all symbols within the sequence:

```

cat theta2_rna.txt | tttt -c "v:||v!:"
18

```

To confirm if TEA is telling us the correct thing, we can also count the characters in that sequence (assuming we pasted just the sequence symbols into the file and nothing else, and no delimiters between them), using the standard Linux/Unix command “wc” as such:

```

cat theta2_rna.txt | wc -c
18

```

⁷**FUTURE WORK:** Provide a basic, in-built capability within the TEA instruction set, to allow for the automatic computation of a text sequence characteristic, $h(\hat{\Theta})$.

Such is the power of the sequence analysis we can do with the TEA language, without any sophisticated tools. Thus shall we analyze all the other sequences in our problem set, and to simplify things, we shall merely tabulate the analysis results for all six sequences as such:

na-Seq	Unspecific Sequence Symbol Set: $\hat{\psi}(\Theta)$	Closest Parent Symbol Set	Modal Sequence Statistic, $\hat{\Theta}$ and Symbol Distribution: $\nu(\rho \in \Theta)$						$\nu(\Theta)$	Codon Count: $\nu(\Theta^*)$ $\approx \frac{\nu(\Theta)}{3}$
$\Theta 1_{DNA}$	$\langle T, A, C, G \rangle$	ψ_{DNA}		C	T	G	A		12	4
				4	3	3	2			
$\Theta 2_{RNA}$	$\langle A, U, G, C, T \rangle$	ψ_{na}		G	A	C	U	T	18	6
				6	4	4	3	1		
$\Theta 3_{DNA}$	$\langle A, T, G, C \rangle$	ψ_{DNA}		G	A	T	C		16	$5.\overline{33}$
				6	5	4	1			
$\Theta 4_{DNA}$	$\langle T, A, C, G \rangle$	ψ_{DNA}		C	T	G	A		12	4
				4	3	3	2			
$\Theta_{insulin}$	$\langle A, T, G, C \rangle$	ψ_{DNA}		G	C	T	A		329	$109.\overline{66}$
				108	105	60	56			
Θ_{rbcl}	$\langle A, U, G, C \rangle$	ψ_{RNA}		U	G	A	C		3168	1056
				1355	1207	305	301			

Table 6.1: A First Analysis of the 5 na-Sequences from **Section 6.1**

To re-iterate the essential steps used to arrive at **Table 6.1**, note that the following methods leveraging TEA programs were used to arrive at the results in the computed columns:

1. *Column 2*: **Unspecific Symbol Set**:

Computing Unspecific Symbol Set

```
1  b:
```

2. *Column 4*: **Sequence Characteristic**:

Computing Sequence Characteristic

```

1 v:vSEQ | v:vANA:{ } | u! : | v:vMS | l:lMS | y:vMS | d! : ^ . | v:vSY |
  y:vMS | d! : ^ . | v:vMS | y:vSEQ | d*! : vSY | g : | v : | v! : | v:vSYN
  | g* : { } : vSY:vSYN | x* : vANA | v:vANA | y:vMS | f : ^ $ : lFIN |
  j : lMS | l : lFIN | y:vANA

```

3. Column 5: Sequence Cardinality:

Computing Sequence Cardinality

```

1 v : | v! :

```

4. Column 6: Codon Count:

Computing Codon Count

```

1 v : | v! : | x! : / 3 | r . :

```

Those TEA programs shall work without modification for any case in which you have a sequence and need to evaluate its properties as above, whether via the TEA command-line interface with `tttt` or via TEA on the WEB⁸ or anywhere [in-between].

6.3 SCENARIO A — the anagram distance measure

Given some two particular instances of na-Sequences, Θ_{1_n} and Θ_{2_n} , not necessarily the same ($(\Theta_{1_n} = \Theta_{2_n}) \vee (\Theta_{1_n} \neq \Theta_{2_n})$), but which are of the same length, $\mathfrak{u}(\Theta_{1_n}) = \mathfrak{u}(\Theta_{2_n}) = n$, and with similar symbol sets, $(\psi_{na}(\Theta_{1_n}) \cong \hat{\psi}_{na}(\Theta_{2_n})) \wedge (\mathfrak{u}(\psi_{na}(\Theta_{1_n})) = \mathfrak{u}(\psi_{na}(\Theta_{2_n})) \leq n)$, how to quantify how similar or dissimilar they are?

So, in this scenario, we are basically given some two particular na-Sequences, of the same length, and which have similar symbol sets, and are tasked with how to quantify their similarity or dissimilarity.

Looking at our example sequences above in **Table 6.1**, only two sequences satisfy the necessary conditions — Θ_1 and Θ_4 , which both have length **12** and which have similar [natural/base] symbol sets under base-na⁹ — $\psi_{na}(\Theta_1) = \psi_{na}(\Theta_4) = \langle A, C, G, T \rangle$, but also their unspecific sequence symbol sets are already similar.

Actually, this case is very telling concerning what might actually happen with na-Sequences — essentially, we note that, despite the two sequences **actually being different** — for example, Θ_1 ends with codon TTC, while Θ_4 ends with

⁸For TEA on the WEB, you might for example take the sequence contents as we have seen them before — just the contents of for example text file `theta1.dna.txt`, and just copy-paste them into the input section on the interface — “TEA Input”, and for the TEA program, copy-paste that into the “TEA Code” section, after which you then hit the “RUN” button, and the results shall appear in the “TEA Output” section. Refer to [43] or visit <https://tea.nuchwezi.com> for more.

⁹Formally defined in **Definition 3**

CAT, and yet, looking at their breakdown analysis in **Table 6.1**, we see that **they almost seem to be the same!** They have the same sequence symbol set, the same parent symbol set, the same modal sequence and symbol distribution, the same sequence length and thus same number of codons! So, how exactly can we quantifiably distinguish between such sequences in real life?

At this juncture, it definitely might make sense to recall what the purpose of the **Anagram Distance Measure**[3] is.

The ADM, $\tilde{A}(\Theta \rightarrow \Theta^*) \geq 0$ is essentially a measure that allows us to tell if any two sequences, Θ and Θ^* , that might contain the same exact distinct symbols, but possibly in different proportions and/or order, are different or not.

So, since we can't use the usually sufficient **modal sequence statistic**[5] to distinguish between these two actually different sequences, let us attempt to compute their anagram distance instead¹⁰.

¹⁰It shall be **very important** to bring to mind here, the fact that, the cases where comparing sequences or datasets using the ADM might not seem obvious, but, as we already saw in a detailed attempt to distinguish between actually different sequences using many various traditional statistical measures — of variation moreover — that all failed to show a difference between some two test sequences — see **Table 1** in [5], one shouldn't take the ADM any less serious even for non-numerical data such as analysis of nucleic acid sequences!

NOTE:**A Quick Recap Concerning Interpreting ADM, $\tilde{A}(\Theta, \Theta^*)$**

The Anagram Distance Measure (ADM), associated testing method, background and justifications for the new statistic were first laid out in the seminal paper[3] introducing that measure and its theory. That work was later advanced in the transformatics paper[5], and essentially, we can note that:

- Given any two sequences, Θ and Θ^* , that ideally should be of the same length, and with the same^a sequence symbol set, then we compute the anagram distance between them via the formula:

$$\tilde{A}(\Theta \rightarrow \Theta^*) = \frac{1}{\nu(\Theta)} \times \sum_{i=1}^{\nu(\Theta)} |I(\omega_i, \Theta^*) - i| \quad (6.7)$$

- And that we can interpret the results — which shall always be a positive real number ($\mathbb{R}^+$) as such:

$$\tilde{A}(\Theta \rightarrow \Theta^*) = \begin{cases} 0, & \Theta = \Theta^*, \text{no differences} \\ < 1, & \Theta \approx \Theta^*, \text{different in at minimum two positions} \\ = 1, & \Theta \neq \Theta^*, \text{all members shifted by exactly 1 position} \\ > 1, & \Theta \ll \Theta^*, \text{potential indication there is chaos.} \end{cases} \quad (6.8)$$

For a quick primer, on how this works out, assume we have $\Theta_1 = \langle a, b, c \rangle$, $\Theta_2 = \langle a, c, b \rangle$, $\Theta_3 = \langle b, c, a \rangle$, $\Theta_4 = \langle c, a, b \rangle$, $\Theta_5 = \langle b, a, c \rangle$. Then we can see that:

1. $\tilde{A}(\Theta_1, \Theta_2) = \frac{1}{3}(|1-1| + |2-3| + |3-2|) = \frac{2}{3} < 1$
2. $\tilde{A}(\Theta_1, \Theta_3) = \frac{1}{3}(|1-3| + |2-1| + |3-2|) = \frac{4}{3} > 1$
3. $\tilde{A}(\Theta_1, \Theta_4) = \frac{1}{3}(|1-2| + |2-3| + |3-1|) = \frac{4}{3} > 1$
4. $\tilde{A}(\Theta_1, \Theta_1) = \frac{1}{3}(|1-1| + |2-2| + |3-3|) = \frac{0}{3} = 0$
5. $\tilde{A}(\Theta_1, \Theta_5) = \frac{1}{3}(|1-2| + |2-1| + |3-3|) = \frac{2}{3} < 1$

However, to see the case when $\tilde{A} = 1$, we can use the sequences: $\Omega_1 = \langle a, b, c, d, e, f \rangle$ and $\Omega_2 = \langle b, a, d, c, f, e \rangle$. So, that we have:

$$\tilde{A}(\Omega_1, \Omega_2) = \frac{1}{6}(|1-2| + |2-1| + |3-4| + |4-3| + |5-6| + |6-5|) = \frac{6}{6} = 1$$

Concerning this last case, it might be worth noting that the only way to have that result for a sequence, is to have each element swapped with its immediate neighbor, without wrapping-around such as we saw happen in Θ_1 Vs Θ_3 for example. Also, the only way this could happen, is if the sequence's cardinality is even — *for those interested, we employ some of these ideas in **Chapter 8** — the two **Lu-Shuffle Algorithms**.*

^aOr more accurately, “similar composition” symbol sets — such as $\psi(\Theta) \cong \psi(\Theta^*)$ — or perhaps similar natural symbols sets as per some base β — $\psi_\beta(\Theta) = \psi_\beta(\Theta^*)$ — with this condition being most important for cases where the two sequences being compared aren't of same length or same symbol distribution, so that then, a more accurate approximate analysis might be applied to their modal sequence statistics — which only makes sense if both MSS contain the same exact symbols irrespective of their member ordering.

So, if we return to our na-Sequence analysis, we see that for our tricky case comparing $\Theta 1$ Vs $\Theta 4$, that if we compute their anagram distance, then we have the results:

$i = I(\omega_i, \Theta 1)$	1	2	3	4	5	6	7	8	9	10	11	12
$\omega_i \in \Theta 1$	T	A	C	G	G	G	C	A	T	T	C	C
$\omega_i^* \in \Theta 4$	T	A	C	T	C	C	G	G	G	C	A	T
$I(\omega_i \in \Theta 1, \Theta 4) = I(\omega_i, \Theta 4)$	1	2	3	7	8	9	5	11	4	12	6	10
$ I(\omega_i, \Theta 4) - i $	0	0	0	3	3	3	2	3	5	2	5	2

Table 6.2: A Tabular Analysis of $\Theta 1$ Vs $\Theta 4$ so as to compute their Anagram Distance

It shall be worth noting concerning how we derive the values of the updated position of ω_i in the compared/resultant sequence, $\Theta 4$, via $I(\omega_i, \Theta 4)$, that we are actually careful to follow the definition of that function (see **Note** on **Page 5** in [5]), with the useful detail that since both sequences contain repeated/duplicated symbols despite their similar length, that we assign to some symbol, such as the first ‘T’ in $\Theta 1$, the index of the first ‘T’ in $\Theta 4$, and second ‘T’ in $\Theta 1$, the index of the second ‘T’ in $\Theta 4$ — no skipping ahead, no juggling them up¹¹.

And thus, we can see readily that:

$$\tilde{A}(\Theta 1 \rightarrow \Theta 4) = \frac{1}{12}(0+0+0+3+3+3+2+3+5+2+5+2) = \frac{28}{12} = 2.3\bar{3} > 1 > 0 \quad (6.9)$$

And so, despite the measures and first analysis in **Table 6.1** having shown that these two nucleic acid sequences didn’t have any significant differences, and yet, using ADM as in **Table 6.2** and **Equation 6.9**, we find that they are appreciably **significantly different**! Their ADM being 2.3, it tells us the second sequence potentially has elements in the first sequence shifted to new locations in the sequence, potentially by more than just 1 position. Thus, despite all their other similarities, they do have some differences that we can measure and pin to some telling numbers.

6.4 SCENARIO B — the sequence characteristic [statistic]

Given some two particular instances of na-Sequences, $\Theta 1_n$ and $\Theta 2_k$, still not necessary the same, and of different lengths, $(\mathcal{U}(\Theta 1_n) = n) \neq (\mathcal{U}(\Theta 2_k) = k)$, but with similar symbol sets, $(\hat{\psi}_{na}(\Theta 1_n) \cong$

¹¹Thus, we can generally say for any two sequences, Θ and Θ^* under ADM analysis, that for computing the $I(\omega_i, \Theta^*)$ for terms from the source, Θ , even where ω_i occurs multiple times in either sequence, even if at different positions, that we assign the first instance of ω_i , the value of $I(\omega_i, \Theta^*)$ equal to the position of the first instance of ω_i in Θ^* , the second instance of ω_i , the position of the second instance of ω_i in Θ^* , etc. So as to properly/correctly compute the ADM, but also for any other cases in using the **position-index function** in say logic or formulations as we do in many scenarios concerning Transformatics.

$\hat{\psi}_{na}(\Theta_{2k}) \wedge (\varrho(\psi_{na}(\Theta_{1n})) = \varrho(\psi_{na}(\Theta_{2k})) \leq \text{Min}(n, k))$, how to quantify how similar or dissimilar they are?

So, given the conditions of this scenario and our sample sequences as summarized in **Table 6.1**, we shall pick sequences Θ_1 and Θ_3 , which both have the same sequence symbol set under base-na, and which have different cardinalities; 12 and 16 respectively.

Good enough, we have already worked through some of the essential details of the analysis method in **Section 6.3 — Scenario A**. However, for the present scenario, we can sum up how to conduct our comparative analysis thus:

1. First, conduct tabular analysis so as to see how the two sequences contrast against each other by the criteria:
 - (a) Their unspecific¹² sequence symbol sets — such as $\hat{\psi}(\Theta_1)$ Vs $\hat{\psi}(\Theta_3)$.
 - (b) Their closest parent/containing/superset symbol set — meaning, find some other symbol set, ψ^* , such that $\psi^* \supset \psi(\Theta_1) \wedge \psi^* \supset \psi(\Theta_3)$ — equivalently, that $\psi^* \supset \psi(\Theta_1) \cup \psi(\Theta_3)$.
 - (c) Their modal sequence statistic — such as $\vec{\Theta}_1$ Vs $\vec{\Theta}_3$.
 - (d) Their **sequence characteristic** — more about this soon: essentially, $\vec{h}(\vec{\Theta}_1)$ Vs $\vec{h}(\vec{\Theta}_3)$.
 - (e) Their cardinality — that is, $\varrho(\Theta_1)$ Vs $\varrho(\Theta_3)$

We have already seen how to do all the above analysis steps and why, in the First Analysis described in **Section 6.2**.

2. So, **if none** of the analyses in the first step help show the **quantifiable similarities** or **quantifiable differences** between the two sequences, then proceed to using the more subtle Anagram Distance Measure on them. How to do this we have already covered well in **Scenario A**.
3. **In case the ADM also can't find any differences between the two sequences** after having already failed to find any such quantifiable differences with the earlier analyses, then most likely, their only differences are **cosmetic** — such as simply naming the same sequence differently, and thus the two sequences under analysis can be considered to be **quantifiably similar**.

So, first, note that, from **Table 6.1**, that even though the two sequences we are looking at, Θ_1 and Θ_3 have the same **specific sequence symbol set**¹³ (under base-na but also with base-DNA): $\hat{\psi}_{na}(\Theta_1) = \hat{\psi}_{na}(\Theta_3) = \langle A, C, G, T \rangle$, and yet, their **unspecific sequence symbol sets** are different!

Thus, their first difference is in their unspecific sequence symbol sets: $\hat{\psi}(\Theta_1) = \langle T, A, C, G \rangle$ and yet $\hat{\psi}(\Theta_3) = \langle A, T, G, C \rangle$. So, at minimum, we know that they

¹²The **Unspecific Symbol Set** concept is first introduced in **Definition 4** of the o-SSI paper[4]

¹³The concept of the **specific symbol set** is first introduced in **Definition 3** of [4], but also further discussed in **Question 4** of [41].

have quantifiable difference in the [first-occurrence] ordering of their similar symbols within either sequence just by this analysis — especially because these particular symbol set types respect order of first occurrence within the sequence they summarize.

If we must continue to still analyze the two sequences, we can further note that they have the same closest¹⁴ parent symbol set, ψ_{DNA} . And so, that's a similarity.

And then we come to the matter of their modal sequence statistics. For this case, we see that we have the values:

- $\overset{>}{\Theta}1 = \langle C, G, T, A \rangle$
- $\overset{>}{\Theta}3 = \langle G, A, T, C \rangle$

So, this alone can tell us that the two sequences differ quantifiably in terms of either their relative distribution of members/symbols, or that they differ in the order of first occurrence of their symbols at worst — essentially, the last, iff the symbols have ties on their relative frequencies in either original sequences.

Also, note that, the modal sequence statistic (MSS), is definitely computed readily by considering the relative frequency of terms within the sequence under analysis, and so, when we look at the MSS column in **Table 6.1**, we see that below each of the terms for the MSS, we also display the associated symbol frequencies. This is important as we are to see hereafter...

Talking of which, the next analysis we could conduct, but which is closely related to the previous one, is the **sequence characteristic**.

Concerning this, note that, unlike **Scenario A** where the two sequences we analyzed had the same exact MSS, and yet, assuming $\Theta1$ and $\Theta4$ had some difference in the frequency of their terms despite having the same MSS, the MSS-analysis alone wouldn't have identified that. And so, before we proceed, let us also introduce/define the sequence characteristic measure.

¹⁴I say “closest parent symbol set” because, for several scenarios in analyzing sequences, one can encounter cases where two sequences have symbols that belong to more than one symbol set where that other symbol set is larger than the sequence symbol set. For example, the case of having to decide if a sequence such as “101” belongs to base 2, base 3, base 10 or even base-36! Or if we have the sequence “AUG” that might span the RNA-symbol set, but also ψ_{na} , and talking of which, we must note that, in principle at least, all na-Sequences could also be classified as sequences of terms from the Latin-Alphabet — the symbol set ψ_{az} !

Definition 8 (The **Sequence Characteristic**, $\hbar(\overset{\rhd}{\Theta})$). If a sequence Θ has the modal sequence statistic^a $\overset{\rhd}{\Theta} = \langle \prod_{\omega_i \in \psi(\Theta)} \omega_i, \rangle$, so that we can express it as a string concatenation of its distinct symbols in their relative order of highest frequency and first occurrence such as $\omega_1\omega_2\omega_3 \dots \omega_k$ for some k the size of $\overset{\rhd}{\Theta}$, then, if for each ω_i we also know its corresponding frequency in Θ , such as f_i for ω_i , then writing the frequency next to the symbol such as $\omega_i f_i$ for all terms in $\overset{\rhd}{\Theta}$ produces a string that contains both the information about the unique modal sequence of Θ , but also the information about how frequent each symbol occurred. We shall call such an expression the **Sequence Characteristic**, and shall denote it as $\hbar(\overset{\rhd}{\Theta})$, so that we can then write for Θ , its corresponding sequence characteristic as:

$$\hbar(\overset{\rhd}{\Theta}) = \prod_{\omega_i \in \overset{\rhd}{\Theta}} \omega_i \cdot f_i \quad (6.10)$$

Note that, for any Θ , if $\mathcal{L}(\Theta) = n$ and $\mathcal{L}(\omega_i \in \Theta) = f_i$, then $1 \leq f_i \leq n \quad \wedge \quad \sum_{1 \leq i \leq \mathcal{L}(\overset{\rhd}{\Theta})} f_i = n$.

^aRefer to rigorous definition: **Definition 14**

Closely related, and since we have already seen such a computation and its application in **Step#4** of our **First Analysis** using the TEA programming language, we might as well formally define a generic machine that can compute $\hbar(\overset{\rhd}{\Theta})$ for any sequence.

Transformer 5 (The **Sequence Characteristic Generator**, gSC).

$$\Theta \xrightarrow{O_{gSC}(\Theta)} \Theta^*; \quad \Theta^* = \hbar(\overset{\rhd}{\Theta}) = \prod_{\omega_i \in \psi(\Theta)} \omega_i \cdot f_i = \prod_{\rho_i \in \overset{\rhd}{\Theta}} \rho_i \cdot \mathcal{L}(\rho_i \in \Theta)$$

So, we note that, like in the example we saw in generating $\hbar(\overset{\rhd}{\Theta}1)$ for $\Theta1$ in our **First Analysis** — which returned the characteristic string as **C4T3G3A2**, we can then see that, by using the information in **Table 6.1**, that for the two sequences we are comparing, we have their sequence characteristics as such:

- $\hbar(\overset{>}{\Theta}1) = \mathbf{C4T3G3A2}$
- $\hbar(\overset{>}{\Theta}3) = \mathbf{G6A5T4C1}$

And thus, since those two measures aren't the same either, again, we have yet another evidence to conclude the two sequences are quantifiably different.

It should be worth noting, that in contrast to the previous scenario, the sequence characteristic¹⁵ for $\Theta 1$ and $\Theta 4$ are equivalent (as expected?).

6.5 SCENARIO C

Given some two potentially different, particular instances of na-Sequences, $\Theta 1_n$ and $\Theta 2_k$, of different lengths, n and k , and with different symbol sets, $((\psi_{na}(\Theta 1_n) \neq \hat{\psi}_{na}(\Theta 2_k)) \vee (\mathcal{V}(\psi_{na}(\Theta 1_n)) \neq \mathcal{V}(\psi_{na}(\Theta 2_k))))$, how to quantify how similar or dissimilar they are?

Without repeating ourselves nor wasting time, note that, from the cases we have in **Table 6.1**, this scenario might be applicable to sequences such as $\Theta 1$ and Θ_{rbcl} for example. And, from the analysis procedure we have already encountered in **Scenario B**, we can definitely use any of several available ways to quantify their similarities or differences. However, most useful perhaps, might be to just look at their sequence characteristics:

- $\hbar(\overset{>}{\Theta}1) = \mathbf{C4T3G3A2}$
- $\hbar(\overset{>}{\Theta}_{rbcl}) = \mathbf{U1355G1207A305C301}$

Which, **automatically disqualifies any allegations that the two sequences are similar**, and which, further [can/does] tell us they differ in not only their symbol sets, but also in their symbol distribution if they actually do.

Also, and important to note, we see that, by having a **characteristic**¹⁶ computed, we can not only tell at a glance which symbols occur most frequently in a sequence, which symbols only occur once or with similar frequencies but with telling ordering, but also, be able to readily compute the actual cardinality of the sequence being summarized by the characteristic — so, with $\hbar(\overset{>}{\Theta}_{rbcl}) = \mathbf{U1355G1207A305C301}$, we can compute the length of the **RbcL** by just summing up the corresponding f_i terms in $\hbar(\overset{>}{\Theta}_{rbcl})$: $1355 + 1207 + 305 + 301 = \mathbf{3168}$. Such is the relevance of the **sequence characteristic** statistic — see **Definition 8**.

It is such a handy and robust measure or rather, statistic, when comparing especially large or arbitrarily sized sequences *at a glance*. Good enough, we have

¹⁵The **Sequence Characteristic** becomes yet another important contribution to the study and analysis of sequences that can serve special purposes than just the related Modal Sequence Statistic, and which, though derived from it, expresses different sequence summarizing information that is important in scenarios such as genetic sequence analysis as we have just seen — **more importantly, it is an info-lossless version of the modal sequence statistic; a more descriptive summary statistic for a sequence**. Definitely, this measure can, as with other measures we have developed in transformatics, be applied in any mathematical or scientific field and not just statistics or genetics such as in this case.

¹⁶Yes, sometimes we might just talk of a **characteristic** when we mean a “sequence characteristic”.

seen that there is a simple/short TEA program¹⁷ to help automatically compute and display that measure for any [well formatted] sequence, so, interested researchers and students can just adapt/build on that in the future.

6.6 SCENARIO D — the population characteristic measure

How, when given a particular na-Sequence, Q_k , and two collections/samples of other sequences $(\Theta_1, \Theta_2, \Theta_3, \dots, \Theta_m)$ and $(\Omega_1, \Omega_2, \Omega_3, \dots, \Omega_n)$ of potentially dissimilar lengths, m and n respectively, and that belong to labeled or classified populations --- POPA and POPB respectively, to determine which of the two populations the query sequence, Q_k , best-belongs to (so we classify it under the associated label), as well as determine which of the particular sample's member sequences --- from that chosen collection it is closest to.

To begin with, and unlike the other scenarios we have already dealt with, it seems like the biggest concern here is to compare some given sequence against not just one other sequence, but an entire collection of them — essentially, it is like having to quantify the distance between a particular sequence and some collection — perhaps a cluster, of other sequences.

So how might we go about solving this using the tools already at our disposal?

NOTE:

Some Useful Ideas Concerning Classification Problems

Though we don't intend to borrow many or any ideas from what others would do or how existing [external or independent] methods might provide a solution to our classification problem, we shall call-out one useful general concept that is well placed in the Oxford Dictionary of Computing in relation to this kind of problem:

Decision Surface A (hyper) surface in a multidimensional state space that partitions the space into different regions. Data lying on one side of a decision surface are defined as belonging to a different class from those lying on the other. Decision surfaces may be created or modified as a result of a learning process and they are frequently used in machine learning, pattern recognition, and classification systems.

— Oxford Dictionary of Computing[44]

First, let us deal with the matter of associating our query sequence, Q_k , with one of several sequences or collections of them at our disposal.

6.6.1 Measuring Proximity to a Set of Sequences, Ω^n

So, if we have a set of n different sequences of arbitrary composition and lengths:

$$\Omega^n = \langle \Omega_1, \Omega_2, \Omega_3, \dots, \Omega_n \rangle \quad (6.11)$$

¹⁷In fact, the sequence characteristic is featured among the TEA standard programs that one might access [and even run] either via the TEA WEB IDE: <https://tea.nuchwezi.com> or via the TEA project repository[9].

And which we might also express more elaborately via [sparse¹⁸] matrix form as:

$$\Omega^n = \begin{bmatrix} \Omega 1 \langle \prod_{i=1}^{\mathcal{L}(\Omega 1)=\alpha} a_{1i} \rangle, \\ \Omega 2 \langle \prod_{i=1}^{\beta} a_{2i} \rangle, \\ \Omega 3 \langle \prod_{i=1}^{\chi} a_{3i} \rangle, \\ \vdots \\ \Omega n \langle \prod_{i=1}^{\zeta} a_{ni} \rangle, \end{bmatrix} = \begin{bmatrix} \Omega 1 \langle a_{11}, a_{12}, \dots, a_{1\alpha} \rangle, \\ \Omega 2 \langle a_{21}, a_{22}, \dots, a_{2(\beta-1)}, a_{2\beta} \rangle, \\ \vdots \\ \Omega n \langle a_{n1}, a_{n2}, \dots, a_{n\zeta} \rangle, \end{bmatrix} \quad (6.12)$$

Then, given some query sequence such as Q_k , and the consequences of **Theorem 2** in [5]¹⁹, we can safely classify the query sequence as **belonging to** or **being appreciably close** to a given population, if we find that computing the anagram distance between the query and the population via their representative modal sequence statistics results in an ADM value of 0, or if, where there are two or more populations to compare against, that we select that population whose ADM against the query sequence's MSS is appreciably close enough to 0 — or perhaps the least when comparing against multiple populations.

With that useful theory then, and given we already know how to compute an MSS, $\overset{>}{Q}_k$, for any particular sequence²⁰ Q_k , we instead better start by solving the matter of how to compute an MSS for a set of sequences — a **representative population modal sequence statistic**.

So, assuming we have $Q_k = \langle a, d, c \rangle$, $\Omega 1 = \langle a, b, c, d, e, f \rangle$ and $\Omega 2 = \langle a, b, a, d, c \rangle$, and so that $\Omega^n = \langle \Omega 1, \Omega 2 \rangle$, we wish to compute $\overset{>}{\Omega}^n$, a population modal sequence statistic representing or summarizing the membership and frequency distribution across the two sequences. So, how should we go about [correctly] computing $\overset{>}{\Omega}^n$?

STRATEGY 1: Since we have multiple sequences, and that they potentially even have some uncommon terms and dissimilar lengths, a meaningful strategy might be to not attempt to compute the population MSS directly from the in-

¹⁸Especially given that some of the rows might mostly contain missing elements where other rows have values, given the sequences (one per row) are of unequal length.

¹⁹Concerning “Equivalence of Distributions under Modal Sequence Analysis” — that if any two sequences otherwise so unlike in literal membership, ordering, size, etc, instead end up having similar or equal modal sequence statistics, then we can safely conclude they possibly belong to the same probability distribution.

²⁰In this work, check **First Analysis**, but also **Section 4.1** and especially **Definition 1** in [5] concerning how to compute the modal sequence statistic. Also consult **Definition 14** in the appendix.

dividual sequences, and instead use their sequence characteristics. That is, we would compute $\hbar(\vec{\Omega_1})$, $\hbar(\vec{\Omega_2})$, ... etc. and then using these, generate a combined **population characteristic**, $\hbar(\vec{\Omega^n})$. The essential definition shall help make things clearer:

Definition 9 (The **Population Characteristic**). *Given a collection, Θ^n , of n sequences of arbitrary length and composition, the representative **population characteristic**, that summarizes all the sequences within that population, denoted $\hbar(\vec{\Theta^n})$, is derived from the sequence characteristics of the contained sequences, $\hbar(\vec{\Theta_1})$, $\hbar(\vec{\Theta_2})$, ... , $\hbar(\vec{\Theta_n})$ as such:*

$$\hbar(\vec{\Theta^n}) = \prod_{\forall \omega_i \in \psi(\Theta^n)} \omega_i \cdot \mathbb{1}(\omega_i \in \Theta^n) \quad (6.13)$$

Where $\forall \Theta_\alpha \in \Theta^n$

$$\mathbb{1}(\omega_i \in \Theta^n) = \sum_{\forall \hbar(\vec{\Theta_\alpha})} f_{\omega_i} \quad (6.14)$$

And $\forall i, j \in \mathbb{N} : I(\omega_i, \vec{\Theta^n}) < I(\omega_j, \vec{\Theta^n}) \implies f_{\omega_i} \geq f_{\omega_j}$ for the frequency terms in $\hbar(\vec{\Theta^n})$. $\square$

Thus having computed the representative population characteristic for our sample/collection/list/set of sequences as $\hbar(\vec{\Omega^n})$, we then proceed to extract or reduce it to just the **representative population modal sequence statistic** (RPMSS)²¹, $\vec{\Omega^n}$. Essentially, this strategy is summarized by the following transformations²²

Transformation 16. $\Omega^n : \langle \Omega_1, \Omega_2, \dots, \Omega_n \rangle \xrightarrow{O_{SC}} \langle \hbar(\vec{\Omega_1}), \hbar(\vec{\Omega_2}), \dots, \hbar(\vec{\Omega_n}) \rangle \xrightarrow{O_{pc}} \hbar(\vec{\Omega^n}) \xrightarrow{\text{extract-mss}} \vec{\Omega^n}$

²¹Any careful analyst shall realize that computing or deriving the RPMSS as in the given process, or as per **Definition 6.13**, shall make the most sense, because, any other way would either ignore the important per-sequence symbol distribution information, or might just not be efficient or robust/trustworthy enough. Plus, where the frequencies don't matter — such as when each symbol only appears once across all sequences, then working from the per-sequence MSS would help generate a RPMSS that is close-enough to the relative ordering of terms in each member sequence with minimal or no effort.

²²We are using the variable Ω and not Θ as was used in **Definition 6.13** to especially relate this transformation to the example we opened with in this section, but the concepts should stay clear and consistent even if we had used the former sequence symbols.

And with the RPMSS, $\vec{\Omega}^n$, and having computed the MSS of the query sequence, $\vec{Q}_k$, we can then proceed to compute the proximity between the query and the population via a measure such as the ADM: $\tilde{A}(\vec{Q}_k, \vec{\Omega}^n)$, and if there were more than one population to compare the query against, use the computed ADM values as a **decision surface** to help objectively decide which of the analyzed populations the query most likely belongs to.

Concerning this method too, it is important to note that, given there might be cases in which the query sequence and the population contain several uncommon symbols — i.e. $\psi(Q_k) \setminus \psi(\Omega^n) \neq \emptyset$ or that $\psi(Q_k) \cap \psi(\Omega^n) = \emptyset$ — in the later case, there being no need to conduct any further analysis. Otherwise, we might need to adjust the modal sequences for both the query and population so that we can correctly compute their associated ADM. Thus, we might proceed by eliminating from both $\vec{Q}_k$ and $\vec{\Omega}^n$, those uncommon terms, so that the ADM we use as our decision surface is computed thus: $\tilde{A}(\approx \vec{Q}_k \rightarrow \approx \vec{\Omega}^n)$.

EXAMPLE:

How to Compute $\tilde{A}(\approx \vec{Q}_k \rightarrow \approx \vec{\Omega}^n)$

Assume we use the given sequences: $Q_k = \langle a, d, c \rangle$, $\Omega 1 = \langle a, b, c, d, e, f \rangle$ and $\Omega 2 = \langle a, b, a, d, c \rangle$, and so that $\Omega^n = \langle \Omega 1, \Omega 2 \rangle$, then:

- $\vec{Q}_k = \langle a, d, c \rangle$
- $\vec{h}(\vec{\Omega} 1) = a1b1c1d1e1f1$
- $\vec{h}(\vec{\Omega} 2) = a2b1d1c1$
- $\vec{h}(\vec{\Omega}^n) = a3b2c2d2e1f1 \implies \vec{\Omega}^n = abcdef$

However, given $\vec{\Omega}^n \setminus \vec{Q}_k \neq \emptyset$ or $\vec{\Omega}^n \cap \vec{Q}_k = \{a, d, c\}$, then we must adjust as such:

- $\approx \vec{\Omega}^n = acd$

And so that we can then compute $\tilde{A}(\vec{Q}_k \rightarrow \approx \vec{\Omega}^n) = \frac{1}{3}(|1-1| + |2-3| + |3-2|) = \frac{2}{3} < 1$.

If there is no other population to compare the query against, we can safely conclude the query is appreciably close enough to the population, otherwise we also compute the ADM between the query and the other populations, and then pick the one with the smallest/least ADM.

STRATEGY 2: The process outlined via the first strategy for solving the first problem in **Scenario D** might work well and be convincing enough for most practical and theoretical purposes, however, it is not the only plausible route to a good solution. Especially because we are mostly concerned with how to

obtain the RPMSS for a collection of potentially widely dissimilar sequences — a very possible case if we are for example dealing with realistic collections of DNA sequence readings/scans of say a person’s genome, the genome of some newly identified species with scanty details, or even the case of attempting to obtain an RPMSS for an entire set of individual organisms — e.g human members of a clan or particular family tree, etc. So, even though we might not delve into it here, there is a proposal to compute $\hat{\Omega}^n$ from n sequences via computing the population’s **genome sequence**.

This process would proceed somewhat like this:

- Using the provided collection of sequences Ω^n , and the process outlined in **Section 7.2.3**, especially via processing such a collection via the **Genome Sequencer** machine/transformer, we shall obtain a representative summary, well-aligned and potentially complete [na-]sequence Ω_{gs} that represents the entire collection/population specified.
- By the **Identity Genome Sequence Law**, we trust that such a sequence shall be unique for any two distinct collections or populations.
- And thus, having obtained Ω_{gs} , we then compute the RPMSS not from the individual sequences in a population, but instead from their representative genome sequence. That is to say:

$$\textbf{Transformation 17. } \Omega^n \xrightarrow{O_{tGS}} \Omega_{gs} \xrightarrow{O_{mss}} \hat{\Omega}_{gs} \approx \hat{\Omega}^n$$

And then we can use the obtained $\hat{\Omega}^n$ and the query’s $\hat{Q}_k$ to determine how close they are via an anagram distance measure as we have already seen in earlier examples and analysis.

Of course, concerning how we obtain the very crucial **RPMSS**, $\hat{\Omega}^n$ when we have a collection of sequences, Ω^n , it might be worth checking (practically so), which route or strategy is better; such as via **Transformation 16** which proceeds via the computation of the sequence characteristics of the individual member sequences so as to finally derive the population characteristic from them Vs **Transformation 17** that instead leverages the concept of sequence alignment so as to reduce the collection to a single representative sequence (a genome sequence), from which then the RPMSS is then derived. The two strategies call for different kinds of skills and sequence processing methods, and it might be that one is perhaps technically simpler than the other, however, theoretically, they seem to both be plausible approaches to the solution²³.

²³**FUTURE WORK:** Determine, both practically and theoretically, whether computing the RPMSS of a given collection of [na-]sequences is better when approached using the population characteristic method, or via the population genome sequence method. And to also ascertain whether either method results in the same or similar RPMSS for the same exact sequences, and if not, when either method is more preferable? For example, would the genome sequence method for computing a RPMSS make sense when being applied to sequences that aren’t genetic in nature, or even when they are genetic in nature, but not belonging to the same entity?

6.6.2 Measuring Proximity between a Q_k and Members of a Sequence Population, Ω^n

For answering the final aspect of **Scenario D**, we merely can proceed via the following Algorithm:

Algorithm 1 (The Closest Sequence Search Algorithm).

1. **COMPUTE** the MSS for Q_k — i.e. $\vec{Q}_k$.
2. Since we are looking for the **sequence closest** to Q_k , then given we know $\vec{Q}_k$, we should derive a special symbol set²⁴ from its MSS: $\psi^*(Q_k) = \hat{\psi}(\vec{Q}_k)$.
3. **INITIALIZE** an empty collection, Closest Sequence Tuples, **CST** := [].
4. **INITIALIZE** **ADM_CST** := ∞ .
5. **INITIALIZE** an empty collection, Aproximately Closest Sequence Tuples, **ACST** := [].
6. **INITIALIZE** **ADM_ACST** := ∞ .
7. **FOREACH** sequence, Ω_i in Ω^n :
 - (a) **COMPUTE** MSS $\vec{\Omega}_i$
 - (b) **COMPUTE** MSS-SS $\psi^*(\Omega_i) = \hat{\psi}(\vec{\Omega}_i)$
 - (c) **INITIALIZE** **ADM_i** := 0.
 - (d) **IF** $\psi^*(\Omega_i) \setminus \psi^*(Q_k) = \emptyset$:
 - i. **COMPUTE** $\tilde{A}(\psi^*(\Omega_i) \rightarrow \psi^*(Q_k))$ and store that in **ADM_i**
 - ii. **IFF** **ADM_i** < **ADM_CST**:
 - A. **add tuple** $\{i, ADM_i\}$ to **CST**.
 - B. **UPDATE**: **ADM_CST** := **ADM_i**.
 - (e) **ELSE/Otherwise**:
 - i. **ADJUST** $\psi^*(\Omega_i)$ and $\psi^*(Q_k)$ to eliminate uncommon terms and derive temporary approximations $\approx \psi^*(\Omega_i)$ and $\approx \psi^*(Q_k)$ from them.
 - ii. **COMPUTE** $\tilde{A}(\approx \psi^*(\Omega_i) \rightarrow \approx \psi^*(Q_k))$ and store that in **ADM_i**
 - iii. **IFF** **ADM_i** < **ADM_ACST**:
 - A. **add tuple** $\{i, ADM_i\}$ to **ACST**.
 - B. **UPDATE**: **ADM_ACST** := **ADM_i**.

²⁴Because we wish to preserve both information about the unique composition of our sequence, but also the relative frequency of the elements it contains, it is wiser that we work with a symbol set type such as **the unspecific symbol set of the sequence's modal sequence statistic**. We shall also compute this for the individual sequences in the collection we wish to compare the query against. This here is a semantic/logical optimization approach, otherwise, a naïve, less expensive method might be to merely proceed by analysing just the unspecific symbol sets of the sequences themselves — however this approach, especially for the case of having a statistically correct/sound neighborhood/proximity measurement would be fragile and very sensitive to order of first-occurrence of symbols in the sequences being analyzed.

8. **IF** *CST* is not empty:

(a) **SORT** *CST* in ascending order of the second term in each contained tuple: $\{i, ADM_i\}$ so that the tuple with the **lowest** *ADM* is the **first** in *CST*.

(b) From Ω^n , **RETURN** sequence with index *i* in that topmost tuple as the solution.

9. Otherwise **PROCESS** *ACST*:

(a) **SORT** *ACST* in ascending order of the second term in each contained tuple: $\{i, ADM_i\}$ so that the tuple with the **lowest** *ADM* is the **first** in *ACST*.

(b) From Ω^n , **RETURN** sequence with index *i* in that topmost tuple as the solution.

□

6.7 Concerning Optimal Symbolic Storage of na-Sequences in Databases

- Computing the Gödel Number for a Sequence Θ_n based on its Characteristic $\tilde{h}(\Theta_n)$



Figure 6.1: Some proposal to use Gödel's idea of abstracting formulae such as formal symbolic sequences using distinct natural numbers.

na-Sequences, for which we might have a collection of them, Θ^n , could have applied to them, a system of optimal or optimizable storage in practice, especially via electronic formats and protocols such as with flat text files, relative database systems and any well structured **operating memory-space structures** — vaults in TEA[29], JSON-dictionaries on the Web[45], YAML files on local file-systems, etc. In this section, we could explore with the same *na*-Sequences introduced in **Section 6.1**, for ways how their corresponding optimal hash keys might be

computed as part of an optimal-sequence lookup mechanism in bio-automata or perhaps in some na-Sequence database.

Concerning Storage of a na-Sequence $\theta_i \in \Theta_k^n$ such that Θ has n sequences partitioned into k subsequences²⁵ — essentially, a higher-order sequence.

We then might wish to have some mechanism — such as the following generator:

Transformer 6 (A Sequence Lookup System, $sLS(Q_m, \Theta^n)$). :

$$\Theta^n \xrightarrow{sLS(Q_m, \Theta^n)} \theta_i;$$

$$\varprojlim(Q_m) = m \leq \varprojlim(\theta_i) \quad \wedge \quad (\ddot{o}(Q_k) \approx \ddot{o}(\theta_i) \quad \vee \quad \tilde{A}(\vec{Q}_k \rightarrow \vec{\theta}_i) \leq \epsilon)$$

□

By **Transformer 6**, we can search for and express or return, a complete na-Sequence $\theta_i \in \Theta^n$, when presented with some query na-sequence Q_m of length m .

Thus, for optimal lookup mechanisms — concerns about space and [polynomial] time for looking up and returning θ_{na} from some **na-database**, $D\langle \theta_{i_{key}}, \theta_{i_{val}} \rangle^*$, such that, instead of directly searching the collection for the sequence with value $\theta_i = \theta_{i_{val}} \approx Q_m$, directly/naively using Q_m — the **user provided query**, we use the provably shorter — in terms of storage space, but also [*more intelligent-friendly* — in terms of sequence-expressing-information] higher **entropy encoding** structure, such as with a **hash key** for each θ_i as well as any Q_k , computed as:

$$\Gamma(id(\theta_i)) = \ddot{o}\left(\prod_{\omega_i, f_i \in \vec{h}(\theta_i)} \omega_i \cdot f_i\right) \cong \ddot{o}(\vec{h}(R_{\Theta_k^n})) \quad (6.15)$$

— see definition of $R_{\Theta_k^n} \approx \Theta_n$ in **IGS Law (Section 7.4)**, while the special sequence operator $\ddot{o}(\Theta_n)$ computes and returns a **Gödel Number**[44]²⁶, $\ddot{o}(\Theta_n) : \psi_{\Theta_n}^k \times \mathbb{N}^k \rightarrow \mathbb{N}$ that is derived optimally using the characteristic, $\vec{h}(\vec{\Theta}_n)$, for Θ_n a sequence of length n , and the first k prime numbers such that $k = \varprojlim(\vec{\Theta}_n) \quad \wedge$

²⁵This is just the proposed concept of the storage representation for an individual entire genome sequence; just one of many possibilities. Particular implementations might vary in actual sequence expression (in storage). For nucleic acid operating systems, we know that na-Sequences are in natural storage, when expressed as DNA (with partitions or access-abstractions able to discern individual nucleic acids, codons, genes, chromosomes, double-helices of chromosomes, etc), however, while in use for processes such as RNA-oriented sequence processing (see **Section 9.1**), are operated on, not in bulk or whole-at-once, but in subsequences of some finite length — such as k in this formal specification, and $k = 3$ for typical, natural protein synthesis in ribosomes as an example.

²⁶Based on the concept of **Gödel numbering** — “A one-to-one mapping (i.e. an injection) of the symbols, formulas, and finite sequences of formulas of the formal system onto some subset of the natural numbers. The mapping must be such that there is an algorithm that, for any symbol, formula, or finite sequence of formulas, identifies the corresponding natural number; this is the *Gödel number* of that object.”[44]. Gödel numbers are a method developed by Kurt Gödel in 1931[46] to encode symbols, formulas, and entire proofs from formal mathematical systems as unique natural numbers. This encoding allows syntactic objects to be treated arithmetically[47].

$\forall \omega_i \in \vec{h}(\vec{\Theta}_n) : \ddot{o}(\omega_i) : \ddot{o}(i) = f_i \times (p_i \in \mathbb{P})$ — p_i is the i -th prime²⁷ starting from 2 and spanning $\mathbb{N}$.

Computing a hashkey based on a Gödel number is just one approach to the optimal way to implement a sequence lookup mechanism with **Transformer 6**. Among the **attractive properties** of this method, is the realization that, unlike the case of using the **ADM**, the Gödel number approach boils down to comparing sequences using pure numbers and to which mere basic arithmetic tests can be applied than using sequences directly or sequence projections (themselves still potentially non-numeric sequences) to which we can not readily apply search or measurements by pure arithmetic means.

Also, note that, with the Gödel number approach, and if we proceed to compute the identifier or hash-key of some sequence Θ — i.e $\Gamma(id(\Theta))$ — via computation of its corresponding Gödel number, $\ddot{o}(\Theta) \in \mathbb{N}$, as per the following formulation:

$$\Gamma(id(\Theta)) = \ddot{o}(\Theta) = \prod_{f_i \in \vec{h}(\vec{\Theta})} f_i \times p_i \quad (6.16)$$

So that, if we have any two sequences, Q and Θ , and that we wish to establish if they are potential neighbors, or symbolically related, one possibility, and a very simple technique of elimination, would be that, if we trust that the length of either Q or its symbol set $\psi(Q)$ is shorter than that of the target sequence Θ — i.e if $\varkappa(Q) \leq \varkappa(\Theta)$ or better, that $\varkappa(\psi(Q)) \leq \varkappa(\psi(\Theta)) \leq \varkappa(\Theta)$, then, it would save lots of processing time, if we merely compute the symbol set of the short query, $\psi(Q)$, and from that, compute the Gödel numbers of each of the query symbols, i.e $\ddot{o}(\omega_i)$ — which assumes that for any distinct $\omega_i \in \psi(Q)$ or $\omega_i \in \psi(\Theta)$, we have a consistent method of assigning corresponding Gödel numbers (for example, by ensuring that we map from $\omega_i \in \psi_* \rightarrow p_i \in \mathbb{P}$ for $\psi_* \supset \psi(Q) \cup \psi(\Theta) \quad \forall Q, \Theta$), such as via $\ddot{o}(\omega_i) = p_i : I(\omega_i, \psi_*) = I(p_i, \mathbb{P}) = i$ — and so that, via the property:

$$\Gamma(id(\Omega)) \mod \ddot{o}(\omega_i) = 0 \quad \forall \omega_i \in \psi_* \cap \psi(\Omega) \quad (6.17)$$

We know that if any two sequences have some uncommon symbol, then we shall find that the corresponding Gödel number for that symbol cannot be a factor of one of these sequences's corresponding Gödel number-based identifier or hash-key. Otherwise, two sequences sharing the same symbol set shall always be guaranteed to have common Gödel number factors for each of their common symbols and

²⁷ $\mathbb{P}$ is the set of all prime numbers.

corresponding database hash-keys or identifiers, and thus, we can leverage this criteria to quickly eliminate unlikely candidates in a proximity or similarity lookup test for a database or other high-density sequence storage system via leveraging a **basic arithmetic modulo operation test** instead of other expensive proximity or similarity tests.

However, even that method utilizes the transformatics concept of summarizing arbitrary sequences by their **sequence characteristic** first, and consequently, their **modal sequence statistic**, so as to later arrive at the pure number hash keys²⁸. However, as we see in the constraints and specification of that generator-transformer, the other option is to compute the solution using the ADM applied to keys based on just the modal sequence statistics of the query and the target sequences in Θ^n as we have already gotten well acquainted with in much of this chapter.

Finally, given that the hashkey or identifier key for some sequence Θ as per the Gödel number method might result in a potentially large natural number that is derived from the product of several natural numbers (f_i) and prime numbers (p_i), as per **Equation 6.16**, of course, unless a clever method for keeping these keys short is enforced²⁹, otherwise, given what we know of the way that the length of number expressions grow in sequence cardinality under multiplication for pure numbers[2]:

NOTE:

Theorem 5 (Cardinality under Multiplication for Pure Numbers). *Given any two **pure numbers** τ and σ , the cardinality of their product $\vartheta(\tau \times \sigma)$ is bounded thus^a*

$$\text{Max}(\vartheta(\tau), \vartheta(\sigma)) \leq \vartheta(\tau \times \sigma) \leq \vartheta(\tau) + \vartheta(\sigma) \quad (6.18)$$

iff $\tau \neq 0$ and $\sigma \neq 0$ otherwise

$$C_{\tau \times \sigma} = 1 \quad (6.19)$$

^aNote that in this reformulation of **Theorem 15** of GTNC[2], that we have replaced use of the original cardinality notation C_Θ in favor of the new, transformatics notation $\vartheta(\Theta)$, but the concepts and underlying logic remain the same.

It means that for hashkeys generated via **Equation 6.16**, the size of the resulting expressions is bounded thus:

²⁸**FUTURE WORK:** In later analyses and perhaps further development of this idea, it might help to ascertain whether or not projecting arbitrary sequences to pure numbers would still internally preserve useful distance heuristics such as how the ADM can hint at sequences that are quantifiably and especially lexically similar/close to each other, while, with Gödel numbers, we have not clearly or robustly explored all such nuances, and there could be a possibility that perhaps, unless, as we have somewhat constrained our derivation methods here, it might be possible to arrive at multiple pure numbers for the same exact sequence when the formulation for computing a sequence's Gödel number is altered across storage or sequence look-up systems.

²⁹Such as, when for starters, we limit our sequence alphabets to only a minimal finite symbol set such as ψ_{na} , ψ_{az} or perhaps for arbitrary text sequences, something like either ψ_{36} or the ascii character set, ψ_{ascii} , since, by having a small alphabet or symbol set, also the corresponding largest prime number, p_i , that we would encounter when computing the corresponding Gödel number-based keys would be thus constrained — especially since for any such symbol set, we want to ensure that $I(\omega_1, \psi_*) = I(p_1, \mathbb{P})$ where we know that $p_1 = 2, p_2 = 3, p_3 = 5, \dots$

$$Max(\vartheta(f_i), \vartheta(p_i)) \leq \vartheta(\Gamma(id(\Theta))) \leq \sum \vartheta(f_i) + \vartheta(p_i) \quad (6.20)$$

For all $f_i \geq 1$ — which is all f_i we shall encounter for any sequence, since there is no way a symbol's frequency entry, f_i — for say some $\omega_i \in \Theta$ or from an associated symbol set, $\omega_i \in \psi_*$ — would end up in our Gödel number formulation (**Equation 6.16**) unless it at least corresponded to at minimum one symbol instance in the underlying sequence Θ and thus at least $f_i = 1$ in the corresponding $\hbar(\overset{>}{\Theta})$.

Chapter 7

The Mathematics of Genome Sequencing: Sequence Alignment, The Modal Sequence, The Sequencing Machine and the Identifier Genome Sequence Law

7.1 Relevant Background

First, we shall mostly assume there is essentially no reliable mathematical formalisms or theory concerning the important matter of genome sequencing¹. And so, shall set out to construct and develop such a formalism ourselves, from what we already know.

Second, from the theoretical and exploratory reviews we have conducted leading up to this moment, and what some authorities[49] say about the subject, we know that genome sequencing as a practice leveraging computational and mathematical principles and concepts, or rather, when approached from a predominantly mathematical theory perspective, still has some ambiguities and unresolved problems that we have identified and intend to iron out especially in this chapter.

These for example include an overall lack of proper mathematical formalisms around issues such as:

1. How [genome] sequencing works or what it is exactly; how can it be described computationally?
2. If we treat of genome sequencing using the language of transformatives, and think of it in terms of processing some collection of sequences to arrive at some resultant genome sequence, how should one arrive at such a resultant sequence then?
3. What is a “consensus sequence”[49]?
4. What is meant by “a most representative genome sequence” or perhaps an

¹For those that do not know, a formal definition is presented in **Definition 11**.

NOTE:**On Sequencing**

It is important that we have some general appreciation of the core concept about which this chapter is dedicated; *sequencing*. For this matter, we shall want to bring to mind one of the popular definitions of the term as employed in computer science for example:

Sequencing The procedure by which ordered units of data (octets or messages) are numbered, transmitted over a communications network (which may rearrange their order), and reassembled into the original order at their destination.

— Oxford Dictionary of Computing[44]

That is the closest concept or definition I could glean from my collection of academic literature and books in my private library at Nuchwezi Research Lab, however, this other definition, obtained via Microsoft's Copilot might also help shed more light on the particular concept of concern here: *genome sequencing*:

Genome Sequencing^a[48]: Genome sequencing is the process of determining the complete DNA (or RNA) sequence of an organism's genome. It identifies the precise order of nucleotide bases (A, T, C, G) across all or most of the genetic material, providing a comprehensive "blueprint" of the organism's hereditary information. This technique is fundamental in biology, medicine, and biotechnology, enabling insights into genetic variation, disease mechanisms, and evolutionary relationships.

— Microsoft Copilot[32]

^aAlso see: https://en.wikipedia.org/wiki/Whole_genome_sequencing

identifying **na-Sequence** given some living organism or particular aspects of it?

5. Could we (perhaps conceptually, or theoretically) talk of sequencing a population instead of an individual organism, an entire species or perhaps (sic) non-living things too? How?
6. Could we pin down a genome sequence or some na-Sequence responsible for say the expression² of some particular protein, some genetic mutation, a behavioral trait, anatomical or psychological anomalies and pathologies, the material basis of particular or general diseases, incapacities, etc. using a mathematical theory or language? How?

Seeing as the matter of determining the actual na-Sequence that correctly identifies a person or thing might be hinging on the case of attempting to reconstruct a book by mathematically piecing together disparate pieces of it — some in alignment, some out of alignment, some, supersets of others, some prefixes or suffixes of others, etc.³, we then start to come to some appreciation of how best we might

²Related, but not to be treated of in this chapter but instead in **Chapter 10**, is the matter of mathematically approaching how the expression — visual, material or spatial, of any meaningful *genome* sequence — could be for a basic organism, an aspect of it, or even the case of *anomalous entities* such as viruses, mere proteins or such, could be approached.

³This analogy shouldn't come as a surprise, when compared to how we found it best to describe DNA using a figurative metaphor

approach the problem's solution.

Also, though we might not immediately surface such philosophical speculations here, the difficulties and problems thus raised above, might likewise apply to or at least hint at the challenges there might be, in attempting to think of or practically resolve the matter of “computationally sequencing anything” — the idea being, to not limit the concept of *sequencing* to only living things or living organisms only, but to also think generally and abstractly **concerning sequencing anything expressible in consensus reality via some kind of specification sequence**⁴.

Thus, we might want to also think of, keep in mind, the possibility of sequencing the peculiar, miniscule aspects of consensus reality — sub-atomic particles that make up atoms that make up molecules that make up living things on the miniature scale, and then all the way up to planets, stars and galaxies on the grand, cosmic and universal scale. One might wonder, and correctly so, is such sequencing possible? Does it/would it make sense? Is it necessary or even useful? These, and related problems are among the concerns we shall attempt to address in the rest of this chapter, but also, bits of what we can not immediately answer, perhaps we shall tackle in later chapters or future works⁵.

7.2 3 Core Problems to Solve

We shall develop our theory in bits, starting with the most basic, most fundamental, and then basing on that, to construct higher aspects of the necessary mathematical structures and system.

The method is simple; we have distilled 3 core problems whose solutions would culminate in the formal and mathematical definition of genome sequencing. The 3 problems are

1. **PROBLEM 1:** Computing the best or rather correct common subsequence given any two or more potentially dissimilar sequences.
2. **PROBLEM 2:** Computing the correct sequence that resolves conflicts and correctly expresses variations at each particular position (essentially a column) in the alignment matrix depicting a definite side-by-side alignment of two or more sequences with an overlap in the alignment position being considered.
3. **PROBLEM 3:** Computing the Genome Sequence.

(see **Introduction**), but also based on how some authorities seem to be teaching this subject to their students[49].

⁴Or perhaps an “identifying sequence”.

⁵**FUTURE WORK:** Explore, philosophically, but also mathematically, whether or not it makes sense to think of the [genome] sequencing of arbitrary things; not necessarily only living/biological entities, but also any coherent material entities such as particular atoms, molecules,... all the way to entire ecosystems, planets and star-systems perhaps! For example, given that the manifestation of biological lifeforms on some particular planets within a star system heavily relies on some initial conditions inherent in the early formation of the star [system] itself, could it be that something fundamental in matter could underlie all manifestation or expression of existence other than just biological life, and that we might somehow be able to pin it down to some kind of identifier sequence? A genome for an entire planet or star system?

We shall tackle each of these problems in its own section, one by one, until we arrive at the original objective of this chapter, in **Section 7.3**.

7.2.1 PROBLEM G1: Computing Maximum Common Subsequence: $\Theta_1 \diamond \Theta_2$

Problem 1 (*Computing MCS:*). : $\Theta_1 \diamond \Theta_2$] Given sequences $\Theta_1 = \langle k, l, m, o, p, x, y, z \rangle$ and $\Theta_2 = \langle b, c, a, y, z, k, l, m, o, p \rangle$, find the longest common subsequence they share — also to be denoted $\Theta_1 \diamond \Theta_2$, their **Maximum Common Subsequence**.

Solution 1. We shall specify a transformer, $tMCS(\Theta_1, \Theta_2)$ that produces the required solution from the input sequences as such:

Transformer 7 (The **tMCS** Transformer). $\langle \Theta_1, \Theta_2 \rangle \xrightarrow{O_{tMCS}(\Theta_1, \Theta_2)} \Theta_1 \diamond \Theta_2$;
 $\Theta_1 \diamond \Theta_2 = \Theta_2 \diamond \Theta_1 \quad \wedge \quad 0 \leq \nu(\Theta_1 \diamond \Theta_2) \leq \text{Max}(\nu(\Theta_1), \nu(\Theta_2))$
 $\wedge \quad \forall \omega \in \Theta_1 \diamond \Theta_2 \quad \exists \omega \in \Theta_1 \wedge \exists \omega \in \Theta_2$
 $\wedge \quad \forall \omega_i, \omega_j \in \Theta_1 \diamond \Theta_2 : I(\omega_i, \Theta_1 \diamond \Theta_2) < I(\omega_j, \Theta_1 \diamond \Theta_2)$
 $\implies \quad I(\omega_i, \Theta_1) < I(\omega_j, \Theta_1) \quad \wedge \quad I(\omega_i, \Theta_2) < I(\omega_j, \Theta_2)$
 $\wedge \quad \forall \Theta_1 \diamond \Theta_2, \quad \Theta_1 \diamond \Theta_2 \subset \Theta_1 \quad \wedge \quad \Theta_1 \diamond \Theta_2 \subset \Theta_2$

□

Solution 1 is easier said than done. However, we shall attempt to formally and rigorously express it by formalizing what exactly the $tMCS(\Theta_1, \Theta_2)$ transformer does when expressed as a computable algorithm. Basically, we are to compute the MCS via **Algorithm 2** specified hereafter.

Algorithm 2 (The **tMCS** Algorithm⁶).

1. **INITIALIZE** the global maximum common sequence, O_g^m , to the empty sequence⁷.
2. **GIVEN** a collection of n sequences, $\Theta^n = \{\Theta_1, \Theta_2, \dots, \Theta_n\}$.
3. **START** by computing the cardinality of each sequence, $\Theta_i \in \Theta^n$, and note the **largest sequence cardinality**, $\text{Max}(\nu(\Theta_i) \quad \forall \Theta_i \in \Theta^n) = \text{Max}(\nu(\Theta_i))$, for the longest sequence, Θ_m such that $\text{Max}(\nu(\Theta_i)) = \nu(\Theta_m)$.
4. **COMPUTE** the cardinality of a sequence that can exactly contain the given sequences each concatenated to the other, but with the longest sequence duplicated. Essentially, compute:

$$C_R = \nu(\Theta_m) + \sum_{\forall \Theta_i \in \Theta^n} \nu(\Theta_i) \quad (7.1)$$

⁶**IMPORTANT:** Note that, even though the MCS transformer, **Transformer 7**, is defined for just two sequences, and yet, the tMCS algorithm based off of it is designed to handle any arbitrary number of sequences and not just a pair.

⁷Refer to **Foundation Idea D.1.1**

5. **SORT** the collection of sequences, Θ^n , in descending order⁸ of the sequence cardinality. Let the sorted sequence collection be denoted $\hat{\Theta}^n = \langle \Theta_m, \Theta_{2*}, \dots, \Theta_{n*} \rangle$ — where the first item in $\hat{\Theta}^n$, Θ_{1*} is essentially Θ_m .
6. **SELECT** the first two sequences from $\hat{\Theta}^n$, such as Θ_m and Θ_{2*} — or rather/— generally, $\hat{\Theta}^n[1]$ and $\hat{\Theta}^n[2]$, as the two sequences we shall use to search for and derive the potential MCS.
7. **POSITION** the two sequences, Θ_m and Θ_{2*} within two adjacent arrays $A_1^{C_R}$ and $A_2^{C_R}$ of equal length C_R — for simplicity, we shall just denote them as A_1 and A_2 , in such a way that Θ_m occupies in its containing array A_1 , positions from m to $2m$, but for simplicity's sake⁹, which we shall just treat as though 1 to m :

$$A_1 : [1, m]$$

While Θ_{2*} occupies positions in the range¹⁰:

$$A_2 : [(1 + I(\Theta_m[m], A_1), \quad I(\Theta_m[m], A_1) + \nu(\Theta_{2*})]$$

Or, equivalently, the range:

$$A_2 : [m + 1, \quad m + \nu(\Theta_{2*})].$$

Which we can also express visually as:

A_1	...	$a_{1,1}$	$a_{1,2}$	...	$a_{1,m}$					...
A_2	...					$a_{2,1}$	$a_{2,2}$	...	$a_{2,\nu(\Theta_{2*})}$	...

8. **UPDATE** the two sequence alignment arrays as such:

⁸**FUTURE WORK:** We especially prefer descending order by sequence cardinality because, as we shall come to appreciate in the rest of the tMCS algorithm, it is more likely that the MCS is discovered or developed based on the longest common [sub-]sequence (in the longest sequences in the collection) than if we had started out with the shortest sequences. In fact, it also is somewhat very logical, since, the longest sequence shall most likely also contain elements and subsequences that shorter sequences have or are derived from than vice versa! Thus we can converge towards the MCS faster that way. Also, this is *the optimistic strategy* — believing that the most common subsequence shall most likely already exist in the longest sequences we are analyzing — which makes sense for genome sequencing given we are typically analyzing sequences from the same entity or individual or species, and that DNA or genetic code for the same individual or entity stays consistent across all or most of the samples from that entity. But otherwise the pessimist route might be to start out with the shortest sequences in the collection (esp. where we are dealing with totally antagonistic entities). But is this always optimal? Should it stay this way even when we are attempting [genome] sequencing for non-biological or non-genetic sequences?

⁹We shall later see (in **Step#8(c)**) that it is useful to position Θ_m this way, because then, the second sequence's positions are all that need to be changed — items being shifted leftwards in A_2 relative to A_1 , while those of the longest sequence in A_1 remain fixed. But to keep the mathematical formalisms simple, we shall especially write our expressions as though elements of Θ_m were positioned from index 1 in A_1 — these details especially matter to programmers and software engineers that shall want to translate the algorithm into working computer program source-code for various different languages and type-systems.

¹⁰We write “ $\Theta_{2*}[1]$ ” in our formulation here, to mean the first element in Θ_{2*} — i.e. position 1 has value 1, and the n th position has value n , not usual (computer-science) $n - 1$

- (a) **KEEP** the first array, A_1 , unchanged (since it contains the longest sequence, $\Theta_{1*} \in \hat{\Theta}_n$).
- (b) **SET** $j = z$, where z is the **shift step size** — simplest scenario, $z = 1$.
- (c) **SHIFT** each element in the second sequence by j -steps to an earlier position in array A_2 , so that the two sequences Θ_m and Θ_{2*} have an overlap across the alignment arrays/matrix in positions spanning range:

$$A_1 : [m - j, \quad m]$$

and so that Θ_2 shall be occupying the updated range:

$$A_2 : [m + 1 - j, \quad m + \mathfrak{L}(\Theta_{2*}) - j]$$

- (d) **SCAN** the overlapping sections in both sequences, in positions $[m - j, m]$ and by comparing each item in that range — i.e. $\Theta_m[i]$ Vs $\Theta_{2*}[i]$, determine the longest common subsequence, or rather the **MCS**, thus:

Assume the overlap is of length¹¹ ϕ such that the overlap can be visually depicted as such:

$\approx \Theta_m$	$a_{1,m-\phi+0}$	$a_{1,m-\phi+1}$	$a_{1,m-\phi+2}$	$\cdots$	$a_{1,m-\phi+j}$
$\approx \Theta_2$	$a_{2,m-\phi}$	$a_{2,m-\phi+1}$	$a_{2,m-\phi+2}$	$\cdots$	$a_{2,m}$
$(\Theta_m[i] == \Theta_{2*}[i])?$	1	1	0	$\cdots$	1
$\approx (\Theta_m[i] == \Theta_{2*}[i]) \approx O^\phi$	$f(\Theta_m[i], \Theta_{2*}[i])$	$a_{1,m-\phi+1}$	0	$\cdots$	$f(a_{1,m}, a_{2,m})$

CREATE another array of maximum length ϕ , i.e. O^ϕ , which contains at most ϕ elements generated thus:

- $O^\phi[i]$ contains 0 if the corresponding two elements in Θ_m and Θ_{2*} didn't match. Otherwise it contains the matching element value from the first sequence. Essentially, resolve what goes into O^ϕ at each of the overlap indices — $i : i \in [m - \phi, m]$ — via the function:

$$f(\Theta_m[i], \Theta_{2*}[i]) = \begin{cases} \Theta_m[i] & \Theta_m[i] == \Theta_{2*}[i], \\ 0 & \Theta_m[i] \neq \Theta_{2*}[i] \end{cases} \quad (7.2)$$

- **TRUNCATE** O^ϕ to its longest non-zero/non-empty subsequence, and **SET** that as $\bar{O}^\phi$.

¹¹Essentially, since the overlap spans the range $[m - j, m]$, then $\phi = m - (m - j) = j$

- **IFF** $\vartheta(\bar{O}^\phi) > \vartheta(O_g^m)$, then **UPDATE** or replace O_g^m with $\bar{O}^\phi$
 $\implies O_g^m := \bar{O}^\phi$
 - (e) **UPDATE** $j \implies j := j + z$
 - (f) **IFF** $j = \vartheta(\Theta_m) + \vartheta(\Theta_{2*})$, then **GOTO**¹² **Step#9**
 - (g) **ELSE ITERATE** from **Step#c**
9. **UPDATE** Θ_{2*} as such:
- **COMPUTE** index of next comparison sequence, k , as such:

$$k = I(\Theta_{2*}, \hat{\Theta}^n) + 1 \quad (7.3)$$

- **IFF** $k > n$, then **RETURN**¹³ O_g^m .
 - **ELSE LET** $\Theta_{2*} := \hat{\Theta}^n[k]$
10. **IFF** $\vartheta(\Theta_{2*}) < \vartheta(O_g^m)$, then **RETURN**¹⁴ O_g^m .
11. **ELSE ITERATE** from **Step#7**

□

Applying The Transformer and Associated Alogirithm

And so, applying **Algorithm 2** to our example sequence collection:

We see that:

1. Initially, our MCS, $O_g^m = \langle \rangle$.
2. For our given sequences, we have our collection $\Theta^n = \Theta^2$ as such:

$$\Theta^n = \langle \Theta_1, \Theta_2 \rangle = \langle \langle k, l, m, o, p, x, y, z \rangle, \langle b, c, a, y, z, k, l, m, o, p \rangle \rangle \quad (7.4)$$

3. $\vartheta(\Theta_1) = 8$, and $\vartheta(\Theta_2) = 10$, so that then, Θ_2 is our longest sequence, and thus $\Theta_m = \Theta_2$.

¹²At this juncture, we shall have essentially finished sliding the second sequence, Θ_{2*} , past all the elements of the first sequence so that at $j = \vartheta(\Theta_m) + \vartheta(\Theta_{2*})$ there is nomore overlap possible, and so we can move on to the next sequence.

¹³Basically, we have reached the end of our sorted list of sequences and whatever we currently have as MCS must be the solution.

¹⁴Basically, once we find that the current MCS is longer than any of the remaining sequences in the collection, we might as well stop searching and return it as the final solution.

4. As for the size of our alignment array, we have that:

$$C_R = 10 + (10 + 8) = 28 \quad (7.5)$$

5. And as for the sorted collection, we have that $\hat{\Theta}^n = \langle \Theta_2, \Theta_1 \rangle$, which implies that initially, $\Theta_{2*} = \Theta_1$.
6. As for how the two sequences are located in the alignment matrix or arrays, we have the following visual depiction initially:

i	...	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
A_1	...	b	c	a	y	z	k	l	m	o	p								
A_2	...											k	l	m	o	p	x	y	z

At which point, the MCS between these two sequences, $\Theta_m \diamond \Theta_{2*} = O^\phi = \langle \rangle$.

7. We find that there shall be no interesting overlaps until when we reach $j = 5$, at which point, we shall have the following alignment configuration:

i	...	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	...
A_1	...	b	c	a	y	z	k	l	m	o	p						...
A_2	...						k	l	m	o	p	x	y	z			...
$a_1[i] == a_2[i]?$	...	0	0	0	0	0	1	1	1	1	1	0	0	0	0	0	...

At which point, the MCS between these two sequences, $\Theta_m \diamond \Theta_{2*} = O^\phi = \langle k, l, m, o, p \rangle$, and that $\nu(O^\phi) = 5 > \nu(O_g^m)$, and so then we can update the global MCS so that $O_g^m = \langle k, l, m, o, p \rangle$.

8. At $j = \nu(\Theta_m) = 10$, we find that the second sequence shall entirely be positioned at the start of the longest sequence, and that we should have the second sequence entirely overlapping with the first, and at which point, we shall have the following alignment configuration for our example:

i	...	1	2	3	4	5	6	7	8	9	10	...
A_1	...	b	c	a	y	z	k	l	m	o	p	...
A_2	...	k	l	m	o	p	x	y	z			...
$a_1[i] == a_2[i]?$	...	0	0	0	0	0	0	0	0	0	0	...

At which point, the MCS between these two sequences, $\Theta_m \diamond \Theta_{2*} = O^\phi = \langle \rangle$, and that $\nu(O^\phi) = 0 < \nu(O_g^m)$, and so then we cannot update the global MCS so that, **still**, $O_g^m = \langle k, l, m, o, p \rangle$.

9. At $j = 13$, we find that we have another interesting overlap, and at which point, we shall have the following alignment configuration for our example:

i	$\dots$	1	2	3	4	5	6	7	8	9	10	11	12	13	14	$\dots$
A_1	$\dots$				b	c	a	y	z	k	l	m	o	p		$\dots$
A_2	$\dots$	k	l	m	o	p	x	y	z							$\dots$
$a_1[i] == a_2[i]?$	$\dots$	0	0	0	0	0	0	1	1	0	0	0	0	0	0	$\dots$

At which point, the MCS between these two sequences, $\Theta_m \diamond \Theta_{2*} = O^\phi = \langle y, z \rangle$, and that $\nu(O^\phi) = 2 < \nu(O_g^m)$, and so then we cannot update the global MCS so that, **still**, $O_g^m = \langle k, l, m, o, p \rangle$.

10. We finally stop when $j = \nu(\Theta_m) + \nu(\Theta_{2*}) = 10 + 8 = 18$, and at which point we shall find that we have not encountered any other overlap longer than our earlier MCS, and so that, given we also have no more sequences to compare against, we must **return the final MCS** as $O_g^m = \langle k, l, m, o, p \rangle$.

Thus, after applying **Algorithm 2** to our two-member sequence collection, we find that the MCS solution we arrive at is $\langle k, l, m, o, p \rangle$ for that particular higher-order sequence. We can confirm that the solution conforms to the specifications of **Transformer 7**, such as noting that:

- From $\langle \Theta_1, \Theta_2 \rangle$, we have $\Theta_1 \diamond \Theta_2 = \Theta_2 \diamond \Theta_1 = \langle k, l, m, o, p \rangle$.
- We see that $\nu(\Theta_1 \diamond \Theta_2) = 5$, so that $0 \leq \nu(\Theta_1 \diamond \Theta_2) \leq \text{Max}(10, 8)$
- We also see that all elements in $\Theta_1 \diamond \Theta_2$ also exist in either sequences.
- And also that the relative order of elements in $\Theta_1 \diamond \Theta_2$ preserves the relative order of corresponding symbols in both Θ_1 and Θ_2 .
- And that finally, we see that $\Theta_1 \diamond \Theta_2$ is a sub-sequence of both Θ_1 and Θ_2 .

7.2.2 PROBLEM G2: Computing Overlap Resultant Sequences: $\Theta_1 * \Theta_2$

Problem 2 (*Computing ORS: **Overlap Resultant Sequence***). : $\Theta_1 * \Theta_2$
 Given two overlapping/well-aligned sequences Θ_1 and Θ_2 of the same length but potentially different terms at any position in the range $[1, \nu(\Theta_1)]$, how to compute a resultant sequence $\Theta_1 * \Theta_2$ that is most meaningful for DNA sequences?

Solution 2. Assuming $\Theta_1 = \langle \prod_{i=1}^n a_i, \rangle$ and $\Theta_2 = \langle \prod_{i=1}^n b_i, \rangle$, construct Θ^* via the following *tORS* transformer:

Transformer 8 (The **tORS** Transformer).

$$\langle \langle \Theta_1 \langle \prod_{i=1}^n a_i, \rangle, \Theta_2 \langle \prod_{i=1}^n b_i, \rangle \rangle \xrightarrow{O_{tORS}(\Theta_1, \Theta_2)} \Theta^* = \Theta_1 * \Theta_2;$$

$$\nu(\Theta_1 * \Theta_2) = \nu(\Theta_1) = \nu(\Theta_2) = n$$

$$\wedge \quad \Theta^* = \langle \prod_{i=1}^n f(a_i, b_i), \rangle$$

for $f(x, y)$ **the ORS resolver function** defined as such:

$$f(x, y) = \begin{cases} x, & x == y, \text{ terms are the same} \\ x, & x = \neg(y), \text{ one term is the complement of the other} \\ y & \text{otherwise.} \end{cases}$$

□

Concerning that succinct solution, note that we have covered sequence complements in **Chapter 5** and especially for DNA or na-Complements in **Section 5.1**. We shall not delve into the intricacies of translating **Transformer 8** into a working generic algorithm such as we did for **Problem G1**, because we have already well illustrated how the process works — see or compare **Algorithm 2** to **Transformer 7**, and also see how we have applied both the algorithm and underlying transformer to some illustrative test-cases in **Section 7.2.1**. Thus, it should not be hard for the serious student, researcher or genetics engineer to apply, verify and/or extend the above proposed solution to their particular needs.

Finally, note that being able to compute the **ORS** is quite useful, as it underlies the computation of the consensus overlapping sequence from which a genome sequence is then derivable as we see in the next section.

7.2.3 PROBLEM G3: Computing the Genome Sequence (GS): $(\Theta_1 \diamond \Theta_2)^*$

Problem 3 (*Computing GS: **Genome Sequence***). : $(\Theta_1 \diamond \Theta_2)^*$] Given any two sequences, Θ_1 and Θ_2 , that are the elements of a sequence collection, Θ^n , compute their **resultant genome sequence**, denoted $(\Theta_1 \diamond \Theta_2)^*$, that contains at core, their longest common subsequence (the MCS), $O^m = (\Theta_1 \diamond \Theta_2)$, potentially padded on either side by the specially computed prefix and suffix subsequences from either input sequence as such:

$$(\Theta_1 \diamond \Theta_2)^* = \Theta_1^a \cdot (\Theta_1^b * \Theta_2^b) \cdot (\Theta_1 \diamond \Theta_2) \cdot (\Theta_1^c * \Theta_2^c) \cdot \Theta_2^a \quad (7.6)$$

And where the special terms are derived and named as such:

- Θ_1^a is the non-overlapping section of Θ_1 before the **COS** — to be introduced soon.

- $(\Theta_1^b * \Theta_2^b)$ is a consensus sequence from Θ_1 and Θ_2 such that its elements are the result of applying the **ORS resolver function** to compute $f(\Theta_1[i], \Theta_2[i])$ as specified in **Transformer 8**. Thus, it is one of the overlapping resultant sequences (ORS) resulting from Θ_1 and Θ_2 — this particular one occurring **before the MCS**.
- $(\Theta_1 \diamond \Theta_2)$ is the **maximum common subsequence**, MCS, O_g^m , of Θ_1 and Θ_2 . It is the heart of the computed genome sequence.
- $(\Theta_1^c * \Theta_2^c)$ is the other ORS from Θ_1 and Θ_2 , this one occurring **after the MCS**.
- Θ_2^a is the non-overlapping section of Θ_2 after the **COS**.

COS, $\overline{\Theta_1 \Theta_2}$, which is the **Consensus Overlapping Sequence**, padded on either side by the non-overlapping sections of either Θ_1 or Θ_2 , has the following properties:

- $\overline{\Theta_1 \Theta_2} = (\Theta_1^b * \Theta_2^b) \cdot (\Theta_1 \diamond \Theta_2) \cdot (\Theta_1^c * \Theta_2^c)$
- $\Theta_1 \approx \Theta_1^a \cdot \overline{\Theta_1 \Theta_2}$
- $\Theta_2 \approx \overline{\Theta_1 \Theta_2} \cdot \Theta_2^a$

So that we could rewrite **Equation 7.6** as such:

$$(\Theta_1 \diamond \Theta_2)^* = \Theta_1^a \cdot \overline{\Theta_1 \Theta_2} \cdot \Theta_2^a \quad (7.7)$$

Finally, we know that the cardinality of the genome sequence, $\mathcal{L}((\Theta_1 \diamond \Theta_2)^*)$, is bounded thus:

$$\mathcal{L}(\Theta_m) \leq \mathcal{L}((\Theta_1 \diamond \Theta_2)^*) \leq C_R - \mathcal{L}(\Theta_m) \quad (7.8)$$

As per **Equation 7.1** and the sensibilities of GTNC[2], and which inequality, in its most general form (applicable to a genome sequence derived from more than just two sequences) we might also depict as¹⁵:

$$\text{Max}(\mathcal{L}(\Theta_i)) \leq \mathcal{L}(\diamond(\Theta^n)) \leq \sum_{\forall \Theta_i \in \Theta^n} \mathcal{L}(\Theta_i) \quad (7.9)$$

¹⁵**NOTE:** we write $\diamond(\Theta^n)$ in **Equation 7.9** instead of $\Theta_1 \diamond \Theta_2$ to mean the MCS resulting from processing all (potentially more than just two) sequences in some collection or higher-order sequence Θ^n . And so that, even though it results from a higher-order sequence, $\diamond(\Theta^n)$, just like $\Theta_1 \diamond \Theta_2$, is a first-order (or rather, “flat”) sequence.

7.3 The Genome Sequencer

Definition 10 (A Genome Sequencer). A solution to **Problem 3**, is a special sequence processor, $\tilde{T}(\Theta^n)$, that can compute and return a resultant sequence, R_{Θ^n} , also known as the **genome sequence**, defined as^a

$$R_{\Theta^n} = (\Theta_m \diamond (\hat{\Theta}^n \setminus \Theta_m))^* \quad (7.10)$$

Such that Θ_m is the longest sequence in Θ^n — a collection of n sequences or $\hat{\Theta}^n$, its version with member sequences sorted in descending order of sequence cardinality, and that $\tilde{T}(\Theta^n)$ produces/generates/computes R_{Θ^n} by iteratively computing and updating the genome sequence in $n-1$ iterations over $\hat{\Theta}^n$ as such

$$\begin{aligned} \textbf{Transformation 18. } & R_{\hat{\Theta}^n}^0 \xrightarrow{O_{tGS}(R_{\hat{\Theta}^n}^0, \hat{\Theta}^{n-1}, 1)} R_{\hat{\Theta}^n}^1 \xrightarrow{O_{tGS}(R_{\hat{\Theta}^n}^1, \hat{\Theta}^{n-2}, 2)} R_{\hat{\Theta}^n}^2 \\ & \xrightarrow{O_{tGS}(R_{\hat{\Theta}^n}^2, \hat{\Theta}^{n-3}, 3)} R_{\hat{\Theta}^n}^3 \dots \xrightarrow{O_{tGS}(R_{\hat{\Theta}^n}^{j-1}, \hat{\Theta}^{n-j}, j)} R_{\hat{\Theta}^n}^{j+1} \dots \xrightarrow{O_{tGS}(R_{\hat{\Theta}^n}^{n-2}, \hat{\Theta}^1, n-1)} R_{\hat{\Theta}^n}^n \end{aligned}$$

To produce the final genome sequence, $R_{\Theta^n}^n$ via the **tGS Transformer** defined as such:

Transformer 9 (The tGS Transformer, $\tilde{T}(\Theta^n)$).

$$\begin{aligned} & R_{\hat{\Theta}^n}^j \xrightarrow{O_{tGS}(R_{\hat{\Theta}^n}^{j-1}, \hat{\Theta}^{n-j}, j)} R_{\hat{\Theta}^n}^{j+1}; \\ & R_{\hat{\Theta}^n}^0 = \Theta_m \quad \wedge \quad R_{\hat{\Theta}^n}^{j+1} = (R_{\hat{\Theta}^n}^j \diamond \hat{\Theta}^{n-j}[1])^* \quad \forall j \geq 1 \end{aligned}$$

□

and $R_{\hat{\Theta}^n}^{j+1}$ is the resultant genome sequence after processing $\hat{\Theta}^n$ with the first j sequences removed from it, and with the previous genome sequence, $R_{\hat{\Theta}^n}^j$ passed to it, as the current best candidate genome sequence so as to produce that next genome sequence $R_{\hat{\Theta}^n}^{j+1}$. Also, for a sequence collection of n member sequences, we write R_{Θ^n} to also mean $R_{\hat{\Theta}^n}^n$ as in **Equation 7.10**.

^aNote that in our formalism here we express the **sequence difference** of $\{\Theta_m\}$ from Θ^n as $\Theta^n \setminus \Theta_m$ — essentially using a set-theoretic notation even though we are actually dealing with sequences here. This, so as to keep the notation simple, but otherwise, later also appeal to the same concept when considering the remaining higher-order sequence after the first j member sequences have been processed and removed from it, with the notation Θ^{n-j} .

Thus, **Transformer 9**, $\tilde{T}(\Theta^n)$, is a genome sequencer, and is essentially a transformer that reduces a matrix of na-Sequences to a resultant sequence, R_{Θ^n} , that is the resultant genome sequence of a specie, population, or any natural entity¹⁶ represented by or expressed by the [potentially incomplete or approximate] genome sequences in the collection Θ^n .

It shall be worth noting that, for the special case where we merely have a collection of just two sequences from which to compute the genome — such as when $\Theta^n = \langle \Theta_1, \Theta_2 \rangle$ for some two [na-]sequences, Θ_1 and Θ_2 , we shall readily find that applying **Transformer 9** to that pair of sequences reduces to the case we have encountered in **Problem 3**, and that then, we find that:

$$R_{\Theta^n} = (\Theta_m \diamond (\hat{\Theta}^n \setminus \Theta_m))^* = (\Theta_1 \diamond \Theta_2)^* = \Theta_1^a \cdot \overline{\Theta_1 \Theta_2} \cdot \Theta_2^a \quad (7.11)$$

Finally, it shall help to put it out clearly, what our working definition of **Genome Sequencing** is, given what we now know.

¹⁶In principle though, it is not that we can not apply this theory to the computing of genome sequences for artificial or rather conceptual entities too.

Definition 11 (Genome Sequencing). When a collection of n -Sequences, Θ^n , is reduced to a single n -Sequence, Θ_{gs} , via a process that correctly aligns all the sequences in Θ^n such that the resultant Θ_{gs} is the best representative consensus sequence given all the variations in Θ^n , we call Θ_{gs} the **genome sequence** of Θ^n , and we know that if some other sequence, Ω_{na} is considered to be a legitimate genome sequence of Θ^n , then it must be equivalent to Θ_{gs} , where Θ_{gs} can be computationally produced from Θ^n , via **Transformer 9** as such:

Transformation 19. $\Theta^n \xrightarrow{\tilde{T}(\Theta^n)} \Theta_{gs}$

and equivalently:

$$\Theta_{gs} = \tilde{T}(\Theta^n) = R_{\Theta^n} \quad (7.12)$$

We call the process of reducing Θ^n to R_{Θ^n} , **genome sequencing**, and can use it to test or prove if indeed some sequence Ω_{na} is a legitimate genome sequence of Θ^n — essentially, testing if the two sequences belong to the same entity or perhaps species, by running the test:

$$0 \leq \tilde{A}(R_{\Theta^n} \rightarrow \Omega_{na}) \leq \epsilon \quad \vee \quad 0 \leq \tilde{A}(\Theta_{gs} \rightarrow \Omega_{na}) \leq \epsilon \quad \implies \quad \Omega_{na} \cong \Theta_{gs} \quad (7.13)$$

Otherwise $\Omega_{na} \neq \Theta_{gs}$, for $\epsilon \leq 1$.

7.4 The Identity Genome Sequence (IGS) Law

Law 3 (The Identity Genome Sequence Law). The Identity Genome Sequence, $R_{\Theta^n}(\Omega) : \mathbb{N} \times \psi_{DNA}$, is derived from some collection of sequences, $\Theta^n \langle \prod_{i=1}^n \theta_i \rangle$ such that $\theta_i \subset \Omega \vee \theta_i \approx \Omega$ are approximations of Ω , the longest known genome sequence of the entity, whose exact genome sequence would be Ω , and which can uniquely identify it. For sufficiently large n , and where the sample sequences θ_i were read or generated correctly by scanning or sequencing the entity, then

$$R_{\Theta^n}(\Omega) \approx \Omega_m \quad (7.14)$$

Where Ω_m is the known best approximation of the genome sequence that correctly identifies any instance of a member of the kind Ω .

NOTE:**On Consequences of IGS**

One potentially significant consequence of **Law 3** is that every distinct living thing — unique or distinct animal, plant, eukaryote or prokaryote, has a distinct identifying genome sequence, Ω , that we can discover and approximate via significantly many readings of particular or representative [sub-]sequences of the DNA of the observable expressions of Ω .

However, given that in reality it might not be exactly possible to exhaustively compute the correct value of R_{Θ^n} , nor tell exactly when that approximation approaches Ω exactly, it makes one wonder whether we can ever accurately know what it is that exactly underlies the expression of our reality — living things or not^a.

^aFirst, because, for currently known living things, the sheer complexity of correctly or accurately sequencing a genome for even the simplest organisms is constrained by the sheer amount of large resources required to correctly process millions to billions of nucleotides, and that typically, it is not possible to obtain an entire genome sequence in a single reading. And then, note that there is the queer possibility that a kind of genome sequencing might be applicable to also inanimate/non-living things too! But that's a philosophical discussion for another day.

Chapter 8

Random Sequence Generators (RSGs)

In many problems involving intelligent or non-linear processing of sequences, we might come across the need for either a random number, a random symbol or a random sequence of these, spanning some known/particular symbol set. This for example could be how we solve the problem: **Given the DNA symbol set ψ_{DNA} , how to generate a random DNA sequence of length n that conforms to some particular specie's genome or DNA-distribution?**¹.

So, to help solve such critical and fundamental problems once and for all, and in a generic, transformativ-way, we shall **specify some fundamental abstract machines** — entropy sources being first and core, especially for the derivation of or construction of stochastic generator machines and algorithms. These can then be used to solve any specific problem concerning the generation of a random sequence of a specific length given some finite symbol set, but also that we might leverage this in cases where the randomness needs be biased towards some specific distribution. And thus, we can apply such to the generation of random na-Sequences for arbitrary ends.

Abstract Sequence Generation:

The ultimate purpose of this chapter though, is to give us both a mathematical formalism, but also a computational and ideological one, underlying the notion of abstract sequence generation — especially where the purpose is to arrive at either truly random, or otherwise Gaussian sequences.

In the following sections then, starting with consideration of how to re-organize a room without taking anything out — essentially, shuffling sequences; we shall construct and formalise a theoretical and computation implementation of the

¹Solving such a problem might for example allow us the means or theory via which we might manifest random instances of a particular genome or species of interest using say the methods of synthetic biology, modern genetic engineering or *unnatural selection(?)*.

First Lu-Shuffle Algorithm², but also other **abstract computing systems** — sequence transformers all of them, that culminate in the specification of a **Random Sequence Generator, RSG**, as specified and implemented in **Section 8.3.7**.

8.1 The First Lu-Shuffle Algorithm (FLSA)

For this algorithm, for which we shall start by specifying its signature, the most important property it has, is that it merely takes a sequence of some symbols or elements, Θ , of length $n = \mathcal{L}(\Theta)$, as the input or data to be shuffled, and then just one other parameter, k — that is called the “**partitioning index**”, and that **should lie in the range** $1 \leq k < n$, and that $\mathcal{L}(\Theta) \geq 2$ — this last condition justified by the fact that it wouldn’t make any sense trying to shuffle a sequence of either one or no members using the partitioning method employed in the algorithm.

8.1.1 A Mathematical and Computational Specification of the FLSA Algorithm

Algorithm 3 (The **First Lu-Shuffle Algorithm**: $\text{flsa}(\Theta, k_0)$).

1. **GIVEN** source sequence Θ such that $n = \mathcal{L}(\Theta)$.
2. **GIVEN** input parameter k_0 that is **Initial Partitioning Index**.
3. **IF** $n < 2$ **RETURN** Θ unmodified.
4. **LIMIT** initial partitioning index using cardinality of input sequence, i.e

$$k = k_0 \mod n \quad (8.1)$$

5. **IF** $k < 1$ **UPDATE**: $k = 1$.
6. **INITIALIZE** the **resultant sequence**, Θ^* with Θ .

$$\Theta^* = \Theta_{\text{input}} = \Theta \quad (8.2)$$

7. **WHILE TRUE**:

- (a) **BREAK IF** $k = n$
- (b) **SET** partitioning index, $p_{\text{index}} = k$
- (c) **SET** partition start, $p_{\text{start}} = 0$
- (d) **SET** sub-partition end/index, $sp_{\text{index}} = \text{CEIL}(\frac{p_{\text{index}}}{2})$

²**FUTURE WORK:** It is very possible that other shuffling algorithms exist out there that would have served our purposes here, however, our own research and specific needs inspired original ideas and methodologies that underlie our own approach to the problem as laid out in most of this chapter. And so, both the FLSA and SLSA should be considered to be original inventions. However, it shall be relevant to benchmark these against earlier or alternative methods out there so as to ascertain whether or not they are better than, just as good or perhaps not as effective or as efficient as the state of the art. Moreover, and as one shall establish while reading or reviewing their development in this chapter, they are founded on sound principles, and actually do provably get the work done, and are demonstrably performant.

- (e) **SET** left sub-partition, $\Theta_l = \{\omega_i \mid \omega_i \in \Theta^* \wedge i \in [p_{start}, sp_{index}]\}$
- (f) **SET** right sub-partition, $\Theta_r = \{\omega_i \mid \omega_i \in \Theta^* \wedge i \in [sp_{index}, p_{index}]\}$
- (g) **SET** unpartitioned sequence, $\Theta_{rest} = \{\omega_i \mid \omega_i \in \Theta^* \wedge i \in [p_{index}, n]\}$
- (h) **SET** swapped sequence, $\Theta_{swap} = \Theta_r \cdot \Theta_l$
- (i) **IF** k is **ODD**: $(k \bmod 2) = 1$:
 - i. **UPDATE resultant sequence**, $\Theta^* = \Theta_{rest} \cdot \Theta_{swap}$
- (j) **ELSE IF** k is **EVEN**: $(k \bmod 2) = 0$:
 - i. **UPDATE resultant sequence**, $\Theta^* = \Theta_{swap} \cdot \Theta_{rest}$
- (k) **INCREMENT** k : $k = k + 1$

8. **RETURN** resultant sequence, Θ^* .

□

8.1.2 Analysis and Examples of Applying FLSA

Using the basic, yet important sequence, ψ_{10} (refer to **Equation 10.1**) as the input, especially given we leverage such in many aspects of applied transformatomics and genetics engineering in this work, we see hereafter, a trace of the accurate **invariant**³ variants (actually, computed and consistent anagrams) of that set — all of them **base-10 o-SSIs**[4], when it is shuffled using **Algorithm 3** and various **meaningful values**⁴ of the **initial partitioning index** k , starting from 1 and *preferably* up to $\frac{n}{2} = 5$ — in the illustrations hereafter, we have also included values up to $n - 1$, just to drive the point home, that the algorithm shall return a result with some **k-grams**⁵ of up to $k = \frac{n}{2}$ in the result for any $k > \frac{n}{2}$ — those large unshuffled subsequences might spoil the randomness of the resultant as you shall see when inspecting the examples we present.

³Basically, for a given particular input sequence such as ψ_{10} , we shall **always** get the same exact resultant sequence for a particular k , so that, it feels as though the algorithm randomizes the input sequence, but in a predictable and consistent manner, and this has its relevance in many problems as we shall come to appreciate.

⁴I say “meaningful” because, given how the algorithm works — shuffling the input sequence by partitioning it up into chunks and then swapping their order, the partitioning index, which determines where the swapping starts, **would not make much sense** for values greater than $\frac{\mathcal{U}(\Theta)}{2}$, and for values less than 1. Also, the algorithm doesn’t make sense for any value of k when $\mathcal{U}(\Theta) \leq 1$. Luckily, the algorithm as specified in **Algorithm 3** is designed so that it checks and applies the necessary remedies for meaningless values of k — such as setting $k = k \bmod n$ for $k > n$ and $k = 1$ for $k < 1$. It returns Θ unmodified for cases where $\mathcal{U}(\Theta) \leq 1$.

⁵The more common term is usually “n-grams”

```

GIVEN s=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
k=0 | s=[9, 7, 4, 0, 5, 3, 8, 2, 1, 6]
k=1 | s=[9, 7, 4, 0, 5, 3, 8, 2, 1, 6]
k=2 | s=[8, 6, 3, 9, 4, 2, 7, 1, 0, 5]
k=3 | s=[8, 6, 3, 9, 4, 2, 7, 0, 1, 5]
k=4 | s=[5, 3, 0, 6, 1, 7, 4, 8, 9, 2]
k=5 | s=[5, 1, 2, 6, 3, 7, 4, 8, 9, 0]
k=6 | s=[0, 8, 9, 1, 5, 2, 6, 3, 4, 7]
k=7 | s=[3, 8, 9, 4, 2, 5, 6, 0, 1, 7]
k=8 | s=[9, 1, 2, 3, 8, 4, 5, 6, 7, 0]
k=9 | s=[9, 5, 6, 7, 8, 0, 1, 2, 3, 4]

```

Figure 8.1: A Tabulation Trace of Shuffling the Base-10 n-SSI Sequence with increasing values of k , the **Initial Partitioning Index** for Algorithm 3, the **First Lu-Shuffle Algorithm**.

```

GIVEN s=[1, 2]
k=0 | s=[2, 1]
k=1 | s=[2, 1]

```

```

GIVEN s=[1, 2, 3]
k=0 | s=[3, 2, 1]
k=1 | s=[3, 2, 1]
k=2 | s=[2, 1, 3]

```

```

GIVEN s=[1, 2, 3, 4]
k=0 | s=[1, 4, 3, 2]
k=1 | s=[1, 4, 3, 2]
k=2 | s=[4, 3, 2, 1]
k=3 | s=[4, 3, 1, 2]

```

Figure 8.2: A Tabulation Trace of Shuffling basic base-case scenarios to help highlight how **FLSA** works.

However, it is not just ψ_{10} that shall help us appreciate this algorithm. **Figure 8.2** does help cast more light on how FLSA works, using other sequences,

particularly shorter than what we started out with, while, in the other examples hereafter, we consider not only longer, but also non-numerical, alphabetical, and specifically, genetic-code sequences too!

Moreover, note that, after looking at the examples in **Figure 8.1** and **Figure 8.2**, one might somewhat start to comfortably say that the **best range for the partitioning index k** , is possibly $1 \leq k \leq \frac{n}{4}$ — though, based on analysis of the algorithm itself, any values of k in the range $1 \leq k \leq \frac{n}{2}$ should also get us provably well shuffled resultant sequences.

And thus we come to the case of generating shuffled or randomized na-Sequences based on well-known, useful starter sequences (not necessarily symbol sets, or deduplicated sequences, but still useful ones nonetheless). For example, Θ_{2na} from **Equation 10.23** is the input sequence in our next illustration...

```

GIVEN s=['A', 'C', 'G', 'T', 'U', 'A', 'C', 'G', 'T', 'U']
k=0 | s=['U', 'G', 'U', 'A', 'A', 'T', 'T', 'G', 'C', 'C']
k=1 | s=['U', 'G', 'U', 'A', 'A', 'T', 'T', 'G', 'C', 'C']
k=2 | s=['T', 'C', 'T', 'U', 'U', 'G', 'G', 'C', 'A', 'A']
k=3 | s=['T', 'C', 'T', 'U', 'U', 'G', 'G', 'A', 'C', 'A']
k=4 | s=['A', 'T', 'A', 'C', 'C', 'G', 'U', 'T', 'U', 'G']
k=5 | s=['A', 'C', 'G', 'C', 'T', 'G', 'U', 'T', 'U', 'A']
k=6 | s=['A', 'T', 'U', 'C', 'A', 'G', 'C', 'T', 'U', 'G']
k=7 | s=['T', 'T', 'U', 'U', 'G', 'A', 'C', 'A', 'C', 'G']
k=8 | s=['U', 'C', 'G', 'T', 'T', 'U', 'A', 'C', 'G', 'A']
k=9 | s=['U', 'A', 'C', 'G', 'T', 'A', 'C', 'G', 'T', 'U']

```

Figure 8.3: A Tabulation Trace of Shuffling the na-Sequence Θ_{2na} from **Equation 10.23** with **Algorithm 3 (FLSA)**.

A closely related set of random na-Sequences would be that generated with **Algorithm 3** from the palindrome sequence Θ_{palna} from **Equation 10.24**:

```

GIVEN s=['A', 'C', 'G', 'T', 'U', 'U', 'T', 'G', 'C', 'A']
k=0 | s=['A', 'G', 'U', 'A', 'U', 'T', 'C', 'G', 'C', 'T']
k=1 | s=['A', 'G', 'U', 'A', 'U', 'T', 'C', 'G', 'C', 'T']
k=2 | s=['C', 'T', 'T', 'A', 'U', 'G', 'G', 'C', 'A', 'U']
k=3 | s=['C', 'T', 'T', 'A', 'U', 'G', 'G', 'A', 'C', 'U']
k=4 | s=['U', 'T', 'A', 'T', 'C', 'G', 'U', 'C', 'A', 'G']
k=5 | s=['U', 'C', 'G', 'T', 'T', 'G', 'U', 'C', 'A', 'A']
k=6 | s=['A', 'C', 'A', 'C', 'U', 'G', 'T', 'T', 'U', 'G']
k=7 | s=['T', 'C', 'A', 'U', 'G', 'U', 'T', 'A', 'C', 'G']
k=8 | s=['A', 'C', 'G', 'T', 'C', 'U', 'U', 'T', 'G', 'A']
k=9 | s=['A', 'U', 'T', 'G', 'C', 'A', 'C', 'G', 'T', 'U']

```

Figure 8.4: A Tabulation Trace of Shuffling the na-Sequence Θ_{palna} from **Equation 10.24** with **Algorithm 3 (FLSA)**.

Having looked at those examples and illustrations, finally note that, it is of course easiest to follow or analyze (by eye), what is going on when some initial/input sequence is being shuffled via the FLSA when the input sequence started out as a well ordered sequence in some natural or common ordering — such as when we are dealing with ψ_{10} , ψ_{az} or perhaps ψ_{36} . Especially note how easy it shall be to appreciate the algorithm when the analysis is conducted using naturally ordered initial sequences such as ψ_{10} that are also entirely numerical (see **Figure 8.1**).

But, all that aside, because the algorithm is generic and works on any sequence of symbols, it won't matter even if what is passed in as input is a sequence of letters in a typical language such as English; that might contain not just letters and numbers, but also symbols, punctuation and white-space! It shall still work, and the only critical factors that matter shall be that the input is a finite sequence, and that the value of the initial partitioning index is suitably set.

Hereafter, we shall shift attention to an implementation of **Algorithm 3** using actual executable computer programming source code so that then it is easier for anyone to reproduce these examples we have just looked at, but also that is becomes easy for analysts and engineers to adapt or profile the algorithm vis-a-vis what other/similar algorithms do.

8.1.3 The Source Code for FLSA

And the essential source code — compatible with **Python 3**, is as provided in **Listing 8.1**

Listing 8.1: The FLSA

```

1 #!/usr/bin/env python3
2
3  def flsa(s, k0):
4      n = len(s)
5      if n < 2:
6          return s,k0 #essentially, nothing to shuffle
7      # make k meaningful...
8      k = k0 % n
9      k = 1 if k < 1 else k # can't partition between nothing
10     new_s = s
11     while True:
12         if k == n: #can't partition between nothing
13             break
14         p_index = k
15         p_start = 0
16         sub_p_index = int(0.5 * p_index)
17         #print(f"At k={k} |"
18         #      +f" [{p_start}-{sub_p_index}][{sub_p_index}-"
19         #      +f"{p_index}][{p_index}-{n}]"")
20         sub_p_l = new_s[p_start:sub_p_index]
21         sub_p_r = new_s[sub_p_index:p_index]
22         rest_p = new_s[p_index:n]
23         swapped = sub_p_r + sub_p_l # first swap
24         if k%2 == 1: # k is odd
25             new_s = rest_p + swapped # second swap
26             #print(f"At k={k} s -->"
27             #      +f" [{p_index}-{n}][{sub_p_index}-"
28             #      +f"{p_index}][{p_start}-{sub_p_index}] == {new_s}")
29         else:
30             new_s = swapped + rest_p # soft swap
31             #print(f"At k={k} s --> [{sub_p_index}-"
32             #      +f"{p_index}][{p_start}-"
33             #      +f"{sub_p_index}][{p_index}-{n}] == {new_s}")
34         k += 1 # so we advance from k-gram to (k+1)-grams
35     return new_s,k
36
37 #---[ Uncomment Lines Below to Run Examples ]
38 #---[Some Further Experiments]
39 #s = [1,2,3] # a good base-case scenario..
40 #s = [0,1,2,3,4,5,6,7,8,9] # the base-10 n-SSI
41 #s = ['A','C','G','T','U','A','C','G','T','U'] # na-SS+na-SS
42 #s = ['A','C','G','T','U','U','T','G','C','A'] # na-SS+complement(na-SS)
43 #---[Generate Lu-Shuffle Look-Up Tables]
44 #k = len(s)
45 #s_k = 0
46 #print(f"GIVEN s={s}")
47 #while s_k < k:
48 #    lu_shuffle_a(s,s_k) # for FLSA
49 #    s_k += 1

```

Figure 8.5: The First Lu-Shuffle Algorithm implemented using Python

Concerning the source code sample for the **FLSA** as shown in **Listing 8.1**, realize that we have chosen to leave explanatory notes and debugging/tracing code (though commented out) within the sample, mostly to help newcomers learn to

read and appreciate the algorithm's implementation properly, but also that we help any nonbelievers or critics of the **FLSA** to try it out themselves and also look at what happens in the guts of the algorithm while it computes the final resultant. A clean and succinct version though, is also shared hereafter, in **Listing 8.2**. This shall help many to also be able to apply or use the algorithm by paper and pen because it is that simple, though unimaginably powerful.

Listing 8.2: The FLSA

```

1  #!/usr/bin/env python3
2
3  def flsa(s, k0):
4      n = len(s)
5      if n < 2:
6          return s, k0
7      k = k0 % n
8      k = 1 if k < 1 else k
9      new_s = s
10     while True:
11         if k == n:
12             break
13         p_index = k
14         p_start = 0
15         sub_p_index = int(0.5 * p_index)
16         sub_p_l = new_s[p_start:sub_p_index]
17         sub_p_r = new_s[sub_p_index:p_index]
18         rest_p = new_s[p_index:n]
19         swapped = sub_p_r + sub_p_l
20         if k%2 == 1:
21             new_s = rest_p + swapped
22         else:
23             new_s = swapped + rest_p
24         k += 1
25     return new_s, k

```

Figure 8.6: A clean version of the Python3 implementation of the FLSA

8.2 The Second Lu-Shuffle Algorithm (SLSA)

This second algorithm — the Second Lu-Shuffle Algorithm (SLSA), is a variation of and enhancement of **Algorithm 3** — the First Lu-Shuffle Algorithm. Also, and **IMPORTANT to NOTE**; this second algorithm combines with the first algorithm at each invocation, so that it takes the output of **FLSA** and further transforms it to produce the (*more randomized*) resultant sequence as we see in the following algorithm specification and illustrative examples that follow thereafter.

Moreover, just like the FLSA, the SLSA also has a simple signature that essentially accepts just a finite sequence, Θ , of length n , and a corresponding initial partitioning index parameter, k , that can take on values in the range $1 \leq k < n$.

It returns a resultant sequence of length n , Θ^* , that is essentially a well randomized version of the input sequence — essentially, a consistent anagram of Θ relative to a particular value of k .

8.2.1 The Mathematical and Computational Specification of the SLSA Algorithm

Algorithm 4 (The **Second Lu-Shuffle Algorithm**: $\text{slsa}(\Theta, k_0)$).

1. **GIVEN** source sequence Θ such that $n = \nu(\Theta)$.
2. **GIVEN** input parameter k_0 that is **Initial Partitioning Index**.
3. **IF** $n < 2$ **RETURN** Θ unmodified.
4. **LIMIT** initial partitioning index using cardinality of input sequence, i.e

$$k = k_0 \mod n \quad (8.3)$$

5. **IF** $k < 1$ **UPDATE**: $k = 1$.
6. **INITIALIZE** the shuffled sequence, Θ^* with the output of invoking **Algorithm 3** using the given parameters as such:

$$\Theta^*, k^* = \text{flsa}(\Theta, n - k) \quad (8.4)$$

IGNORE k^* , while Θ^* becomes the **initial shuffled resultant sequence** for the rest of this algorithm.

7. **WHILE TRUE**:

- (a) **BREAK IF** $k = n$
- (b) **SET** partitioning index, $p_{\text{index}} = k$
- (c) **SET** partition start, $p_{\text{start}} = 0$
- (d) **SET** sub-partition end/index, $sp_{\text{index}} = \text{CEIL}(\frac{p_{\text{index}}}{2})$
- (e) **SET** left sub-partition, $\Theta_l = \{\omega_i \mid \omega_i \in \Theta^* \wedge i \in [p_{\text{start}}, sp_{\text{index}}]\}$
- (f) **SET** right sub-partition, $\Theta_r = \{\omega_i \mid \omega_i \in \Theta^* \wedge i \in [sp_{\text{index}}, p_{\text{index}}]\}$
- (g) **SET** unpartitioned sequence, $\Theta_{\text{rest}} = \{\omega_i \mid \omega_i \in \Theta^* \wedge i \in [p_{\text{index}}, n]\}$
- (h) **SET** swapped sequence, $\Theta_{\text{swap}} = \Theta_r \cdot \Theta_l$
- (i) **IF** k is **ODD**: $(k \mod 2) = 1$:
 - i. **UPDATE resultant sequence**, $\Theta^* = \Theta_{\text{rest}} \cdot \Theta_{\text{swap}}$
- (j) **ELSE IF** k is **EVEN**: $(k \mod 2) = 0$:
 - i. **UPDATE resultant sequence**, $\Theta^* = \Theta_{\text{swap}} \cdot \Theta_{\text{rest}}$
- (k) **INCREMENT** k : $k = k + 1$

8. **RETURN** resultant sequence, Θ^* .

□

So, important to note that **SLSA** invokes **FLSA** with the **decreasing initial partitioning index** $n - k$ as shown in **Equation 8.4** as an alteration in how it initializes its resultant sequence, Θ^* — one of the key differences it has from the **FLSA**⁶.

8.2.2 Analysis and Examples of Applying SLSA

Using the same exact sequences, and in the same order as we utilized in **Section 8.1.2** when analyzing the **FLSA**, we shall come to appreciate readily, not just that the two algorithms, though they perform similar functions, produce **entirely different results for the same exact sequence and same exact partitioning index parameter**, but also that, there are some good reasons for why one might prefer one or the other.

IMPORTANT: The Consistency of FLSA and SLSA

Note that, though both the **FLSA** and **SLSA** randomize any sequence they are given in their respective results/outputs — essentially, that they produce anagrams of their inputs, and yet they are both **consistent** — the same input sequence with a particular k shall always produce the same exact expected shuffled sequence post-transform; for a particular value of the parameter k , both algorithms consistently produce a particular anagram for a given input sequence.

Note that **Figure 8.7** corresponds to the earlier table in **Figure 8.1**.

⁶That's actually all the difference between SLSA and FLSA! It is subtle, but makes all the difference as you shall establish in our analysis of SLSA.

```

GIVEN s=[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
k=0 | s=[4, 2, 8, 9, 0, 7, 3, 6, 5, 1]
k=1 | s=[4, 2, 8, 9, 0, 7, 3, 6, 5, 1]
k=2 | s=[7, 5, 3, 0, 8, 2, 6, 1, 9, 4]
k=3 | s=[1, 6, 4, 7, 2, 9, 0, 3, 8, 5]
k=4 | s=[2, 1, 0, 6, 8, 3, 5, 4, 7, 9]
k=5 | s=[7, 1, 2, 4, 6, 8, 3, 9, 0, 5]
k=6 | s=[5, 9, 2, 3, 7, 0, 4, 6, 1, 8]
k=7 | s=[9, 1, 5, 4, 3, 2, 7, 8, 6, 0]
k=8 | s=[5, 6, 3, 9, 0, 4, 2, 7, 1, 8]
k=9 | s=[6, 3, 8, 2, 1, 9, 7, 4, 0, 5]

```

Figure 8.7: A Tabulation Trace of Shuffling the Base-10 n-SSI Sequence with increasing values of k , using **Algorithm 4 (SLSA)**, the **Second Lu-Shuffle Algorithm**.

```

GIVEN s=[1, 2]
k=0 | s=[1, 2]
k=1 | s=[1, 2]

```

```

GIVEN s=[1, 2, 3]
k=0 | s=[3, 1, 2]
k=1 | s=[3, 1, 2]
k=2 | s=[2, 3, 1]

```

```

GIVEN s=[1, 2, 3, 4]
k=0 | s=[4, 2, 1, 3]
k=1 | s=[4, 2, 1, 3]
k=2 | s=[1, 2, 3, 4]
k=3 | s=[2, 3, 1, 4]

```

Figure 8.8: A Tabulation Trace of Shuffling basic base-case scenarios to help highlight how **SLSA** works and how it relates to **FLSA**.

Note that **Figure 8.9** corresponds to the earlier table in **Figure 8.3**.

```

GIVEN s=['A', 'C', 'G', 'T', 'U', 'A', 'C', 'G', 'T', 'U']
k=0 | s=['U', 'G', 'T', 'U', 'A', 'G', 'T', 'C', 'A', 'C']
k=1 | s=['U', 'G', 'T', 'U', 'A', 'G', 'T', 'C', 'A', 'C']
k=2 | s=['G', 'A', 'T', 'A', 'T', 'G', 'C', 'C', 'U', 'U']
k=3 | s=['C', 'C', 'U', 'G', 'G', 'U', 'A', 'T', 'T', 'A']
k=4 | s=['G', 'C', 'A', 'C', 'T', 'T', 'A', 'U', 'G', 'U']
k=5 | s=['G', 'C', 'G', 'U', 'C', 'T', 'T', 'U', 'A', 'A']
k=6 | s=['A', 'U', 'G', 'T', 'G', 'A', 'U', 'C', 'C', 'T']
k=7 | s=['U', 'C', 'A', 'U', 'T', 'G', 'G', 'T', 'C', 'A']
k=8 | s=['A', 'C', 'T', 'U', 'A', 'U', 'G', 'G', 'C', 'T']
k=9 | s=['C', 'T', 'T', 'G', 'C', 'U', 'G', 'U', 'A', 'A']

```

Figure 8.9: A Tabulation Trace of Shuffling the na-Sequence Θ_{2na} from **Equation 10.23** with **Algorithm 4 (SLSA)**.

Note that **Figure 8.10** corresponds to the earlier table in **Figure 8.4**.

```

GIVEN s=['A', 'C', 'G', 'T', 'U', 'U', 'T', 'G', 'C', 'A']
k=0 | s=['U', 'G', 'C', 'A', 'A', 'G', 'T', 'T', 'U', 'C']
k=1 | s=['U', 'G', 'C', 'A', 'A', 'G', 'T', 'T', 'U', 'C']
k=2 | s=['G', 'U', 'T', 'A', 'C', 'G', 'T', 'C', 'A', 'U']
k=3 | s=['C', 'T', 'U', 'G', 'G', 'A', 'A', 'T', 'C', 'U']
k=4 | s=['G', 'C', 'A', 'T', 'C', 'T', 'U', 'U', 'G', 'A']
k=5 | s=['G', 'C', 'G', 'U', 'T', 'C', 'T', 'A', 'A', 'U']
k=6 | s=['U', 'A', 'G', 'T', 'G', 'A', 'U', 'T', 'C', 'C']
k=7 | s=['A', 'C', 'U', 'U', 'T', 'G', 'G', 'C', 'T', 'A']
k=8 | s=['U', 'T', 'T', 'A', 'A', 'U', 'G', 'G', 'C', 'C']
k=9 | s=['T', 'T', 'C', 'G', 'C', 'A', 'G', 'U', 'A', 'U']

```

Figure 8.10: A Tabulation Trace of Shuffling the na-Sequence Θ_{palna} from **Equation 10.24** with **Algorithm 4 (SLSA)**.

And thus, one shall observe that unlike the FLSA that had some noticeable convergence for values of $k > \frac{n}{2}$, for SLSA, all values of $1 \leq k < n$ produce appreciably well randomized resultant sequences with the exception being cases having very small values of n ; essentially, one notices that FLSA and SLSA might perform similarly well for small sequences and similar values of k , however, **the larger the size of the sequence, the better the results that one obtains for any value of k when using the SLSA than not.**

8.2.3 The Source Code for SLSA

And the essential source code — compatible with **Python 3**, for the SLSA is as provided in **Listing 8.3**

Listing 8.3: The SLSA

```

1  #!/usr/bin/env python3
2
3  #SLSA
4  def slsa(s, k0):
5      n = len(s)
6      if n < 2:
7          return s,k0
8      k = k0 % n
9      k = 1 if k < 1 else k
10     new_s = flsa(s,n-k)[0] #invoke FLSA with k0=n-k
11     while True:
12         if k == n:
13             break
14         p_index = k
15         p_start = 0
16         sub_p_index = int(0.5 * p_index)
17         sub_p_l = new_s[p_start:sub_p_index]
18         sub_p_r = new_s[sub_p_index:p_index]
19         rest_p = new_s[p_index:n]
20         swapped = sub_p_r + sub_p_l
21         if k%2 == 1:
22             new_s = rest_p + swapped
23         else:
24             new_s = swapped + rest_p
25         k += 1
26     return new_s,k
27
28 #FLSA
29 def flsa(s, k0):
30     n = len(s)
31     if n < 2:
32         return s,k0
33     k = k0 % n
34     k = 1 if k < 1 else k
35     new_s = s
36     while True:
37         if k == n:
38             break
39         p_index = k
40         p_start = 0
41         sub_p_index = int(0.5 * p_index)
42         sub_p_l = new_s[p_start:sub_p_index]
43         sub_p_r = new_s[sub_p_index:p_index]
44         rest_p = new_s[p_index:n]
45         swapped = sub_p_r + sub_p_l
46         if k%2 == 1:
47             new_s = rest_p + swapped
48         else:
49             new_s = swapped + rest_p
50         k += 1
51     return new_s,k
52
53
54 #TEST:
55 #s,k = [0,1,2,3,4,5,6,7,8,9],8
56 #rs=slsa(s,k)
57 #print(f"For s={s} and k={k}:\ns*={rs}") # show test results

```

Figure 8.11: The Second Lu-Shuffle Algorithm implemented using Python

8.3 Generating Realistic Random na-Sequences

In the previous sections we have considered the matter of taking a particular sequence (of any symbols or elements, including symbol sets) and then worked on ways to generate random sequences, particularly their anagrams, based off of these. Thus, focus was specifically on randomizing an existing sequence, and not necessarily about generating entirely new sequences with say new symbol distributions different from what was in the source/input sequence. In this section though, we wish to instead turn our attention towards the other related problem — generating a previously non-existent random sequence of arbitrary length, based off of just a small initial “seed” sequence such as just a basic symbol set. We shall especially want to treat of the matter of how to generate such sequences, when they are DNA, RNA or otherwise na-Sequences.

Thus, starting from the fundamental case of **generating a SINGLE RANDOM DNA symbol** — such as randomly generating one of the DNA-nucleobases from ψ_{DNA} or doing the same for RNA, shall be where our adventures start.

8.3.1 How to Generate a Random Nucleotide & The Random Symbol Generator

In this section, we are going to tackle two related problems — the generation of a random symbol (such as just one element from some symbol set or sequence), and then the case of how to generate a random sequence given some starter or seed sequence. So, basically, we are interested in implementing a **Random Symbol Generator**(RSG), but also which would very likely underlie the implementation of a **Random Sequence Generator** (RSG), and both of which are problems or concepts related to the popular matter of a **Random Number Generator** (RNG). Especially though, we are interested in developing a mechanism via which we can generate random na-Sequences such as believable random DNA sequences or genetic code.

So, first of all, given that we already have the necessary source symbol sets well defined⁷, the matter of how to generate a single **random letter** from a particular symbol set (also sometimes referred to as “alphabet”) shall be as follows; first, we shall merely define an abstract machine using the notation and semantics of transformatics.

⁷Refer to **Chapter 4** for the case of DNA and RNA.

Transformer 10 (The Random Sequence Generator (RSG), $RSG(k, \Theta_n)$).

$$\Theta_n \xrightarrow{O_{RSG}(k, \Theta_n)} \Theta_k^*;$$

$$\begin{aligned} \varprojlim(\Theta_n) = n \quad \wedge \quad \varprojlim(\Theta_k^*) = k \quad \wedge \quad \psi(\Theta_k^*) \subseteq \psi(\Theta) \\ \wedge \quad \forall x, y \in \mathbb{N}, \quad \Omega : \mathbb{N} \times \psi_y \quad \text{such that:} \end{aligned}$$

$$RSG(x, \Omega) = SELECT(x, SHUFFLE(PROTRACT(\psi_y, k \times 2 \times \varprojlim(\psi_y)))) \quad (8.5)$$

Where;

- $\psi_y = USYMBOLSET(\Omega)$
but in a more robust case, we can also randomize the symbol set to be used in generating the random sequence, by generating ψ_y thus:

$$\psi_y = SHUFFLE(USYMBOLSET(\Omega)) \quad (8.6)$$

- **USYMBOLSET**(Θ) is a function that reduces the input argument Θ , which is expected to be a sequence of some symbols — unique or not, to its **unspecific symbol set** — $\hat{\psi}(\Theta)^a$.
- **PROTRACT**(Θ, α) is a sequence transformer^b that operates on a source sequence of [possibly] distinct n symbols, Θ , such that the resultant sequence is of length α and spans the source sequence symbol set.
- **SHUFFLE**(Θ) is a function such as a generalization of **slsa**($\cdot$) from **Algorithm 4** that returns a shuffled/randomized instance of any input sequence such as Θ that it is given.
- **SELECT**(x, Θ) is a function that can return the first x terms from Θ as the result of its invocation for some $1 \leq x \leq \varprojlim(\Theta)$.

□

^aSee **Definition 4** in [4]

^bNote that, just like the first parameter could be any sequence of any symbols, also, the second parameter could be any positive number, and further that, the function shall operate on the input sequence so as to **either shrink/truncate it** — for $\alpha < \varprojlim(\Theta)$ — or **grow/multiply it** — for $\alpha > \varprojlim(\Theta)$ — relative to the specified **multiplier** size, α .

So, with the essential mathematics out of the way as depicted in **Transformer**

10, we can then try to see how this would work practically, with some real tangible computer programs that implement that higher-order⁸ generator-transformer specification.

8.3.2 The USYMBOLSET Function

We shall go bit by bit. So, first, here is what the **USYMBOLSET** function looks like in Python:

Listing 8.4: The USYMBOLSET

```

1  #!/usr/bin/env python3
2
3  #---[ USYMBOLSET(s) ]
4  def usymbolset(s):
5      n_s = len(s)
6      if n_s <= 1:
7          return s
8
9      # the next commented-out code would kinda work,
10     # if we just wanted a dirty and pythonic solution, but
11     # it doesn't respect order of first occurrence!
12     #resultant = list(set(s))
13
14     #our CORRECT solution for unspecific symbol set..
15     resultant = []
16     for i in range(n_s):
17         s_i = s[i]
18         if not (s_i in resultant):
19             resultant.append(s_i)
20     return resultant
21
22 #---[ TEST USYMBOLSET ]
23 #s = [0,1,2,3,4,5,6,7,8,9]
24 # s = [0,1,2,3,4,5,6,7,8,9] --> usymbolset(s+s+s+s)
25 #--> [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
26 #s = [1,0,1,1,1,1,1,0,1,1] --> usymbolset(s) --> [1,0]
27 #s = "3.1415926535898" # --> usymbolset(s) -->
28 #['3', '.', '1', '4', '5', '9', '2', '6', '8']
29 #s = ['A', 'C', 'G', 'T', 'U', 'U', 'T', 'G', 'C', 'A'] #w/
    duplication
30 # usymbolset(s) --> ['A', 'C', 'G', 'T', 'U']
31 #result = usymbolset(s)
32 #print(result)

```

Figure 8.12: The **USYMBOLSET**(Ω) unspecific symbol set generator implemented using Python

As one can tell from the commented-out test-cases for that **USYMBOLSET**(Ω) implementation, indeed, our code conforms to what we expect of the unspecific symbol set — refer to **Question 4** of [41].

⁸The **RSG** is indeed a great example of a higher-order transformer, because, as you can tell from the specification of the underlying **RSG**() function in **Equation 8.5**, its resultant sequence is derived from the chaining of several other sequence transformers in a manner similar to what is formalized in **Section D.2.9**.

8.3.3 The PROTRACT Function

And here is what the **PROTRACT** function looks like in Python:

Listing 8.5: The PROTRACT

```

1  #!/usr/bin/env python3
2
3  #---[ PROTRACT(s,n) ]
4  def protract(s,n):
5      n_s, n = len(s), int(n)
6      if n_s < 1:
7          return s
8      if n < n_s:
9          return s[:n]
10     if n == n_s:
11         return s
12     multiple = int(n / n_s)
13     target_n = n_s * multiple + 1
14     resultant = s
15     i = 1
16     while i < target_n:
17         resultant = resultant + s
18         i += 1
19     return resultant[:n] # could potentially, also compress
20
21 #---[ TEST PROTRACT ]
22 # s = [0,1,2,3,4,5,6,7,8,9], n = 3 --> [0,1,2]
23 # s = [0,1,2,3,4,5,6,7,8,9], n = 23 -->
24 # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2]
25 #result = protract(s,23)
26 #print(result)

```

Figure 8.13: The **PROTRACT**(Θ, n) sequence generator-transformer implemented using Python

Thus, we see that the **PROTRACT**(Θ, n) function, when given a value of n less than the size of the input sequence, shall merely truncate the sequence to the specified size. While, if the value of n is larger than the size of the input sequence, it shall produce a resultant that somewhat contains the input sequence concatenated to itself multiple times, but such that the final resultant is again truncated to the exact specified and required output length, n .

8.3.4 The Random Number Generator, RNG Function

Before we proceed to the **SHUFFLE** function, we shall want to first specify and test an important utility that would otherwise make our implementation dependent on someone else's random sequence generator — thus rendering all our efforts here almost useless!

The critically important function is **RNG**(ll, ul) that essentially takes two parameters — ll , the **lower limit** and ul , the **upper limit**, both of which are expected to be integers, and using which, the function is supposed to generate a

random number n that lies within the range:

$$ll \leq n \leq ul \quad (8.7)$$

It is such a **critically important routine** (see author's earlier work on the matter[1]), however, for the interests of keeping things simple, for now, we shall settle with a basic implementation that only relies on the **System Time** functions as our **source of entropy**⁹.

So, without delving into formally specifying our **Random Number Generator**, **RNG**, here, the associated definition of this utility as implemented using Python is as follows:

Listing 8.6: The RNG

```

1  #!/usr/bin/env python3
2  import time
3
4  #---[ RNG(ll,ul) ]
5  def rng(l,u):
6      lu = max(l,u)
7      ll = min(l,u)
8      lu = lu + 1 if ll == lu else lu
9      part_k = int((lu - ll) * 0.5)
10     candidates = flsa(list(range(ll,lu+1)),part_k)[0] #invokes FLISA
11     n = len(candidates)
12     pick = int(flsa(str(time.time()).replace('.', ''),1)[0])%n #because we need
        an entropy source!
13     return candidates[pick]
14
15 #---[ TEST RNG ]
16 #for r in [[0,10],[0,1],[0,100],[1900,1999]]:
17 #    print(f"RNG From {r[0]}-{r[1]} -> {random_ng(r[0],r[1])}")

```

Figure 8.14: The **RNG**(ll, ul) random number generator implemented using Python

In the following demonstration, we can indeed ascertain, via a review of various illustrative invocations of our program — such a simple and beautiful RNG! — that indeed, it does get the job perfectly done and is such an indispensable little gem!

⁹**FUTURE WORK:** While, and perhaps in the near future, we continue to seek for theory and practical ways to generate entropy algorithmically — which *might be* futile, we shall also perhaps need to implement our own entropy generators interfacing with entropy sources in nature for more naturally stable, trustworthy sources of randomness.

Listing 8.7: A TESTING Our RANDOM NUMBER GENERATOR

```

RNG From 0-10 -> 6
RNG From 0-1 -> 0
RNG From 0-100 -> 56
RNG From 1900-1999 -> 1984

RNG From 0-10 -> 7
RNG From 0-1 -> 1
RNG From 0-100 -> 46
RNG From 1900-1999 -> 1950

RNG From 0-10 -> 6
RNG From 0-1 -> 0
RNG From 0-100 -> 11
RNG From 1900-1999 -> 1995

```

Figure 8.15: A text dump from testing our RNG(*ll*, *ul*) that only utilizes System Time as entropy source, and uses our First Lu-Shuffle Algorithm to generate True Random Numbers

Before we proceed, note that, just like with all our earlier programs, the RNG implementation we just finished testing hardly relies on any external libraries or functions except the `time.time()` utility we needed for providing an entropy source¹⁰, and then our own `flsa()` that we use to randomize the items in the specified range that we then pick from. As for which shuffling algorithm to use, of course, we could have used the **SLSA**, however, so as to keep the implementation very efficient, we chose to use the **FLSA** since we pass in a cleverly chosen partitioning index that is computed such that it lies somewhat around $\frac{n}{2}$ when we derive it from the two user-specified RNG parameters as such:

$$k = \frac{ul - ll}{2} \quad (8.8)$$

And as per our analysis of FLSA and SLSA, we know that with such a value of the initial partitioning index, then FLSA shall get us just the right kind of anagram as might be obtained using SLSA¹¹. Refer to **Listing 8.6**, but also

¹⁰**IMPORTANT FUTURE WORK:** Realize when studying our use of the `time.time()` function as an entropy source that we couldn't just merely use the value it returns without any extra transforms on it — which ideally is the current time in seconds and milliseconds since epoch — because, when the `RNG()` function is then invoked multiple times within the same second, it is very likely that the result of using this entropy source would then stay the same (as our own tests did prove with earlier implementations of this function — such as merely using `pick = int(time.time())%n` that only *sluggishly* changes — with almost no changes in the order and relative distribution of some digits (in the seconds part of the underlying time sequence) for hours, days or even months!). Thus we had to tweak the output of that function further as you can see with the extra massaging of its output via `pick = int(flsa(str(time.time()).replace('.', ''), 1)[0])%n`, so that then, even when the value the time function returns doesn't change [significantly] across invocations, we are sure that the value we pick shall still be well randomized since we apply the FLSA shuffle algorithm to the output of the time function nonetheless! The other alternative would be to not even bother with involving any mangling of the time value, such as by anagrammatizing it before use, but instead by merely doing away with the bit that mostly stays stable or stagnant across multiple invocations on the millisecond spectrum — essentially, leveraging an approach such as `pick = int(str(time.time()).split('.')[1])%n`. However, even that approach, despite not involving complex procedures such as the shuffler `flsa()`, and yet, if the invocation resolution window is small enough even on the millisecond level, it shall take us back to the same original problem we set out to avoid! And so, we should settle with the chosen Lu-Shuffle work-around for now. That hack is very crucial!

¹¹So, it happens that, when we have little or no control over the initial partitioning index parameter, *k*, that is required for the two shuffling algorithms FLSA and SLSA, that then, for optimal guarantee that we shall always obtain fine sequence anagrams, then the SLSA is preferable. But where we know what we are doing, such as when we can dynamically compute the *k*, we might as well just default to the FLSA as demonstrated here.

Section 8.1.2 and Section 8.2.2 for details.

8.3.5 The SHUFFLE Function

And here is what the **SHUFFLE** function looks like in Python:

Listing 8.8: The SHUFFLE

```

1  #!/usr/bin/env python3
2
3  #---[ SHUFFLE(s) ]
4  def shuffle(s):
5      n_s = len(s)
6      if n_s < 2:
7          return s
8      random_k = rng(1,n_s)
9      shuffled_s,k = slsa(s,random_k) #invokes SLSA
10     random_k2 = rng(1,int(n_s*0.5))
11     shuffled_s,k = flsa(shuffled_s,random_k2) #invokes FLSA
12     return shuffled_s
13
14 #---[ TEST SHUFFLE ]
15 #s = [0,1,2,3,4,5,6,7,8,9]
16 # s = [0,1,2,3,4,5,6,7,8,9] --> [9, 3, 4, 7, 0, 5, 2, 1, 6, 8],
17 # [4, 6, 0, 3, 5, 2, 7, 9, 1, 8], etc.
18 #s = ['A' , 'C' , 'G' , 'T' , 'U']
19 #--> ['C', 'G', 'T', 'U', 'A'],
20 # ['U', 'G', 'T', 'C', 'A'], etc.
21 #result = shuffle(s)
22 #print(result)

```

Figure 8.16: The **SHUFFLE**(Θ) sequence randomizer implemented using Python

Definitely, just like we didn't *originally* want to implement our own basic random number generator (see **Line#8** and **Line#10** in **Listing 8.8**), we could also have used an external shuffle algorithm — such as with the standard python random utility in `s=[1,2,3];rng.shuffle(s);s` that might return `[3,1,2]` for example, but we chose to use our own shuffle algorithms as developed in the previous sections (see **Line#9** and **Line#11** that invoke , first, the **SLSA**, and then the **FLSA**, `slsa(·)`, defined in **Listing 8.3**), so those who need to cut dependencies might pick a leaf from our solution, or perhaps port our algorithms to any programming language as is.

Also, unlike invoking `slsa( $\Theta$ ,k)` directly, the provided sequence randomizer utility function takes care of generating random parameters for the **Lu Shuffle Algorithm**, by randomly picking legitimate values of k from 1 to $\mathcal{U}(\Theta)$, so that the **SHUFFLE** function can be invoked without having to explicitly pass any arguments other than just the sequence one wishes to have shuffled.

Also, note that **Listing 8.8**, like the ones we have looked at before, provides some (commented out) code that might give one an idea how to use the function,

or how the potential results of invoking it with certain sequences might look like. Otherwise, the function itself is very minimal, with the essential code spanning just about 9 lines of code!

8.3.6 The **SELECT** Function

And as for the **SELECT** function required by **Transformer 10** (the **RSG**), the corresponding definition for that function in Python is as shown:

Listing 8.9: The **SELECT**

```

1  #!/usr/bin/env python3
2
3  #---[ SELECT(n,s) ]
4  def select(n,s):
5      n_s, n = len(s), int(n)
6      resultant = []
7      if n <= 0:
8          return resultant
9      n = n if n <= n_s else n_s
10     resultant = s[:n]
11     return resultant
12
13 #---[ TEST SELECT ]
14 #s = [0,1,2,3,4,5,6,7,8,9]
15 #s = [0,1,2,3,4,5,6,7,8,9] --> select(2,s) --> [0,1],
16 # select(12,s) --> [0, 1, 2, 3, 4, 5, 6, 7, 8, 9], etc.
17 #s = ['A', 'C', 'G', 'T', 'U']
18 #select(3,s) --> ['A', 'C', 'G'], etc.
19 #result = select(12,s)
20 #print(result)

```

Figure 8.17: The **SELECT**(n, Θ) sequence sampler function implemented using Python

Thus, the `select( $n, s$ )` function shall return the first n elements from the specified sequence s , and when the specified selection criteria n lies outside the cardinality of the sequence; such as when $n \leq 0$, it returns the empty sequence. Otherwise, when $n > \nu(s)$, it returns the entire sequence as is.

8.3.7 The **RSG** Function

Having seen how to implement the individual essential components that make the **Random Sequence Generator (RSG)**, $RSG(k, \Theta_n)$, defined in **Transformer 10** work, we can then go ahead and actually implement a re-usable, and generic random sequence generator or **even a random symbol generator** based off of that.

As with the sample codes offered in earlier sections of this chapter, we shall merely default to implementing a Python-compliant function, but the semantics

and code could of course be adapted for other programming languages by interested engineers, students and researchers.

Listing 8.10: The RSG

```

1  #!/usr/bin/env python3
2
3  # import/add earlier utilities as necessary...
4  # of course: flsa(s,k), slsa(s,k) and rng(ll,ul) first
5  # then usymbolset(s), protract(s,n), shuffle(s) and select(n,s)
6
7  #---[ RSG(n,s) ]
8  def rsg(n,s):
9      n_s,n = len(s),int(n)
10     resultant = []
11     if n == 0 or n_s == 0:
12         return resultant
13     # first, compute the u-symbol set
14     s_usymbolset = usymbolset(s)
15     n_us = len(s_usymbolset)
16     # then chain the sequence transformers...
17     resultant = shuffle(s_usymbolset) # randomize the symbol set
18     resultant = protract(resultant,n*2*n_us)
19     resultant = shuffle(resultant)
20     # then pick only as much as was asked for...
21     resultant = select(n,resultant)
22     return resultant

```

Figure 8.18: The **RSG**(n, Θ) random sequence generator function implemented using Python

Interesting to note, we could also take a program such as we have in **Listing 8.10**, and somewhat express it (albeit in summary or abstract form), using the semantics and notation of transformatics. Normally, it would be the case that the mathematical specification must or would precede the procedural implementation, however, nothing should stop us from working in reverse too! And so, the essential higher-order RSG transformer as thus implemented can be expressed mathematically as such:

Transformer 11 (The **RSG**(Θ, k)). $\langle \Theta, k \rangle \xrightarrow{RSG(\Theta, k)} \Omega$;

$\Omega = \langle \rangle$ iff $\perp(\Theta) = 0 \vee k = 0$

Otherwise, Ω is derived via the following transformer chain:

Transformation 20. $\langle \Theta, k \rangle \xrightarrow{O_{usymbolset(\Theta),k}} \langle \Theta_1^*, k \rangle \xrightarrow{O_{shuffle(\Theta_1^*),k}} \langle \Theta_2^*, k \rangle \xrightarrow{O_{protract(\Theta_2^*, k),k}} \langle \Theta_3^*, k \rangle \xrightarrow{O_{shuffle(\Theta_3^*),k}} \langle \Theta_4^*, k \rangle \xrightarrow{O_{select(k, \Theta_4^*)}} \Theta_5^* = \Omega$

$$\wedge \quad \forall \omega \in \Omega \quad \exists \omega \in \Theta \quad \wedge \quad \psi(\Omega) \subset \psi(\Theta) \quad \wedge \quad \perp(\Omega) = k \quad \square$$

Of course, the careful reader or observer should have already noticed that

we now have two transformatics specifications of the **RSG** transformer; **Transformer 10** and **Transformer 11**. Moreover, both exactly say the same thing concerning this transformer, however, they say it differently and that's among the beauties or peculiarities of the mathematics of transformatics!

For example, the earlier transformer specification was more verbose (almost filling an entire page) and was expressed mostly using natural language/English. However, this final, later version is suitably succinct and terse, and should help demonstrate how such formalisms leveraging transformatics might be evolved over time to become clearer, more general, less ambiguous, terse and precise!

But yet again, neither version is less useful — for example, where **Transformer 11** sacrifices verbosity for generality and terseness, it might leave some people without any understanding for how the actual transformation might be implemented or what the internal transforms depicted in **Transformation 20** might be interpreted as. While, **Transformer 10** does offer sufficient details about how the internals of the transformer work or how they might be realized — almost so that it most closely reflects what we find depicted in the **RSG** implementation as we see it expressed using Python in the following section (where the RSG is implemented in its complete form). Thus, depending on context, audience and needs, sometimes a transformer might be written one way and not the other, as long as we don't leave much doubt about what it is we are specifying or expressing using the mathematics and language of transformatics — especially when expressing or applying sequence transformers.

Finally, this should be yet another great example in this book, and treatment of transformatics, for how, the mathematics and notions of transformatics are very handy when it comes to abstracting several otherwise intricate or complex matters in domains such as computing and general mathematics.

The Complete RSG Implementation

Note that **Listing 8.10** focused on just the $\text{RSG}(n,s)$ function, which is indeed the basic implementation of our sequence generator at the highest level. However, if we were to have to fill-in all the gaps — such as having an implementation that can work with all referenced functions in a single file, then the following code listing would be what we instead need:

The RSG (complete implementation)

Listing 8.11: Complete RSG

```

1  #!/usr/bin/env python3
2  #####
3  # RSG: Random Sequence Generator
4  #-----
5  # A complete standalone reference
6  # implementation in Python 3.
7  # COPYRIGHT: Fut. Prof. JWL
8  # FEEDBACK: jwl@nuchwezi.com
9  # Nuchwezi Research (nuchwezi.com)
10 #####
11
12 import time
13
14 ##---[ FLsa(s,k) ]
15 def flsa(s, k0):
16     n = len(s)
17     if n < 2:
18         return s,k0
19     k = k0 % n
20     k = 1 if k < 1 else k
21     new_s = s
22     while True:
23         if k == n:
24             break
25         p_index = k
26         p_start = 0
27         sub_p_index = int(0.5 * p_index)
28         sub_p_l = new_s[p_start:sub_p_index]
29         sub_p_r = new_s[sub_p_index:p_index]
30         rest_p = new_s[p_index:n]
31         swapped = sub_p_r + sub_p_l
32         if k%2 == 1:
33             new_s = rest_p + swapped
34         else:
35             new_s = swapped + rest_p
36         k += 1
37     return new_s,k
38
39 #---[ SLsa(s,k) ]
40 def slsa(s, k0):
41     n = len(s)
42     if n < 2:
43         return s,k0
44     k = k0 % n
45     k = 1 if k < 1 else k

```

```

46     new_s = flsa(s,n-k)[0] #invoke FLSA with k0=n-k
47     while True:
48         if k == n:
49             break
50         p_index = k
51         p_start = 0
52         sub_p_index = int(0.5 * p_index)
53         sub_p_l = new_s[p_start:sub_p_index]
54         sub_p_r = new_s[sub_p_index:p_index]
55         rest_p = new_s[p_index:n]
56         swapped = sub_p_r + sub_p_l
57         if k%2 == 1:
58             new_s = rest_p + swapped
59         else:
60             new_s = swapped + rest_p
61         k += 1
62     return new_s,k
63
64 #---[ RNG(l1,u1) ]
65 def rng(l,u):
66     lu = max(l,u)
67     ll = min(l,u)
68     lu = lu + 1 if ll == lu else lu
69     part_k = int((lu - ll) * 0.5)
70     candidates = flsa(list(range(ll,lu+1)),part_k)[0] #invokes FLSA
71     n = len(candidates)
72     pick = int(flsa(str(time.time()).replace('.', ''),1)[0])%n #time-entropy
73     return candidates[pick]
74
75 #---[ SHUFFLE(s) ]
76 def shuffle(s):
77     n_s = len(s)
78     if n_s < 2:
79         return s
80     random_k = rng(1,n_s)
81     shuffled_s,k = slsa(s,random_k)
82     random_k2 = rng(1,int(n_s*0.5))
83     shuffled_s,k = flsa(shuffled_s,random_k2)
84     return shuffled_s
85
86 #---[ USYMBOLSET(s) ]
87 def usymbolset(s):
88     n_s = len(s)
89     if n_s <= 1:
90         return s
91     resultant = []
92     for i in range(n_s):
93         s_i = s[i]
94         if not (s_i in resultant):
95             resultant.append(s_i)
96     return resultant
97
98 #---[ PROTRACT(s,n) ]
99 def protract(s,n):
100     n_s, n = len(s), int(n)
101     if n_s < 1:
102         return s
103     if n < n_s:
104         return s[:n]

```

```

105     if n == n_s:
106         return s
107     multiple = int(n / n_s)
108     target_n = n_s * multiple + 1
109     resultant = s
110     i = 1
111     while i < target_n:
112         resultant = resultant + s
113         i += 1
114     return resultant[:n] # could potentially, also compress
115
116 #---[ SHUFFLE(s) ]
117 def shuffle(s):
118     n_s = len(s)
119     if n_s < 2:
120         return s
121     random_k = rng(1,n_s)
122     shuffled_s,k = slsa(s,random_k)
123     random_k2 = rng(1,int(n_s*0.5))
124     shuffled_s,k = flsa(shuffled_s,random_k2)
125     return shuffled_s
126
127 #---[ SELECT(n,s) ]
128 def select(n,s):
129     n_s, n = len(s), int(n)
130     resultant = []
131     if n <= 0:
132         return resultant
133     n = n if n <= n_s else n_s
134     resultant = s[:n]
135     return resultant
136
137 #---[ RSG(n,s) ]
138 def rsg(n,s, DEBUG=False):
139     n_s,n = len(s),int(n)
140     resultant = []
141     if n == 0 or n_s == 0:
142         return resultant
143     # first, compute the u-symbol set
144     if DEBUG:
145         print(f'input sequence: {s}')
146     s_usymbolset = usymbolset(s)
147     if DEBUG:
148         print(f'u-symbolset: {s_usymbolset}')
149     n_us = len(s_usymbolset)
150     # then chain the sequence transformers...
151     resultant = shuffle(s_usymbolset) # randomize the symbol set
152     if DEBUG:
153         print(f"shuffled u-symbolset: {resultant}")
154     resultant = protract(resultant,n*2*n_us)
155     if DEBUG:
156         print(f"protracted: {resultant}")
157     resultant = shuffle(resultant)
158     if DEBUG:
159         print(f"shuffled: {resultant}")
160     # then pick only as much as was asked for...
161     resultant = select(n,resultant)
162     if DEBUG:
163         print(f"selection:{n}: {resultant}")

```

```

164         return resultant
165
166     #---[ TEST RSG ]
167     #s = [0,1,2,3,4,5,6,7,8,9,0,1]
168     #s = [0,1,2,3,4,5,6,7,8,9] --> rsg(0,s) --> [],
169     # rsg(1,s) --> [6], [1], [0], etc.
170     # rsg(3,s) --> [2, 9, 5], [0, 1, 2], [5, 6, 5], etc.
171     #s = ['A', 'C', 'G', 'T', 'U']
172     # rsg(1,s) --> ['T'], ['A'], ['G'], etc.
173     #s = ['A', 'T', 'C', 'G']
174     # rsg(3,s) --> ['C', 'G', 'A'], ['G', 'T', 'C'] # random DNA codons!
175     #result = rsg(3,s,False)
176     #print(f"RSG result: {result}")
177     # what of generating sentences from words?
178     # e.g pass in a glossary
179     # rsg(10,['CAT','DOG','PEN','ACE','SKY','EYE','IS','THAT','NO','YOU'])
180     # --> ['YOU', 'IS', 'NO', 'DOG', 'SKY', 'THAT', 'ACE', 'EYE', 'CAT', 'PEN']

```

Of course, as shown in the final comments in the **RSG** code listing in **Listing 8.11**, we can see that our basic random sequence generator can actually be used to **generate random ANYTHING!** In that complete listing, unlike the earlier RSG specification in **Listing 8.10**, we have also enhanced the function with an extra optional keyword parameter, **DEBUG**, that expects a boolean value (**True** or **False**), and so that, by setting that flag to “True” — the boolean value for **true** in Python, we then allow the function to print-out the internal transformation results as it processes the final resultant sequence. Thus, we might appreciate how it works for various inputs based on example illustrative debugging outputs as shown hereafter...

Example 1: DEBUG Output of RSG: Generating a random digit 3-gram

```

1 input sequence: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1]
2 u-symbolset: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
3 shuffled u-symbolset: [9, 6, 1, 4, 2, 8, 0, 7, 5, 3]
4 protracted: [9, 6, 1, 4, 2, 8, 0, 7, 5, 3, 9, 6, 1, 4, 2, 8,
               0, 7, 5, 3, 9, 6, 1, 4, 2, 8, 0, 7, 5, 3, 9, 6, 1, 4, 2, 8,
               0, 7, 5, 3, 9, 6, 1, 4, 2, 8, 0, 7, 5, 3, 9, 6, 1, 4, 2, 8,
               0, 7, 5, 3]
5 shuffled: [0, 3, 9, 5, 6, 3, 4, 5, 3, 1, 1, 2, 0, 0, 4, 4, 5,
             4, 9, 9, 4, 8, 0, 7, 3, 6, 8, 6, 0, 7, 1, 5, 3, 5, 6, 9, 2,
             8, 1, 6, 2, 8, 2, 2, 7, 9, 4, 3, 6, 5, 7, 2, 8, 1, 0, 7, 1,
             7, 8, 9]
6 selection:3: [0, 3, 9]
7 RSG result: [0, 3, 9]

```

Example 2: DEBUG Output of RSG:: Generating a random letter

```

1 input sequence: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
2 u-symbolset: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z']
3 shuffled u-symbolset: ['j', 'p', 'l', 'm', 'v', 'q', 'r', 'c', 'a', 'g', 't', 'h', 'w', 'e', 'y', 'z', 'x', 's', 'b', 'o', 'f', 'u', 'k', 'i', 'n', 'd']
4 protracted: ['j', 'p', 'l', 'm', 'v', 'q', 'r', 'c', 'a', 'g', 't', 'h', 'w', 'e', 'y', 'z', 'x', 's', 'b', 'o', 'f', 'u', 'k', 'i', 'n', 'd', 'j', 'p', 'l', 'm', 'v', 'q', 'r', 'c', 'a', 'g', 't', 'h', 'w', 'e', 'y', 'z', 'x', 's', 'b', 'o', 'f', 'u', 'k', 'i', 'n', 'd']
5 shuffled: ['o', 'x', 'g', 'f', 't', 'p', 'h', 'h', 'l', 's', 'c', 'e', 'b', 'u', 'a', 'k', 'd', 'w', 'w', 'j', 'd', 'n', 'k', 'z', 'u', 'm', 's', 'q', 'i', 'c', 'y', 'r', 'g', 'n', 'i', 'a', 'v', 'r', 't', 'y', 'x', 'e', 'b', 'o', 'j', 'z', 'f', 'p', 'l', 'm', 'v', 'q']
6 selection:1: ['o']
7 RSG result: ['o']

```

Example 3: DEBUG Output of RSG:: Generating a random DNA Codon

```

1 input sequence: ['A', 'T', 'C', 'G']
2 u-symbolset: ['A', 'T', 'C', 'G']
3 shuffled u-symbolset: ['C', 'T', 'A', 'G']
4 protracted: ['C', 'T', 'A', 'G', 'C', 'T', 'A', 'G', 'C', 'T', 'A', 'G', 'C', 'T', 'A', 'G', 'C', 'T', 'A', 'G']
5 shuffled: ['T', 'G', 'A', 'C', 'A', 'C', 'C', 'C', 'T', 'T', 'C', 'A', 'G', 'A', 'T', 'A', 'G', 'T', 'G', 'C', 'T', 'G', 'A', 'G']
6 selection:3: ['T', 'G', 'A']
7 RSG result: ['T', 'G', 'A']

```

In fact, we can appreciate how useful this RSG is, at generating random sequences, if we turn off debugging, combine the results into a string, and then perform multiple invocations to get us a sequence of random sequences as we see in the next output that shows a sequence of random DNA codons it generates when we invoke it 10 times in a row with the same parameters as in the last example...

Example 4: Generating 10 random DNA Codons

```
1 10 random DNA codons via RSG(3,['A','C','T','G']):
2
3 ['CGC', 'AGT', 'GGA', 'TCG', 'CGC', 'GTC', 'CGA', 'ATC', 'CTA',
4  'GTC']
5
6 ['AAT', 'AGG', 'GTC', 'CTC', 'CGG', 'AAT', 'AAC', 'CTC', 'TCT',
7  'AGT']
8
9 ['CCC', 'CTC', 'TGC', 'AAT', 'ATC', 'ACG', 'GAG', 'AAT', 'GTT',
10 'CCC']
```

But not just that! We have already seen that it could be used to generate random numbers — whether individual digits such as 1, 5 or 0,.. but also sequences or n-grams of them — just depending on what kind of sequence we pass in as the source, how long we wish the output to be and what we do with the result. Moreover, note that **RSG** could be used to compute long DNA sequences using a clever invocations such as:

```
N=3*200; print(''.join(str(c) for c in
rsg(N,['A','T','C','G'])));
```

Which would get us compelling na-Sequences and potentially useful genetic codes as shown in the following listing that leverages that little code-golf snippet shown above...

Listing 8.12: A RANDOM DNA SEQUENCE

```

TATACGGCAAGCGATGAGATATCAAGCTAATCGATAGGCAACGAGACACCAGGAGACGCGT
CAGGCAACAAGGAATTAAGTCACGCGTCAAGCTATGAGATACTAAGCAACACGGTAGGCAA
CGAGACAATAGGAGATGCGCCAGGCGACAAGGAATCAAGTTATCCGACAAGCAATGAGATA
GTAAGCGACGCGGTAGGCAACGAGACATAAGGAAACGCGCCAGGCCACAAGGAATCAAGTT
ATACGGCAAGCGATGAGATATCAAGCTATTTCGATAGGCAACGAGACACCAGGAGTCGCTTC
CGTCAACATCGAACTCATTCCCGCCTCAACCTATGCCATACTAACCAATACCGTTGCCAAC
GATACAACTGCAGTTGCTCCCGTAGACATCGATCCCATTTCTCACACAACCATTGCCATAG
TAACCGAGGACGTTGCCAGCGATACATTTGCAATTGCTCCCGTACGCATCGCTCACATCTT
GAGTGCTATAGTCGCTACGTTTCATCTTCTGTACTGTCAACGCTACGCCCGTCGTGGCTTCT
GTGACCACTGCTTACATCCTCGATTCTATGTTTCGCTACACGCATCATAACT

```

Figure 8.19: A Random DNA Sequence of exactly 200 random codons as generated using our **RSG**(n, Θ) random sequence generator algorithm.

And finally, realizing that much of what is exciting about today's use of computing and especially artificial intelligence concerns popular applications such as the likes of **ChatGPT** that does interesting text-generation based on user provided prompts — including not just the ability to generate stories, essays, code snippets and even DNA sequences, but also ascii-art, music scores, etc. We shall then want to note here that, our basic **RSG**(n, s) function — a higher-order sequence transformer, can likewise do some surely compelling feats such as when we use it to generate some random prose based on a carefully selected list of input words:

A Random Prose Generator based on the RSG

```

1 print('\n'.join([' '.join(rsg(10,['CAT','DOG','PEN','
    ACE','SKY','EYE','IS','THAT','NO','YOU'])) for i in
    range(1,6)]))

```

Which code then gives us some stupid but entertaining stories and prose such as...

Example 5: Random Prose based on RSG

```

1 YOU ACE NO ACE SKY NO THAT CAT THAT ACE
2 YOU ACE EYE NO SKY THAT CAT THAT THAT CAT
3 THAT PEN IS NO DOG YOU SKY THAT IS EYE
4 YOU ACE NO ACE SKY NO THAT CAT THAT ACE
5 PEN THAT NO ACE IS ACE CAT SKY CAT IS

```

or perhaps...

Example 6: Random Prose based on RSG

```
1 NO DOG THAT YOU IS EYE DOG IS SKY NO  
2 PEN THAT EYE IS DOG EYE SKY CAT ACE EYE  
3 YOU CAT IS THAT ACE IS EYE THAT THAT SKY  
4 THAT DOG THAT YOU ACE YOU NO PEN CAT CAT  
5 ACE NO IS DOG NO EYE YOU ACE CAT IS
```

Thus you must note that the sheer significance of having a properly working generator-transformer such as the **RSG** is difficult — almost impossible, to explain¹².

¹²Even though we might not exhaust all ways that a random symbol or random sequence generator might be utilized in GENETICS, we shall never the less come to appreciate some of the utilities underlying our RSG, but also the RSG itself, in other parts of the book such as in **Chapter C** and in **Chapter 9**.

Chapter 9

Gene Expression in Living Organisms Leveraging Genetic Code (DNA → *mRNA* → *Protein* → Organism)

In earlier parts of this book, we have looked at the stuff that underlies a special kind of code from which biological life manifests. This *genetic code* is a kind of natural computer program that contains and specifies special instructions by which the mechanisms of nature¹ process na-Sequences so as to decide how to bring forth particular kinds of life or life-functions.

Code: A rule for transforming a message from one symbolic form (the source alphabet) into another (the target alphabet), usually without loss of information. The process of transformation is called encoding and its converse is called decoding.

— The Oxford Companion to the Mind[19]

¹**FUTURE WORK:** We have a potentially fundamental philosophical problem here; The rather *weird* aspect of this entire gene and genome expression mechanism underlying the manifestation of biological life from genetic code, is that, the very mechanisms or material substratum via which such genetic processing proceeds — the cells and proteins and protein factories, etc. that process DNA and RNA code — are themselves results of DNA or built up from genetic code! It is such a fine contrast from how artificial computing works; physical computers (the equivalent of physical organisms) are made of silicon and metals and plastic, etc. **but** the software that runs them (equivalent of genetic code) is perhaps best understood as just the electricity (emergent property of the physical matter) flowing through the physical machine in particular ways, and which, from some levels of abstraction might even seem as though it were *immaterial* — a good analogy for relating software to the “spirit” or “soul” that underlies material lifeforms, but though, that too somewhat doesn’t work well, because, software can be pinned down to the action of say electrons and ions in a physical computer, just like the materials making up the physical computer itself are likewise fundamentally made of similar stuff!

And then there’s the other major problem; it somewhat feels like there is a chicken-egg problem here... a question, if not perhaps in the domain of bio-physics, perhaps one of philosophy; what happened before DNA was, and before there was genetic code to specify the formation of life’s building blocks? Is it that DNA had to come first, or that there existed life in some form, and then genetic code manifested from that as a later refinement of what could have started out as a purely haphazard process? And so that, if it proceeded thus in the beginning, why can’t it be possible to create or manifest biological life from anything else but genetic code [anymore]?

And then of course there is the interesting matter of [genetic] code executing genetic code! Reminds one of Lutalo’s **UPLT** — Universal Programming Language Theory — applicable to software and modern [digital] computers; *that all programming is text-processing and that programs are essentially text processing text*[51]! It feels like, for biological life, all “biological software” or “biological processing” is nothing other than genetic code executing [other] genetic code!

In a later section (**Chapter 10**) we shall deal with [whole] genome expression — a kind of na-Sequence processing that results in not just bits, but an entire living organism, and shall especially deal with a particular hypothetical system of genome expression as well as use the modal sequence concept introduced earlier on, as the basis for exploring the expression or manifestation of an entire organism from just its basic genetic code. In this section though, we shall focus on the [lesser] processes that underlie whole genome expression; particularly, actual **gene expression** in real and natural biological systems, and shall specifically focus more on expression at a molecular level — essentially, at the level of protein manifestation via gene expression.

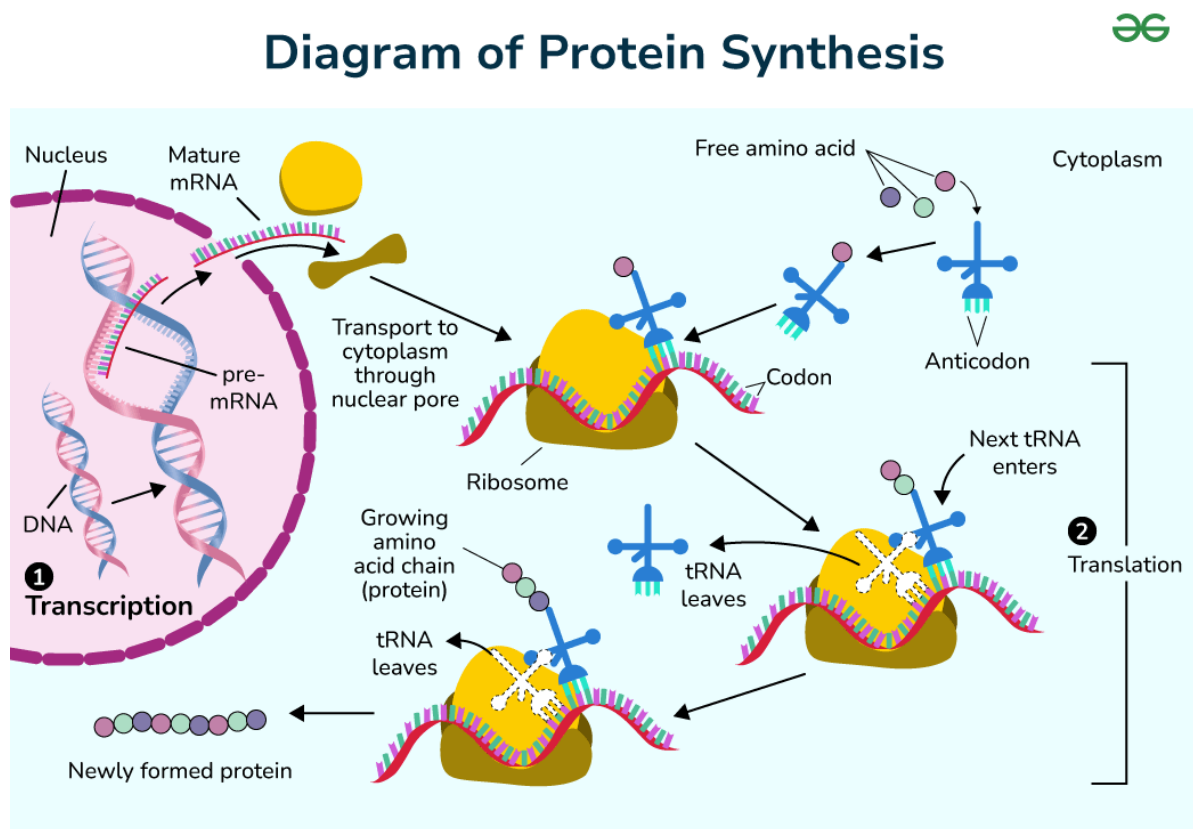


Figure 9.1: An illustration of the gene translation process in a cell via protein-factories known as ribosomes (Illustration credit: GeeksForGeeks.org[11])

It shall be interesting to note that for genetic code in natural automata (nature-like bio-automata² and generally living things), the most important reason the genetic coding language exists, is so that the body/host-organism system can

²My research assistant — thanks **Microsoft Copilot**, did bring it to my attention that the term “bio-automata” is “usually reserved for models or conceptual representations of biological systems — especially those designed to simulate behaviors, growth patterns, or decisions-making processes using predefined rules, like in cellular automata or agent-based modeling.” And thus, much as I often find it attractive to use the term — as an umbrella term including actual living organisms (which, from the perspective of the computer scientist in me, are still correctly classifiable under the “biological automata” category since they actually operate on some [inherent] natural [software] programs in their DNA just like artificial automata do [based on digital software]), however, perhaps it is indeed correct to relax any such usages moving forward.

produce required materials as and when they are needed or demanded for. This for example means producing new or extra body tissue in a still growing organism or in one with any damaged or missing tissue, and essentially, such productions are about the synthesis of particular molecules in the body's system that are basically proteins, and from which then, other materials are then [later] formed or regulated. The diagram in **Figure 9.1** is a basic illustration of the protein formation process for living organisms — eukaryotes especially.

Definition 12 (The DNA and RNA Codon Symbol Sets, ψ_{DNA}^* and ψ_{RNA}^*). If Θ_n is a DNA sequence of length n — meaning, it contains n nucleotides or n symbols from ψ_{DNA} , then, if each triplet of symbols in Θ_n is replaced by its corresponding 3-gram/CODON, then, we can re-write Θ_n as a sequence of terms from a special symbol set ψ_{DNA}^* :

$$\psi_{DNA}^* = \langle \alpha_1, \alpha_2, \dots, \alpha_{64} \rangle \quad (9.1)$$

Such that:

$$\mathfrak{U}(\psi_{DNA}^*) = \mathfrak{U}(\psi_{DNA})^3 = 4^3 = 64 \quad (9.2)$$

And that each distinct codon has a corresponding standard name associated with either its corresponding amino-acid or whether it is a STOP-codon[19][40]. We shall refer to ψ_{DNA}^* as the **DNA Codon Symbol Set**.

Further, since RNA sequences are similar to DNA sequences with just the symbols 'T' swapped with 'U' (see **Definition 2** and **Transformer 12**), we can also expect that for RNA sequences, there would be an equivalent symbol set for RNA-codons as such:

$$\psi_{RNA}^* = \langle \beta_1, \beta_2, \dots, \beta_{64} \rangle \quad (9.3)$$

And the names of the codons in ψ_{RNA}^* are similar to those in ψ_{DNA}^* by the following rule:

$$name(\beta_i) = name(\alpha_i) \quad \forall \beta_i \in \psi_{RNA}^* : \beta_i = O_{mRNA-encode}(\alpha_i) \quad (9.4)$$

For simplicity's sake, we can assume the following summarization of the basic process that fully and correctly breaks down the typical ordeal:

First, we shall assume that given the protein is just a chain of amino-acids, we might as well just think of it as though it were a sequence of some terms, and thus, in keeping with the notation from transformatics, we might just refer to such a protein with our usual typical **resultant sequence** symbol — Θ^* — especially when we are talking about it as a result of some prior processing of another kind of sequence.

And so, given that these proteins are actually nearly direct/1-to-1 mappings from the corresponding DNA sub-sequence code of a finite length, we might then refer to the DNA sequence that encodes the instructions for producing Θ^* with just the basic typical transformatics **source sequence** symbol: Θ — more con-

veniently, because we wish to also talk of the length of the sequence, we might preferably write the DNA sequence code of length $\mathbf{n}$ (meaning for example, **it contains exactly $\mathbf{n}$ DNA-codons**), as Θ_n . So, for example we might more clearly express Θ_n as such:

$$\Theta_n = \langle \alpha_{ij}, \rangle; \alpha_{ij} \in \psi_{DNA}^* \quad \forall j \in [1, n], i \in [1, 64] \quad \wedge \quad n \in \mathbb{N} \quad (9.5)$$

Equation 9.5 being just a sometimes preferable way to write the same exact sequence as:

$$\Theta_n = \langle a_1, a_2, a_3, \dots, a_i, \dots, a_{n-1}, a_n \rangle; \forall i \quad \exists a_i \in \psi_{DNA}^* \quad \wedge \quad n \in \mathbb{N} \quad (9.6)$$

We have defined the special symbol sets ψ_{DNA}^* in **Definition 12** and ψ_{DNA} in **Definition 1**, and as for ψ_{DNA}^* , we of course know that it essentially is the set of the distinct 64 codons (see **Table 1** in [19]) that *especially* encode amino acids, and which were first introduced in **Chapter 1**. Another way to expound on this is by saying that:

$$\Theta_n = \{a_i \mid a_i = \prod_{\rho \in \psi_{DNA}}^3 \rho \quad \wedge \quad \forall i \in [1, n], n \in \mathbb{N} \quad \wedge \quad \psi_{DNA} = \langle A, C, G, T \rangle\} \quad (9.7)$$

Thus we might encounter a gene such as Θ_4 composed of exactly 4 codons as shown below:

$$\Theta_4 = \langle \langle A, T, G \rangle, \langle A, A, A \rangle, \langle T, T, A \rangle, \langle T, A, G \rangle \rangle \quad (9.8)$$

Which might also be equivalently expressed as a flattened sequence if the fact that the nucleobases it contains are always read in triplets/3-grams/tuples of 3 at a time isn't that important. So that we can merely write it as:

$$\Theta_{4 \times 3} = \langle A, T, G, A, A, A, T, T, A, T, A, G \rangle \quad (9.9)$$

So we imply by that expression, that under the flat-structure notation, the sequence has exactly $4 \times 3 = 12$ elements. By this fact and the observation that the previous nested notation merely helps to group together each codon's members within a meaningful sub-sequence, and that the order is otherwise maintained in the flat-sequence structure, we might then also equivalently express the same actual DNA code sequence as an $n \times 3$ matrix as shown below:

$$\Theta_{4 \times 3} = \begin{pmatrix} A & T & G \\ A & A & A \\ T & T & A \\ T & A & G \end{pmatrix} \quad (9.10)$$

Whether to actually express it as a $3 \times n$ matrix or as $n \times 3$ might be up to the particular taste of the mathematician or scientist, but otherwise, we know that the sequence Θ in any of the forms above is essentially a **gene program**³ or a potential segment of one, that could be used to guide a DNA code processor such as a ribosome, to construct a corresponding protein based on the equivalent transcribed mRNA code sequence — in this case Θ_4^* , which we might express into mRNA code as such:

$$\Theta_4^* = \langle \langle A, U, G \rangle, \langle A, A, A \rangle, \langle U, U, A \rangle, \langle U, A, G \rangle \rangle \quad (9.11)$$

Which is what we would obtain after a necessary **DNA** $\rightarrow$ **mRNA** transform attainable via a DNA sequence transformer we might define as such:

Transformer 12 (DNA to mRNA Encoder). $\Theta_n \xrightarrow{O_{mRNA-encode}(\cdot)} \Theta_n^* ;$
 $\forall a_i \in \Theta_n = \langle a_i, \rangle : n \in \mathbb{N}, \quad a_i \in \psi_{DNA} \equiv \{A, T, C, G\},$
and if $\exists a_i \in \Theta_n : a_i = T \implies \exists a_i^* \in \Theta_n^* : a_i^* = U$
Otherwise $a_i = a_i^* \implies \perp(a_i = T \in \Theta_n) = \perp(U \in \Theta_n^*) \quad \wedge \quad \perp(\Theta_n) = \perp(\Theta_n^*) = n.$

Thus, though the gene in its DNA form comprised of the sequence of amino-acid codes named as in **Table 9.1**, and yet, the resultant sequence after applying **Transformer 12** to Θ_4 would be as illustrated in **Table 9.2**.

So, note that the names (and code-names) of the mRNA encoded codons stay the same as those of their corresponding DNA-encoded codons in both tables — this is actually generally/conventionally so. But also, note that the functions of the individual codons in either scenario are likewise expressed the same. So, this is because, when the genetic code is actually being executed (such as in standard

³Indeed, it is right, that any legit natural gene program targeting a codon processor can be expressed as a $n \times 3$ matrix.

i	a_i	Amino Acid (Code-name)	Function
1	ATG	Methionine (Met)	Start Codon: initiates translation
2	AAA	Lysine (Lys)	Basic amino acid
3	TTA	Leucine (Leu)	Non polar amino acid
4	TAG	Stop (Amber)	Terminates translation

Table 9.1: Amino-Acid Codes and Names in Θ_4 , a DNA-encoded gene

i	a_i	Amino Acid (Code-name)	Function
1	AUG	Methionine (Met)	Start Codon: initiates translation
2	AAA	Lysine (Lys)	Basic amino acid
3	UUA	Leucine (Leu)	Non polar amino acid
4	UAG	Stop (Amber)	Terminates translation

Table 9.2: Amino-Acid Codes and Names in Θ_4^* , a mRNA encoded gene

protein-synthesis⁴), the processor (the ribosome) merely operates on the mRNA-encoded gene and not directly on the original DNA-encoded code sequences.

Also, important to note, the processor only produces an amino acid (as part of the protein synthesis program), only after having encountered a “start” instruction, and we know that such instructions are the kind encoded by **start codons**, of which the most universally utilized START-codon is **ATG/AUG** known as Methionine, but also other rare-scenario⁵ START-codons include the mRNA codes GUG and UUG — used as such in prokaryotes, and then AUU and AUA, used as such in humans only.

And then, the processor will stop the protein construction task once it encounters a gene instruction of the “stop” kind. These are encoded using the **stop codons**, and these are strictly any one of: TAA/UAA (Ochre), TAG/UAG (Amber) and TGA/UGA (Opal)[50].

⁴**NOTE** that, in reality, protein synthesis is just the tip-of-the-iceberg for what DNA is actually useful for. **Chapter A** in the appendix of this book does cover, in summary, 6 other major functions of DNA in a biological system that the reader might wish to be aware of.

⁵They are used less frequently, mostly in prokaryotes and some organelles. When used as START codons, they still recruit the initiator tRNA and translate as **methionine**, not their usual amino acid[32]

That said, further note that, after processing the gene, and after encountering a stop-codon, the ribosome (also understood as the “protein factory”) is then triggered to detach (from the “assembly line”) and then release/return the final assembled protein thus far. These resultant proteins are basically just a chain of **actual amino acids** generally starting with the Met-amino acid.

And then, further note that, in case any codons were encountered before the AUG (or rather *start-codon*), these shall then be merely skipped — they are not processed or would not translate into any product such as the usual case of producing an amino-acid (this, even if they would normally have triggered the production of some amino acid).

These concepts and facts are the vital, fundamental truths known to underlie gene expression and the processing of gene programs, however, in the following sections, we shall undertake the task of further refining and rigorously, formally specifying what exactly goes on when such genetic code processing happens in a living organism or cell.

9.1 The Protein Manufacturing Algorithm

So, overall, we might sum up this critically important protein generation process with a convenient formalism such as with a protein-production algorithm expressed as with a flow-chart — depicted in **Figure 9.2** hereafter.

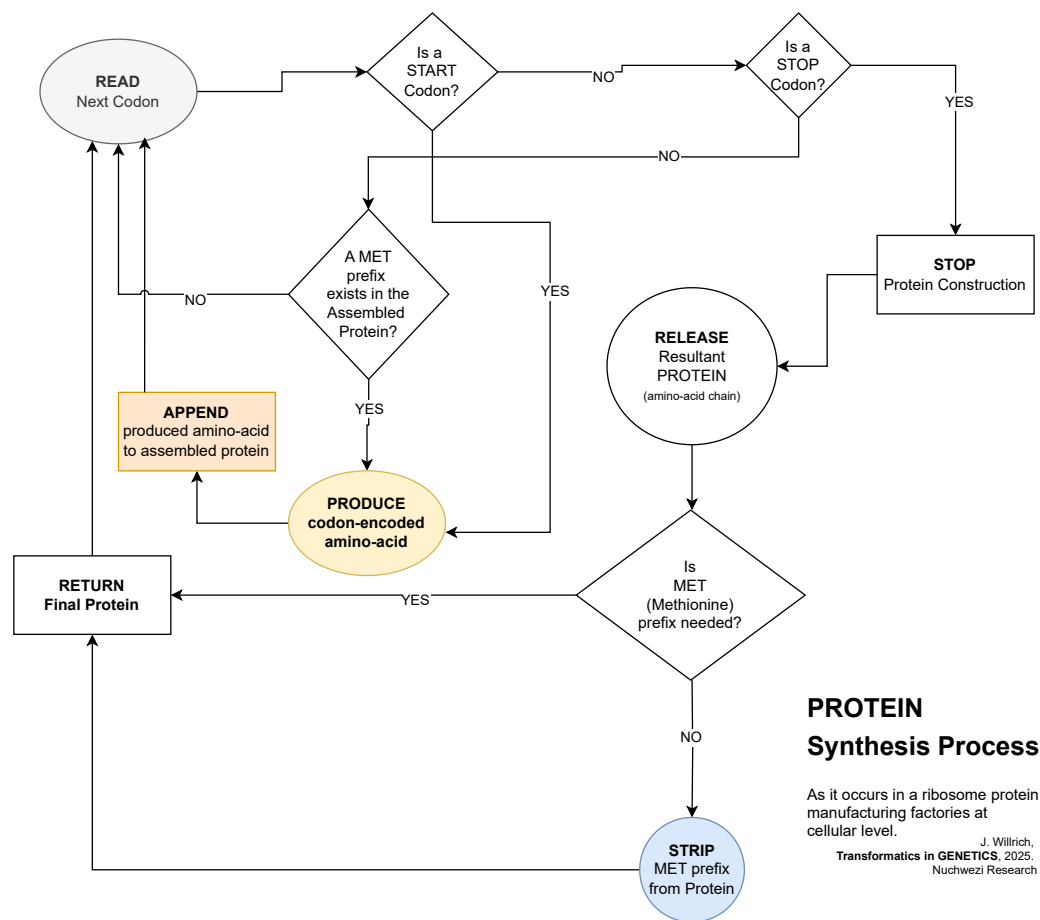


Figure 9.2: Flow-Chart summarizing the Ribosome-based Protein-Synthesis Process in a Living Cell

That process which is depicted in **Figure 9.2** is how the ribosome protein-manufacturing factory operates at a cellular level as depicted using a **Flow Chart Diagram** — meaning, the states of the operating environment as well as those of the operator (the ribosome) are interlaced with decision-making scenarios so as to bring to mind the logic behind how the process proceeds. However, in a different diagram — the **Ribosome State Machine** as depicted in **Figure 9.3**, we clearly abstract everything else away and focus on what actually happens from the point-of-view of the gene code sequence processor — the ribosome it self.

In a way, that state machine not only depicts the various states the ribosome shall be in while operating on incoming gene-code sequences (kind of *requests for solutions/solution-instances/proteins* to problems/specifications/genes) and then how it goes about producing the out-going amino-acid sequences (the proteins). It might even start to feel like the ribosome is a kind of programmable 3D-printer, which, when presented with the specifications of a particular 3D-specification-sequence, knows to process it (translate it) and then produce the

required/specified [3D-]object that is, in the context of biological systems we are looking at here, essentially some protein⁶.

THE RIBOSOME

State Machine

$S \rightarrow S^* \rightarrow [S^*]^*$

"God is our best teacher of Programming I Trust!"

J. Willrich,
Transformatics in GENETICS, 2025,
Nuchwezi Research

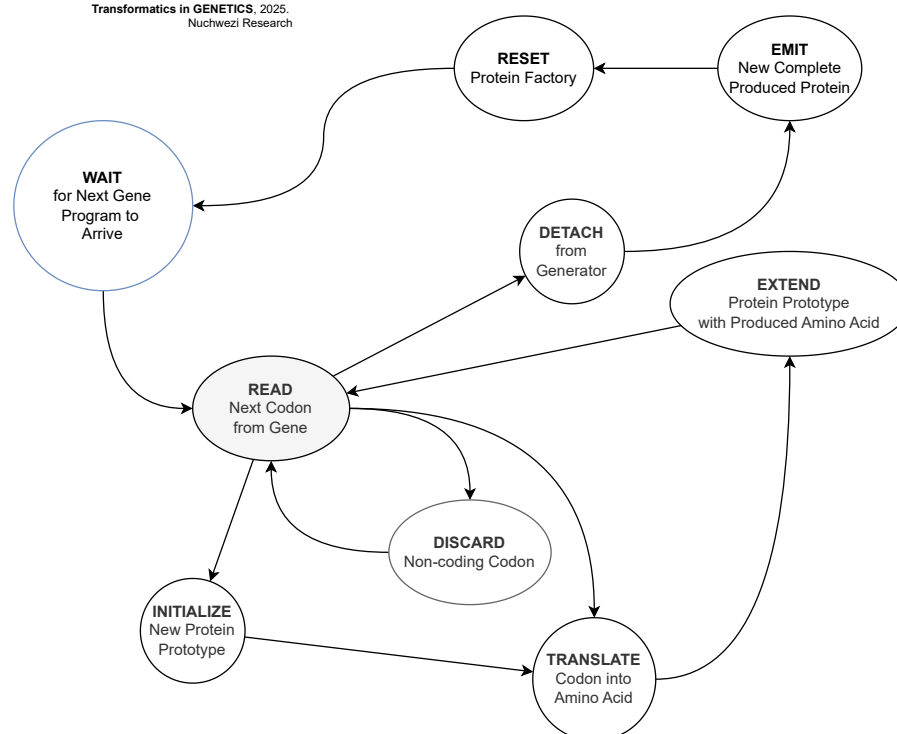


Figure 9.3: The RIBOSOME State Machine

Before concluding this section on biological computing systems of the sequence generator kind[7], it should be worth noting that, as depicted in **Figure 9.3**, the process that is naturally found in every living thing's tiniest cells (excluding the special case of viruses as we already saw in **Chapter 1**), could be — as with any legitimate state machine, used to implement a proof-of-concept [artificial] ribosome processor (or a *ribosome computer*) — a way to implement bio-automatons that behave like or function like a ribosome; which can process code similar to DNA and mRNA for starters, but also which, generally, operate on n-gram sequences to produce other (possibly more complex) n-gram sequences.

Thinking up a **robotic ribosome** might not be something that is immediately required in contemporary medicine or *in-organism*⁷ computing systems, but might

⁶**FUTURE WORK:** Of course, for someone with a background in computer science and who also has interest in designing not just computer programs but also new kinds of computers — abstract machines or not, studying the ribosome from a computing theory and computer architecture perspective — such as; so one appreciates what Instruction Set the ribosome employs; how the ribosome compares to say a Von-Neumann architecture machine; is it Turing Complete or not? How might a pure-text processing language such as TEA[51] implement a ribosome simulation in say a web-browser environment[45]? Or that, failing at that, at least allow for the creation of a protein generator or even an entire organism generator as a simulation of how gene code sequences can be translated into sequences of multi-dimension objects? etc.

⁷Meaning something like “invasive”, nano or embedded computing of a hybrid/bio-artificial kind perhaps.

be a concept worth adapting or exploring, for the design and implementation of a self-contained robotic factory for the manufacturing of otherwise complex [multi-form] products producible via use of a sequence-processing, assembly-line kind of method, just like how proteins are produced in a bio-cell by the ribosome, and with the entire process regulated or guided merely using instructions in the form of a sequence [of few symbols from some finite, minimalistic alphabet], spanning a special kind of “bio-instruction set”.

Thus, extending this idea of creating things based on genetic code, and not necessarily something to do with **genetic programming**, we might for example think of “a code sequence from which a complete, particular type of car can be manufactured at will” or “a sequence from which certain kinds of sequential-form/sequence-based⁸ artificial, but also organic creatures, implements and bots might be systematically assembled at will or on-demand”. It might call for special, yet-to-be kinds of “programmable printers” — which, unlike mere printers of *things on paper*, and not just like contemporary 3D-printers that only know to produce plastic variants of models they are fed with, might be the kind of printers with which we might entertain the idea of being able to *print a human*⁹. Such might also be the kind of essential aspects of some machinery with which we might **teleport**¹⁰ a cow or a banana, etc. It might be interesting and worth it, to explore, as we look into the far-future, where, with humanity’s ability to travel and survive in remote and “unnatural” worlds away from their biological home environments (such as Earth’s biosphere), whether or not mankind might not be compelled to have to develop new kinds of machines that would not only print out ideas on paper, but also be able **to print complete food ready-to-eat**, medicines for particular kinds of ailments, certain kinds of companion or life-supporting creatures, artificially re-producing rare or endangered species, etc. Essentially, the means to survive on/in alien worlds by leveraging smarter, general-purpose sequence processors and genetic-code-based material might become a thing not of fiction but of necessity and fact.

Talking of which, the next section shall help us start to appreciate this perspective of using the case of genetic code sequences and the ribosome sequence processor, to think of and reason about artificial, conceptual or hypothetical bio-machines that like the bio-cell, can produce complex artefacts via the pro-

⁸Vertebrates anyone(?)

⁹**FUTURE WORK:** Though we might touch on it in a future paper on philosophy — say one dealing with *Computational Mysticism*[52], it might be interesting to air-out the author’s illuminating view that unlike most other bio-automata such as beasts in the wild or fish in the seas, and definitely not as with silicon-based automatons such as GPT-powered modern *disincarnate* artificial entities — nor the likes of the ZHA qAGI[53], that the human in particular, has this peculiar attribute to them, that, apart from just their material substratum as any physical robot might possess or need — and which is say the domain of “material producers” such as the ribosome is, and away from their conceptual/software substratum too, they seem to, or perhaps arguably, also possess a preternatural layer of existence that perhaps is or might not have anything to do with their DNA/material-blueprint(?) A better or more precise classification term for such [*preter*]-intelligent -mata/matter(?) might be the still underground term and concept of a *Psymaton* or **Psymata** — bits of this line of inquiry have already been touched upon in the Psymaz Interview[54]; essentially, we need to answer the question of whether the “spirit” or “mind” in man is exactly just a result or consequence of their exact genetic code blueprint. Bits of this discussion have also been touched upon by the author, in their treatment of Psychology and how transformatics might help shed light on problems typically outside of the realm of strict-biological or anatomical studies — see [6].

¹⁰Refer to the appendix, **Definition 19** for some treatment of this concept.

cessing of some kind of instruction sequences — essentially, [re-]programmable bio-automata.

9.2 Gene Expression in Living Things and for Bio-Automata via Ribosomes

One might begin by wondering: **using the concepts from transformatics and pure mathematics, how might we model or express a general and realistic ribosome?**

A plausible and meaningful solution to this problem would be to begin by acquiring or developing *a rigorous and correct working definition of what a ribosome is*. The answer would follow directly from that, thus our next definition; a formal specification of a ribosome in any system natural or artificial:

Definition 13 (A Ribosome). *Assuming we re-write a sequence of DNA code in terms of the [ordered] sequence of nucleotides it contains, as in **Equation 9.7** re-written as in **Equation 9.12**:*

$$\Theta_n = \{a_i \mid a_i = \prod_{\rho \in \psi_{DNA}}^3 \rho \quad \wedge \quad n \in \mathbb{N}\} \quad (9.12)$$

Θ_n would then be any sequence of DNA-codons, and of length n , equivalent to a first-order sequence of nucleotides of length $n \times 3$. We can then produce mRNA-codons from Θ_n as per **Transformer 12**, so that we produce a new mRNA-codon sequence of length n that is generated as such:

$$\begin{aligned} \textbf{Transformation 21. } \Theta_n &\xrightarrow{O_{mRNA-encode}(\cdot)} \Theta_n^*; \\ \Theta_n^* = \{a_i^* \mid a_i^* &= \prod_{\rho \in \psi_{RNA}}^3 \rho\} \quad \wedge \quad \mathcal{L}(\Theta_n) = \mathcal{L}(\Theta_n^*) = n \end{aligned}$$

And with Θ_n^* thus produced, we can then merely generate the corresponding sequence of amino-acids, denoted as $[\Theta_n^*]^*$, via the following mRNA to amino-acid transformer:

Transformer 13 (mRNA to Amino-Acid Translator).

$$\Theta_n^* \xrightarrow{O_{mRNA-translate}(\cdot)} [\Theta_n^*]^* ;$$

1. $\mathcal{L}([\Theta_n^*]^*) < \mathcal{L}(\Theta_n^*)$ because¹¹ we only count each codon in the source once, and as per the rules of gene processing/transcription, all non-coding codons (introns) — basically, codons not able to be translated into an amino-acid given the state of the gene processor¹² don't contribute to the generated resultant sequence in terms of sections it contains — with the exception of the special “Met” codon¹³ that might or might not be retained in the resultant sequence even though it is automatically included as the first produced amino-acid in any legitimate gene sequence transcription result.
2. The entire sequence $[\Theta_n^*]^*$ is a kind of 3-Dimension molecule based on the chain of amino-acids it contains, and is technically referred to as a protein.

□

A **Ribosome** then, is any combination of transformers that can result in $[\Theta_n^*]^*$ when presented with just Θ_n as per the two intermediate transformer processes **Transformer 12** and **Transformer 13**, and whose overall processing algorithm is as depicted in **Figure 9.2** and its corresponding state machine as in **Figure 9.3**.

So, overall, a ribosome is any machine that can implement the higher-order transformer defined as in **Transformer 14**:

Transformer 14 (The Protein Generator (A Ribosome)).

$$\begin{aligned} \Theta_n &\xrightarrow{O_{mRNA-encode}(\cdot)} \Theta_n^* \xrightarrow{O_{mRNA-translate}(\cdot)} [\Theta_n^*]^*; \\ \mathcal{L}([\Theta_n^*]^*) &< (\mathcal{L}(\Theta_n^*) = \mathcal{L}(\Theta_n) = n) : \\ \psi_\Theta &\subseteq \psi_{DNA} \quad \wedge \quad \psi_{\Theta_n^*} \subseteq \psi_{RNA} \quad \wedge \quad \psi([\Theta_n^*]^*) \subseteq \psi_{amino-acids} \end{aligned}$$

And thus, we can finally merely call any machine capable of implementing the protein generator in **Transformer 14** as a **Ribosome**.

¹¹**FUTURE WORK:** Verify, either empirically or theoretically, whether indeed this inequality is universally true or whether there is a computable upper limit to the length of the produced amino-acid sequence relative to the length of the gene program sequence that describes its production.

¹²See **Figure 9.2** and **Figure 9.3**

¹³A START-codon also counted among genuine “exons” — coding codons.

Chapter 10

Transformatics and the Lu-Genome Expression System in Bio-Automata



Figure 10.1: A blackboard snapshot from the author's **Blackboard Adventures** (<https://t.me/bclectures>) series on Telegram, on **20th AUG, 2025**, depicting two **LGES** example genome expressions, one for $\vec{\Theta} = \langle 218 \rangle$, the other for the **PFA** segment configuration $4_{pf} \otimes 6_{oz}$

In this chapter, we are going to do a lot of creative work besides just the core mathematical analysis and modeling. We are going to explore a phenomena somewhat

similar to encountering a computer program's source-file, which, when processed in the right environment, somewhat has the program source code transform [itself?] into some dynamic object complete with a distinct appearance, an identity and the ability to somewhat replicate or reproduce itself!¹

Genome Expression: The collective expression profile of all genes within a genome, including coding and non-coding regions, across different conditions, cell types, or developmental stages. It is a systems-level view - measuring how the entire genome behaves dynamically. This includes: mRNA expression of all genes, noncoding RNA activity, epigenetic regulation, chromatin accessibility and transcriptional networks. In essence, gene expression is local, while genome expression is global.

— Microsoft Copilot[32]

We shall use a **thought experiment** and *hypothetical cases* leveraging **exemplary** genetic code modeled using special sequences that are treated as **modal sequences** at minimum.

Applying Transformatics:

Away from what we have already encountered in earlier parts of the book; such as using transformatics to make sense of the differences between organisms based on their genetic code sequences; an even more exciting application of the theory and practical methods of transformatics would be in conceptualizing, analyzing and explaining the important biological, physical, chemical, informational, mathematical, and *aesthetic* concept of **genome expression**; how a sequence of “symbols” transforms into something sublime — perhaps only remotely akin to the sequence from which it first originated.

First, we shall define a basic cipher that can encode the basic digits (of base-10)

¹A good metaphor for those coming to Genetics with a sane background in fiction and Hollywood entertainment — such as the stuff, thrilling studies and adventures of **Professor Walter Bishop**, Agent Olivia Dunham, and Peter Bishop in the sci-fi series *Fringe*. Except, ours isn't really science fiction... Bits of this theory and exploratory pragmatic work might come off as abstract and merely conceptual, but it is all grounded in factual sensibilities and concrete mathematics.

as some unique, but non-digit forms (essentially, with non-digit *glyphs*). This, so when we speak of a living organism — for example a pine-apple or a feline, we don't expect that in nature, or rather, that by physically dissecting the organism, that one shall find trapped or stored inside it, the literal genetic code symbols (such as those in ψ_{na}), but rather, that they shall find natural literal expressions or instances of genetic code's **material basis** — such as the chemical base molecules that define nucleobases, or nucleotides, and also complex molecular structures such as transcribed amino-acids post-RNA translation of DNA expressions.

So, in our hypothetical genome expression model — the **Lu-Genome Expression System** (LGES), we are going to expect and express elements from our basic hypothetical DNA symbol set — an alphabet² we shall call **Base-Omega**.

$$\psi_{\Omega} = \langle 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \rangle \equiv \psi_{10} \quad (10.1)$$

Which, especially for curiosity's sake, but also, for creative and expressive reasons, is meant to allow us to explore a peculiar genetics model based not on just some four base symbols as is the case in ψ_{DNA} , but which allows us to look at any base-10 number expression — essentially any *decimal sequence* — as a potential genetic code sequence — with **each digit expressing a distinct hypothetical nucleic acid**.

For example, we might describe some fictitious organism — for simplicity's sake, perhaps, just a virus, that we shall name **the Euler Virus**³ or just “veuler”, as having its entire⁴ genome sequence encoded as just:

$$\Omega_{veuler} = 27182818 \quad (10.2)$$

However, that is just the DNA-side of the story. As in real nature, we shall also need an extra, *different*, alphabet or symbol set for expressing genetic code in its *intermediate form* — as genome expression typically proceeds via expression of DNA into mRNA first of all, and then finally into functions such as proteins, via which and from which the final organism is then fully expressed. So, in our case, the intermediate expressions shall be expressed using what we are going to

²In our mathematics, and especially in Transformatics, we conventionally talk of a **symbol set** where other literature might talk of an *alphabet*.

³I called it that, simply because of the fact that the associated genome sequence is based on the important natural physical constant — 2.718281828459, the **Euler number**, that is one of those irrational numbers that also happens to be the base of the natural logarithm and which arises naturally in many areas of mathematics, especially in calculus, complex analysis, and probability theory, and which number I trust, is very likely to be of relevance to geneticists too.

⁴For keeping the analysis simple, we of course chose to truncate the expression to just the first 8 significant digit symbols of the natural constant, but in future treatments, might as well explore using longer versions of the sequence.

refer to as the **Ozin Cipher**⁵, which we are to describe hereafter — especially because its symbols are potentially less familiar to most than would be the case of the decimal-digit based Omega alphabet specified above.

10.1 The Ozin Genetic Code Cipher

With the background we have obtained thus far, note that, the special **transcription level** expression of our genetic code originally expressed via ψ_Ω — as **omega genetic code**, shall then be transcribed into the intermediate **ozin genetic code** that spans the symbol set ψ_{oz} , so that then, it is mapped from the genetic code's [original] storage expression into the actionable, intermediate form⁶, via a mapping as depicted in **Figure 10.2**.

⁵More about this in **Appendix B**.

⁶For those coming to Genetics from fields such as Computer Science and Software Engineering, this idea of nature requiring genetic information to first be translated into some intermediate form so as to be operated on, shall likewise reflect ideas such as what we find being utilized in many modern software operating environments[?] — such as say the .NET common-language runtime — a virtual machine kind of platform, that takes program code in its usual natural form such as in C#, F# or VB, and then translates it into IL (Intermediate Language) first, which then the CLR (Common Language Runtime) knows to compile (Just-In-Time/JIT), so as to have it executable platform-independently as native machine code. Similar ideas are also found in SOEs such as Java's, when code in languages such as JRuby, Kotlin, Scala, and Groovy need to first be translated into the JVM intermediate language (Java bytecode), so that they then can take advantage of the cross-platform/platform-indepdent JVM (Java Virtual Machine)[32]. These ideas of essential language transforms are thus common in artificial software operating environments, but also underlie the execution of “biological software” in the form of genetic code.

The OZIN CIPHER

Originally a "secret hand"
developed at Nuchwezi Research
as part of occult and
cryptographic studies circa
2016.

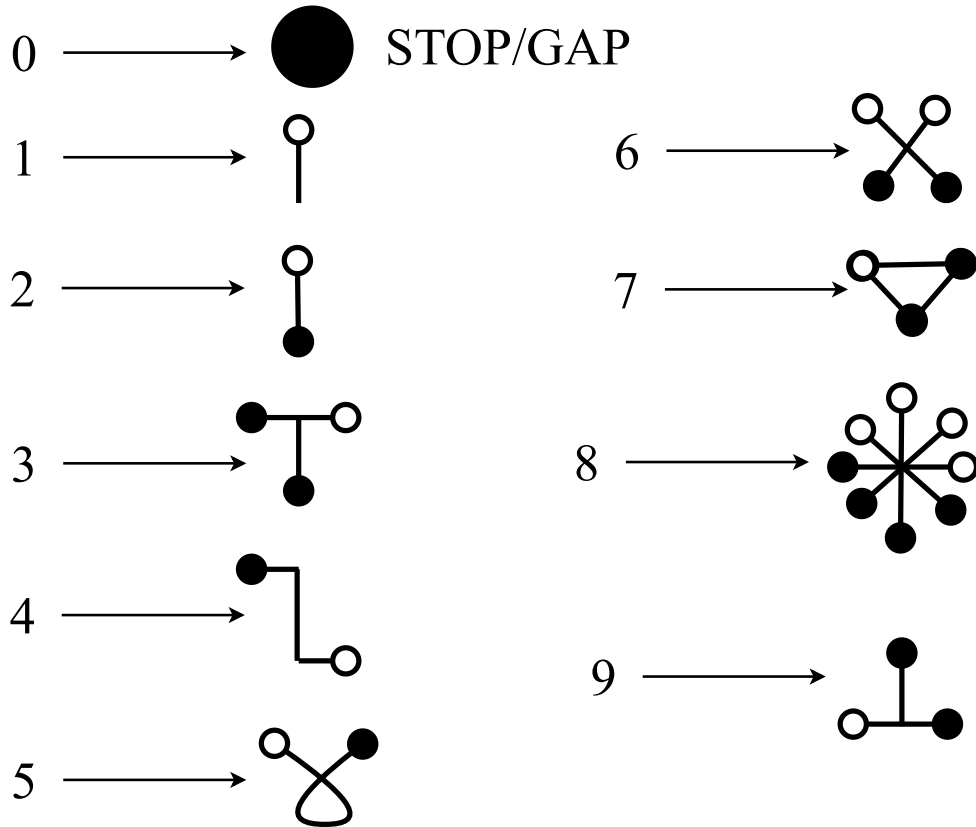


Figure 10.2: The OZIN Cipher[12], a mapping from Decimal Symbols (our base- Ω) to Symbols for Intermediate Genetic Code.

Without further ado, note that we might interpret or transcribe from base- Ω DNA into our base-OZ RNA expression as such:

If for some organism such as that expressed via $\Omega_{veuler} = 27182818$, we wish to encode the equivalent intermediate [physical] expression, then we iterate through each symbol in the source code sequence, and re-write it as the equivalent or corresponding symbolic structure in OZIN item-wise, and this, being done in a way so as to **center the ozin glyphs along a backbone structure** of just a mere line that starts with a tiny dot and ends with the last ozin glyph from the source sequence thus being transcribed. That is the basic LGES protocol for transcription from omega genetic code into ozin genetic expressions.

Essentially, for our Euler Virus, the LGES transcription protocol would result in an expression such as what we see depicted in **Figure 10.3**.

The Euler Virus

Encoded in base- Ω as just

27182818

and in base-OZ as

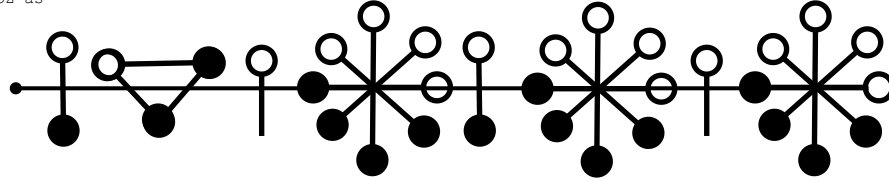


Figure 10.3: The equivalent OZIN expression of the hypothetical Euler Virus genome sequence.

For another organism, which we might just refer to as the **HiFinelle** — particularly because it actually is based off of our favorite base-10 o-SSI⁷ — the **Hi-Fi o-SSI**[3], and thus, its corresponding genetic code at rest is as:

$$\Omega_{HiFinelle} = 8649137520 \quad (10.3)$$

And which, after we have it transcribed into intermediate OZIN genetic expression, shall appear as shown in **Figure 10.4**

The HiFinelle

Encoded in base- Ω as just

8649137520

and in base-OZ as

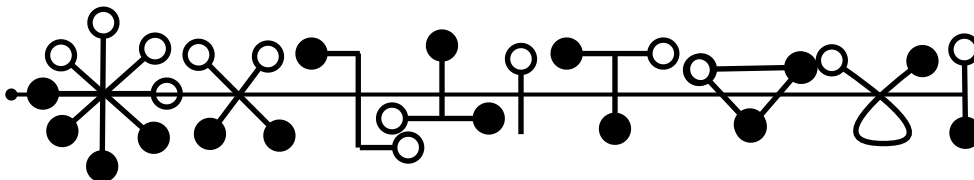


Figure 10.4: The equivalent OZIN expression of the HiFinelle genome sequence.

Of course, in our simple genome expression system, the occurrence of that last

⁷Refer to **Theorem 3** and **Proposal D.2.1** for related treatments.

“0” symbol in the HiFinelle genome sequence tells us to STOP or just exclude that symbol with a GAP in case any other symbols follow thereafter. It is our STOP-codon equivalent.

Another genome sequence,

$$\Omega_3 = 0123026 \quad (10.4)$$

Might help illustrate that point — see **Figure 10.5**

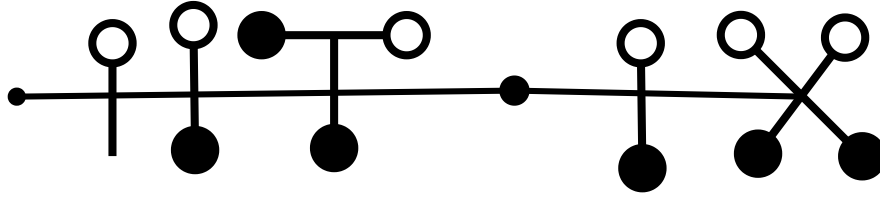


Figure 10.5: The equivalent OZIN expression of the Ω_3 genome sequence with multiple STOP-codon elements in the original genetic code, including some in the middle of the code.

10.1.1 Ω -to-OZIN Genetic Code Transcription Rules

So, in general, we might sum up our gene transcription rules (the *LGES transcription protocol*) as such:

Algorithm 5 (The Ω -to-OZIN Genetic Code Transcription Algorithm).
Assuming we have an Ω -base encoded genetic code sequence Θ_Ω of some length $n > 0$.

1. **INITIALIZE** the transcription by constructing a backbone-structure that essentially is just a line with a dot at the head/start. That structure shall encode the initial or baseline transcribed sequence $OZ(\Theta_\Omega)$.
2. **FOREACH**/Next symbol ω_i in Θ_Ω :
 - (a) **USE** the transcription mapping: $\psi_\Omega \rightarrow \psi_{oz}$ as depicted in **Figure 10.2**, to **ENCODE** ω_i in the equivalent form for the intermediate code.
 - (b) **IF** $\omega_i = 0$:
 - i. **IF** $OZ(\Theta_\Omega)$ was still empty:
 - A. Simply **SKIP** to the next symbol in Θ_Ω (jump to **Step#2**).
 - ii. **ELSE**:
 - A. **APPEND**/Insert a GAP in $OZ(\Theta_\Omega)$
 - B. Then **JUMP** to the next symbol in Θ_Ω (jump to **Step#2**).

(c) **ELSE**:

i. **APPEND** that encoded version of ω_i along the backbone structure, $OZ(\Theta_\Omega)$, at the next vacant position before the end ($\leq i < n$).

ii. Then **JUMP** to processing the next symbol in Θ_Ω (jump to **Step#2**).

3. **TRUNCATE** the backbone-structure after the final term in $OZ(\Theta_\Omega)$.

4. **RETURN** $OZ(\Theta_\Omega)$ as the final transcribed version of Θ_Ω .

10.1.2 Examples of Ω -Genetic Code Transcription

So, now that we have the general transcription rules as in **Algorithm 5**, we can proceed to explore how to leverage them.

First, note that, in **Chapter 5**, we have looked at not just the interesting idea of [genetic code] sequence complements, but have also explored several applications of the idea in **Section 5.2**, including the concept of the equivalence of sequences under the complement transform. Talking of which, assuming we had a special genome sequence that is made up of subsequences that are complements of each other (such as we might call **palindromic na-Sequences**) — e.g.⁸ Θ_{pal} defined below, that is actually a palindrome of the two DNA codons — **ATG** (Methionine) and **CAA** (Glutamine) — the complements being **GTA** (Valine) and **AAC** (Asparagine)[40].

$$\Theta_{pal} = \langle A, T, G, C, A, A, A, A, C, G, T, A \rangle \quad (10.5)$$

If we express Θ_{pal} in our special numeric **base- Ω** , and then into the visual **base-OZ**, we shall get something like Ω_{pal} ⁹:

$$\Omega_{pal} = \langle 1, 2, 3, 3, 2, 1 \rangle \quad (10.6)$$

And which, if we transcribe it into the intermediate OZIN code, we get:

⁸As in all other uses in this work — *exempli gratia*

⁹Simplified, so we can easily visualize it in base-OZ

cipher introduced in **Section 10.1**, but shall also introduce another formalism in the form of decoding position within a sequence (such as genetic code in this case), as information about how to express or process the associated symbols. This idea, of leveraging *subtle instructions* embedded in genetic code but which might not be explicit as the actual nucleic acid combinations (such as amino-acid encoding-codons), can be justified when we consider that the genetic code processor (such as the ribosome in our main context here — see **Section 9.2**), has a way of interpreting sequences not just by what they contain, but also based on the context within which they manifest — basically, that the order of occurrence of sequences does matter when their actual interpretation or execution happens.

For example, though generally, the appearance of the RNA codon, **AAA**, would cause the ribosome to synthesize the corresponding amino-acid Lysine (see **Table 9.1**), and yet, if such a codon appears in a sequence where no earlier instruction compels the ribosome to **START** manufacturing a protein, the appearance of that codon shall be useless — or rather, it shall be interpreted differently merely because of being in the *wrong* context. We saw this in **Chapter 9**, and especially via the Ribosome’s protein-manufacturing process diagram (**Figure 9.2**) as well as the State Machine (**Figure 9.3**) that help bring this out clearly. For example, if we modify the gene-program in **Equation 9.11** as such:

$$\Theta_4^* = \langle \langle A, A, A \rangle, \langle A, U, G \rangle, \langle U, U, A \rangle, \langle U, A, G \rangle \rangle \quad (10.7)$$

Then, where we would have expected the ribosome to manufacture a protein whose structure includes the AAA/Lysine amino-acid, the result/effect of our modified sequence shall actually be different from what the original gene-program would have produced. In particular, since the **START**-codon, AUG (Methionine) appears after AAA, the effective protein to be synthesized would be as equivalent to just the program:

$$\bar{\Theta}_4^* = \langle \langle A, U, G \rangle, \langle U, U, A \rangle, \langle U, A, G \rangle \rangle \quad (10.8)$$

Which would produce the protein with just **UUA**(Leucine) in it! So, positioning of symbols or terms within genetic code does matter, and does encode some important gene or even genome expression information that we can’t take for granted, thus the following suggested system for how to express post-RNA genetic code sequences.

10.2.1 The Platonic Form Cipher and Position-Sensitive Genetic Code Translation

Just like we developed a system for how to map numbers (base-Omega) to some non-trivial expressions that we equate to a kind of intermediate genetic code (the OZIN-encoded expressions), we shall similarly start by considering a way to map (still decimal-encoded) numbers to spatial-structures¹¹.

The PLATO CIPHER

Inspired by the related concepts of the Platonic Forms (philosophy) and Platonic Solids (geometry). This cipher maps digit N to a polygon formed by joining the N vertices obtained after a circle spanning 360 degrees is divided into N equal parts. Copyright jw1@NUCHWEZI.com

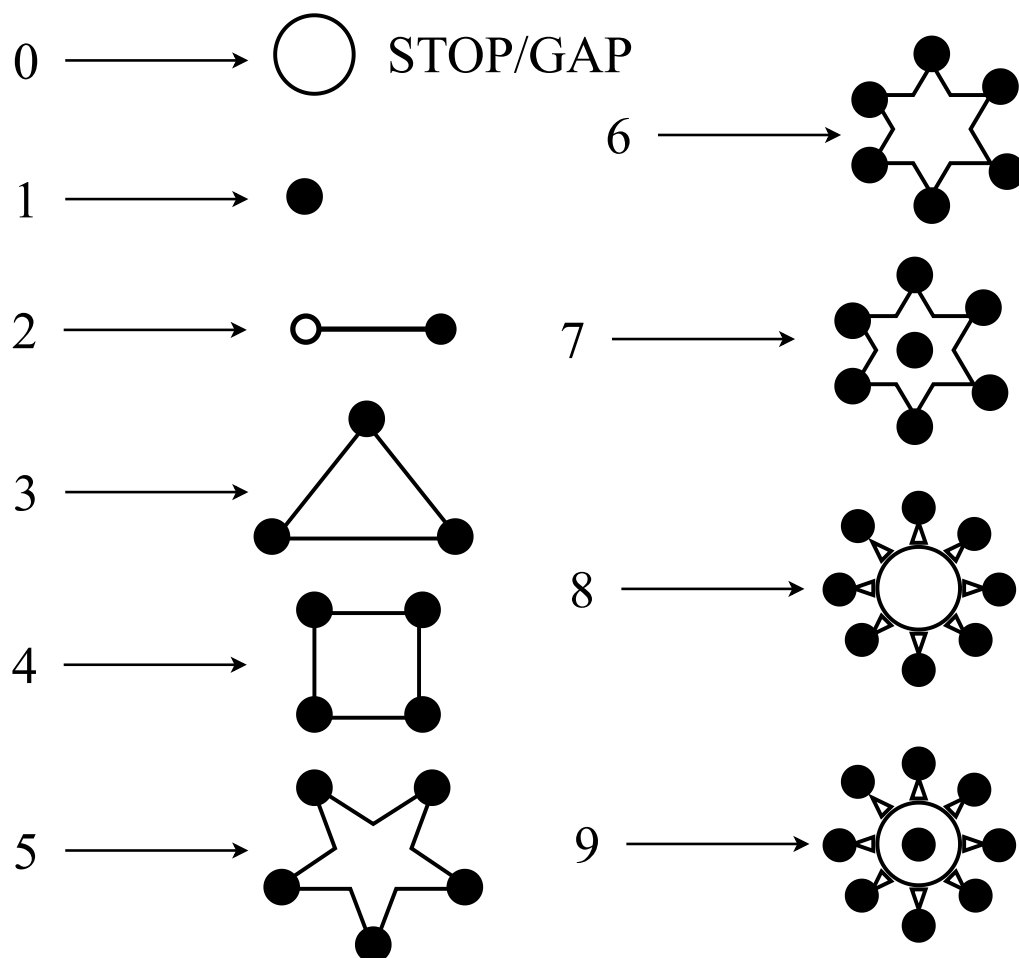


Figure 10.7: The PLATONIC Cipher[13] a mapping from Decimal Symbols (our base- Ω) to Spatial-Geometric Forms (ψ_{pf}).

So, with the exception of “0”, which again is treated as an instruction to ‘skip’ or ‘stop’ within our genome expression system¹², we shall adopt the mapping from

¹¹Especially because, and essentially that, when compared to the code that expresses them, proteins and the organism taken as a whole, *occupy more space* and are more *spatially-elaborate* than does just the basic [compact] DNA code from which they are expressed.

¹²Perhaps, and as we shall adopt here, we shall interpret ‘0’, especially *within* and not at the start or end of a sequence, to mean

numbers to spatial-structures as depicted in **Figure 10.7** in what we are calling the **PLATONIC Cipher System**[13].

How does it work?

10.2.2 ψ_Ω -to- ψ_{pf} : PLATONIC Form Genetic Code Translation Rules

So, in general, we might sum up our start to finish gene (and genome) expression protocol with the post-transcription rules included, and process as such:

Algorithm 6 (The Ω -to-PLATONIC Form Genetic Code Expression Algorithm). *Assuming we have an Ω -base encoded genetic code sequence Θ_Ω of some length $n > 0$, we shall start by constructing the **Intermediate Form Affix** (IFA), and finalize with the entire genome expression as **Final Genome Expression** (FGE), $\boxed{OZ(\Theta_\Omega)}^*$. The process, step-by-step, is as follows:*

1. **INITIALIZE** the expression with a transcription, by using **Algorithm 5** to **CONSTRUCT** the intermediate code expression of Θ_Ω — essentially compute $OZ(\Theta_\Omega)$, which is the **IFA**.
2. So, once we have the **IFA**, $OZ(\Theta_\Omega)$, **PROCEED** to use it to generate the rest of the [intended FGE] sequence expression as follows:
 - (a) **INITIALIZE** FGE as just:

$$\boxed{OZ(\Theta_\Omega)}^* = \langle\langle OZ(\Theta_\Omega) \rangle\rangle \quad (10.9)$$

*This ensures we have the **IFA** as the prefix of our **FGE**, whose only other element at this moment is a **still empty subsequence**. In the spatial expression form, this shall merely be a backbone or tree structure¹³ with **IFA** as its head or initial [child] node, and with the other part... the terminal node (or “alternate child branch” when thinking in terms of trees or graphs) as just an empty node that is ready to have more units appended to it. Betwixt these, there must be a **single GAP/JOINT extension** — essentially the “root” if thinking in terms of trees — to bridge the two parts (IFA and PFA) that shall make up the FGE. That [still] empty sequence node represents the **PFA**¹⁴. So, it also means, at this juncture, **PFA** is just:*

not just a GAP as was the case in **Algorithm 5**, but as a JOINT. It somewhat makes biological/anatomical sense.

¹³For those not aware, realize that in the past, we have worked on a rigorous treatment of how transformativ sequence algebras relate to other earlier ideas such as tree and graph algebras, and this for example would explain how or why we can encode a spatial expression such as a tree or graph using sequence notation as we are doing here. Details are covered well in [71].

¹⁴Platonic Form Aspect

$$\boxed{OZ(\Theta_\Omega)} = \langle \rangle \quad (10.10)$$

- (b) **COMPUTE** the **IFA MSS**¹⁵, $OZ(\overset{\triangleright}{\Theta}_\Omega)$ — basically, or simply, compute the MSS of the original sequence ($\overset{\triangleright}{\Theta}_\Omega$), then convert that into OZIN.
- (c) **COMPUTE** m , the cardinality of the **IFA MSS, $m = \perp(OZ(\overset{\triangleright}{\Theta}_\Omega))$**
- (d) **COMPUTE** the **Platonic Form Encoding Key**(PFEK), $\boxed{\psi_{pf}}_m$, corresponding to m and based on the **SLSA Algorithm**¹⁶
- (e) **FOREACH** symbol/term ω_i^* in **IFA MSS**, $OZ(\overset{\triangleright}{\Theta}_\Omega)$:
- i. **USE** the **symbol set translation mapping**¹⁷ $\psi_\Omega \rightarrow \boxed{\psi_{pf}}_m \rightarrow \psi_{pf}$ where that final term corresponds to the mapping depicted in **Figure 10.7**, to translate ω_i^* into the equivalent **Platonic Form**, $\boxed{\omega_i^*}$, for the final expression as such:
 - ii. **IF** $\omega_i^* \equiv 0$: Operating on the **PFA**:
 - A. **APPEND**/Insert a **JOINT** onto the **PFA**:

$$\boxed{OZ(\Theta_\Omega)} = \boxed{OZ(\Theta_\Omega)} \cdot \boxed{\omega_i^*}_{joint} \quad (10.11)$$

- B. **PROCEED** to the next symbol in **IFA MSS**, $OZ(\overset{\triangleright}{\Theta}_\Omega)$ (jump to **Step#2(e)**).
- iii. **ELSE: RESOLVE** the correct $\boxed{\omega_i^*}$ for ω_i^* by using the relation¹⁸

¹⁵We shall render the **PFA** based on $OZ(\overset{\triangleright}{\Theta}_\Omega)$ because it allows us to not express Θ_Ω naively at the genome or gene level as we finalize the **FGE** construction. But also, so that, from an information-theoretic perspective, and as we shall demonstrate, that the completed genome expression is somewhat a statistical summary of what the original genome sequence contained — **the idea that the appearance of an animal, a plant or a virus depicts or expresses a summary of its underlying DNA code**. Thus, we approach expression in the LGES, not via a simple 1-to-1 translation of the underlying genetic code into the final spatial form, but instead via first computing its summary in form of $\overset{\triangleright}{\Theta}_\Omega$, and then translating that into the final spatial/geometric forms as per the $\psi_\Omega \rightarrow \psi_{pf}$ transform rules specified hereafter. This process shall help suitably model the gene/genome expression processes that follow the initial transcription steps in natural gene or genome expression, and so this step is a very critically important aspect of the LGES protocols.

¹⁶The **SLSA**, **Second Lu-Shuffle Algorithm** is well introduced and explained in **Section 8.2**, and it basically is about generating an exact anagram of the input sequence, corresponding to a particular partitioning index — in this case (and by **LGES** convention, denoted) m , the size of the **IFA MSS**.

¹⁷So, instead of directly translating from the base- Ω expression into base-PLATO/ ψ_{pf} , we instead use the parameter m which shall be constant for a particular genetic code sequence, so that it gets us a new/random but consistent alternative anagram of the symbol set ψ_Ω — essentially, an anagram of the base-10 digits in this model, and so that, if in ψ_Ω the symbol “1” occupied position 2 and yet, for a given m such as 3, it shall occupy position 8! For ψ_{10} , these anagrams are captured in **Table C.1**. But how we exactly use them in LGES is explained in a later step in the present algorithm, so read on...

¹⁸Concerning how to actually arrive at the correct $\boxed{\omega_i^*}$ via the correct $\boxed{\psi_\Omega}_m$ for a given **IFA MSS**, refer to **Appendix C** that covers how to compute or look-up the necessary **platonic form encoding key**.

$$\boxed{\omega_i^*} = \rho_j \in \psi_{pf} + j \times \omega_i^* : j = \boxed{j} : I(\boxed{j}, \boxed{\psi_{pf}}_m) = I(\omega_i^*, OZ(\overset{\triangleright}{\Theta}_\Omega)) \quad (10.12)$$

The core; **LGES-PFA** component resolution protocol; which, as we shall be using in other parts of the literature, we might also write using **PFA component configuration notation** as such:

$$\boxed{\omega_i^*} = (\rho_j \in \psi_{pf}) + j \times (\omega_i^* \in \psi_{oz}) = j_{pf} \otimes (\omega_i^*)_{oz} \quad (10.13)$$

To mean, we express $\boxed{\omega_i^*}$ by writing the j -th platonic form it corresponds to, with the associated ozin term ω_i^* attached to it j -times.

A. **PLACE** that encoded version, $\boxed{\omega_i^*}$, along the existing **PFA** structure at the next vacant position ($\mathfrak{v}(\Theta_\Omega) + i$). Expressible and approached as we did in **Equation 10.11**, updating/extending and overriding the current **PFA** as:

$$\boxed{OZ(\Theta_\Omega)} = \boxed{OZ(\Theta_\Omega)} \cdot \boxed{\omega_i^*} \quad (10.14)$$

Where the special **PFA** term, $\boxed{\omega_i^*}$, as per **Equation 10.12**, is basically the **PLATONIC Form** corresponding to position j in $\boxed{\psi_{pf}}_m$ — an **anagrammatized pf-symbol set** relative to m and which is used to look-up the **correct indices** for the **corresponding** geometric structure components from ψ_{pf} , the symbol set of the final form of the **PFA**, but also with the twist that the intermediate form ω_i^* is attached to that geometric structure j times — as depicted in example¹⁹ **Figure 10.8**, with each expression of ω_i^* attached to one of the j **appendage spots** on the shape for $\boxed{\omega_i^*}$.

B. **PROCEED** to the next symbol in **IFA MSS**, $OZ(\overset{\triangleright}{\Theta}_\Omega)$ (jump to **Step#2(e)**).

¹⁹It shall be interesting to note (but also a great, quick illustration), that based on the **PFEK** by **SLSA** — see **Table C.1**, that assuming we had [an actually *strange*] organism whose equivalent Ω -encoded DNA sequence were just the sequence $\Theta = \mathbf{666}$, its corresponding **IFA** characteristic would be just $h(\overset{\triangleright}{\Theta}) = \mathbf{63}$, the corresponding **MSS** just $\langle \mathbf{6} \rangle$, and so its **PFEK** look-up key, $m = \mathfrak{v}(\overset{\triangleright}{\Theta}) = 1$, so that, its corresponding $\boxed{\psi_{pf}}_1 = \langle \mathbf{4} \rangle$, and so that for its **BODY** component in the associated full genome expression, the entire **PFA** would just be $\boxed{\omega_1^*} \Rightarrow 4_{pf} \otimes 6_{oz}$ — which is exactly what we see in this picture in **Figure 10.8**!

(f) Now that the **PFA** is complete, **UPDATE** the **FGE** as such:

$$\mathbf{FGE} = \mathbf{IFA} \cdot \mathbf{PFA} \quad \Leftrightarrow \quad \boxed{OZ(\Theta_\Omega)}^* = OZ(\Theta_\Omega) \cdot \boxed{\omega}_{joint} \cdot \boxed{OZ(\Theta_\Omega)} \quad (10.15)$$

3. **TRUNCATE** the final-structure after the final term in **FGE**.
4. **RETURN FGE**, $\boxed{OZ(\Theta_\Omega)}^*$ as the final transcribed and translated version of Θ_Ω . The culmination of our conceptual genome expression process.

□

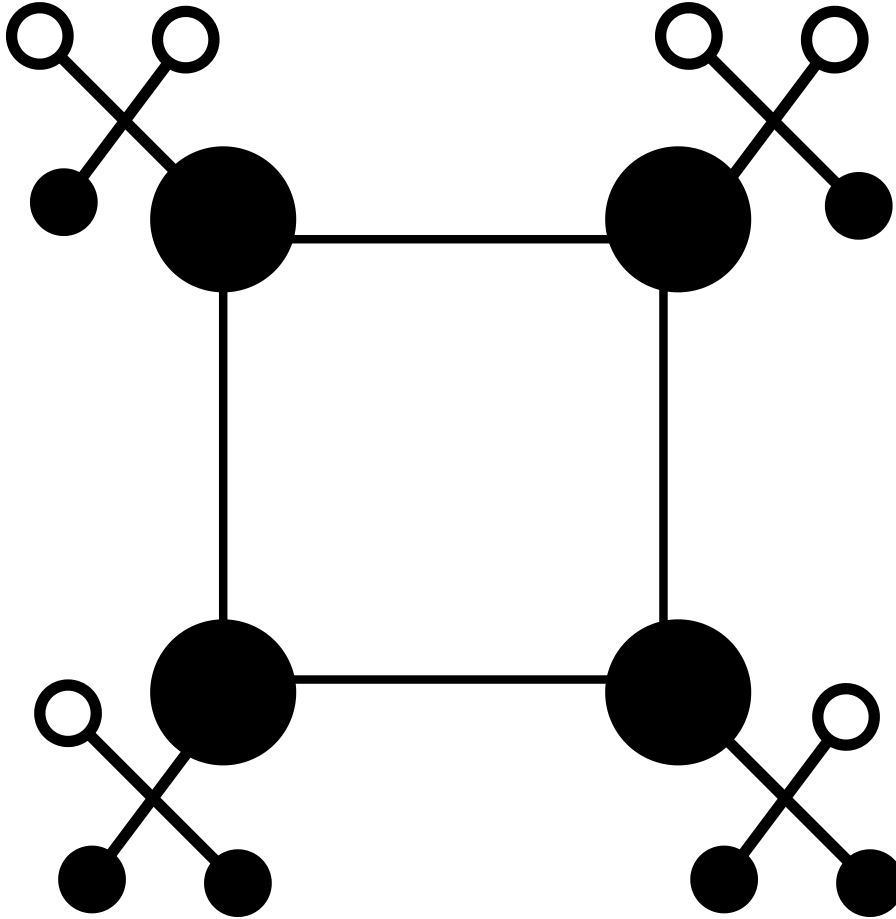


Figure 10.8: An example of a single **amino-acid** or rather, the **PFA** component of a **FGE** equivalent to one **amino-acid** segment expressed using the **LGES** system. This particular expression corresponding to a rendering of the configuration $4_{pf} \otimes 6_{oz} = 4_{pf} + 4 \times 6_{oz}$.

After looking at some examples of actual genome expressions rendered using

the **LGES**, perhaps most reviewers of **Algorithm 6** shall come to appreciate that it is not only non-trivial, but is likewise plausible given the sensibilities of how actual expression occurs in nature/reality²⁰ — see a natural expression as in **Figure 1.1**, and that it encapsulates the ideals of both distinctiveness but also diversity for the minutest of organic creatures that are just as basic as mere genetic code with a basic encapsulating body (such as viruses), but also complex organisms belonging to distinct families of prokaryotes and eukaryotes.

10.2.3 Examples of Complete Lu-Genome Expressions

We get to see the first example of the output of processing some na-Sequence expressed originally in base- Ω such as for Ω_3 from **Equation 10.4**, in its final expression form as in **Figure 10.9**[55] — more importantly, since we have already seen its **IFA** (the **FGE** prefix) in **Figure 10.5**, we note that, we essentially see the update as the result of expressing the entire **FGE** via the **IFA**, **IFA MSS** and then corresponding **PFA** as in the following transformativ derivation

$$\begin{aligned}
 &\textbf{Transformation 22. } \Omega_3 = \langle 0, 1, 2, 3, 0, 2, 6 \rangle \xrightarrow{>} \tilde{\Omega}_3 = \langle 0, 2, 1, 3, 6 \rangle \\
 &\implies m = \mathcal{U}(\tilde{\Omega}_3) = 5 \implies \boxed{\psi_{pf}}_5 = \langle 2, 1, 0, 6, 8 \rangle \\
 &\implies \boxed{OZ(\Omega_3)} = \langle 2_{pf} \otimes 0_{oz}, 1_{pf} \otimes 2_{oz}, 0_{pf} \otimes 1_{oz}, 6_{pf} \otimes 3_{oz}, 8_{pf} \otimes 6_{oz} \rangle \\
 &\approx \langle GAP, 1_{pf} \otimes 2_{oz}, GAP, 6_{pf} \otimes 3_{oz}, 8_{pf} \otimes 6_{oz} \rangle \\
 &\implies \boxed{OZ(\Omega_3)}^* = \langle 0, 1, 2, 3, 0, 2, 6 \rangle_{oz} \cdot \langle GAP, 1_{pf} \otimes 2_{oz}, GAP, 6_{pf} \otimes 3_{oz}, 8_{pf} \otimes 6_{oz} \rangle
 \end{aligned}$$

The corresponding complete rendered **FGE** for Ω_3 would then be as shown in **Figure 10.9**.

²⁰**FUTURE WORK:** That said, it shall be up to others — botanists, zoologists, molecular biologists, virologists, etc. to either confirm or refute whether or not **LGES** does indeed echo the reality in nature or whether it is a theory only applicable to a subsection of reality — *which, if not all?*

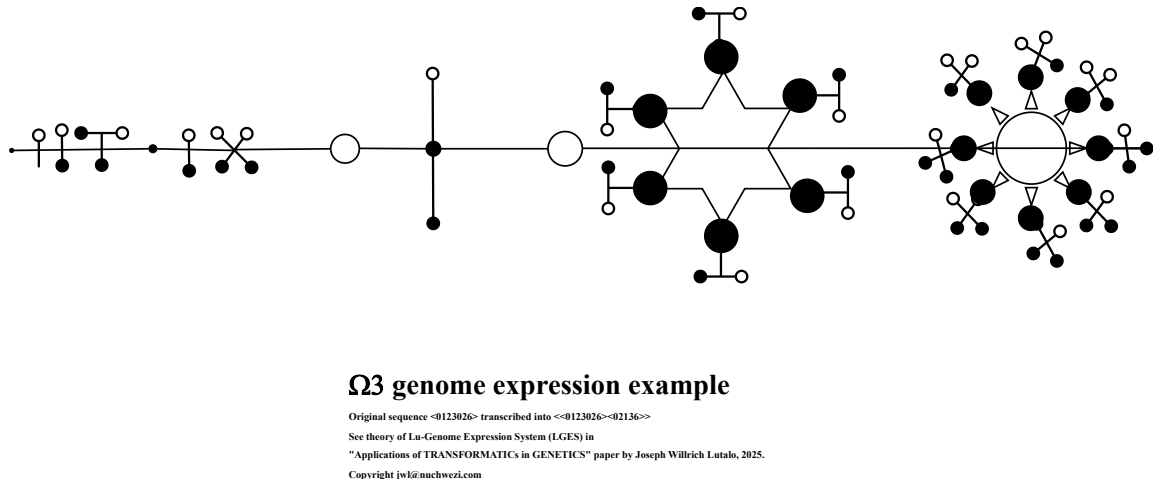


Figure 10.9: The equivalent LGES complete genome expression of the Ω_3 genome sequence (from **Equation 10.4**)

Though we won't delve into exploring all kinds of possible *interesting* scenarios with regards to the **LGES**, we can look at one other example and then move on. Also, it shall be interesting and worth noting — especially for those who wish to explore the **LGES** algorithm, that a mathematical derivation such as depicted in **Transformation 22**, captures the gist of all there is to **LGES**, and that any other case can merely adapt such a formalism so as to arrive at the visual-spatial expression of some Ω -encoded genetic or arbitrary sequence.

From the example hypothetical genome sequences we have encountered, perhaps let us revisit **Equation 10.2** and see what the **Euler Virus** might look like when fully expressed in our system.

So, first of all, note that we already did the first-leg of **Algorithm 6**, and we have seen what the **FGE** prefix — the **IFA**, would look like in **Figure 10.3**²¹. So, like we did for Ω_3 in the last exploration, let us start by looking at what the numeral-equivalence of the LGES-processed Ω_{veuler} would be like. For brevity, we can sum up the entire production/transformation that leads to the complete **FGE** with correct prefix and suffix, and thus $\boxed{OZ(\Omega_{veuler})}^*$ as:

²¹**FUTURE WORK:** Talking of which, for the well-read reader, and especially students and explorers of not just esoteric languages but especially occult languages and systems — such as one finds in **Occult Philosophy**[56], not only the coincidence that our OZIN-expressions look indeed *occult* — first, nothing to do with any remote association with classic children's occult stories such as *Wizard of Oz*, but also because of what one might find utilized in occult systems such as John Dee's Enochian[54][30]. And then there's the peculiarities one might identify with both the **IFA** and **PFA** expression method being used here — the later, expressed in a somewhat more familiar language (because the polygons at the core of the PLATONIC-FORM-cipher are indeed commonplace in both mathematics and mysticism), but ultimately, the experienced observer might appreciate that complete **LGES** expressions/figures such as **Figure 10.9** look much like the traditional/ancient mystical glyphs known as **Mandalas** (*oriental esotericism*) to modern scholars such as Carl G. Jung that enjoyed and advocated for these kinds of inquiries[57] and perhaps, as the author himself has explored in works such as the novel "Rock 'N' Draw"[58], these figures so much remind one of **Voodoo Veves** — in that novel, depicted as enchantment geomantic/geomancy forms etched or drawn upon rocks or on the bare ground as in child-play, but which, upon interaction with a conscious mind, are capable of causing altered states of consciousness! Perhaps, in a later work on either philosophy or mysticism by the author (see earlier works of this kind in [59], but also [6] and esp. [30]) we shall want to take time off and properly explore this exciting dimension of human expression and culture, especially since, unlike what might have ever come before, our symbolic programs, unlike voodoo veves (*african esotericism*) or chaos magick sigils (*western esotericism*), are rooted in two objective fields of inquiry — mathematics and biology (and not mere speculation, divine inspiration nor mere aesthetics) — thus we might even start talking of **genetic veves**. And no, though the acronym **IFA** reminds one of the *Ifa* esoteric system of Western Africa, and yet, the author didn't have that in mind while working on the LGES at first.

$$\begin{aligned}
\textbf{Transformation 23. } \Omega_{veuler} &= \langle 2, 7, 1, 8, 2, 8, 1, 8 \rangle \rightarrow \hbar(\Omega_{veuler}) = \mathbf{83221271} \rightarrow \\
\Omega_{veuler} &= \langle 8, 2, 1, 7 \rangle \implies m = \mathfrak{u}(\Omega_{veuler}) = 4 \implies \boxed{\psi_{pf}}_4 = \langle 1, 6, 4, 7 \rangle \\
\implies \boxed{OZ(\Omega_{veuler})} &= \langle 1_{pf} \otimes 8_{oz}, 6_{pf} \otimes 2_{oz}, 4_{pf} \otimes 1_{oz}, 7_{pf} \otimes 7_{oz} \rangle \\
\implies \boxed{OZ(\Omega_{veuler})}^* &= \langle 2, 7, 1, 8, 2, 8, 1, 8 \rangle_{oz} \cdot \langle 1_{pf} \otimes 8_{oz}, 6_{pf} \otimes 2_{oz}, 4_{pf} \otimes 1_{oz}, 7_{pf} \otimes 7_{oz} \rangle
\end{aligned}$$

Note that, in the derivation depicted in **Transformation 23**, one of the most tricky parts is computing that FGE suffix, $\boxed{OZ(\Omega_{veuler})}$, the **PFA** — which is formalized/defined by *especially*²² **Equation 10.12**. So, once that is done, and we have our final numeral-**FGE** as such:

$$\boxed{OZ(\Omega_{veuler})}^* = \langle 2, 7, 1, 8, 2, 8, 1, 8 \rangle_{oz} \cdot \langle 1_{pf} \otimes 8_{oz}, 6_{pf} \otimes 2_{oz}, 4_{pf} \otimes 1_{oz}, 7_{pf} \otimes 7_{oz} \rangle \quad (10.16)$$

Of course, concerning **Equation 10.16**, note that, or rather, keep in mind that concerning the **PFA** component, $\boxed{OZ(\Omega_{veuler})}$, the corresponding associated sequence in numeral form is the **IFA MSS**, $OZ(\Omega_{veuler}) = \Omega_{veuler} = \langle 8, 2, 1, 7 \rangle$. And thus, using **the necessary** transcription and translation ciphers as well as the multiplication/linear-combination rules required by **Equation 10.12** when treating of the **PFA**, we then generate the final genome expression for the **Euler Virus** under the **LGES** that looks as in **Figure 10.10**.

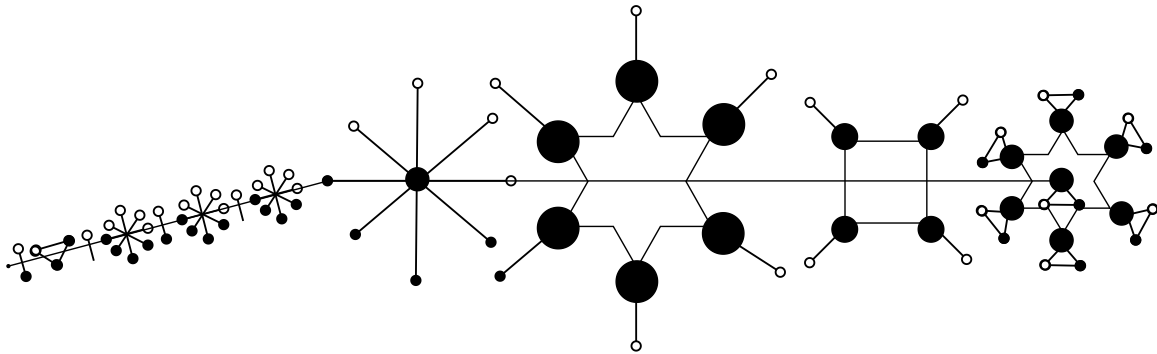


Figure 10.10: The equivalent LGES complete genome expression of the Ω_{veuler} (Euler Virus) genome sequence (from **Equation 10.16**)

Now, concerning that rendering of the genome sequence expressed in **Equation 10.16**, it might be worth noting that unlike the earlier examples of **LGES**

²²**Equation 10.11** is what covers the special symbol '0', that is treated as a JOINT/GAP in the **PFA** and as STOP/GAP in **IFA**.

expressions, we have taken the liberty to exploit that special joint that kick-starts the **PFA**, or rather, which separates the **IFA** from the **PFA** in a **FGE** expression. So, just like an organism with a physical joint might sometimes appear as angled or curved, so we have chosen to depict the Ω_{veuler} with its one joint²³ well utilized. However, and important to note; it is not just joints that might appear different across different renderings of the same basis genome sequence. So, in **Figure 10.11**, we see another rendering of the same genome sequence expression, with readily noticeable differences being the IFA-PFA joint in its neutral position (we also simplified and fused it with one of the edge-nodes of the $1_{pf} \otimes 8_{oz}$ term in both diagrams), and then the second term in the PFA (corresponding to $7_{pf} \otimes 2_{oz}$ or rather $7_{pf} + 7 \times 2_{oz}$) with the 2_{oz} appendages depicted mostly aligned horizontally — basically, there's still room for some creativity while rendering, but perhaps the future and more research might help decide on what is optimal/ideal. At the moment, it is mostly the mathematics, a sane dose of and principles of aesthetics and design determining much of the final outputs. But also, trusting that in nature, systems are at their *most basic form*, when in states of lowest kinetic energy or highest potential energy²⁴.

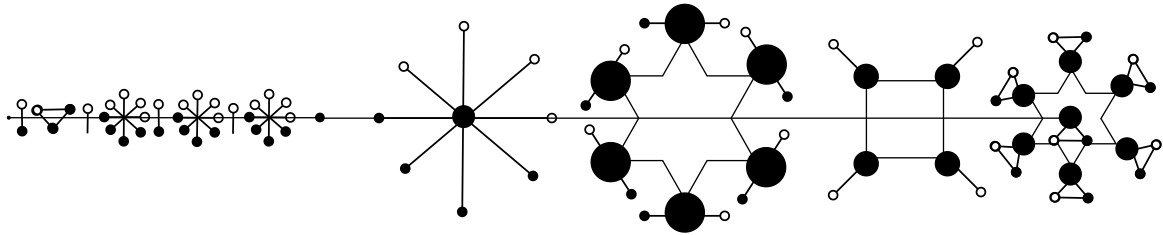


Figure 10.11: An **alternative rendering** of the same sequence as Ω_{veuler} originally depicted as the **LGES** genome expression in **Figure 10.10**

And talking of scaling nodes, in the future, we might want to explore whether in such a system as **LGES** in which the expression is sensitive to measures such as the **IFA MSS**, whether it might make sense to scale each subsequent pf-node in the **PFA** based on its position within the sequence? Though we didn't have such a rule included in **Algorithm 6**, and yet, our example renderings here have somewhat taken advantage of that plausible logic.

That said, note that, we started this work with a photograph of a pineapple ready to harvest (see **Figure 1.1**), and though the associated discussion²⁵ did bring up the matter of how it might inspire a genome expression system in which the prefix is the actual DNA that renders the rest of the organism (fruit, plant-

²³Definitely, and interesting to explore, a genome sequence with multiple **JOINT** points in it might be how an organism with say multiple appendages such as legs, hands and a head that can move independently of the core/thorax of the body is possibly arrived at. The interested student should go ahead and explore more non-trivial Ω -base sequences just to see what's possible.

²⁴Such as, not finding trees that grow with their stems at an angle to the ground or animals with curved legs or walking on 3 legs or with the legs all at the back — for 4-legged animals, etc... Thus, though I didn't cite any authority on this matter (except perhaps nature), I trust that basic physics applied to anatomy and physiology might help justify the sensibilities we only seem to be applying from basic understanding of physics and nature here — *natural axioms* and *instincts* perhaps.

²⁵Checkout video/mini-lecture concerning this particular organism and what might be gleaned about genetic expressions from it, via author's YouTube Channel: <https://youtube.com/shorts/yTJzBEpw7Z4>

base and roots in this case of the pineapple), it is left as an exercise to the reader to decide whether example genome sequence expressions/renderings such as in **Figure 10.10** depict this pineapple-inspired philosophy correctly.

NOTE:**On Why Place-Notation Expression Systems Matter**

As a further justification for why we chose to use a place-notation sensitive genetic code expression system in our thought experiment, consider that, just like with modern numbers and number systems — of which the Arabic number system is considered one of the best since ancient times, the occurrence of a symbol within an expression is interpreted not just by what it quantifies, but also by where in the expression it is situated.

For good measure, a few enlightening quotes from the priceless work on the evolution of number systems since ancient times, by Raymond S. Nickerson, shall help cement this ideology:

Aristotle attached considerable significance to the fact that human beings can count: it is this ability, he claimed, that demonstrates our rationality.

As Brainerd (1973) has pointed out, in most Western nations today we expect students, by the time they reach their early teens, to be much more competent with numbers than was an educated adult Greek or Roman of 2,000 years ago. This fact arises in no small measure from the elegance and power of the scheme that we now use to represent numbers.

In short, a price of the increased abstractness of the Arabic system is an obscuring of some of the key principles on which numbers are based. To an ancient Egyptian, the fact that $3+4=7$ was apparent from the relationship between the Egyptian number representing 7 and those representing 3 and 4. There is no hint of such a relationship, however, when this equation is expressed in Arabic notation. The price has been worth the paying, however.

If you invented a unique symbol to represent each of the possible quantities of stones (including the case of no stones) up to that of your standard and decided to use the position of that symbol to indicate the pile to which it refers, you would have what we refer to today as a place-notation system for representing numbers.

The relative ease with which one can count or compute depends to no small degree on the characteristics of the system that is used to represent number concepts.

—Raymond S. Nickerson, *Counting, Computing, and the Representation of Numbers*, 1988[60]

And finally, it shall be worth noting that this notion also underlies the present author's use of the **Position-Index measure** ($I(\omega_i, \Theta)$), first formalized in the Base-36 paper[61], and which, among many important applications it has, is its use in generalizing the computation of decimal values of arbitrary number expressions in any base, its use in transformatics to express sequence summaries using modal sequences[5], etc.

10.3 The Extended Lu-Genome Expression System (x-LGES)

If you look at the genome sequence expressions that we have encountered in **Section 10.2.3**, and compare that to the case of our pineapple in **Figure 1.1** that opens the introductory chapter, you shall notice that, just in case we assume our **LGES** protocols to be sufficient for the expression of not just bits of an organism, but also relevant to the expression of **the complete organism**, then, it might seem like something is still missing, and thus the **Extended Lu-Genome Expression System** (x-LGES).

So, what exactly is missing from our genome expressions thus far? We shall call out a few telling points to help drive the point home:

1. As has been stressed already, and as is the case that we wish our science to not be constructed in ivory towers alienated from facts and reality in nature, we must borrow a leaf from how actual DNA is utilized to express organisms in reality. For example, it is not that one genome expresses the hands, another the heart, and yet another the eyes! Well, of course, in a particular organism's DNA, there shall be specific subsequences that best define the expression of particular features or aspects of an organism, however, overall, a single, complete genome sequence is and should be sufficient for the complete expression of an entire organism with all its various attributes and distinct aspects well catered for.
2. And then, returning to nature as our authority on genome expression, we note that, most organisms — plants or animals, typically manifest in a **PREFIX-THORAX-SUFFIX/HEAD-BODY-ROOT** form. Looking at our **LGES** renderings in **Section 10.2.3**, we see that the **IFA** aspect properly caters for the “PREFIX/HEAD” component — it somewhat contains the entire [genome] sequence just encoded in base-OZ²⁶. Then we have the “THORAX/BODY” bit. It feels plausible to equate that with the protracted spatial-expressions of the **PFA**. But what of the “SUFFIX/ROOT” component? And talking of suffixes in natural genome expressions; we can appreciate that for humans, birds and most animals, we can talk of **legs**; for aquatic animals especially, such as fish, we might talk of **tail-fins**; for insects and all various kinds of arthropods, we can not only talk of a body form that's generally made up of chained bits or adjoined segments such as in our **LGES**, but that they generally terminate in a kind of suffix we might consider to be **an abdomen**. And then finally, for plants, it's obvious they typically terminate in **roots** — and especially for the case of food-crops, we find that these roots contain sufficient *bio-information* to allow us to reproduce the original

²⁶Interesting to say, “the brains” of the expressed organism, especially since it has the complete genome almost unaltered, and in its *actionable form*.

plant organism from just those suffixes²⁷.

So, with those observations and criticisms of the original **LGES**, we can provide a solution in form of the following algorithm that extends **Algorithm 6**:

10.3.1 ψ_{Ω} -to- $(\psi_{oz}$ - ψ_{pf} - $\psi_{oz})$: The **x-LGES** Algorithm

To render a complete genome expression using this extended algorithm, and starting from the original genome sequence encoded as a base- Ω expression, we proceed as such:

Algorithm 7 (The **x-LGES** Algorithm). *Assuming we have an Ω -base encoded genetic code sequence Θ_{Ω} of some length $n > 0$.*

1. **INITIALIZE** the final expression by constructing the initial two parts of the **FGE** using **Algorithm 6** to construct the **Intermediate Form Affix (IFA)** of Θ_{Ω} and the **Platonic Form Aspect (PFA)** — so that, at the end of this step, the **FGE** thus far shall be as:

$$\boxed{OZ(\Theta_{\Omega})}^* = OZ(\Theta_{\Omega}) \cdot \boxed{\omega}_{joint} \cdot \boxed{OZ(\Theta_{\Omega})} \quad (10.17)$$

2. **EXTEND** that initial **FGE** with just a **single GAP/JOINT extension**, so that we then have the **FGE** as such:

$$\boxed{OZ(\Theta_{\Omega})}^* = \boxed{OZ(\Theta_{\Omega})}^* \cdot \boxed{\omega_i^*}_{joint} = OZ(\Theta_{\Omega}) \cdot \boxed{\omega}_{joint} \cdot \boxed{OZ(\Theta_{\Omega})} \cdot \boxed{\omega_i^*}_{joint} \quad (10.18)$$

3. Then, **USE** the **IFA MSS** from **Step#2(b)/Algorithm 6**, that is $OZ(\Theta_{\Omega})^>$, as the **Genome Expression Suffix (GXS)** to extend the current **FGE** so that in simplified form, we have:

$$\boxed{OZ(\Theta_{\Omega})}^* = \mathbf{IFA} \cdot \mathbf{PFA} \cdot \mathbf{GXS} = OZ(\Theta_{\Omega}) \cdot \boxed{OZ(\Theta_{\Omega})} \cdot OZ(\Theta_{\Omega})^> \quad (10.19)$$

²⁷Though not all root-crops might readily be grown from their root tubers, some good examples from observation and experience author has obtained practicing domestic crop-husbandry include irish potatoes (*obutakuli*), yams (*ebihuna*), onions (*obutunguru*), ginger (*entangahuzi*) and (*binyobwa*) ground-nuts! And talking of which, for the aspect of being able to re-grow an organism from its affixes — head/prefix or root/suffix, the interesting case of the sweet-potato, would be a fun case; despite being a root-crop as those mentioned before, attempts to re-grow it say by re-burying a sweet-potato tuber, is so futile! Only way is to take the leaves (prefixes) and partially-bury them in soil, so as to re-grow the complete sweet-potato plant!

Which, if qualified with the necessary encoding bases is as such:

$$\boxed{OZ(\Theta_\Omega)}^* = OZ(\Theta_\Omega)_{oz} \cdot \boxed{OZ(\Theta_\Omega)}_{pf \cup oz} \cdot OZ(\Theta_\Omega)_{oz}^> \quad (10.20)$$

And if we express this as the update to **Equation 10.18**, we have:

$$\boxed{OZ(\Theta_\Omega)}^* = OZ(\Theta_\Omega) \cdot \boxed{\omega}_{joint} \cdot \boxed{OZ(\Theta_\Omega)} \cdot \boxed{\omega_i^*}_{joint} \cdot OZ(\Theta_\Omega)^> \quad (10.21)$$

Which tells us that the final **FGE** is the original **FGE** extended with just one extra **JOINT** and the **GXS/IFA MSS** encoded in base-OZIN.

4. **TRUNCATE** the final-structure after the final term in **FGE**.
5. **RETURN** that final resultant **FGE**, $\boxed{OZ(\Theta_\Omega)}^*$, as the final genome expression projection of Θ_Ω . The culmination of our extended genome expression system.

□

We shall come to appreciate this updated protocol in **Algorithm 7**, by taking a few of the **FGE** examples we looked at earlier, and re-render them using the **x-LGES** to see what difference it makes.

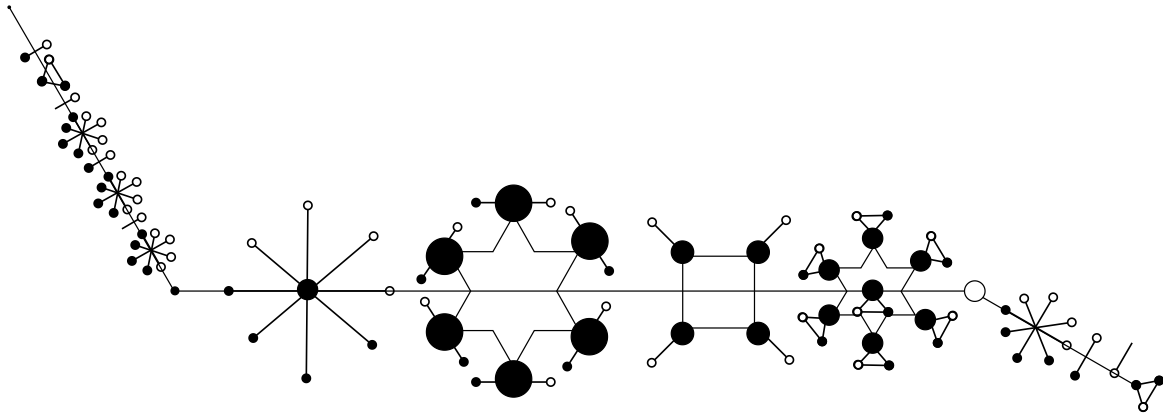


Figure 10.12: The **x-LGES** form rendering of the Ω_{vulner} , the **Euler Virus**, now with **HEAD**, **BODY** and **ROOT** segments.

The first example is with the notorious **Euler Virus** originally defined in **Equation 10.2**, and whose updated genome expression rendering as shown in **Figure 10.12** shall start to paint the new picture for you, if you compare it to

the original OZIN-expression of its equivalent **mRNA** as shown in **Figure 10.3**, and then, that same organism as the basic **LGES**-form in **Figure 10.11** which shows the full genome expression minus the extension **GXS SUFFIX/ROOT**.

10.4 Actual DNA Sequences Expressed in the Extended Lu-Genome Expression System

So, now that we have finished looking at what could happen with genome expression under the hypothetical LGES protocols, one might wonder, and correctly so, how might we use that system to attempt to visualize actual na-Sequences such as DNA or RNA sequences or perhaps just their subsequences?

On Barriers to Entry:

Concerning working with the **LGES** or **x-LGES** as it is at the moment, the entry point barrier is that **the system is well defined for sequences expressed in base-10 only**. That means, unless your sequence is expressed using symbols from ψ_{10} , there is no direct way to leverage the theory we have just presented. So, that's where we need put our attention first, with regards to processing DNA or RNA leveraging our thought experiment — also applies to any other kinds of sequences one might wish/attempt to visually-spatially express using our system.

Good enough, we have had extensive background concerning transformation of sequences across symbol sets in earlier chapters and works by the author[61][1]²⁸, and so, we can just re-iterate some important points here.

- We know that any na-Sequence shall span ψ_{na} , which has cardinality 5 (refer to **Law 1**), and yet, $\mathcal{L}(\psi_{10}) = 10 > \mathcal{L}(\psi_{na})$.
- For either DNA (ψ_{DNA}) or RNA (ψ_{RNA}), their corresponding symbol set cardinalities are just **4** and **4** respectively, which still doesn't map well to the base-10 or rather ψ_{Ω} symbol set that the Lu-Genome Expression System requires.

So, using sequences we have already encountered, such as Θ_4^* in **Equation 10.7** — an RNA sequence, or its original DNA form (as in **Equation 9.9**) how, should we go about expressing it in equivalent decimal or essential Ω -form?

TODO: Continue indexing starting from here

²⁸Also see [62], J. Bossé and William Cook on general base-conversions for numerical expressions (only).

TODO: proceed from here...

10.4.1 Converting Between Nucleic Acid Code and Numeric Codes

First, given the constraints we have seen above, one plausible solution might be to start with a naive²⁹ encoding that merely maps every element in ψ_{DNA} to a corresponding term in ψ_{10} **based not on the quantity of the symbol — since na-Sequences aren't made of 'literal numbers', but rather, based on the position of the symbol within the DNA symbol set.** So that, we have the following basic encoding-transformer:

Transformer 15 (ψ_{DNA} to ψ_{10} **DNA-Decimal Encoder**).

$$\Theta \xrightarrow{O_{dna-dec-encode}(\cdot)} \Theta^*; \quad \forall \omega \in \Theta : I(\omega, \Theta) = k, \exists \rho \in \Theta^* : I(\rho, \Theta^*) = k \implies \rho = I(\omega, \psi_{DNA})$$

And so, using that **Transformer 15**, we can process any DNA sequence and map it to corresponding Ω -form (or rather *decimal form*) as such:

$$\begin{array}{cccc} \left[\begin{array}{c} A \\ \uparrow \\ 1 \end{array} & \begin{array}{c} C \\ \uparrow \\ 2 \end{array} & \begin{array}{c} G \\ \uparrow \\ 3 \end{array} & \begin{array}{c} T \\ \uparrow \\ 4 \end{array} \right] \\ \left[\begin{array}{c} 1 \\ \downarrow \\ A \end{array} & \begin{array}{c} 2 \\ \downarrow \\ C \end{array} & \begin{array}{c} 3 \\ \downarrow \\ G \end{array} & \begin{array}{c} 4 \\ \downarrow \\ T \end{array} \right] \end{array}$$

With that basic mapping, and our example DNA sequence from **Equation 9.9**) we see that the corresponding decimal-encoded DNA sequence shall merely be as in the following transformation:

Transformation 24. $\Theta = \langle A, T, G, A, A, A, T, T, A, T, A, G \rangle$

$$\xrightarrow{O_{dna-dec-encode}(\cdot)} \langle 1, 4, 3, 1, 1, 1, 4, 4, 1, 4, 1, 3 \rangle$$

Essentially, the symbol set for DNA sequences encoded in decimal shall only span the special symbol set we might refer to as **DNA-Digits Symbol Set**, and which we might denote as ψ_{DD} , and which is just:

$$\psi_{DD} = \langle 1, 2, 3, 4 \rangle \quad (10.22)$$

²⁹I think it is naive, because, using the position-index operator $I(\omega, \Theta)$ in the encoding definition, it isn't clear how we shall ever resolve for the digit '0' given DNA has only 4 distinct symbols.

So, with ψ_{DD} and knowing the mapping from DNA to Decimal, we could also work backwards from decimal sequences back into DNA sequences. So, for example, assuming we took the interesting **Hi-Fi o-SSI** from **Equation 10.3**, and wanted to map it to an equivalent DNA sequence, we might proceed by combining the consequences of **Theorem 4** and **Transformer 15**. Thus, we might start by resolving to the following approximate numerical sequence:

Transformation 25.

$$\Omega_{HiFinelle} = \langle 8, 6, 4, 9, 1, 3, 7, 5, 2, 0 \rangle \rightarrow \langle 8, 6, T, 9, A, G, 7, 5, C, 0 \rangle$$

However, as warned earlier on, transformations via **Transformer 15** would be somewhat *weak* when dealing with the reverse case — decimal-to-DNA decoding, especially because not all symbols in ψ_{10} shall have meaningful equivalent terms in ψ_{DNA} . So, better we start by solving that problem instead.

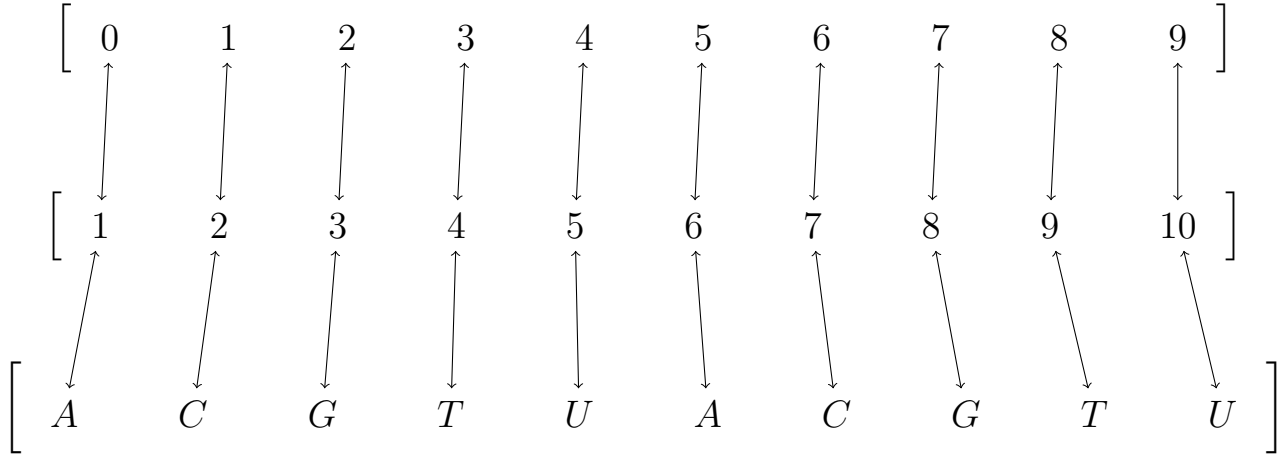
Thus, with some *necessary tweak* — that, instead of mapping from ψ_{DNA} to ψ_{10} and vice-versa, we take ψ_{na} which has exactly 5 elements and which is a superset of ψ_{DNA} ³⁰, and then map any symbol from ψ_{na} to ψ_{10} and vice-versa as per the following non-trivial transformer:

Transformer 16 (ψ_{na} to ψ_{10} na-to-Decimal Decoder).

$$\begin{aligned} &\Theta \xrightarrow{O_{na-dec-decode}(\cdot)} \Theta^*; \\ &\forall \omega_i \in \psi_{na} : I(\omega_i, \psi_{na}) = i \implies \exists d_j \in \psi_{10} : I(d_j, \psi_{10}) = j = i \quad \vee \quad I(d_j, \psi_{10}) = \\ &\quad \perp(\psi_{na}) + i \implies \omega_i \leftrightarrow d_j \\ &\wedge \quad \forall \alpha_k \in \Theta \quad \exists d_k \in \Theta^* : I(\alpha_k, \Theta) = I(d_k, \Theta^*) = k \quad \wedge \quad I(d_k, \psi_{10}) = \\ &\quad I(\alpha_k, \psi_{na}) \vee I(d_k, \psi_{10}) = \perp(\psi_{na}) + I(\alpha_k, \psi_{na}) \implies \alpha_k \leftrightarrow d_k \quad \square \end{aligned}$$

So that, by **Transformer 16**, we arrive at complete decimal-to-DNA and decimal-to-RNA or generally decimal-to-na-Sequence symbol encoding-decoding is as shown in the following diagram:

³⁰Refer to **Definition 4**



Of course, the necessary tweak here, especially for target sequences specific to either DNA or RNA and not both, the tricky terms ‘T’ and ‘U’ need to be dealt with as necessary — possibly with a basic swapping of all ‘U’ with ‘T’ for an na-Sequence targeting a DNA processor, and vice-versa for those targeting strict RNA processors.

Also, that mapping offers us a basic protraction na-Sequence that is a basic multiplication of the ψ_{na} symbol set by 2. Because it maps nicely to terms in ψ_{10} , both sequences having equal length, we could use it in various problems such as in generating random na-Sequences or mapping random base-10 o-SSI sequences to corresponding na-Sequences — or virtually any decimal sequence of any length.

That said, let us set that special sequence aside for later use in the following equation:

$$\Theta_{2na} == \psi_{na} \cdot \psi_{na} = \langle A, C, G, T, U, A, C, G, T, U \rangle \quad (10.23)$$

However, and also related, we might also come to appreciate a related sequence, Θ_{palna} , that is a palindrome derived from ψ_{na} , and definitely, with the same length as ψ_{10} .

$$\Theta_{palna} = \psi_{na} \cdot \neg(\psi_{na}) = \langle A, C, G, T, U, U, T, G, C, A \rangle \quad (10.24)$$

That said, and back to our **x-LGES** problem...

10.4.2 Revisiting Numeric Sequences

Before we proceed with actual na-Sequences, we shall first revisit hypothetical genome sequences expressed in numeric form, and then shall use the experience we garner to handle processing of actual na-Sequences later.

So, returning to that interesting **Hi-Fi o-SSI** sequence — $\Omega_{HiFinelle} = 8649137520$, only partially translated into DNA-code in **Transformation 25**, if we are to convert it to an equivalent na-Sequence using the new information we have, we would obtain the results:

Transformation 26. $\Omega_{HiFinelle} = \langle 8, 6, 4, 9, 1, 3, 7, 5, 2, 0 \rangle \xrightarrow{O_{na-dec-decode}(\cdot)} \langle T, C, U, U, C, T, G, A, G, A \rangle = TCUUCTGAGA$

Which, if we wish to constrain it to just the DNA symbol set, would be as:

Transformation 27. $\Omega_{HiFinelle} = 8649137520 \xrightarrow{O_{na-dec-decode}(\cdot)} TCUUCTGAGA \xrightarrow{O_{mRNA-encode}(\cdot)} UCUUCUGAGA \xrightarrow{O_{DNA-encode}(\cdot)} TCTTCTGAGA$

And that last transformation was made possible by chaining several transformers, some of which we have already encountered in earlier discussions, so that the final sequence is strictly DNA-encoded. Thus, our $\Omega_{HiFinelle}$ can be mapped from base- Ω into base-DNA and vice-versa as we have seen. These ideas can definitely then be used to process any other numeric sequence when there is need to map it to or treat it as a legitimate na-Sequence.

However, for the case of our gene expression system, we really don't care or need the na-Sequence expressions to begin with. And so, given we already have the numeral equivalent of $\Omega_{HiFinelle}$, we can directly advance to rendering the organism that might ensue from processing it via **x-LGES**.

Note that, for this particular sequence, we had already dealt with its **IFA** prefix encoded in base-OZIN — see **Figure 10.4**. Thus, proceeding via a meaningful **transformatic-distillation** such as we did for the Euler Virus in **Transformation 23**

Transformation 28. $\Omega_{HiFinelle} = \langle 8, 6, 4, 9, 1, 3, 7, 5, 2, 0 \rangle \rightarrow \overset{>}{h}(\Omega_{HiFinelle}) = \overset{>}{81614191113171512101} \rightarrow \Omega_{HiFinelle} = \langle 8, 6, 4, 9, 1, 3, 7, 5, 2, 0 \rangle$
 $\Rightarrow m = \mathfrak{u}(\Omega_{HiFinelle}) = 10 \Rightarrow \boxed{\psi_{pf}}_{10} = \langle 6, 3, 8, 2, 1, 9, 7, 4, 0, 5 \rangle$
 $\Rightarrow \boxed{OZ(\Omega_{HiFinelle})} = \langle 6_{pf} \otimes 8_{oz}, 3_{pf} \otimes 6_{oz}, 8_{pf} \otimes 4_{oz}, 2_{pf} \otimes 9_{oz}, 1_{pf} \otimes 1_{oz}, 9_{pf} \otimes 3_{oz}, 7_{pf} \otimes 7_{oz}, 4_{pf} \otimes 5_{oz}, 0_{pf} \otimes 2_{oz}, 5_{pf} \otimes 0_{oz} \rangle$
 $\Rightarrow \boxed{OZ(\Omega_{HiFinelle})}^* = \langle 8, 6, 4, 9, 1, 3, 7, 5, 2, 0 \rangle_{oz} \cdot \langle 6_{pf} \otimes 8_{oz}, 3_{pf} \otimes 6_{oz}, 8_{pf} \otimes 4_{oz}, 2_{pf} \otimes 9_{oz}, 1_{pf} \otimes 1_{oz}, 9_{pf} \otimes 3_{oz}, 7_{pf} \otimes 7_{oz}, 4_{pf} \otimes 5_{oz}, 0_{pf} \otimes 2_{oz}, 5_{pf} \otimes 0_{oz} \rangle \cdot \langle 8, 6, 4, 9, 1, 3, 7, 5, 2, 0 \rangle_{oz}$

And without actually wasting time or space rendering the visual equivalence of that organism, among interesting things to note from **Transformation 28** is that unlike in **Transformation 23**, we have applied the extended LGES algorithm — **Algorithm 7**, so that the final expression in that transformation specifies the HEAD, BODY and ROOT of the organism. Also, and interesting to

note, that despite this particular sequence having no exciting **IFA MSS** — since all the symbols in the original sequence have frequency 1 (which is why we see the organism’s HEAD and ROOT looking exactly the same³¹ *in numeric form*), though, because of the consequences of how the BODY component is generated, we see a potential mid-BODY-JOINT — **PFA** node $0_{pf} \otimes 2_{oz}$, that would definitely make the corresponding **FGE** interesting to look at. Also, given 0_{oz} maps to “STOP/GAP”, the final term in the **PFA**, $5_{pf} \otimes 0_{oz}$, would potentially look something like **a hand** perhaps — it is a geometric structure of exactly 5 vertices (just like on a human palm), and like it, with *discontinuities* at the appendage points/vertices³².



Figure 10.13: An example of an actual organism from nature — an Earthworm, for which, like the x-LGES rendering of the sequence $\Omega_{HiFinelle}$, results in a genome expression for a creature that almost looks the same at the HEAD and ROOT. Image sourced from Wikimedia Commons[14].

10.4.3 x-LGES applied to named DNA Sequences

Now that we have attained sufficient experience with expressing pure numeric sequences, we can return to actual na-Sequences, and checkout some potentially enlightening scenarios.

First, we shall want to have a way to assign meaningful names to our sequences.

³¹I won’t show many pictures here except perhaps **Figure 10.13**, but some kinds of **millipedes** and **centipedes** have this strange symmetrical anatomy that makes it difficult to distinguish the head from the tail. Earthworms too! Someone suggested the concept of the *Ouroboros* from esoteric folklore, that exhibits a kind of self-referential form, however, in my local folklore and vocabulary, we have a peculiar snake-like creature known as *namunungu* that exhibits this peculiar form too.

³²Jokes aside, normally in esoterica, *magical wands* are depicted as being a rod that terminates in a 5-vertex shape — typically a pentagram, so a creative mind might start to think of the $\Omega_{HiFinelle}$ creature as a potentially **magical [wand-like] creature!**

For example, the sequence from **Transformation 24**. Instead of just calling it Θ , we could adapt a name from its corresponding na-Sequence, and then use that to distinguish it from other na-Sequences we shall want to compare it against.

Without caring whether it actually has a standard or conventional name based on its actual na-composition as depicted, we could adopt a system such as picking the first letters of its contained codons in the order they appear in the sequence, and then assign it a name based on the resulting acronym. So, we for example know from **Table 9.1** originally associated with that sequence, that the corresponding codons by the code-names (also see **Equation 9.8**) are as follows:

$$\Theta = \langle Met, Lys, Leu, Amber \rangle \quad (10.25)$$

So that we can just obtain a name for the sequence using the above suggested strategy as such:

Transformation 29. $\langle Met, Lys, Leu, Amber \rangle \rightarrow \langle M, L, L, A \rangle \rightarrow MLLA$

Thus, we might formally refer to our gene-program with a proper name as Θ_{MLLA} or just the **MLLA**-gene program — where no confusion might arise, we shall just refer to it as Θ_{ml} .

So, how would this program's corresponding sequence appear if it were expressed in the **x-LGES**?

We have its corresponding Ω -form expression in **Transformation 24**, so that³³

$$\Theta_{ml} = \langle 1, 4, 3, 1, 1, 1, 4, 4, 1, 4, 1, 3 \rangle \quad (10.26)$$

From that, we can leverage the pre-rendering calculus we have already encountered, well-applied in **Transformation 28**, and so that we can then readily apply **Algorithm 7** to Θ_{ml} . The necessary derivation of the corresponding genome expression shall be as in:

³³**NOTE** that, when encoding from na-Sequences constrained to just ψ_{DNA} or ψ_{RNA} , then, the corresponding Ω -base sequences shall always only span ψ_{DD} (see **Equation 10.22**), but this need not be the case when mapping from ψ_{na} to ψ_{Ω} .

Transformation 30. *First, we wish to show the na-Sequence-to-Ω preamble leading us into how LGES would be applied to a sequence originally expressed as DNA code:*

$$\begin{aligned} \Theta &= \langle \langle A, T, G \rangle, \langle A, A, A \rangle, \langle T, T, A \rangle, \langle T, A, G \rangle \rangle \xrightarrow{\text{flatten}} \\ \langle A, T, G, A, A, A, T, T, A, T, A, G \rangle &\xrightarrow{O_{dna-dec-encode}} \langle 1, 4, 3, 1, 1, 1, 4, 4, 1, 4, 1, 3 \rangle = \\ \Theta_{ml} \end{aligned}$$

So that then, the rest of the LGES derivation is as usual...

$$\begin{aligned} \Theta_{ml} = \langle 1, 4, 3, 1, 1, 1, 4, 4, 1, 4, 1, 3 \rangle &\rightarrow \hbar(\overset{\triangleright}{\Theta_{ml}}) = \mathbf{164432} \rightarrow \overset{\triangleright}{\Theta_{ml}} = \langle 1, 4, 3 \rangle \implies \\ m = \varkappa(\overset{\triangleright}{\Theta_{ml}}) = 3 &\implies \boxed{\psi_{pf}}_3 = \langle 7, 5, 3 \rangle \\ \implies \boxed{OZ(\Theta_{ml})} &= \langle 7_{pf} \otimes 1_{oz}, 5_{pf} \otimes 4_{oz}, 3_{pf} \otimes 3_{oz} \rangle \\ \implies \boxed{OZ(\Theta_{ml})}^* &= \langle 1, 4, 3, 1, 1, 1, 4, 4, 1, 4, 1, 3 \rangle_{oz} \cdot \langle 7_{pf} \otimes 1_{oz}, 5_{pf} \otimes 4_{oz}, 3_{pf} \otimes 3_{oz} \rangle \cdot \\ \langle 1, 4, 3 \rangle_{oz} \end{aligned}$$

FUTURE WORK: *There is perhaps one **critical** quirk worth noting concerning such derivations as this: in cases when the underling na-Sequence contained some non-coding codons (introns) such as the final sequence $\langle T, A, G \rangle$ in this case, should we eliminate such from the LGES input sequence when converting from na-Sequence into base-Ω or should we not? This is important because, as one can tell when studying the following LGES derivation preamble, the final IFA and consequently the entire FGE would be altered if we had kicked out some of the original na-Sequence components:*

$$\begin{aligned} \Theta &= \langle \langle A, T, G \rangle, \langle A, A, A \rangle, \langle T, T, A \rangle, \langle T, A, G \rangle \rangle \xrightarrow{\text{exclude_introns}} \langle \langle A, T, G \rangle, \langle A, A, A \rangle, \langle T, T, A \rangle \rangle \xrightarrow{\text{flatten}} \\ \langle A, T, G, A, A, A, T, T, A \rangle &\xrightarrow{O_{dna-dec-encode}} \langle 1, 4, 3, 1, 1, 1, 4, 4, 1 \rangle = \Theta_{ml}^* \neq \Theta_{ml} \end{aligned}$$

And the corresponding, fully rendered **FGE** would be as depicted in **Figure 10.14** hereafter.

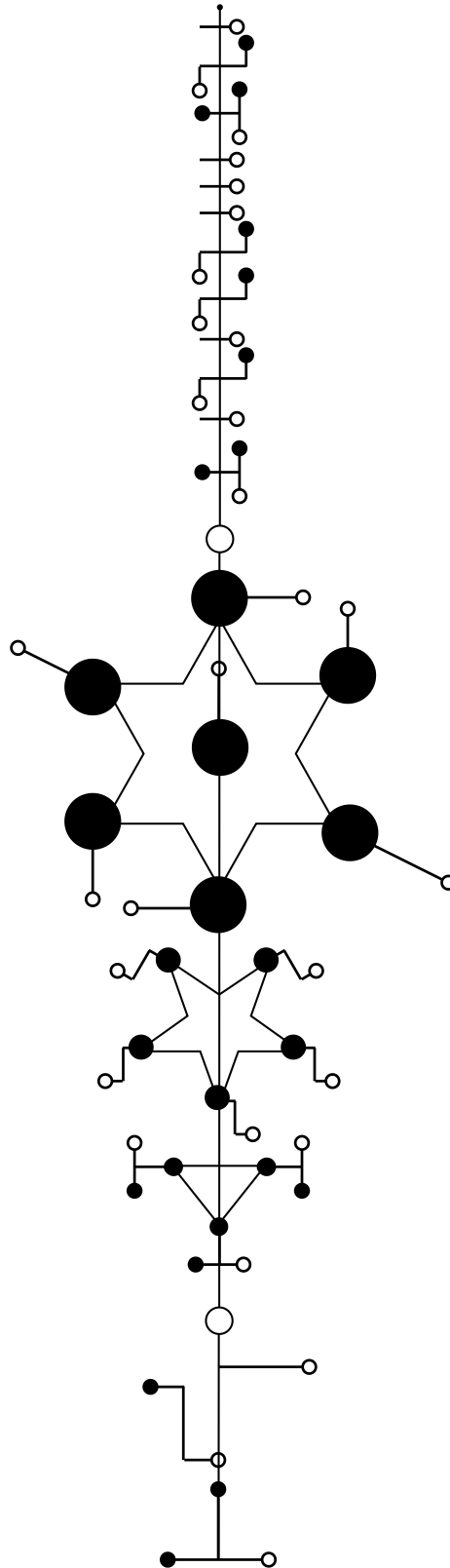


Figure 10.14: The **x-LGES form rendering** of the sequence Θ_{mlla} , depicting its complete **FGE** with **HEAD**, **BODY** and **ROOT** segments.

Thus we conclude our treatment of **TRANSFORMATICS** in **GENETICS**.

Chapter 11

Conclusion

In this book¹, we have advanced our knowledge concerning the important and fundamental subject of genetics with new theory and practical ideas previously only held informally or totally unknown. We have explored several useful and interesting ways by which the newly proposed mathematical field of inquiry, known as **TRANSFORMATICS** — potentially a sub-field of/fusion of mathematical statistics and linear algebra², might be applied in the sub-field of the biological sciences known as **GENETICS**.

Though much of the work was mathematical in nature, we have also had lots of useful theory presented and some only proposed, in the form of conceptual and especially philosophical ideas relating to how to understand or explore genetics through the lens of a sequence analyst or processor.

In terms of concrete **Key Metrics and Contributions** this single work has brought to surface:

1. **16 Working Definitions** were developed in relation to various concepts in analytical genetics, sequence analysis and sequence processing in general (including 3 *Extra Definitions* not directly under Genetics, but also meant to demonstrate how Transformatics could be leveraged/applied — in Physics: refer to Appendix **Chapter F**), and these include:
 - (a) The **na-Sequences** — see **Definition 4**
 - (b) The **START-Codons** — see **Definition 6**
 - (c) The **STOP-Codons** — see **Definition 7**
 - (d) The **Sequence Characteristic** — see **Definition 8**
 - (e) The **Population Characteristic** — see **Definition 9**
 - (f) The **Genome Sequencer** — see **Definition 10**
 - (g) The **Ribosome** — see **Definition 13**

¹Originally, first a paper, then a monograph, and finally a mini-treatise!

²As in the conclusion of the original transformatics paper[5], the author strongly feels that in the future, and with further investments in research, development of theory and applications, transformatics might become its own sub-field of mathematics, distinct from both statistics and linear algebra, but also, though inspired by sequence processors in computer science, also more general than or distinct from that field too.

2. **4 Theorems** were developed and presented for the first time, spanning genetic sequences and then sequence complements in general. These include...
 - (a) **Complement Sequences** — see **Theorem 2**
 - (b) **Complement of an o-SSI is an o-SSI** — see **Theorem 3**
 - (c) **Equivalence of Sequences** under the Complement Transform — see **Theorem 4**
3. **3 Laws** were distilled and presented for the first time:
 - (a) **Law 1:** Concerning the nucleic acid identifier symbol set, ψ_{na} .
 - (b) **Law 2:** Concerning Introns: Non-coding gene-programs.
 - (c) **Law 3:** The **Identity Genome Sequence Law**.
4. **16 Transformers** were developed and presented for the first time, including...
 - (a) The formal definition of the **Ribosome** as a **Protein Generator (Transformer 14)**
 - (b) The **n-Gram Generator (Transformer 1)**
 - (c) The **Sequence Look-up System** that leverages Gödel numbers for efficient DNA-database queries (**Transformer 6**)
 - (d) The **Random Sequence/Symbol Generator** that can construct random DNA/RNA sequences, but also any arbitrary random³ sequence (**Transformer 10**)
 - (e) A transformer for converting between pure numbers (in base-10) into DNA/RNA code, and vice-versa — the ψ_{na} to ψ_{10} **na-to-Decimal Decoder (Transformer 16)**

and many more simple and some complex, but essential transformer utilities and specifications.

5. **30 Transformations** were presented to help advance generic as well as particular computational solutions to many kinds of basic problems in sequence analysis, processing and several concerning gene/genome expression problems, including some special ones such as:
 - **Transformation 18** that generalizes the intricate process of generating a correct **consensus genome sequence** via **genome sequencing**.
 - **Transformation 28** that serves as a good template for the intricate process underlying the **derivation of a Full Genome Expression** from a plain sequence using the **x-LGES**.

³We could have wanted to generalize this generator so that it can also handle the case of non-random sequence generation — especially since it already bases its generated sequence on the properties of some input sequence, so that for example we can generate sequences that reflect the symbol distribution within the input/sample sequence (something akin to how a ChatGPT model can generate new text based on/related to some samples with say non-uniform symbol distributions), but we chose to not do that within the context of this present work, and shall leave that for a future publication and treatment.

6. **8 Algorithms** were developed and presented for the first time, including:
 - (a) **Algorithm 1: The Closest Sequence Algorithm.**
 - (b) **Algorithm 2: The tMCS Algorithm** for computing the Maximum Common Subsequence.
 - (c) **Algorithm 3: The First Lu-Shuffle Algorithm (FLSA):** `lu_shuffle_a( $\Theta, k$ )`.
 - (d) **Algorithm 4: The Second Lu-Shuffle Algorithm (SLSA):** `lu_shuffle_b( $\Theta, k$ )`.
 - (e) **Algorithm 5: The Ω -to-OZIN Genetic Code Transcription Algorithm.**
 - (f) **Algorithm 6: The Ω -to-PLATONIC Form Genetic Code Expression Algorithm** — chosen as core of **LGES**, leverages **PFEK** translation keys to process BODY/THORAX aspect of **FGE**.
 - (g) **Algorithm 7: The x-LGES Algorithm** — extends **LGES** with processing of a ROOT/SUFFIX component.
7. **3 Tabular Analyses** were presented, including **Table 6.1** that depicts how to perform comparative statistical analysis of DNA, RNA or any na-Sequence using some measures and methods from transformatics.
8. **Overall, a total of 7 Tables** were presented, including the two critically important **PFEK** tables **Table C.1** and **Table C.2** for helping in computing predictable shuffles/anagrams of the base-10 n-SSI using the **SLSA** and **FLSA** algorithms respectively.
9. **2 Original Ciphers** — very rare and universally usable cryptographic keys were presented as part of the **Lu-Genome Expression System**, as well as proper justification for their practical relevance and sufficient historical background and example applications of both.
 - (a) The **OZIN Cipher** key depicted in **Figure 10.2**
 - (b) The **PLATO Cipher** key depicted in **Figure 10.7**
10. **3 Core Problems** in the matter/domain of **Genome Sequencing** were formally defined and their theoretical solutions also presented⁴. Refer to **Section 7.2** or **Chapter 7**.
11. **More than 10 Source Code Examples**, some in the TEA language (by the author[9]), and relating to conducting quick analyses of arbitrary na-Sequences via the commandline (refer to **Section 6.2**), and 8 (in **Chapter 8**), related to how to generate arbitrary random sequences using Python and the proposed **RSG Transformer Machine**:
 - (a) **Listing 8.1** — The First Lu-Shuffle Algorithm.

⁴**FUTURE WORK:** We could have delved into actually demonstrating how to use the proposed theory and solutions to conduct actual genome sequencing, especially by hand, however, I have decided to postpone that to a later publication or edition of this book.

- (b) **Listing 8.3** — The Second Lu-Shuffle Algorithm.
 - (c) **Listing 8.4** — Generate Unspecific Symbol Set for Any Sequence.
 - (d) **Listing 8.5** — Multiply any Sequence by a given factor.
 - (e) **Listing 8.6** — Generate a Random Number within a specified range.
 - (f) **Listing 8.8** — Shuffle/Anagrammatize a sequence using no other parameters.
 - (g) **Listing 8.9** — Select a specific number of items from some sequence.
 - (h) **Listing 8.10** — Generate a random sequence of a specified length and whose terms derive from the symbol set of a specified input sequence.
12. And many other quantifiable and unquantifiable major and minor contributions to genetics, mathematics, computer science, philosophy and more.

Thus, we set out to explore ways to leverage as well as further the newly proposed discipline of **Transformatics**, by using it to explain problems in **Genetics**⁵, but also, so as to be able to use it to derive or develop solutions to any such problems we identify and find tractable under the theory and mechanics of sequence transformers.

In this regard, as highlighted in the enumerations above, we have formalized many working definitions of important fundamental concepts (from a transformativ-analytical point-of-view) spanning symbol sets, various statistical measures applicable to sequences, several abstract machines applicable to specific and general problems in genetics and more.

We have distilled some useful general results in the form of theorems, have immortalized some critical and relevant laws relating to genetic code sequences, have extensively demonstrated how the concept of modeling general solutions to various problems can be accomplished using the notion of [re-usable] transformers — defined not in programming code as some might think, but as mathematical operators usable with or without computers. We have demonstrated how such transformers can be leveraged to solve specific or particular practical problems via numerous well-structured transformations.

We have invented and presented some new algorithms that would allow the concepts and solutions we developed in transformatics theory for various problems to be readily turned into computable solutions using any programming language. And in this regard, have also gone ahead to demonstrate the implementation of some practical solutions or fundamental aspects of them using source-code implementations in both TEA and Python that anyone can merely copy from the text and apply to their specific problems or which one could adapt so as to build

⁵Which happens to be one of the major, fundamental sciences that hinges much on the concept of sequences, their processing and analysis.

more elaborate computational solutions to problems not just in genetics, but also for general sequence analysis and processing problems.

We have demonstrated ways to leverage statistical analysis so as to make sense of differences in and quantifiable similarities between two or more sequences — DNA related or not. We have also presented some reference tables that would help anyone attempting to leverage the proposed theory and algorithms of the **Lu-Genome Expression System** to be able to readily do so without having to guess or dive into the daunting process of generating **Platonic Form Encoding Keys** by hand.

Related to the **LGES**, we have also presented and demonstrated for the first time, two very interesting and universally usable [originally esoteric] ciphers that would help not in genetics-related cryptography problems, but in helping both students and researchers of genetics to readily explore and communicate otherwise difficult to express concepts relating to genetic diversity, genetic similarity, the consequences of genetic mutations — additions, subtractions or alterations to genetic sequences, etc. using a visual language that is reminiscent of actual molecular-level expressions resulting from the natural processing of DNA and mRNA code, all the way up to the expression of entire organisms from mere genetic code.

With regards to understanding or explaining how a full genome might be derived from bits of DNA sequence read from or known, concerning an individual organism, an individual species, a genus, an entire family or populations of organisms, we have advanced the literature by developing rigorous mathematical concepts and solutions to the matter of genome sequencing whether performed by hand as in the classical context of Fred Sanger, or with advanced computation such as in modern genome banks and DNA processors.

Overall, this book, though it originally started out as just a paper, and though some would envision it to have been a longer treatise than it actually is, surely has helped lay the ground for much future work in genetics, the mathematical sciences as well as in transformatics itself, so they can take advantage of and leverage the methods, theory and mechanics/calculus of transformatics as presented in this work, and especially so that it makes easier, the definition of new problems, design new solutions to existing as well as new problems, but also, so that it is possible to readily take many ideas previously either intractable or only appreciable in theory, into the practical domain — especially via computational methods leveraging sequence transformers.

It is in the dreams and hopes of the author, that the work that started with the treatment of basic sequences in the form of pure number expressions — see **GTNC**⁶[2], circa 2020, while only getting started with their MSc and at that

⁶The original paper actually first got written circa 2020: <https://scholar.archive.org/work/apn4f26kdzatfodrap7avqafce>

time⁷, only looking at sequences as though they were a concept native to only **number theory**, does not stop with recent advances such as the **LNS**[1] that started to explore these ideas in the context of physics and computer science, and then **ADT** that ushered in the sensibilities of mathematical statistics, but that transformatics and the collective mathematical theories of treating of sequences of symbols via transformers and analytical specifications, gets its own wings and finally flies off the ground into any and all fields of inquiry with useful results and advancements of human understanding and knowledge.

11.1 Future Directions

In this single work, several issues, problems and potential areas of further research and inquiry have been identified, as well as highlighted all throughout the book. For purposes of bringing attention to the key points thus identified, we shall only enumerate them as highlights and in brief:

1. **Pathological Gene Programs:** Given we isolated the START and STOP codons that regulate or constrain how protein synthesis is conducted at the ribosome level, we wish to establish if there might exist malicious gene programs that might exploit the ribosome's na-Sequence processing logic so as to cause problems such as failure to manufacture some necessary proteins or amino-acids (due to absence of or misplacement of START-codons), but also the terrible, pathological case where a ribosome starts to construct a protein (as an amino-acid chain), but fails to return/complete since it is presented with an infinitely long or non-terminating gene program. **Refer to Section 4.2**
2. **Automatic Sequence Characteristic:** Having established the fundamental relevance of the modal sequence statistic, as well as the more elaborate sequence characteristic measure and its usefulness in several aspects of genetic sequence analysis and processing (similarity tests, classification problems, genome expression, etc.), we wish to make sure that there is a simple and straightforward way by which, when presented with a sequence of symbols such as the complete genome sequence of some organism stored in a flat text file, we can automatically compute and present the corresponding sequence characteristic statistic. This, especially via use of the TEA programming language on the command line. **Refer to Section 6.2**
3. **Evaluate Gödel Number Indices for na-Sequences:** Since we realized

⁷Joseph, despite having been very active in the contemporary scientific and mathematical research community (see ORCID: <https://orcid.org/0000-0002-0002-4657>), still hasn't been generally well appreciated — they are still not affiliated with any academic institution or faculty, have not managed to secure a teaching position despite much of their work contributing to global open-learning via the Internet (see <https://t.me/bclectures> and https://www.youtube.com/playlist?list=PL9nqA7nxEPgv_CqH0xIDj2Jf1gHAEV-B-), and despite having expressed interest in pursuing their PhD in either Computer Science or Mathematics at Oxford[16], haven't yet secured a PhD position nor scholarship.

that computation of and use of special indices — in the form of hash keys, corresponding to especially very long na-Sequences (such as those of the human genome that might shoot into millions and up to billions of symbols) might help lessen the space-time complexity with regards to storage, lookup and comparison of na-Sequences as in a database system, we shall want to empirically and theoretically establish if use of Gödel numbers (especially when combined with use of a sequence's modal statistic or sequence characteristic) might offer an optimal solution for storage, lookup and comparison of large or arbitrarily long na-Sequences. **Refer to Section 6.7**

4. **Implementing Entropy Sources:** Especially because, as we have identified in several problems and solutions concerning sequence processing (shuffling, anagrammatizing, sequence generation, etc.), but also in genetics (simulation of meiosis, genome expression, etc) that use of and generation of/harnessing of randomness is fundamentally important, we shall want to ascertain, and especially both in theory and with empirical computable solutions, how we can produce randomness from scratch, and especially in a closed-system or with mere pen and paper. And to ascertain whether such is actually feasible or not. **Refer to Section 8.3.4**
5. **Further Computational Model of The Ribosome:** Since we have already embarked on developing, presenting and exploiting a computational model for the protein synthesis mechanism in living cells, and have managed to not only develop a formal process that captures the gist of how a ribosome works, it raises new and important extra problems for the computer scientist studying the ribosome, and which problems we would want to find reliable answers for. These include the matter of the Instruction Set of the Ribosome Machine, its computer architecture and how it relates to the von Neumann Architecture, whether or not the ribosome is Turing Complete, how we might implement a virtual or robotic ribosome etc. **Refer to Section 9.1**
6. **Concerning the Relationship Between DNA, Psychology and Spirituality:** Though it might not directly fall in the domain of pure genetics, it shall be interesting and very likely rewarding if not enlightening, to study, explore and expound on the matter or problem of whether it is indeed true (or false) that genetics underlies the spiritual-functions in a human? Could genetics explain or underlie peculiarities in human psychology both at an individual as well as from the context of entire communities, families, nations or an entire species? Since artificial (though intelligent) constructs such as robots and AGIs don't share the same material basis as does living things — in the sense that they aren't controlled/limited by nor based on organic tissue, no semblance of genetic code underlying their expression or functions⁸,

⁸Of course, one might argue, and plausibly so, that artificial intelligences are nonetheless designed by and brought to life by

etc. Could it be that they can't exhibit or participate in the same spiritual, psycho-spiritual, or even preternatural phenomena that living things (especially humans), seem to be inseparable from? **Refer to Section 9.1 and Chapter F**

7. **Establish Bounds on Protein or Amino-Acid Chain Relative to Gene Program:** While formalizing the concept of gene expression in living things, and especially when dealing with the matter of how na-Sequences get translated into an amino-acid chain, and thus into a protein, a problem arose; since it is that most of the protein manufacturing process is somewhat linear and especially a matter of 1-to-1 encoding or translation of genetic code into other materials, it would be useful to prove or perhaps clarify on whether it is true that the length of a gene-program (since it is essentially the chain of codons that eventually get translated into the protein) imposes a finite limit on the length of the material (such as a protein or amino-acid chain) that a ribosome can produce from a particular genetic code sequence? **Refer to Section 9.2 and especially Transformer 13**
8. **Genetics and Mysticism, Genetic Veves:** So, despite this being a book on especially the mathematics of genetics and sequence analysis, we did come across important matters (especially for the *humanities* and philosophy) concerning the (not-very surprising) role genetics and especially genetic code might play in humane culture, the arts and traditions across the world. Particularly, while studying and developing the **LGES**, we found that the systematic diagrams — the **FGEs**, derivable from or expressing particular na-Sequences as well as any numeric sequence, have so many similarities to or that they are somewhat reminiscent of concepts such as sacred geometry, voodoo veves, magical seals and sigils, psycho-cultural archetypes, etc. from various contexts, cultures and traditions all over the world. Is it just because of the fact that geometry and geometrical forms are generally universal and cut-across cultures, space and time, or is there something deeper, perhaps not immediately discernible concerning our **Lu-Genome Expression System** that might hint at critical aspects of universal human culture, mysticism as well as spirituality? Could a genetic veve disrupt contemporary voodoo veves or traditional cabalistic seals when considered from the perspective of visual-spatial glyphs that speak to the human subconscious for example? **Refer to Section 10.2.3**
9. **Practical Genome Sequencing:** We have laid down most, if not all of the necessary theoretical and conceptual machinery for how to go about practi-

humans who are controlled or limited by genetics, and also, there is a strong community in computing, that bases the design and implementation of artificial systems on not just code that somewhat is reminiscent of genetic code, but also which directly or indirectly mimics natural/biological systems such as neural networks in living things, but also use of genetic programming for optimizing self-regulating learning algorithms, etc. But, this point here, specifically refers to whether or not such artificial constructs are directly built with or using natural, genetic material... bio-automata constructed out of actual living tissue or synthetic organisms that can exist around or inside living things might otherwise cause us to relax this stressed point.

cally solving the matter of reducing a collection of sequences to their most correct, representative genome sequence. However, in this work, we have not actually gone ahead to demonstrate how this theory shall be applied or utilized, say with sample na-Sequences much as we have done this for most of the other problems and concepts we have tackled in the book. Thus, future work shall need to expound on this with illustrative or representative examples and complete solutions to how to perform actual genome sequencing for genetic code first of all, but also, and given the theory can be applied to any kinds of symbol sequences, for arbitrary scenarios too. **Refer to Chapter 7**

Appendix A

Other Functions of DNA Beyond Protein Synthesis

While most of this book focused on DNA's most well-known function as the material basis for how the protein synthesis process is made possible, but also how it is regulated in nature, and yet, protein synthesis is not all that DNA is useful for! As any newcomer to genetics and nucleic-acid sequences might wonder — **What else is DNA used for apart from the construction of proteins?** And so, I posed this question to my research assistant[32], and the answer here shared, almost verbatim, might help lift others out of darkness too...

Deoxyribonucleic acid is widely known for its role in protein synthesis, however, it also serves several other critical functions in biological systems as enumerated in summary here:

1. Regulation of Gene Expression[63]

- Non-coding DNA (which makes up $\approx 98\text{--}99\%$ of the genome) contains regulatory elements like enhancers, silencers, and promoters.
- These regions control *when*, *where*, and *how much* a gene is expressed — like how a conductor would guide and regulate a symphony's performances.

2. Epigenetic Control[64]

- DNA interacts with histones and chemical tags (like methyl groups) to form chromatin, which can be tightly or loosely packed.
- This packing determines gene accessibility, influencing cell identity and response to environmental factors like stress or diet.

3. DNA Repair and Replication[65]

- DNA contains built-in mechanisms for self-repair and accurate replication, ensuring genetic fidelity across cell divisions.

- Specialized sequences help detect and correct errors, preventing mutations that could lead to disease.

4. Cell Differentiation and Development[66]

- Though every cell has the same DNA, different genes are turned on/off to create neurons, muscle cells, skin cells, etc.
- This selective expression is orchestrated by DNA's regulatory architecture and chromatin dynamics.

So, although all cells share the same DNA, selective gene expression enables differentiation into various cell types such as neurons, muscle cells, and epithelial cells .

5. Evolutionary Adaptation[67]

- Non-coding regions accumulate mutations without affecting protein function, serving as a sandbox for evolutionary innovation.
- These changes can influence species traits, adaptation, and even speciation over time.

6. Biological Memory and Identity[68]

- DNA acts as a biological archive, encoding ancestral information and evolutionary history.
- When considering context-aware symbolic systems and the matter of *cultural abstraction*, then DNA can also be seen as a **modal sequence** that bridges biology and [symbolic] meaning.

Appendix B

Origins of the OZIN Cipher

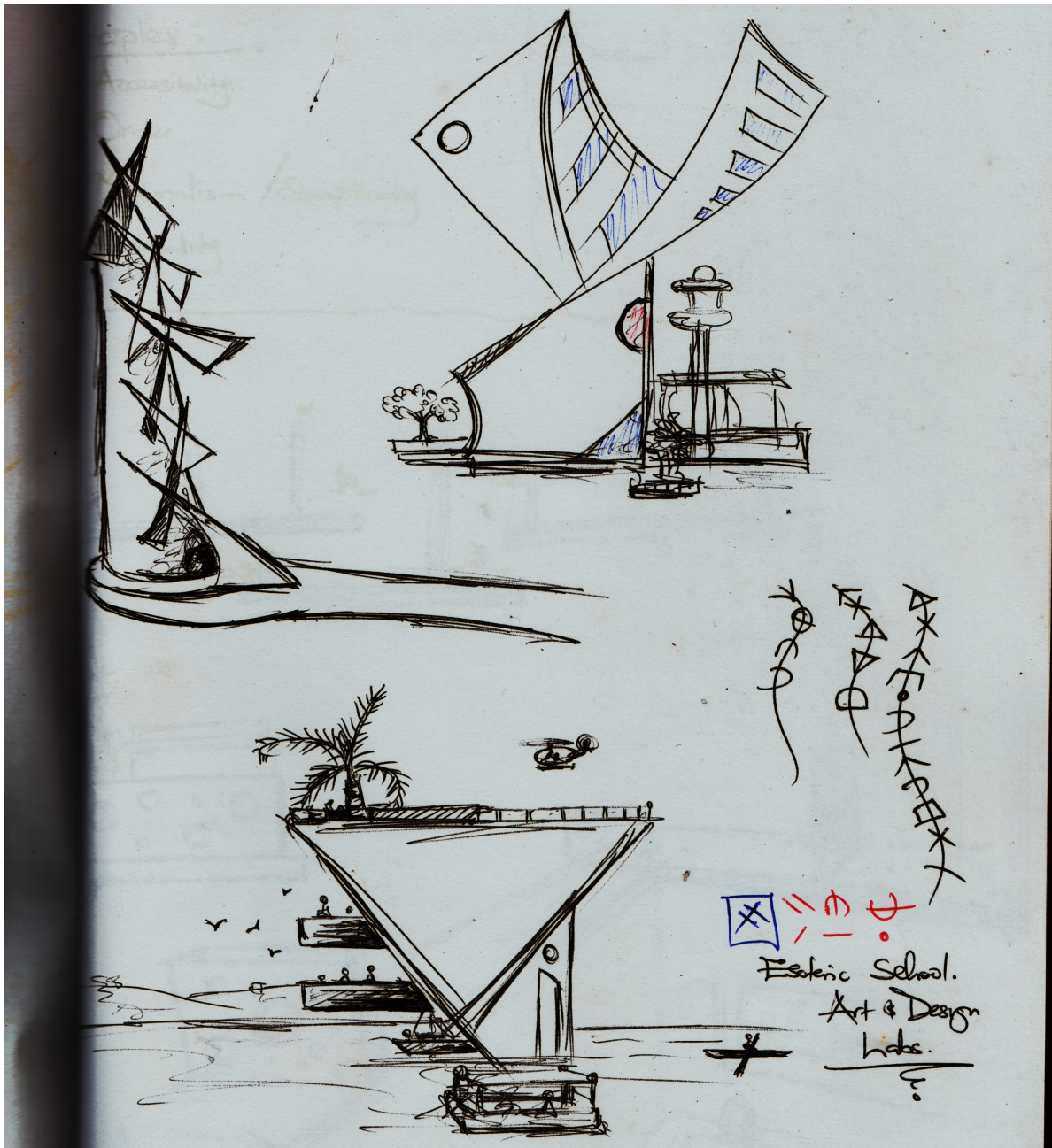


Figure B.1: An excerpt from architectural design notes by the author, from the early days of his home-based research laboratory. In this particular sketch note, we also get to see an idea similar to how gene expressions are actually a chain of glyphs that, as in the **LGES** protocols, are depicted as structures chained to each other along a *backbone structure* symbolically depicted as a line (curved lines in this case). Copyrights due to author and Nuchwezi Research Labs.

For some, the **Ozin Cipher**[12] (see **Section 10.1**) might not seem that important or useful, but, perhaps because not everyone can immediately appreciate the fact that basic [visual and symbolic] cryptography is such a dear part of expression in nature, that whoever is the Architect of Life, has applied it abundantly throughout nature, in many creative ways — or, cast in a different light, that when we (humans) explore nature using our creative and intellectual faculties, we find or see it vastly applied across most of nature; camouflage in reptiles and viruses, the occult languages of birds, telling, but cryptic signatures of various herbs, etc. And these phenomena have indeed inspired many observant minds throughout the ages, to pick a leaf from nature and leverage this diversity and stealthiness in expression via various ideas and in many humane fields; in mathematics and computing, in mundane as well as esoteric philosophy, and yes... in *magick*! Talking of which, for those interested in getting a deeper idea concerning how such a beautiful numeric symbol system — an alternative to the commonplace Arabic Numerals, came to be, one useful resource (and some clues) is the **CODE OGF**[69] and a queer fascination with the **sacredness of numbers** — especially the **kaballah** system!

For Academic or Research Purposes Only.

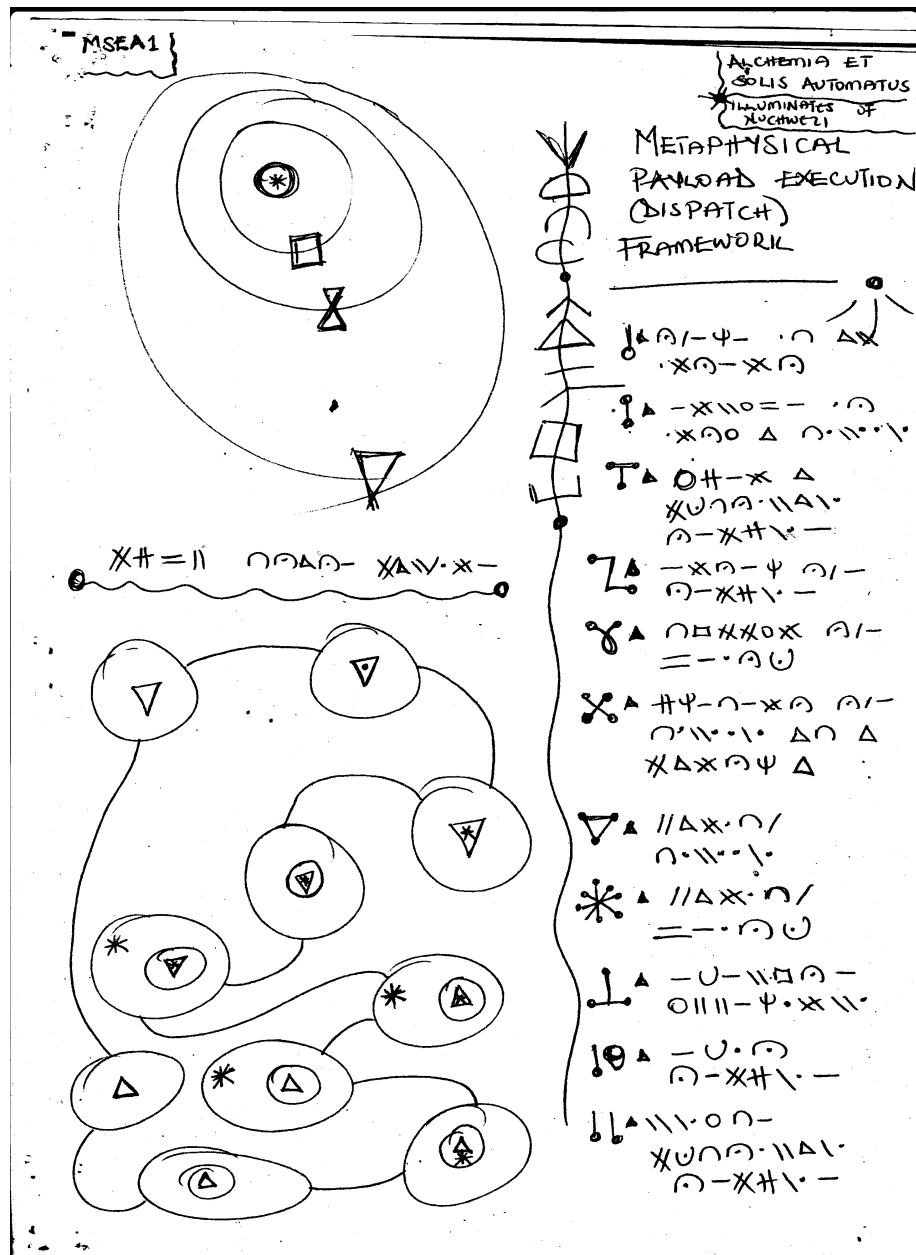


Figure B.2: The OZIN Cipher in use. An excerpt from the Church of Dance Eternal OGF

Figure B.2 is just an excerpt from that manuscript, perhaps not so clear because the original manuscript is handwritten¹, and this is just a scanned version. But hopefully it drives the point home. Also, as seen from that excerpt as well as a closely related excerpt — see **Figure B.3**, just like we might replace numbers with alternative symbols, we also see the Latin Alphabet replaced by an “alien-

¹Some of these notes do have a life of their own though. So, for example, if one wishes to look at a clear scanned version of **Figure B.1**, there is a copy at the author's online Art Portfolio — <https://www.deviantart.com/nemesisfixx/art/Not-So-Impossible-Dwellings-823360670>, and as for **Figure B.3**, a clear version is also curated at <https://www.deviantart.com/nemesisfixx/art/Nu-Druid-Calligraphy-806645934>

Appendix C

Computing Platonic-Form Encoding Keys (PFEKs), $\boxed{\psi_\Omega}_m$, for a Particular na-Sequence Θ and some m

It is one of the tricky, but otherwise readily solvable aspects of the **Lu-Genome Expression System** (LGES), that when constructing the **BODY** aspect of a **Full Genome Expression** (FGE) corresponding to a particular sequence Θ , that we correctly pick or generate the corresponding platonic-form encoding key, $\boxed{\psi_\Omega}_m$, that corresponds to the sequence's characteristic, or rather, the **modal sequence**.

In particular, we use the concept of shuffling/anagrammatizing a sequence relative to some initial/specific input **partitioning index** k — in this case, denoted m , and which is derived as such:

$$m = \varkappa(\overset{\triangleright}{\Theta}) \tag{C.1}$$

So that, for the case of rendering **FGE** basing on some sequence Θ — possibly an na-Sequence, encoded (for **LGES**) using base- Ω , we use the **Second Lu-Shuffle Algorithm** (SLSA)¹ to anagrammatize ψ_Ω **relative to** m , so as to arrive at a correct $\boxed{\psi_\Omega}_m$ that we then use to render the **PFA** for Θ . Essentially, we must compute the transform:

Transformation 31 (Generating the **Platonic-Form Encoding Key**).

$$\psi_\Omega \xrightarrow{slsa(\psi_\Omega, m)} \boxed{\psi_\Omega}_m$$

Where $slsa(\Theta, m)$ works as examples in **Listing 8.3** show.

¹Introduced in **Section 8.2**, and for which, important, example look-up-tables are shown in **Section 8.2.2**.

Good enough, we not only have ready-to-use shuffled versions of $\psi_\Omega = \psi_{10} = \langle 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \rangle$ as shown in **Figure 8.7**, but also, in case one wishes to manually compute or perhaps automatically generate the necessary key for a given m or *any* sequence, the Python program in **Listing 8.3** can be utilized for this purpose.

That said, basing on the algorithm that generated the mappings in **Figure 8.7**, we have ready-to-use **Platonic-Form Encoding Keys** corresponding to particular values of m , and shuffling of ψ_Ω as shown in the following table.

PFEK Look-up Table by SLSA										
$I(\omega_i, \overset{\rightharpoonup}{\Theta})$ $m = \underline{\nu}(\overset{\rightharpoonup}{\Theta})$	1	2	3	4	5	6	7	8	9	10
1	4	2	8	9	0	7	3	6	5	1
2	4	2	8	9	0	7	3	6	5	1
3	7	5	3	0	8	2	6	1	9	4
4	1	6	4	7	2	9	0	3	8	5
5	2	1	0	6	8	3	5	4	7	9
6	7	1	2	4	6	8	3	9	0	5
7	5	9	2	3	7	0	4	6	1	8
8	9	1	5	4	3	2	7	8	6	0
9	5	6	3	9	0	4	2	7	1	8
10	6	3	8	2	1	9	7	4	0	5

Table C.1: Platonic Form Encoding Keys (PFEK) mapped to corresponding values of $m = \underline{\nu}(\overset{\rightharpoonup}{\Theta})$ using **SLSA**

And if one wishes to render their **PFA** — the platonic form aspect, of the **FGE** using the **FLSA**, then the corresponding table would be as follows:

PFEK Look-up Table by FLSA										
$I(\omega_i, \overset{\rhd}{\Theta})$ $m = \nu(\overset{\rhd}{\Theta})$	1	2	3	4	5	6	7	8	9	10
1	9	7	4	0	5	3	8	2	1	6
2	9	7	4	0	5	3	8	2	1	6
3	8	6	3	9	4	2	7	1	0	5
4	8	6	3	9	4	2	7	0	1	5
5	5	3	0	6	1	7	4	8	9	2
6	5	1	2	6	3	7	4	8	9	0
7	0	8	9	1	5	2	6	3	4	7
8	3	8	9	4	2	5	6	0	1	7
9	9	1	2	3	8	4	5	6	7	0
10	9	5	6	7	8	0	1	2	3	4

Table C.2: Platonic Form Encoding Keys (PFEK) mapped to corresponding values of $m = \nu(\overset{\rhd}{\Theta})$ using **FLSA**

However, having the **PFEK** tables is one thing, while, knowing what to use them for, or how to use them is another, and we cover the gist of that, under the next title.

C.0.1 Example of Resolving PFA Components, $\boxed{\omega_i^*}$

We shall not attempt to render a complete **FGE** here since that matter is already well catered for in the associated chapter and section — see **Section 10.2.3**. However, we shall attempt to set straight, the necessary procedures for:

1. Computing m for a given Ω -encoded sequence Θ :
 - (a) Compute the **modal sequence statistic**, $\overset{\rhd}{\Theta}$ — see **Definition 1** in [5].
 - (b) Compute m from the cardinality of the modal sequence as: $m = \nu(\overset{\rhd}{\Theta})$.
2. Picking the correct **PFEK** for that m — i.e. $\boxed{\psi_{pf}}_m$:
 - (a) If using **FLSA**, refer to **Table C.2**, and for any $m = k$, pick the row for which the first column has the value k .
 - (b) Of course, for any $m = k$, the only useful values on that row for doing the **PFEK** operations are the **first k emboldened** values on the row corresponding to m . E.g. for $m = 2$ the necessary key is just $\langle 9, 7 \rangle$, not the entire row.
 - (c) If using **SLSA**, lookup the **PFEK** using **Table C.1**, so that, for $m = 2$, we have $\boxed{\psi_{pf}}_2 = \langle 4, 2 \rangle$

3. Resolving the correct component from $\boxed{\psi_{pf}}_m$ corresponding to a particular element ω_i in the corresponding **IFA MSS**, $\overset{>}{\Theta}$.

(a) So, if our **IFA MSS** was just $\langle 0, 2, 1, 3, 6 \rangle$ such as for Ω_3 from **Equation 10.4**, then, taking note of the special case for any $\omega_i^* = 0$ — such as the first term in this case (see **Step#2(e)ii** in **Algorithm 6**), we merely map values of ‘0’ in the **MSS** to just the **JOINT/GAP** element in **Figure 10.7**), and for the other terms, such as $\omega_3^* = 3$, since the corresponding **PFEK** for $m = 5$ is $\boxed{\psi_{pf}}_5 = \langle 2, 1, 0, 6, 8 \rangle$, then $\boxed{\omega_3^*} = 0$ as per the corresponding look-up mapping we see for such an **MSS**:

$$\begin{array}{ccccc} \left[\begin{array}{ccccc} 0 & 2 & 1 & 3 & 6 \end{array} \right] \\ \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \quad \downarrow \\ \left[\begin{array}{ccccc} 2 & 1 & 0 & 6 & 8 \end{array} \right] \end{array}$$

And so, for the last term in that **MSS**, $\boxed{\omega_5^*} = 8$

4. How to render the final pf-encoded version of ω_i^* , i.e. $\boxed{\omega_i^*}$?

(a) So, now that you know the **PFEK** for your m , such as $\boxed{\psi_{pf}}_5 = \langle 2, 1, 0, 6, 8 \rangle$ from above, and have the correct pf-term for a particular term in the **MSS** such as $\boxed{\omega_5^*} = 8$ in the example above, the corresponding pf-term is the platonic structure from **Figure 10.7** that corresponds to the index 8 — so that would be the **octagon** or rather **octa-gram** with nothing at the center. But also, given the term we are decoding is $\omega_5^* = 6$, then the overall structure would be as per configuration $8_{pf} + 8 \times 6_{oz}$: That is to say:

$$\boxed{\omega_5^*}_{pf} = 8_{pf} \otimes 6_{oz} = 8_{pf} + 8 \times 6_{oz} \quad (\text{C.2})$$

So that the final rendered term for this element alone would be as shown:

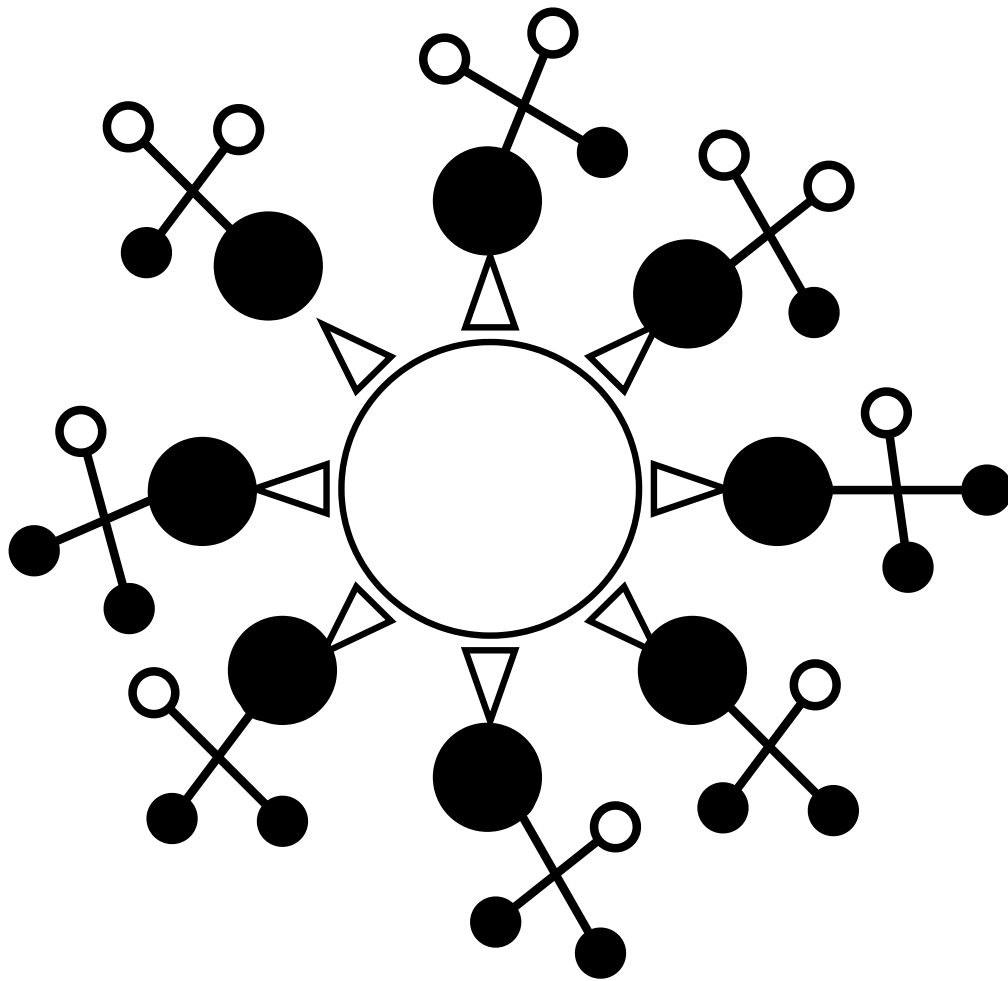


Figure C.1: An example of a single **PFA** component equivalent to the configuration $8_{pf} \otimes 6_{oz}$.

Of course, once one has the correct $\boxed{\omega_i^*}$, they can just attach it to the existing **FGE** chain-structure so as to advance the piece-wise construction the complete **BODY** aspect of a full genome expression for Θ .



FUT. PROF. J. WILLRICH LUTALO

TRANSFORMATICS

101 explained

Appendix D

TRANSFORMATICS 101 - explained:

A rigorous, succinct introduction to all the essential foundations and proposed theory of Transformatics

ABSTRACT¹:

In only **10 general foundational results**[70] — all **based on earlier mathematical research** by the author, as well as **10 extra abstractions** spanning **how transformatics might be leveraged as a basis mathematics** for problems in; number theory and cryptography (o-SSIs); mathematical statistics (ADM, TCR, PCR, MSS, SC, PC); linear algebra (super sequences); computer science (TEA language, transformer-chains or sequence-transformers) and information theory (entropy of a sequence) among others, we distill and establish, in a summarized form, the **core foundations of Transformatics**[5] as its own mathematical theory, branch, or **discipline**. In this **mini-thesis**. Each foundational result presented is likewise accompanied by one or **more references** to some earlier work where the idea or presented concept was first developed, presented, or applied by the author during their research. **Useful notes and commentary** are also included where necessary, and in the extension decade of **proposed abstractions**, also commentary about how the proposals relate to the decade of foundational results likewise is catered for.

Keywords: Foundations, Transformatics, Sequence Analysis, Sequence Transformers, Thesis

D.1 Foundations of a New Mathematics of Sequences

D.1.1 Result 1: The Empty Sequence[1]

$$\Theta = \langle \rangle \implies |\Theta| = 0 \quad \wedge \quad \Theta \equiv \emptyset$$

D.1.2 Result 2: A Symbol Set[2]

$$\psi_\beta^n = \langle \beta_{i \in [1, n]} \rangle : n \in \mathbb{N} : \mathbb{1}(\beta_i \in \psi_\beta) = 1 \quad \forall \beta_i \in \psi_\beta$$

¹Note that this chapter attempts to present [23] verbatim, with the only major difference being the numbering of citations and references, which are co-shared with the main manuscript.

D.1.3 Result 3: A Sequence[1]

$$\Theta^n = \langle \theta_{i \in [1, n]} \rangle : \mathbb{N} \times \psi_\beta : n \geq 1$$

D.1.4 Result 4: Sequence Cardinality[3][2]

$$\forall \Theta^n = \langle \theta_{i \in [1, n]} \rangle : \mathbb{N} \times \psi_\beta \implies \iota(\Theta^n) = |\Theta^n| = n \in \mathbb{N}$$

D.1.5 Result 5: A Sequence Symbol Set[4][2]

$$\psi(\Theta^n) = \langle \theta_{i \in [1, k]} \rangle : k, n \in \mathbb{N} \wedge k \geq 1 : \iota(\theta_i \in \psi(\Theta^n)) = 1 \quad \forall \theta_i \in \Theta^n : k \leq n : \iota(\psi(\Theta^n)) = k$$

D.1.6 Result 6: A Sequence Transformation[5][1]

$$\Theta^n = \langle \theta_{i \in [1, n]} \rangle : \mathbb{N} \times \psi_\beta \rightarrow \Theta^* = \langle \theta_{i \in [1, k]}^* \rangle : \mathbb{N} \times \psi_* : \psi_\beta \neq \psi_* \vee \psi(\Theta^n) \neq \psi(\Theta^*) \vee \overset{>}{\psi}(\Theta^n) \neq \overset{>}{\psi}(\Theta^*)$$

D.1.7 Result 7: A Sequence Transformer[6][5]

$$\Theta^n = \langle \theta_{i \in [1, n]} \rangle : \mathbb{N} \times \psi_\beta \xrightarrow{f(\Theta)=\Theta^*} \Theta^* = \langle \theta_{i \in [1, k]}^* \rangle : \mathbb{N} \times \psi_*; \quad \psi_\beta \neq \psi_* \vee \psi(\Theta^n) \neq \psi(\Theta^*) \vee \overset{>}{\psi}_\beta \neq \overset{>}{\psi}_*$$

D.1.8 Result 8: A Sequence Filter[5]

$$\Theta^n = \langle \theta_{i \in [1, n]} \rangle : \mathbb{N} \times \psi_\beta \xrightarrow{f(\Theta, \psi_{\beta^*})=\Theta^*} \Theta^* = \langle \theta_{i \in [1, k]}^* \rangle : \mathbb{N} \times \psi_{\beta^*}; \\ k \in \mathbb{N} : \psi_{\beta^*} \subseteq \psi_\beta \implies \iota(\psi_{\beta^*}) = k \leq \iota(\psi_\beta) \implies \iota(\Theta^*) \leq \iota(\Theta^n)$$

D.1.9 Result 9: A Sequence Generator[5][7]

$$\Theta^n | \emptyset \xrightarrow{f(\psi_\beta, k)=\Theta^*} \Theta^k = \langle \beta_{i \in [1, k]}^* \rangle : \mathbb{N} \times \psi_\beta : k \in \mathbb{N} : \forall \beta_i \in \Theta^k \implies k \geq 1 \quad \wedge \quad \beta_i \in \psi_\beta = \Psi^\infty \cup \Theta^n$$

D.1.10 Result 10: A Sequence Sampler[8]

$$\Theta^n \xrightarrow{f(\Theta^n, k)=\Theta^k} \Theta^k = \langle \theta_{i \in [1, k]} \rangle : \mathbb{N} \times \psi(\Theta^n) : k, n \in \mathbb{N} \wedge k \geq 1$$

NOTES:

1. **Result 1:** Apart from the concept of the set, that of the **empty set** underlies all of transformatics — it is the ideological basis of all kinds of sequence generation, in particular, when projected unto the **empty sequence**. It allows us to codify states such as emptiness, non-emptiness, equality, identity, **zero** etc.

2. **Result 2:** The **symbol set** underlies of all formulations in need of a way to speak of or quantify the **identity composition** of any sequence.
3. **Result 3:** The **sequence** is the basis concept in transformatics when applied to sequences, or rather, the **algebra of sequences**[71].
4. **Result 4:** **Sequence cardinality** allows us to quantify various measures about a sequence relative to the size of its membership at rest. These might for example be measures such as the entropy of the sequence (**Proposal 10**).
5. **Result 5:** The **sequence symbol set** is an application of the symbol set concept to any particular sequence as opposed to say, talking of the symbol set of a language — an alphabet like ψ_{az} or ψ_{0-9} for example Also, when applied to a particular sequence, such a ordered set might also reflect the order of members in the concerned sequence — somewhat similar to the MSS (**Proposal 5**) then, but not the same.
6. **Result 6:** Underlying all notions of one sequence being *different* from any other sequence is that of **sequence transformations**. Any operation that can alter either the composition or member-ordering of a sequence is essentially a sequence transformer. And when we talk of any two or more sequences in terms of how they differ from each other irrespective of the process or operation that maps them between each other, we then speak of a **sequence transformation**. In the latest (1st November 2025) update to these foundations, we have also introduced use of the concept of the **modal sequence statistic** in some of the specifications such as for the definition of sequence transformation as a concept. This, especially so we can enhance or make more robust, the foundational result. Otherwise, we formally [re-]introduce the concept of the MSS in **Proposal 5**.
7. **Result 7:** The **sequence transformer** concept concretizes the notion of a transformer operation underlying or betwixt any two sequences involved in a sequence transformation. Such transformers might be operations such as those specified on structures such as matrices in linear algebra, or might be as the structure-preserving sequence altering functions as the catamorphisms that Gibbons mentions in relation to transformations on trees[71]. Also realize that in the current formulation of **Result 7**, we employ the concept of the MSS in specifying the final potential kind of sequence transformation possible — where the composition and cardinality of a sequence pre and post transform might stay preserved, but not the relative order of terms within the sequence — a concept also precisely analyzable using mathematics and measures such as the **anagram distance measure** (**Proposal 2**).
8. **Result 8:** **Sequence filters** allow us to specify or treat of scenarios in which one sequence is reduced to a smaller, perhaps more useful sequence — such as the generation of an MSS or any symbol set for that matter, from any

arbitrary sequence. Note that since the 8th October first edition of these foundations, **Result 8** has since been fixed/corrected, so that the current transformer specification for a generic sequence filter is the correct one — also more specific than originally.

9. **Result 9:** Underlying any concept of *creation* is that of bringing things to life.. (*ex nihilo or not*) especially relative to some pattern or specification (such as the generation of words in some specified language or number system or alphabet, etc.), but also, generation of structures or objects of a particular kind or type. Concerning the current general specification of a [sequence] generator, note that first of all, any sequence generator is essentially a sequence transformer to begin with. Moreover, for the fact that the generated sequence might sometimes span an alphabet, type or symbol-set different or larger than that of the source or original reference sequence — such as a seed-symbol-set ψ_β in the presented general case — we also could have expressed the final specification as either $\beta_i \in \psi(\Theta^n) \cup \psi^\infty$ or perhaps $\psi_\beta := \psi_\beta | \psi_\beta \cup \psi(\Theta^n) \cup \psi^\infty | \psi^\infty$ and where we know that $\psi(\Theta^n) \subseteq \Theta^n$.
10. **Result 10:** Being able to **choose** is a fundamental concept in mathematics such as probability theory but also much of statistical experiments[72] or intelligent behavior[6] for example. We might liken a **sequence sampler** to a sequence filter (**Result 8**), but they are not the same thing; they approach the same concept differently; a filter might be the source sequence minus everything not matching a pattern, while the sampler could be modeled as the source sequence minus everything matching a pattern or vice-versa.

D.2 Proposed Abstractions Leveraging Transformatics

Before we kick-off, note that, as in many previous works on transformatics since [5], when we have any sequence $\Theta^n : \forall i \in [1, n] : \exists \theta_i \in \Theta^n$, then, when we wish to talk of the **position index** of some member element or symbol θ_i in Θ^n , such that we mean the member in that sequence that is located or positioned at exactly index i in the range $[1, n]$, then we shall, as introduced and elaborated upon in the introduction section of [5], merely use the notation:

$$I(\theta_i, \Theta^n) = i \quad \forall \theta_i \in \Theta^n \quad (\text{D.1})$$

Further, if we have any two elements from the sequence (whether or not they are equal: $\theta_i = \theta_j$ or not: $\theta_i \neq \theta_j$), and yet, if $i \neq j$, then $I(\theta_i, \Theta^n) \neq I(\theta_j, \Theta^n)$,

and if $i \leq j$, then $I(\theta_i, \Theta^n) \leq I(\theta_j, \Theta^n)$.

As you shall come to appreciate when studying formulations and expressions in transformatics, the position operator or measure, $I(\cdot, \cdot)$ is very fundamental, useful and indispensable especially when treating of (sic) **ordered sequences** or of order in any sequence.

D.2.1 Proposal 1: The o-SSI Sequence from any Sequence[4]

Assume we consider the sequence $\Theta : \mathbb{N} \times \psi_*$ of any length and spanning an arbitrary symbol set ψ_* . Then, in case we wish to reduce Θ to a representative sequence Θ_{ossi} that only contains elements of Θ spanning the symbol set, ψ_β , of some particular base, β ; in which case, Θ_{ossi} would be an **orthogonal symbol set identity** (o-SSI) sequence under base- β , then, the following o-SSI-generator — also **an extractor**² and **a reducer** transformer, defined using the calculus and semantics of transformatics, would suffice³:

Transformer 17 (The o-SSI Extractor). $\Theta \langle \theta_{i \in [1, n]} \rangle : \mathbb{N} \times \psi_* \xrightarrow{O_{filter}(\Theta, \psi_\beta)} \Theta^k \langle \theta_i \rangle : \mathbb{N} \times \psi_\beta = \Theta_{ossi};$
 $0 \leq k \leq \mathcal{U}(\psi_\beta) \leq \mathcal{U}(\Theta) = n \in \mathbb{N} : \forall \theta_i \in \Theta^k \implies \exists \theta_i \in \psi_\beta$
 $\wedge \quad \forall i, j \in \mathbb{N} \wedge \theta_i, \theta_j \in \Theta^k : I(\theta_i, \Theta) \leq I(\theta_j, \Theta) \implies I(\theta_i, \Theta^k) \leq I(\theta_j, \Theta^k)$

First, realize that **Transformer 17** leverages the concept of a sequence filter as first introduced in **Foundational Result 8**. Also, note that, the **o-SSI Sequence**, Θ_{ossi} , of any sequence Θ under β would likewise be related to the concept of the **Specific Symbol Set**⁴, $\hat{\psi}_\beta(\Theta)$, of that sequence, in the sense that, at most, Θ_{ossi} is equivalent to $\hat{\psi}_\beta(\Theta)$ or that their cardinalities and memberships are related as:

$$\mathcal{U}(\Theta_{ossi}) \leq \mathcal{U}(\hat{\psi}_\beta(\Theta)) \leq \mathcal{U}(\psi_\beta) \leq \mathcal{U}(\Theta) \quad (\text{D.2})$$

and as for membership:

$$\psi(\Theta_{ossi}) \subseteq \hat{\psi}_\beta(\Theta) \subseteq \psi_\beta \subseteq \Theta \quad (\text{D.3})$$

²In the sense that, where a sequence already contains the essential symbols spanning some particular symbol set, and even though they might be scattered about or ‘buried’ around in the sequence surrounded by ‘noise’, the function would merely extract those essential symbols [only] from that sequence in their natural order of first occurrence.

³Say, and as almost all transformatics might be in practice — consists of a clear, succinct and complete specification that can readily be translatable into a working or equivalent computer program based on or defined using the chaining of sequence transformers such as in a language as TEA[9].

⁴Refer to **Definition 3** in [4]

D.2.2 Proposal 2: The Anagram Distance Between Any Two Sequences[3]

Occasionally, one shall find that they need to quantify how different some two or more sequences are. In the simplest case, we might merely sort a collection of sequence by the relative size of each sequence, and thus differentiate them by their sequence cardinality for example — $\vartheta(\Theta)$ Vs $\vartheta(\Theta^*)$ for some two sequences involved in a general transformation such as:

$$\textbf{Transformation 32. } \Theta \rightarrow \Theta^* \implies \nabla_{\Theta, \Theta^*} = (\vartheta(\Theta) - \vartheta(\Theta^*));$$

$$\nabla_{\Theta, \Theta^*} = \begin{cases} > 0 & \text{compression} \\ 0 & \text{conservation} \\ < 0 & \text{protraction} \end{cases}$$

or if we only wish to care about absolute distance..

$$\textbf{Transformation 33. } \Theta \rightarrow \Theta^* \implies |\nabla|_{\Theta, \Theta^*} = |\vartheta(\Theta) - \vartheta(\Theta^*)|;$$

$$|\nabla|_{\Theta, \Theta^*} = \begin{cases} > 0 & \text{different} \\ 0 & \text{similar} \end{cases}$$

However **Transformation 32** and **Transformation 33** only cater for comparisons or analysis relative to size of the sequences being compared. And such that $\nabla_{\Theta, \Theta^*}$ or $|\nabla|_{\Theta, \Theta^*}$ shall work just fine where the purpose of the sequence analysis is just concerned with the relative sizes of the sequences. But, and as we established in the **Anagram Distance Theory**(ADT) paper[3], there are many important cases when it is necessary to compare two or more sequences **based on the relative ordering of members in them and not just by their member composition or frequencies**. Thus the **Anagram Distance Measure**(ADM), and which, for any two sequences as presented above, the essential measure would be:

$$\tilde{A}(\Theta \rightarrow \Theta^*) = \frac{1}{\vartheta(\Theta)} \times \sum_{\forall \theta_i \in \Theta}^{\vartheta(\Theta)} |I(\theta_i, \Theta) - I(\theta_i, \Theta^*)| \quad (\text{D.4})$$

Which, as we saw in [3], has the interesting interpretation as:

$$\tilde{A}(\Theta \rightarrow \Theta^*) = \begin{cases} 0, & \Theta = \Theta^*, \\ \leq 1, & \Theta \approx \Theta^*, \\ > 1, & \Theta \ll \Theta^*. \end{cases} \quad (\text{D.5})$$

And otherwise a more elaborate analysis as worked out in **Section 6.3** of [8]:

$$\tilde{A}(\Theta \rightarrow \Theta^*) = \begin{cases} 0, & \Theta = \Theta^*, \text{no differences} \\ < 1, & \Theta \approx \Theta^*, \text{different in at minimum two positions} \\ = 1, & \Theta \neq \Theta^*, \text{all members shifted by exactly 1 position} \\ > 1, & \Theta \ll \Theta^*, \text{potential indication there is chaos.} \end{cases} \quad (\text{D.6})$$

And thus can help us observe/compute, quantify and [objectively] appreciate, the **relative disorder** resulting from one sequence being transformed into another (of potentially similar cardinality and member composition/distribution). In earlier work, we have already demonstrated and argued that the **ADM** is a very important formalism in mathematical statistics and statistical analysis, especially in cases dealing with **measures of variability** (MoVs) where traditional measures such as the range, mean deviation, variance and standard deviation statistics cannot (**or fail to**) catch any actual differences between two or more sequences as we quantifiably/provably demonstrated in [5], but also where simpler difference measures such as just differences in cardinality (such as in **Transformation 32**) aren't sufficient or conclusive enough for the special cases that the ADM caters for; more about this topic has also been covered well in earlier work[8].

D.2.3 Proposal 3: The Transformer Compression Ratio of any Sequence Transformation[5]

In **Transformation 32** we have seen that there might be cases in which a transformation results in a sequence that is observably larger or smaller than the source sequence; **protraction** or **compression** respectively. And as was laid down in [5] — refer to **Section 4.3** of [5] — we can measure the *overall* effect of a transform on the *entire* sequence merely by comparing the sequence cardinalities of the source and resultant sequence as such:

$$\xi(\Theta \rightarrow \Theta^*) = \frac{\underline{\nu}(\Theta^*)}{\underline{\nu}(\Theta)} \quad (\text{D.7})$$

The measure, $\xi(\Theta \rightarrow \Theta^*)$ is called the **Transformer Compression Ratio** (TCR), and when compared against $\nabla_{\Theta, \Theta^*}$, note that TCR is always positive, whereas the later, as we saw in **Transformation 32**, might be negative too; this,

because, well, TCR and $\nabla_{\Theta, \Theta^*}$ or even $|\nabla|_{\Theta, \Theta^*}$ all are based on comparing the cardinalities of the source and resultant sequences involved in a transformation, however, unlike them, TCR is computed via a division operation (thus ratio), while the others are based on a subtraction operation. In fact, TCR spans $\mathbb{R}^+$ as was laid out in [5], and as we saw in that work, given $\nu(\Theta) \geq 0$ for any sequence Θ , and if $\nu(\Theta) \geq 1$ for any **meaningful transformation**⁵, can be interpreted as such:

$$\xi(\Theta \rightarrow \Theta^*) = \begin{cases} 1, & \text{the size of } \Theta \text{ was preserved,} \\ 0, & \Theta \text{ was reduced to nothing,} \\ > 1, & \Theta \text{ was extended/protracted,} \\ < 1, & \Theta \text{ was reduced/compressed.} \end{cases} \quad (\text{D.8})$$

D.2.4 Proposal 4: The Piecemeal Compression Ratio of Any Sequence Transformation[5]

Further, with regards to how we might quantify the effects of some sequence transformation, note that, in cases such as:

Transformation 34. $\Theta : \mathbb{N} \times \psi_\beta \rightarrow \Omega : \mathbb{N} \times \psi_\alpha;$
 $\psi_\beta \neq \psi_\alpha \quad \vee \quad \exists \omega \in \Omega : \omega \notin \Theta$

As explained in [5] — refer to **Section 4.3** of [5] — when the source and resultant sequence do not share any members in common, **and yet we wish to evaluate the deformation aspects of the transform**, then we can only compare them using a measure such as **TCR** — it only cares about sizes and not membership. However, in case they **at least** share 1 member in common, then, we can restrict our comparison to only their common aspects — i.e $\psi(\Theta) \cap \psi(\Theta^*)$, and thus, as laid out in **Definition 3** in [5], we can compute a special measure called the **Piecemeal Compression Ratio(PCR)** as such:

$$\Psi(\Theta \rightarrow \Theta^*) = \frac{1}{\nu(\psi_\Theta \cap \psi_{\Theta^*})} \times \sum_{\forall x \in (\psi_\Theta \cap \psi_{\Theta^*})} \frac{\nu(x \in \Theta^*) - \nu(x \in \Theta)}{\nu(x \in \Theta)} \quad (\text{D.9})$$

And for the case of **Transformation 34**, can be re-written as:

⁵Well, this is somewhat murky ground, because, in fact, it can make sense for the source sequence to have 0 cardinality such as in a transform as $\Theta \xrightarrow{f(\cdot)} \Theta^*$ where for example $f(\cdot)$ is an operator that can output a non-empty sequence even when given an empty input — such as the case of a **sequence generator**!

$$\Psi(\Theta \rightarrow \Theta^*) = \frac{1}{\mathcal{U}(\psi_\beta \cap \psi_\alpha)} \times \sum_{\forall x \in (\psi_\beta \cap \psi_\alpha)} \frac{\mathcal{U}(x \in \Theta^*) - \mathcal{U}(x \in \Theta)}{\mathcal{U}(x \in \Theta)} \quad (\text{D.10})$$

And as was laid out in that foundational paper on transformatics[5], we can interpret **PCR** values as such⁶

$$\Psi(\Theta, \Theta^*) = \begin{cases} \text{undefined}, & \psi_\Theta \cap \psi_{\Theta^*} = \emptyset \implies \text{no members in common,} \\ 0, & \mathcal{U}(x \in \Theta) = \mathcal{U}(x \in \Theta^*) \forall x \in \psi_\Theta \cap \psi_{\Theta^*} \implies \text{conservation,} \\ > 0, & \exists x \in \psi_\Theta \cap \psi_{\Theta^*} : \mathcal{U}(x \in \Theta^*) > \mathcal{U}(x \in \Theta) \implies \text{protraction,} \\ < 0, & \exists x \in \psi_\Theta \cap \psi_{\Theta^*} : \mathcal{U}(x \in \Theta^*) < \mathcal{U}(x \in \Theta) \implies \text{compression.} \end{cases} \quad (\text{D.11})$$

Finally, note that, like $\nabla_{\Theta, \Theta^*}$ and $|\nabla|_{\Theta, \Theta^*}$ that we saw in **Proposal 2**, **PCR** is based on a subtraction operation, however, and in addition to that, like both ADM and TCR, it also has division operations in it that makes it take on fractional number[2] values too. In fact, $\Psi(\Theta, \Theta^*) \in \mathbb{R}$ or equivalently $-\infty \leq \Psi(\Theta, \Theta^*) \leq \infty$, unlike ADM that only spans $\mathbb{R}^+$, and thus, is great for evaluating deformations involving either growth or contraction as we have seen.

D.2.5 Proposal 5: The Modal Sequence Statistic of Any Sequence[5]

For especially situations when we are not interested in comparing one sequence to another, but when we merely wish to better understand some particular sequence — like say, not just its size as could be told by computing the **sequence cardinality**, nor about its membership as we might tell using the **sequence symbol set**, there might be a case when we want to combine those two concepts into one; such as when we wish to **compare the relative frequency of members in a particular sequence**. Essentially, we might be faced with the problem:

Problem 4. *Given*

$\Theta^n : \mathbb{N} \times \psi_\beta : \theta_i, \theta_j \in \Theta^n : \theta_i \neq \theta_j \quad \wedge \quad i, j \in [1, n] : n \in \mathbb{N} : 1 \leq \mathcal{U}(\theta_i \in \Theta^n) < n,$
is it true that $\mathcal{U}(\theta_i \in \Theta^n) \geq \mathcal{U}(\theta_j \in \Theta^n)$?

So, as laid out in **Section 4.1** of [5], we can approach a solution to **Problem 4** by leveraging the concept of the **modal sequence statistic** (MSS). Essentially,

⁶More research needed, with regards to this matter though.

we would compute $\overset{>}{\Theta}^n$, which, by **Definition 1** in that foundation paper on transformatics[5], would give us a resultant sequence based on/derived from the source sequence Θ^n , such that the elements/members of the former exactly span the symbol set of the later/source sequence — i.e. $\psi(\Theta^n)$, and yet, there are not only no duplicates, but that the elements are sorted such that the member/symbol that occurs the most (with highest frequency) in Θ^n , occurs earliest in $\overset{>}{\Theta}^n$ than any other element, or at least (such as where any two elements have the same relative frequency in Θ^n), they occur in their natural order of first-occurrence.

Thus, if we compute a MSS for the sequence specified in **Problem 4** as such:

Transformation 35. $\Theta^n \xrightarrow{O_{mss}(\Theta^n)} \overset{>}{\Theta}^n$

Then, we can solve the problem thus:

Solution 3. $\mathbb{1}(\theta_i \in \Theta^n) \geq \mathbb{1}(\theta_j \in \Theta^n)? = \begin{cases} true & I(\theta_i, \overset{>}{\Theta}^n) \leq I(\theta_j, \overset{>}{\Theta}^n) \\ false & otherwise \end{cases}$

Not only shall we come to appreciate and love the MSS for being a statistical measure first introduced in a work on transformatics[5], but that, like the traditional statistical measure it is inspired by — the **mode** of any dataset/sequence — it is indispensable as a summary statistic, and yet, unlike and though as powerful as the mode, gives us a whole collection of summary statistics in one measure (as a **sequence or vector of modes**, and not merely as a single scalar value which is all that the traditional mode is).

Also, we have argued in several works exploring and applying transformatics — such as in [5], [3] and [8] — that the modal sequence statistic can be more useful to compute than many traditional summary statistics, but also that, it can help us arrive at other, more useful statistics (such as the next two sections are going to tackle), and for domains or problems such as in **statistical artificial intelligence**, machine learning and data mining to name but a few, is one of few core measures that can **greatly simplify decision making problems in an autonomous way**.

Finally, note that, as per the notation and arguments employed in **Definition 8** in [8] when defining and formalizing the related concept of the sequence characteristic, we can succinctly define the modal sequence statistic for any sequence Θ , as such:

Definition 14 (The Modal Sequence Statistic, $\overset{>}{\Theta}$).

Given any sequence Θ that consists of well ordered finite collection of n symbols,

ω_i , possibly with some being repeated — essentially that we have the sequence $\Theta : \mathbb{N} \times \psi_\omega \quad \wedge \quad \mathcal{L}(\Theta) = n$, then the measure:

$$\overset{>}{\Theta} = \langle \prod_{\omega_i \in \psi_\omega} \omega_i \rangle \quad (\text{D.12})$$

is the **modal sequence statistic** (MSS) of the given sequence, and that $\psi_\omega \cong \psi(\Theta)$, such that, we can produce or generate $\overset{>}{\Theta}$ from Θ via the following transformer:

Transformer 18 (The MSS Generator, $O_{mss}(\Theta)$).

$$\Theta \xrightarrow{O_{mss}(\Theta)} \overset{>}{\Theta} = \langle \prod_{\omega_i \in \psi_\omega} \omega_i \rangle;$$

$$\forall \omega_i \in \Theta \quad \exists \omega_i \in \overset{>}{\Theta}$$

$$\wedge \quad I(\omega_i, \overset{>}{\Theta}) < I(\omega_j, \overset{>}{\Theta})$$

$$\implies \quad \mathcal{L}(\omega_i \in \Theta) > \mathcal{L}(\omega_j \in \Theta) \quad \vee \quad I(\omega_i, \Theta) < I(\omega_j, \Theta) \quad \forall i, j \in \mathbb{N}$$

$$\wedge \quad \psi_\omega \cong \psi(\Theta) \cong \overset{>}{\Theta}$$

$$\wedge \quad \mathcal{L}(\psi(\Theta)) = \mathcal{L}(\overset{>}{\Theta}) \leq \mathcal{L}(\Theta) \quad \square$$

D.2.6 Proposal 6: The Characteristic of Any Sequence[8]

In **Proposal 5**, we have encountered the MSS, which is a neat summary statistic applicable to any sequence. However, and as you might notice when we are presented with a sequence such as:

$$\text{golden ratio} = 1.618033988749895 \implies \Theta_{gr} = \langle 1, 6, 1, 8, 0, 3, 3, 9, 8, 8, 7, 4, 9, 8, 9, 5 \rangle \quad (\text{D.13})$$

Which is essentially the golden ratio⁷ reduced to a sequence of just its first 16 significant digits, and which, if we must summarize it using say $\overset{>}{\Theta}_{gr}$ as such:

$$\textbf{Transformation 36. } \Theta_{gr} \xrightarrow{O_{mss}(\cdot)} \overset{>}{\Theta}_{gr} = \langle 8, 9, 1, 3, 6, 0, 7, 4, 5 \rangle$$

⁷Basically, $\frac{(1+\sqrt{5})}{2}$

maps to a summary statistic, its modal sequence, of cardinality 9, and yet, if one wanted to solve the following problem:

Problem 5. *Given the sequence Θ_{gr} in **Equation D.13**, how can we exactly tell if indeed $\mathfrak{u}(3, \Theta_{gr}) > \mathfrak{u}(6, \Theta_{gr})$, or if $\mathfrak{u}(3, \Theta_{gr}) = \mathfrak{u}(6, \Theta_{gr})$?*

Note that though we might attempt to solve **Problem 5** naively by merely counting the terms in the sequence under question, or by merely checking the relative positions of the involved terms in the computed MSS such as in **Transformation 36**, it is not the best way to arrive at a solution. Moreover, and as we have encountered in the previous proposal, and especially since we know that elements that tie on frequency shall occur adjacent to each other in the MSS of a sequence — ignoring their order of first-occurrence — and yet, without having a way to look at the frequencies of each element in the MSS, we might not correctly solve this problem *elegantly* and/or optimally (for very large sequences such as the kind we saw when treating of genetic code sequences[8] that might span millions of terms).

And so, as laid out in a treatise on how transformatics might be applied in the sequence-heavy science of genetics[8], in particular, **Section 6.4** of [8], instead of just the MSS, we can instead compute the related **sequence characteristic** (SC) statistic, and which, as per **Definition 8** in that work, consists of both the same exact elements as in the MSS, but with each distinct symbol followed by/annotated by its relative frequency in the original/source sequence. Essentially, given some sequence Θ , and given we know its modal sequence statistic is $\overset{>}{\Theta}$, the SC is as:

$$\hbar(\overset{>}{\Theta}) = \prod_{\omega_i \in \psi(\Theta)} \omega_i \cdot f_i \quad (\text{D.14})$$

Which, if we apply **Equation D.14** to our example sequence Θ_{gr} , produces the corresponding sequence characteristic as such:

Transformation 37. $\Theta_{gr} \xrightarrow{O_{mss}(\cdot)} \overset{>}{\Theta}_{gr} \xrightarrow{O_{sc}(\cdot)} \hbar(\overset{>}{\Theta}_{gr}) \rightarrow 8_4.9_3.1_2.3_2.6_1.0_1.7_1.4_1.5_1$
 $\rightarrow \mathbf{84.93.12.32.61.01.71.41.51} \rightarrow \mathbf{849312326101714151}$

Note that, in **Transformation 37**, we had the liberty to express the sequence characteristic in 3 different ways other than what was originally demonstrated in [8]. In particular, the alternative expressions for the SC thus shown, shall, such

as in this case when we are dealing with a sequence made of entirely numbers/digits, help to make the SC more readable/understandable; instead of merely concatenating the computed frequency of each symbol to the associated symbol such as the basic definition for the SC calls for (see **Definition 8** in [8]), and yet, without loss of generality, we can redefine the SC as such:

$$\hbar(\overset{>}{\Theta}) = \prod_{\omega_i \in \psi(\Theta)} \omega_i f_i \quad (\text{D.15})$$

So that, as we have seen in the first SC expression in **Transformation 37**, we can merely write the SC for Θ_{gr} as such:

$$\hbar(\overset{>}{\Theta}_{gr}) = 8_4.9_3.1_2.3_2.6_1.0_1.7_1.4_1.5_1 \rightarrow 8_4 9_3 1_2 3_2 6_1 0_1 7_1 4_1 5_1 \quad (\text{D.16})$$

The final form in **Equation D.16** also kicking out the “.” symbols meant to show/imply that we are merely concatenating the terms to each other. In fact, the SC could be also expressed as a sequence of **2-grams**⁸:

$$\hbar(\overset{>}{\Theta}) = \prod_{\omega_i \in \psi(\Theta)} \langle \omega_i, f_i \rangle \quad (\text{D.17})$$

So that, we can again further express the sequence characteristic for the golden ratio as such⁹

$$\hbar(\overset{>}{\Theta}_{gr}) = \langle \langle 8, 4 \rangle, \langle 9, 3 \rangle, \langle 1, 2 \rangle, \langle 3, 2 \rangle, \langle 6, 1 \rangle, \langle 0, 1 \rangle, \langle 7, 1 \rangle, \langle 4, 1 \rangle, \langle 5, 1 \rangle \rangle \quad (\text{D.18})$$

At this juncture, and returning to **Problem 5**:

Solution 4. *Given Θ_{gr} , we can merely compute its sequence characteristic as shown in **Equation D.18**, and by inspecting the entries in that resulting SC for the two symbols under question — ‘3’ and ‘6’, which are known to map to the*

⁸In **Chapter 3** of [8] we already introduced **N-grams** or rather higher-order sequences, but shall revisit them again in **Proposal 8**.

⁹**FUTURE WORK:** feels like we might be able to visually/graphically express the sequence characteristic summary statistic! Especially for sequences of numeric data — essentially, treating the SC in the form shown in **Equation D.17** as coordinates in a Cartesian space!

entries: $\langle \dots \langle 3, 2 \rangle, \langle 6, 1 \rangle \dots \rangle$ in the SC, confirm that actually $\nu(3 \in \Theta_{gr}) > \nu(6 \in \Theta_{gr})$ since $(\nu(3 \in \Theta_{gr}) = 2) > (\nu(6 \in \Theta_{gr}) = 1)$ $\square$

Before we move on, and as we shall come to appreciate when we come to **Proposal 9**, we can solve many problems not only by leveraging the theory and mathematics of transformatics, but also by leveraging its philosophy, which, as one shall establish, underlies how the **TEA** computer programming language[9] works. Essentially, and with reference to the present proposal, note that we can readily and elegantly compute the sequence characteristic of any sequence¹⁰ by leveraging **sequence-transformers** and sequence-processors as shown in the following TEA program that can compute the SC of any input sequence, and produce an SC expression similar to what we saw in **Transformation 37**:

TEA Program: SC-Transformer

```

1 #SC-Computer
2 #i!: {1618033988749895}
3 v: vIN
4 u!: | v: vMSS | v: vSC: {}
5 l: lPROC
6 y: vMSS | d!: ^ . | v: vW
7 y: vMSS | d: ^ . | v: vMSS | y: vW
8 f: ^ $: lEND
9 y: vIN | d*!: vW | v: | v!: | v: vF | v: vD: _ | g*: _: vW: vF: vD | v: vSC1
   |
10 g*: {}: vSC: vSC1 | v: vSC | j: lPROC
11 l: lEND
12 y: vSC | r!: _: . | d: . $
13 # (=8_4_9_3_1_2_3_2_6_1_0_1_7_1_4_1_5_1)

```

Figure D.1: TEA EXAMPLE: **SC-Computer**: transforms any input sequence into its sequence characteristic!

In case you have a particular sequence of interest that you wish to analyze or for which you wish to compute the SC, then, based on the above theory and the sample TEA program we have just looked at, you can go to the **WEB TEA IDE** — <https://tea.nuchwezi.com>, and paste in your sequence into the ‘TEA INPUT’ section (for example paste in your long genetic code sequence, a text file, votes/elections tally data well-encoded etc.), and by running it against the SC-program — which you might load as:

¹⁰Sometimes though, only after slight modifications or clean-up/pre-processing of the input sequence expression such as we saw in **Equation D.13**.

https://tea.nuchwezi.com/?fc=https://gist.githubusercontent.com/mcnemesis/b5b9957179a4a1d43b965959f9bb29fb/raw/sc_generator.tea

You shall then be able to replicate the results and analyses we have conducted in this proposal — for example, applied to election/votes datum, can help readily identify the winner.

D.2.7 Proposal 7: The Population Characteristic of Any Collection of Sequences[8]

In **Proposal 6**, we have dealt with the matter of applying transformatics towards extending as well as simplifying various important aspects of statistics — statistical analysis in particular, and which work might also underlie both **descriptive statistics** (generation of graphs and visualizations summarizing or describing some dataset or sequence) and **computational statistics** (such as in modeling behavior or distributions or implementing statistical artificial intelligence programs that reason about the statistical patterns in a dataset, sequence or entire collections of them).

In this section though, and building upon what we have already encountered, we are to re-iterate the concept of the **Population Characteristic** (PC), that was first presented in the earlier work about transformatics in GENETICS[8]. That book, in **Section 6.6**, presented a problem for which this proposed concept is [part of] the solution. More specifically, and without repeating what was already well discoursed then, we note that the PC has the following useful properties:

1. Given any **flat-structure**/first-order sequence, Θ , the PC is essentially the same as its SC — in terms of what its computation produces.
2. In case two or more sequences are to be reduced to a single characteristic — in which case it becomes the **population characteristic** for the combined set, the PC thus computed shall be equivalent to the SC of a sequence generated from flattening and combining all the involved sequences into a single sequence.
3. Where two or more sequences are involved, and where the SC for each sequence is already known or computed, the PC for the entire collection can readily be derived from the SCs thus already computed as an optimal strategy.

In summary though, and given a collection of n sequences¹¹ — basically a sequence of n -subsequences — denoted Θ^n , we then can compute the PC as such:

¹¹Potentially of arbitrary length and composition.

$$\hbar(\overset{>}{\Theta}^n) = \prod_{\forall \omega_i \in \psi(\Theta^n)} \omega_i \cdot \mathcal{U}(\omega_i \in \Theta^n) \quad (\text{D.19})$$

Where $\forall \Theta_j \in \Theta^n$

$$\mathcal{U}(\omega_i \in \Theta^n) = \sum_{\forall \hbar(\overset{>}{\Theta}_j)} f_{\omega_i} \quad (\text{D.20})$$

And $\forall i, j \in \mathbb{N} : i < j \implies f_i \geq f_j$ for the frequency terms in $\hbar(\overset{>}{\Theta}^n)$.

Note that, as laid out in [8], the PC can be very indispensable in problems **where we need to analyze multiple sequences in-groups or clusters** — such as in classification problems, clustering as part of artificial intelligence systems performing tasks such as **self-modifying maps, unsupervised learning**, etc. All of which are critical matters in statistical artificial intelligence, but also which could be useful in many other areas of applied mathematics and computation.

Before we conclude this section, note that, just like we did with and saw with the SC in **Proposal 6**, we can use the TEA language to compute a population characteristic statistic for arbitrary sequences (after careful and proper pre-processing perhaps). But also, and concerning how we might express the final output/PC, we can chose from several meaningful expression forms as proposed for the SC in **Proposal 6** and depending on where or how the PC expression or statistic thus generated is to be utilized.

D.2.8 Proposal 8: A Super Sequence from One or More Sequences[8]

The idea of treating of sequences in other than their flat-structure form was first presented as part of the discussion in how to apply transformatics in GENETICS[8]; refer to **Chapter 3** of [8]. In that work, we demonstrated how, for cases such as in dealing with problems where a sequence might need to be processed in “chunks” of a fixed size — also known as **n-grams**, we might want to introduce abstractions that can allow us to form smaller-sequences/sub-sequences within a larger/the original flat-structure sequence.

For example, we saw that, in case we have some sequence of symbols Θ^n defined as such:

$$\Theta^n = \langle a_1, a_2, \dots, a_n \rangle \quad (\text{D.21})$$

And that if we wish to re-write the same sequence such that every k terms are grouped in the same sub-sequence, we then can re-write Θ^n differently as such:

$$\Theta^n \rightarrow \Theta^* = \Theta_2(n, k) = \langle a_{11}, a_{12}, a_{13}, a_{1k}, a_{21}, a_{22}, \dots, a_{(i)(k)}, \dots, a_{(\frac{n}{k})(k-1)}, a_{(\frac{n}{k})(k)} \rangle \quad (\text{D.22})$$

Further, and for any arbitrary sequences, we saw that we could leverage a special kind of transformer that can take a flat-structure or lower-order sequence and from it generate a similar, but higher-order sequence (also understood to mean, a sequence of some n-grams/k-grams such that $k > 1$), as such:

Transformer 19 (The **k-GRAM Generator**). $\Theta^n = \langle a_1, a_2, \dots, a_n \rangle \xrightarrow{O_{bundle-k}(\cdot)} \Theta^*$;
 $\Theta^* = \langle \langle a_1, a_2, \dots, a_k \rangle_1, \langle a_{k+1}, \dots, a_{2k} \rangle_2, \dots, \langle a_{(n-k+1)}, \dots, a_{(n-k+k-1)}, a_{(n-k+k)} \rangle_{\frac{n}{k}} \rangle$
 $\forall a_i \in \Theta \quad \exists a_{ij} \in \Theta^* : a_i = a_{ij} \quad \wedge \quad j \in [1, k]$
For some $n, k \in \mathbb{N}$, and that $\mathfrak{u}(\Theta^) = \frac{n}{k} = \text{number of } k\text{-gram subsequences generated from } \Theta$. $\square$*

For future purposes, and without deviating from the theory already presented in earlier works, we might also refer to such produced sequences as Θ^* in **Transformer 19** as a **Super Sequence**.

We shall note that if Θ^* is a super-sequence of/relative-to another sequence, such as the sub-sequence $\langle a_1, a_2, \dots, a_k \rangle_1$ in **Transformer 19**, and which we might denote as Θ_1 , then, we can correctly say that:

$$\Theta^* \supset \Theta_1 \quad \wedge \quad \psi(\Theta^*) \supset \psi(\Theta_1) \quad (\text{D.23})$$

And that generally,

$$\forall \Theta_i \in \Theta^* \quad \implies \quad \Theta^* \supset \Theta_i \quad \wedge \quad \psi(\Theta^*) \supset \psi(\Theta_i) \quad (\text{D.24})$$

Also, and as shown in **Section 6.6.1** of [8], we might also treat of super-sequences as matrices (in the sense of **linear algebra**). In particular, note that if we have a set of n different sequences of arbitrary composition and lengths:

$$\Omega^n = \langle \Omega 1, \Omega 2, \Omega 3, \dots, \Omega n \rangle \quad (\text{D.25})$$

That we might also express them more elaborately via matrix form as¹²

$$\Omega^n = \begin{bmatrix} \Omega 1 \langle \prod_{i=1}^{\alpha} a_{1i} \rangle, \\ \Omega 2 \langle \prod_{i=1}^{\beta} a_{2i} \rangle, \\ \Omega 3 \langle \prod_{i=1}^{\chi} a_{3i} \rangle, \\ \vdots \\ \Omega n \langle \prod_{i=1}^{\zeta} a_{ni} \rangle, \end{bmatrix} = \begin{bmatrix} \Omega 1 \langle a_{11}, a_{12}, \dots, a_{1\alpha} \rangle, \\ \Omega 2 \langle a_{21}, a_{22}, \dots, a_{2(\beta-1)}, a_{2\beta} \rangle, \\ \vdots \\ \Omega n \langle a_{n1}, a_{n2}, \dots, a_{n\zeta} \rangle \end{bmatrix} \quad (\text{D.26})$$

So that, if we have the special case as what **Transformer 19** would produce if given the right kind of input sequence (particularly, that if Θ^n is to be split into k -grams, then, that it has the property that $\frac{n}{k} = m$ is a pure number[2], and so that, we can then map the flat-structure sequence Θ^n to a higher-order sequence, or super-sequence of **dimensions** $k \times m$, so that the resulting mathematical structure can be neatly expressed as a $k \times m$ or $m \times k$ matrix as shown:

$$\textbf{Transformation 38. } \Theta^n = \langle \theta_1, \theta_2, \dots, \theta_n \rangle \xrightarrow{O_{\text{bundle}-k}(\Theta^n)} \begin{bmatrix} \theta_{11}, \theta_{12}, \dots, \theta_{1k}, \\ \theta_{21}, \theta_{22}, \dots, \theta_{2k}, \\ \vdots \\ \theta_{m1}, \theta_{m2}, \dots, \theta_{mk} \end{bmatrix}$$

So that, if $m = k$, then we essentially have a super-sequence that can be expressed as a *square-matrix* of dimensions $m \times m$. Talking of which, once we can or do express a sequence as a matrix¹³, almost all of the mathematical theory of linear-

¹²Notice how the structures in **Equation D.26** look like *column matrices*?

¹³Even as a *sparse matrix* such as in cases where a low-order sequence Θ^n needs be transformed into a super-sequence, but when either $\frac{n}{k}$ is not a pure number, or that there are some sub-sequences that might have empty positions.

algebra does readily apply on such sequences from transformatics! And thus we sum up our proposal on transformatics and super-sequences.

D.2.9 Proposal 9: Programs as Chains of Sequence Transformers (Transformer-Chains)[8][9]

First, note that we already encountered an example of/application of this present proposal in the useful program we encountered at the end of **Proposal 6**; in a program that basically computes the sequence characteristic of any input sequence, and which is essentially a *higher-order transformer* that is composed as a chain of smaller sequence transformers — which is what all or most of **TEA primitive instructions**[43] are), and which transformer then (as the “SC-Generator”) essentially reduces any input sequence to its summary statistic expression, particularly the sequence characteristic.

Otherwise, note that; looking at a TEA program as depicted in **Figure D.1** is helpful, since it reflects how the overall [higher-order] transformer program might be composed out of logically-meaningful sections¹⁴ as a software engineer or computer programmer already familiar with the TEA language might approach the SC problem. However, also note that; that same exact program is also equivalent to the following two versions — both of which kick-out *TEA comments*¹⁵, and **only focus on the individual TEA instructions necessary to make the program work as desired, as well as how they ought be ordered within the program**, but where, the first (**Listing D.2.9**) simplifies the code to just a sequence of TEA instructions one per line, while the second (**Listing D.2.9**) shows the same exact program, in its **minified** form — where, each instruction is delimited from the next by the standard TEA instruction delimiter ‘|’, and nothing else.

TEA Program: SC-Generator

```

1 v : v IN
2 u ! :
3 v : v MSS
4 v : v SC : {}
5 l : l PROC
6 y : v MSS
7 d ! : ^ .

```

¹⁴In TEA parlance and in relation to how most computer scientists might think of the matter of *scoping*, *code-blocks* or *encapsulation*, for TEA, it is mostly all about **l-blocks** or *label-blocks* — also known as “labeled blocks” [43].

¹⁵In the example source-code we shall encounter in this work, but also typically, these are expressions in a TEA program that start with the ‘#’ character, and which makes them inert or non-executable lines/expressions in the program.

```

8  v:vW
9  y:vMSS
10 d: ^ .
11 v:vMSS
12 y:vW
13 f: ^ $ : lEND
14 y:vIN
15 d*! : vW
16 v:
17 v!:
18 v:vF
19 v:vD:_
20 g*:_:vW:vF:vD
21 v:vSC1
22 g*:{ }:vSC:vSC1
23 v:vSC
24 j:lPROC
25 l:lEND
26 y:vSC
27 r!:_ _:.
28 d: . $

```

And the same program in its minified form is as:

TEA Program: SC-Generator Minified

```

1  v:vIN|u!:|v:vMSS|v:vSC:{ }|l:lPROC|y:vMSS|d!: ^ . |v:vW|
   y:vMSS|d: ^ . |v:vMSS|y:vW|f: ^ $ : lEND|y:vIN|d*! : vW|v:|
   v!:|v:vF|v:vD:_|g*:_:vW:vF:vD|v:vSC1|g*:{ }:vSC:vSC1
   |v:vSC|j:lPROC|l:lEND|y:vSC|r!:_ _:.|d: . $

```

As one shall establish while studying the **26 TEA primitives A: to Z:**[43], or if they dive deeper into the potential **182 distinct TEA commands**[43]; apart from branching commands such as **F:** (the “Fork”) and **Q:** (the “Quit”) and commands such as **L:** (the “Label”) that don’t modify or alter the **Active Input(AI)**[43] — meaning, when they are encountered in a TEA program, they merely pass on the current input to the next instruction unmodified, most,

or all of the bulk of TEA programming consists of some sequence processing instruction that accepts some input [sequence], processes it or transforms it, and then returns the resultant [sequence] as the **Instruction Output**(IO).

Thus, and without loss of generality, we can **model or think of any program expressible in the TEA language as a computer program, abstract machine or automaton, expressible as or reducible to just an ordered chain of sequence transformers**. A good demonstration of this is the following basic program, which, when presented with any input text (a sequence of characters), reduces or transforms it into a resultant sequence of characters, preserving their original order, but with the transformation resulting in only UPPER-CASE CONSONANTS [derived from the source/input sequence].

TEA Program: CONSONANT FILTER

```
i:{This is a TEST 123} | z!: | d:[AEIOU] | d!:[A-Z]
```

Which program, when run as is, shall use the explicit hard-coded input it opens with — “This is a TEST 123”, and shall return “THSSTST”. While, if presented with some other input such as “hello world 123?”, shall return “HLLWRLD”.

The following transformation expression shall help illustrate exactly what is happening in that program, or how the different bits/chained sequence transformers come together to compose the higher-order transformer that is what the entire TEA program is:

Transformation 39. $\emptyset \xrightarrow{i:\{This\ is\ a\ TEST\ 123\}} \langle This\ is\ a\ TEST\ 123 \rangle \xrightarrow{z!:} \langle THIS\ IS\ A\ TEST\ 123 \rangle \xrightarrow{d:[AEIOU]} \langle THS\ S\ TST\ 123 \rangle \xrightarrow{d!:[A-Z]} \langle THSSTST \rangle$

and for the input “hello world 123?”:

Transformation 40. $\langle hello\ world\ 123? \rangle \xrightarrow{i:\{This\ is\ a\ TEST\ 123\}} \langle HELLO\ WORLD\ 123? \rangle \xrightarrow{z!:} \langle HLL\ WRLD\ 123? \rangle \xrightarrow{d!:[A-Z]} \langle HLLWRLD \rangle$

So, hopefully, after looking at those two example transformations, as well as the preceeding TEA source-code, you start to appreciate that TEA programs are indeed not only [higher-order] sequence transformers in the strict sense, but are likewise compositions of, chains of, sequence transformers.

Though the **TEA TAZ**[43] is still being worked-on as of this writing, much of what is required to understand or write TEA programs is already covered well in that living document, and so, it must be consulted and utilized by those who wish to understand better, but also apply readily, many ideas encountered while working with or developing ideas using the mathematics of transformatics [in/with computing¹⁶].

D.2.10 Proposal 10: Entropy of a Sequence[1]

Information expressions (IE)[1] are a mathematical structure, $\Omega^{1,\infty} : \mathbb{N} \times \Psi^\infty$, to which **Foundation Idea D.1.3** applies. The sequence $\Theta_{hw} = \langle \text{hello world} \rangle$ is one such example, with the properties:

$$\Theta_{hw} \rightarrow \left\{ \begin{array}{ll} \mathcal{L}(\Theta_{hw}) \rightarrow \mathbb{N} & \text{bits in information} \\ \mathcal{L}(\psi(\Theta_{hw})) \rightarrow \mathbb{N} & \text{cardinality of alphabet} \\ \frac{\mathcal{L}(\psi(\Theta_{hw}))}{\mathcal{L}(\Theta_{hw})} \rightarrow \mathbb{R}^+ & \text{sequence entropy} \\ \psi(\Theta_{hw}) \times \psi(\Theta_{hw})^{[1, \mathcal{L}(\Theta_{hw})]} \\ \vee \quad \psi(\Theta_{hw}) \times \psi(\Theta_{hw})^{[1, \infty]} \rightarrow \mathbb{N} \times \mathbb{N} & \text{derivative languages} \end{array} \right. \quad (\text{D.27})$$

Most important about what we can glean from **Equation D.27** is the fact that we might be able to compute a kind of **information-content** measure for any arbitrary sequence, merely by looking at the length of the sequence Vs the length of the symbol set spanned by that sequence. Essentially, and as shown in that distillation, given any sequence Θ , we could compute a **Sequence Entropy** for it — $\Phi(\Theta)$ as such:

Definition 15 (The **Sequence Entropy**, $\Phi(\Theta)$). *Given any sequence $\Theta \langle \theta_{i \in [1, n]} \rangle : \mathbb{N} \times \psi_*$, we can compute a measure of how much information (**sequence entropy**) there is in the sequence, using the following measure:*

¹⁶Essentially, the TEA language brings to the world, a new programming paradigm based on chaining sequence transformers.

$$\Phi(\Theta) = \frac{\mathcal{L}(\psi(\Theta))}{\mathcal{L}(\Theta)} = \frac{1}{n} \times \mathcal{L}(\psi(\Theta)) \quad (\text{D.28})$$

and if we wish to compute the **relative sequence entropy** — meaning, the amount of information the sequence contains relative to some particular symbol set/alphabet other than the symbol set spanned explicitly by the sequence — such as using ψ_* instead¹⁷ of $\psi(\Theta)$, in which case, the associated measure would be:

$$\Phi_{\psi_*}(\Theta) = \frac{\mathcal{L}(\psi_*)}{\mathcal{L}(\Theta)} = \frac{1}{n} \times \mathcal{L}(\psi_*) \quad (\text{D.29})$$

As far as how a measure such as the **sequence entropy** in **Definition 15** might go, note that, for our example input sequence Θ_{hw} , we can establish that the entropy is $0.727272\overline{72}$ as per:

$$\Phi(\Theta_{hw}) = \frac{\mathcal{L}(\psi(\Theta_{hw}))}{\mathcal{L}(\Theta_{hw})} = \frac{1}{n} \times \mathcal{L}(\psi(\Theta_{hw})) = \frac{1}{11} \times 8 = \frac{8}{11} = 0.727272\overline{72} \quad (\text{D.30})$$

Whereas, and as might be interesting to note given similar ideas such as the **Shannon Entropy**[73], realize that if we alter Θ_{hw} further, say, by introducing more redundancy or duplication of terms¹⁸, such as in the string “hello hello world”, the new sequence entropy shall likewise go lower as shown:

$$\Phi(\langle \text{hello hello world} \rangle) = \frac{1}{n} \times \mathcal{L}(\psi(\Theta)) = \frac{1}{17} \times 8 = \frac{8}{17} = 0.47058823529411764 \quad (\text{D.31})$$

And so, and as expected, since in this last case we increased the length of the sequence, though, we did not introduce any new **distinct symbols** — meaning, the sequence’s symbol set or alphabet remained the same as before ($\mathcal{L}(\psi(\Theta)) = 8$), we then see that indeed, as we would expect of a good entropy measure, $\Phi(\Theta)$ did indeed decrease in value after this sequence transformation.

Of course, if we were to use the **relative sequence entropy** instead, and for our example, using say the English/Latin alphabet (ψ_{az}) as the reference, the

¹⁷For example, for the sequence “hello world”, we might wish to measure the entropy relative to the entire Latin Alphabet (ψ_{az}) instead of just the symbol set of the string ($\psi(\Theta_{hw}) = \{\text{lohe wrd}\}$).

¹⁸Already, Θ_{hw} contains two instances of symbol ‘l’, which thus reduces its *unexpectedness* or *surprise* value, and thus its entropy.

associated entropy measures for the two cases we have considered would be as such:

$$\Phi_{\psi_{az}}(\Theta_{hw}) = \frac{1}{n} \times \mathcal{U}(\psi_{az}) = \frac{1}{11} \times 26 = \frac{26}{11} = 2.36363636\overline{36} \quad (\text{D.32})$$

and for the transformed/modified sequence:

$$\Phi_{\psi_{az}}(\langle \text{hello hello world} \rangle) = \frac{1}{n} \times \mathcal{U}(\psi_{az}) = \frac{1}{17} \times 26 = \frac{26}{17} = 1.52941176470588235294 \quad (\text{D.33})$$

For any particular sequence of interest, the following associated, and generic **WEB TEA** program can be used to compute the sequence entropy ($\Phi(\Theta)$):

TEA Program: Sequence Entropy Computer

```

1 # e.g given input as `hello hello world`
2 v:vIN|v*!:vIN|v:vLIN|y:vIN
3 |u!|v:vMSS|v*!:vMSS|v:vLMSS
4 |g*:{/}:vLMSS:vLIN|v:vCMD|
5 z*:vCMD
6 # returns something like: 0.47058823529411764

```

In general, note that both measures ($\Phi(\Theta)$ and $\Phi_{\psi_*}(\Theta)$) shall always return a non-zero and non-negative value as long as they are applied to some sequence of non-zero length. However, how exactly our proposed entropy measure might relate to the more commonplace, standard information-theoretic Shannon Entropy measure shall be a matter of future research/evaluations¹⁹.

D.3 Conclusion

In summary, we have laid down the essential mathematical theory for how to treat of mathematical structures known as “sequences” as formally defined in

¹⁹**FUTURE WORK:** Establish how the transformativ sequence entropy relates to Shannon’s Entropy measure when applied to sequences of finite length relative to or not some reference alphabet/symbol set.

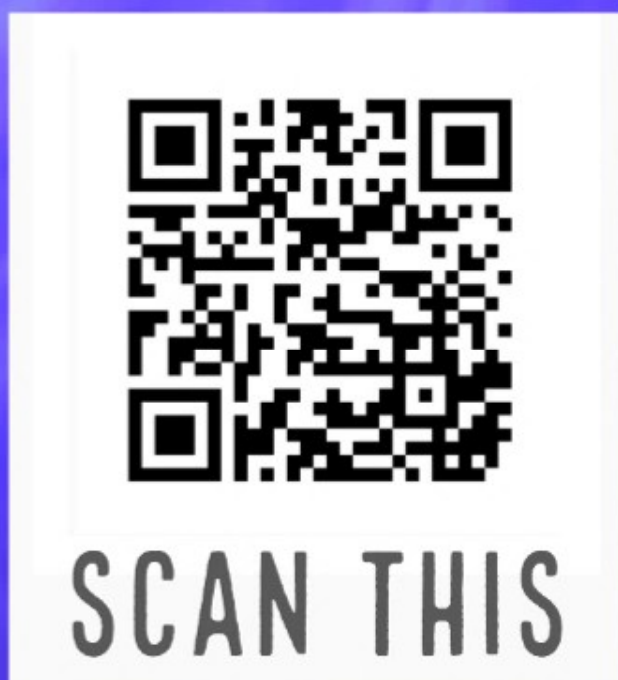
Foundation Idea D.1.3. Via **10 Proposals**, we have seen that we can leverage a mathematical and philosophical abstraction such as the **sequence** and **sequence-transformer** (as especially a transformatics formalism), so that then, we can readily build pragmatic/empirical solutions to any problems for which a solution can be conceptualized, manifested and proven, based on some model of the problem that allows the problem space to be fully specifiable via the use of one or more sequences. That these sequences might also be treated as or considered to be **information expressions**[1] — essentially, *arguments* or parameters — that for example **encode** the character, name, dimensions etc. of the system in which the problem might be rendered tractable/decidable/computable/... so that, then, by applying some particular sequence transformer [program²⁰] — itself, an information expression, though, of **a different kind** than the data upon which it might be applied²¹ — we can then manifest sequences that can modify or predictably operate on other sequences — a mechanism and capability which, when well orchestrated and manifested as is the case of the **TEA computer programming language**[43], brings to life a highly robust and useful arcanum practicum in the form of a living, reusable, [technology] platform (both in theory/formalism) and experience(executable mathematics) that, at foundation, is beautifully based on and comprehensible via the theory of TRANSFORMATICS as laid out in the **10 Foundations** and associated works.

TO CITE:

Lutalo, Joseph Willrich (2025). **TRANSFORMATICS 101 - explained**. I*POW. Thesis. <https://www.academia.edu/144344109/>

²⁰For this case, also equivalently a **definition**, specification, symbol sequence, etc. applicable to **transformers**

²¹Or, as is **the case** in **TEA**; where, as with the programs of a *Von Neumann Architecture* machine, allows the language to allow for a systematic means in which a program can be expressed with explicit data it might process as a default, but also with the possibility of the program operating on information *read* from the environment (when it is available/presented).



FOUNDATIONS
OF
A
NEW
BRANCH
OF
MATHEMATICS

10 Foundational Results → 10 Proposed Abstractions

TRANSFORMATICS
101 explained

2025 | I*POW

Appendix E

Situating TRANSFORMATICS (as a Theory or Field) within Mathematics:

E.1 A Working Definition and Clarification on Fundamental Nomenclature

Definition 16 (Transformatics). *The study of **sequences of symbols**, their transformations and analysis.*

To kick-start our discussion, note that the definition above shall help to distinguish transformatics as its inventor currently treats of it, from any other mathematical line of inquiry, but shall also help explain why past references to and parallels between transformatics and topics such as sequence analysis, sequence transformers, sequence generators and especially a heavy talk about symbol sets and sequences makes all the sense (refer to **Section D.1**). That said, note that we are going to attempt to properly, perhaps somewhat rigorously locate this theory and associated concepts within the vast spectrum of mathematics. However, first, let us clear-up the potential confusion concerning fundamental nomenclature and potential parallels between other fields and sub-fields of mathematics.

In keeping with standard nomenclature, but also applying it well, we shall want to help the reader, and especially the students of Transformatics, to readily distinguish between several somewhat similar, yet distinct concepts and terms from across mathematics, that might help anyone readily make sense of **Definition 16**. So, first, we are going to outline the distinctions and relationships between six foundational mathematical structures: **set**, **sequence**, **series**, **vector**, **matrix**, and **space**. These concepts are central to fields such as linear algebra, analysis, and symbolic systems [74, 75].

Comparison Table

Concept	Ordered?	Duplicates?	Dimensionality	Key Feature
Set	No	No	Abstract	Unordered collection of distinct elements
Sequence	Yes	Yes	1D	Ordered list indexed by $\mathbb{N}$
Series	Yes	Yes	1D	Sum of a sequence (often infinite)
Vector	Yes	Yes	1D (or n D)	Element of a vector space
Matrix	Yes	Yes	2D	Rectangular array of scalars
Space	N/A	N/A	n D (abstract)	Set with algebraic or topological structure

Figure E.1: Comparison of Several Fundamental Mathematical Structures related to Sequences in Transformatics

Definitions and Examples

Set

A *set* is a collection of distinct elements with no inherent order. For example, $\{2, 3, 5\}$ is the same as $\{5, 2, 3\}$. Sets are foundational in logic and topology [76]. Also, and concerning formal expression, note that authors such as Patrick M. Fitzpatrick, when introducing sets in their textbook undergraduate calculus[77], have this to say:

For a set A , the membership of the element x in A is denoted by $x \in A$ or x in A , and the nonmembership of x in A is denoted by $x \notin A$. A member of A is often called a *point* in A . Two sets are the same if and only if they have the same members. Frequently sets will be denoted by braces, so that $\{x \mid \text{proposition about } x\}$ is the set of all elements x such that the proposition about x is true.

That quote gives us a good introduction to the notation most commonly used to treat of sets — a structure that in one way or another, likewise serves immensely in justifying the formalisms likewise used in transformatics. Also, note that, authors like Walter Nef, introducing sets in a linear algebra text[78], uses a generalization

somewhat merely more abstract than the plain-english version above, but still in the same vein, when he writes

$$A = \{x_1, x_2, x_3, \dots\} = \{x; \phi(x)\} \quad (\text{E.1})$$

So that $\phi(x)$ denotes a statement by which the elements of A are characterized¹. Moreover, as we see in **Table E.1**, particularly for **only sets**, the order of members or elements **does not matter**, and also that, **duplication is not allowed**. Thus, we see below, that we might express a set with different orderings of its members, and still correctly refer to the same set:

$$VOWELS = \{a, e, i, o, u\} = \{e, i, o, a, u\} = \{u, o, i, e, a\} = \{x : x \in \psi_{vowels}\} \quad (\text{E.2})$$

Where ψ_{vowels} is the symbol set of only the 5 vowels of the Latin Alphabet.

Sequence

A *sequence* is an ordered list of elements possibly with repetition[32] — for transformatics though, we many times prefer to say “of symbols”. For instance, the Fibonacci sequence is (1, 1, 2, 3, 5, 8, ...), while the sequence of the vowels in the Latin Alphabet is² $\hat{\psi}_{az}(VOWELS) = \psi_{vowels}$ as depicted below:

$$\psi_{vowels} = \langle a, e, i, o, u \rangle \neq \langle e, a, i, o, u \rangle \neq \langle u, o, i, e, a \rangle \quad (\text{E.3})$$

Sequences are central in analysis and recursion theory [75], but also are at the core of transformatics³.

Note how, as in much of the recent literature on transformatics, we mostly prefer to express sequences using the notation $\langle a_1, a_2, \dots \rangle$ and not say $\{a_1, a_2, \dots\}$ — this, especially so we emphasize the fact that **order matters** in transformatics, and so we can **readily differentiate between unordered sets and ordered sets or rather sequences**. Moreover, as shown in **Equation 2.5**, we might sometimes prefer to summarize or generalize sequences using notations such as:

¹Walter’s use of “;” such as we depict in **Equation E.1** might somewhat significantly differ from the notation we have heavily utilized in transformatics when we prefer to either write $\{x : \phi(x)\}$ or even just $\{x \mid \phi(x)\}$, but the semantics remain the same.

²We borrow the formalism of a symbol set over a set, but whose elements are ordered by some criteria, in this case, that of the latin alphabet such as in [5], but also see **Equation 2.2**.

³Refer to [23], **Section D.1** and **Definition 16** among others.

For infinite sequences...

$$S_\infty = \langle \cdots, a_{n-1}, a_n, a_{n+1}, \cdots \rangle = \langle a_i \rangle^\infty \quad (\text{E.4})$$

And for finite sequences as...

$$S_n = \langle a_1, a_2, \cdots, a_{n-1}, a_n \rangle = \langle a_i \rangle^n \quad (\text{E.5})$$

For $i \in \mathbb{N}$.

However, and especially where we know that the elements in a sequence might not necessarily be numbers, or that they might not even be simple elements such as just literal symbols but perhaps symbolic structures of say a complex kind — such as a sequence of codons encoding a gene or perhaps a sequence of words that define or specify a particular phrase such as the command “fire melts ice”⁴, or perhaps, the sequence of instructions in a TEA program (refer to **Section D.2.9**) — in such cases, we shall want to generalize a sequence by incorporating information about the symbol set spanned by its elements. Thus, we might adopt generalizations such as...

$$S_\infty = \langle \cdots, a_{n-1}, a_n, a_{n+1}, \cdots \rangle = \langle a_i \in \psi_S \rangle^\infty \quad (\text{E.6})$$

Or perhaps

$$S_n = \langle a_1, a_2, \cdots, a_{n-1}, a_n \rangle = \langle a_i \in \psi_S \rangle^n = \mathbb{N} \times \psi_S \quad (\text{E.7})$$

or, as preferred in calculus texts such as [77], as..

$$S_n = \langle a_1, a_2, \cdots, a_{n-1}, a_n \rangle = f : \mathbb{N} \rightarrow \psi_S \quad (\text{E.8})$$

And just $S_n = \{a_k\}$ or maybe $S_n = \{f(a_k)\}$ and even the very plain $a_n = f(n)$

⁴Which is indeed different both literally and semantically from “ice melts fire” or perhaps “melts fire ice”, etc.

as commonly used in calculus texts such as [77] and especially in Ron Larson et al.[79]. In these cases, the emphasis, especially because for fields such as calculus, sequences are used to treat of and generalize continuous or discrete functions mapping the counting numbers ($\mathbb{N}$) to some other numbers (especially to real numbers, $\mathbb{R}$), so that, by focusing on the general term only, and using n as the common subscript in the general term, such as when the sequence of square numbers is written as $\{a^2\}$, it is understood that these collections might be finite or even infinite, however, that the key thing is how, when given any particular position index such as $i=1$ or $i=n$ or $i=\infty$, one can correctly derive or compute the corresponding term within the specified sequence. In transformatics though, especially when not dealing with sequences whose terms might be readily generalizable using some simple formula that is a function of just the position index, we might prefer to use other generalization notations as the case best warrants.

Finally, note that, perhaps out of preference and for consistency with earlier literature[2] in transformatics and its development[3][4][5], we usually depict general or arbitrary sequences using the letter Theta — Θ , and its terms or elements using the lowercase version θ such as in:

$$\Theta^n = \langle \theta_i \rangle^n : \theta_i \in \psi_\beta \quad \wedge \quad \text{card}(\Theta) = n \quad \wedge \quad i, n \in \mathbb{N} \quad (\text{E.9})$$

For some symbol set ψ_β that characterizes the elements of the sequence. But, and **this is important**; as with many instances across mathematics — which is just *a language for unambiguously stating [general] truths* — when the context is clear and simplicity is preferable, much of these formalisms and notations do get much more simpler than discussed here, and so that, to the uninitiated, sometimes it might appear as though one is saying one or more things, when in fact, a certain expression or mathematical statement is actually meaning a particular thing⁵.

Series

A *series* is the sum of a sequence[32], however, in drawing parallels with similar formalisms from some mathematics fields such as linear algebra, we might likewise think of series as kinds of **linear combinations**. For example:

⁵As always, care, practice and the *right* background knowledge is necessary when both writing and reading or analyzing mathematical texts, more so in such a new field or theory such as transformatics is.

$$Q = \sum_{n=0}^{\infty} \frac{1}{2^n} = 1 + \frac{1}{2} + \frac{1}{4} + \dots = 2. \quad (\text{E.10})$$

Depicts a series Q , defined as the linear combination of the terms of a sequence whose general term is $\frac{1}{2^n}$.

Series are used in convergence analysis and power series expansions[80], which applications come in handy when computing limits or numerical approximations of continuous and discrete functions such as in calculus, but also, because we have related them to linear combinations in linear algebra, they likewise find varied applications in vector analysis such as in establishing linear independence, linear groups, etc [78].

Also, because series are somewhat conventionally derived from sequences[77], such that if we have a sequence $S = \{a_k\}$, then both $\sum_{k=1}^n a_k$ and $\sum_{k=0}^{\infty} \alpha a_k = \alpha \sum_{k=0}^{\infty} a_k$ for some scalar α are corresponding series based off of it, one might wonder, and correctly so, is a linear combination of the terms of S via multiplication a series as well or not? Basically, if $S = \{a_1, a_2, \dots, a_n\} = \{a_{i \in [1, n]}\}$, and that $\alpha_1, \alpha_2, \dots, \alpha_n$ are scalars, so that

$$S^+ = \alpha_1 a_1 + \alpha_2 a_2 + \dots + \alpha_n a_n = \sum_{i=1}^n \alpha_i a_i \quad (\text{E.11})$$

or that

$$S^* = \alpha_1 a_1 \times \alpha_2 a_2 \times \dots \times \alpha_n a_n = \prod_{i=1}^n \alpha_i a_i \quad (\text{E.12})$$

We know that **Equation E.11**, or rather, S^+ depicts a series or a linear combination over the terms of the sequence S , however, would it also be correct to refer to **Equation E.12** or rather S^* as a series too?

Vector

A *vector* is an element of a vector space, often represented as an ordered tuple or as a sequence of sequences. Vectors are generally used to depict quantities

that have both magnitude and direction[74][79] such as velocity, momentum, and force.

When expressing vectors, we normally want to stress the fact that they consist of one or more components, for which each component corresponds to some particular dimension or direction. For example, a pure scalar such as just the number 5, might become a vector, when it is expressed as $\vec{5}$ or $\overleftarrow{5}$ — in which case we are combining information about direction with information about magnitude, and that our vector consists of just one component or dimension⁶.

When a vector consists of multiple components, such as when a ball's velocity needs to describe its relative speeds in the three dimensions of space — typically denoted x , y and z , we then might want to express the vector as either a dimension oriented sequence, a dimension annotated series or as some kind of matrix (usually as a column matrix), such as in:

$$\vec{v} = \langle 1, 3, 2 \rangle \equiv 1\mathbf{x} + 3\mathbf{y} + 2\mathbf{z} \equiv \begin{bmatrix} 1 \\ 3 \\ 2 \end{bmatrix} \in \mathbb{R}^3 \quad (\text{E.13})$$

That we can use sequences to depict vectors, such as in the first expression used to specify the vector $\vec{v}$ in the above equation might help to align some results or arguments in transformatics with other mathematical fields such as in calculus, mechanics or even linear algebra. This is essentially via the realization that, as long as we understand a vector to depict an ordered set of quantities where each position in the set maps to a particular dimension, that then, if we for example had a vector that only has two components — e.g. a velocity with only components in the x and z dimension, we might then still use the sequence notation common in transformatics, to express it, such as in:

$$\vec{v} = \langle 1, 0, 2 \rangle \equiv 1\mathbf{x} + 0\mathbf{y} + 2\mathbf{z} = 1\mathbf{x} + 2\mathbf{z} \equiv \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} \in \mathbb{R}^3 \quad (\text{E.14})$$

Thus for example, the case of **complex numbers** ($\mathbb{C}$), that consist of a real and

⁶In this case we might for example be describing the velocity of a ball swooshing past us, and are interested in indicating its speed and whether it was headed left or right, and nothing more.

imaginary component, and which are generally written as the linear combination $a + i\mathbf{b}$, might likewise use a sequence-like notation such as just $\langle a, b \rangle$ when the context is clear. In fact, Larson et al. in [79] in their calculus treatment of vectors in the plane has this to say:

The components of $\mathbf{v} = \overrightarrow{PQ}$, where $P = (a_1, b_1)$ and $Q = (a_2, b_2)$, are the quantities [such that] $a = a_2 - a_1$ (x-component) [and] $b = b_2 - b_1$ (y-component), [and so that] the pair of components is denoted $\langle a, b \rangle$.

And thus, when the dimensions aspect is clear, we might sometimes see a vector written as just a sequence such as just $\langle a, b \rangle$ — the **transformativ-way**, or perhaps as a column matrix, $\begin{bmatrix} a \\ b \end{bmatrix}$ — the linear algebraic way.

Matrix

A *matrix* is a two-dimensional array of scalars [32]. In the transformatics language though, we might think of a matrix as a higher-order sequence (see **Section D.2.8**) — essentially, a sequence of sequences. In fact, a more commonplace or familiar approach might be to think of a matrix as a sequence of vectors — this helps appreciate that:

1. A matrix could be reduced to a vector.
2. A matrix could be interpreted as the projection of one or more vectors to a sequence that preserves just their component magnitudes but otherwise respecting the dimensions of each component.
3. Any vector can essentially be re-written as either a column or row matrix.

Matrices are normally described or characterized by their dimensions, so that we find authors such as Walter Nef in [78] talking of matrices as “a rectangular array of numbers”, and that “the matrix A has m rows and n columns” — this, when talking of a structure generalized such as:

$$A = \begin{pmatrix} \alpha_{11} & \alpha_{12} & \cdots & \alpha_{1n} \\ \alpha_{21} & \alpha_{22} & \cdots & \alpha_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots \\ \alpha_{m1} & \alpha_{m2} & \cdots & \alpha_{mn} \end{pmatrix} = \alpha_{ik}$$

Where $k = 1, 2, \dots, n$ and $i = 1, \dots, m$ — essentially, $i, k \in \mathbb{N}$, and further, that “the mn scalars α_{ik} are known as the elements of the matrix”, and that we refer to a matrix that m rows and n columns as an $m \times n$ matrix.

Then, we also know that matrices represent linear transformations and systems of equations [81][78], and this might be appreciated by relating the case on vectors that we saw before this, to how we might map vectors to matrices, as well as use them to express [linear] transformations. For example, assuming we have a starting vector $P = \langle 1, 2 \rangle$ and a resultant vector $Q = \langle 3, 4 \rangle$, we might then talk of the transformation $A = \overrightarrow{PQ}$ resulting from the ordered application of these two vectors as such:

$$A = \begin{pmatrix} \overrightarrow{P} \\ \overrightarrow{Q} \end{pmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

And we would be able to reduce this matrix to a corresponding transformation vector respecting the dimensions as such:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \cdot x + 2 \cdot y \\ 3 \cdot x + 4 \cdot y \end{pmatrix} = (3 - 1)x + (4 - 2)y = 2\mathbf{x} + 2\mathbf{y}$$

So that then, we essentially can express the transform $\overrightarrow{PQ}$ as just the sequence $\langle 2, 2 \rangle$.

Finally, note that, as when we had the debate concerning whether to express sequences in transformatics using braces $\{\dots\}$ or instead angled brackets $\langle \dots \rangle$, note that also, in many fields of mathematics where matrices are encountered, you shall find use of both square brackets $[\dots]$, and brackets $(\dots)$. The choice often

depends on the author and the conventions in the field as long as no confusion arises, otherwise, they might be used interchangeably too.

Space

A *space* is a set with additional structure [32]⁷. Examples include:

- **Vector space**: closed under addition and scalar multiplication.
- **Metric space**: equipped with a distance function.
- **Hilbert space**: complete inner product space.

Spaces provide the abstract setting for geometry, analysis, and quantum mechanics [82].

E.2 Comparison with Other Fields

Before we close our focus on extending what was originally introduced via the Transformatics 101 thesis[23], it shall especially be worth adding a word or two concerning where, across the vast spectrum of all mathematics and the mathematical disciplines and sub-disciplines, we might correctly locate or position our theory of Transformatics. This undertaking might not be something we can fully and rigorously accomplish within the context of chapter-sub-section such as this, however, and especially for completeness's sake, it is better we attempt and kick-start this discussion here, in this vital work — the first to undeniably non-trivially, but also exhaustively apply Transformatics (the other major works being [6] on Psychology, [43] on the TEA language, and more recently, [30] on Western Esotericism).

E.2.1 Relation to Statistics

It shall be helpful to note that, earlier attempts to situate this theory within the spectrum of mathematics — such as when we penned the paper [5], mostly relied on “bias” towards where it is that most of the developed theory at that point would find its closest relatives. Thus, it should not be so shocking that initially, we thought perhaps Transformatics had better be treated as a theory within **mathematical statistics** — especially because, as one can tell looking at the proposed applications of the theory (**Section D.2**), many of the proposed and developed ideas back then indeed resonated well with work in **Statistics**, and

⁷Interesting to note: this kind of phrasing somewhat resonates with what is also typical when talking of an **algebra** — *a set with a specific operation or operations well-defined for it* — refer to **Section E.2.5** for more on this.

that, as with many theories and formalisms in that field, much of what makes statistics what it is, starts by the *special* treatment of sequences known as **data**⁸ — such as when we treat of a variable X , that might taken on values x_i spanning some symbol set (ψ_X) and ordered by the naturals $(\mathbb{N})$ — i.e

$$X = \{x_i : i = 1, 2, \dots, N\} = \{x_i : i \in \mathbb{N}\} = \psi_X \times \mathbb{N} = x_{i \in \mathbb{N}} \in \psi_X \quad (\text{E.15})$$

However, and despite the fact that statistics heavily and naturally relates to not only collecting information that is then encoded using sequences, and which is then processed and interpreted as sequences (of data points, observations, events, probabilities, measurements etc), however, **it is not** the only major field of mathematics that essentially treats of structures in the form of sequences.

E.2.2 Relation to Linear Algebra

The other competing field might for example be **Linear Algebra**⁹; in that field, sequences do show up right at the start (see for example Chapter 1 of [78]), and particularly, they *initially* manifest in the form or under the terminology of **sets**, and then *soon enough* in the [more transformativ] form of matrices of many kinds, and that, very often, much of the theory and formalisms of linear algebra rotate around encoding data as the mathematical structure known as a matrix¹⁰, whether of a simple form such as 1-dimensional finite sequence, or as high-dimensional dense or sparse matrices encoding infinitely large spaces or quantifiably large datasets. That field also presents many processes that deal with reducing such basic or higher-order sequences (**Section D.2.8**), also sometimes likened to or related to “vectors”, to just simple expressions or quantities such as scalars — basic numbers in their simplest forms, via **linear-transformations** of many kinds. And talking of transformations, we find in the field of linear algebra, many kinds of ways that one sequence might be mapped or reduced to another sequence, via processes or computations such as computing the basis (a sequence from which other sequences of similar structure can be derived), endomorphisms (linear transforms that map a type onto itself), inverses (sequences which would result in the identity sequence when multiplied with the source sequence), etc.[78]

⁸Whether as “raw data” or as “coded data”[83]

⁹Majorly[32] attributed to **Hermann Grassmann** the 19th century German polymath that first *created* linear algebra in its modern form when he penned the 1844 work *Die Lineale Ausdehnungslehre* (“Theory of Linear Extension”) which introduced the ideas of vector spaces and linear combinations. Other contributors[32] are Gottfried Wilhelm Leibniz (1693) that introduced early ideas of matrices and determinants; Gabriel Cramer (1750) that published *Cramer’s Rule* for solving n equations in n unknowns; Joseph-Louis Lagrange (late 1700s) that used matrices to study bilinear forms, and finally Giuseppe Peano (1888) that introduced the abstract modern concept of a vector space, in essence, completing the conceptual framework that underlies all of linear algebra.

¹⁰Which we have seen to actually be based on sequences — refer to **Section E.1**

First, concerning sequences in particular, it shall be worth noting that Walter Nef does even explicitly bring up the concept of *a sequence space* — in relation to, and derived from the concept of a real vector space (also to be found in other fields such as Calculus and real analysis), when he writes:

Instead of ordered sets of n real numbers, we can also consider countably infinite sequences of real numbers, i.e., $x = (\xi_1, \xi_2, \xi_3, \dots)$. Addition and multiplication by scalars can be defined... term by term... and in this way, a new real vector space is constructed which we call the space of sequences

— Walter Nef, *Linear Algebra*, 1967[78]

Moreover, and not to confuse transformatics with linear algebra and similar mathematics, note that, whereas they talk of “spaces” as especially sets or families of sets that especially satisfy the **7 axioms of a vector space**, and which especially focus on the operations of addition (of the member sets to other member sets) and multiplication (of scalars by member sets), such that then we can talk of “the space of sequences”, “the space of polynomials”, “the space of linear manifolds”, etc. in transformatics, we are more interested in not assuming anything about the nature of the elements of a set, so that for example, we might not readily talk of the possibility of a meaningful multiplication of a scalar by an element in some set, or that we might not guarantee that the axioms expected of a vector space hold when we apply them to say, sequences of letters (such as in ψ_{az} or ψ_{na}), sequences of TEA instructions, etc. And also that, as introduced in both [5] and [71], we are more interested in operations that can be applied to sequences irrespective of the nature of the elements they contain — for example, **concatenation**, **filtering**, **lateral inversion**, **cardinality** etc. would readily work well on sequences of real numbers, integers, letters of the latin alphabet, sequences of quaternions, etc.

Thus, even though transformations of a linear kind or rather, “linear transformations” in the strict sense, might correctly treat of sequences, vectors and matrices such as when we have two vectors u , v , and z and some scalar c , so that, given some linear transformation $T : V \rightarrow W$ mapping between vector spaces, we have that:

$$T(u + v) = T(u) + T(v) = z \quad (\text{E.16})$$

and that

$$T(cu) = cT(u) \quad (\text{E.17})$$

And so that the transformation $T(\cdot)$ can be treated of as we typically do in transformatics as such:

Transformation 41. $u + v \xrightarrow{T} z$

And that if $T(u) = u^*$ and $T(v) = v^*$, so that $u^* + v^* = z$, then we also expect that:

Transformation 42. $u^* + v^* \rightarrow z$

Or to sum it all up in a linear-algebraic-transformation notation combo¹¹, we have that:

$$(u + v \xrightarrow{T} z) \equiv ((u \xrightarrow{T} u^*) + (v \xrightarrow{T} v^*) = z) \quad (\text{E.18})$$

For those not very familiar with linear algebra, note that **Equation E.18** would for example hold for transformations such as **halving** — but not say **squaring**. For example, if we have $u = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$ and $v = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$, and that we have the operation $T(u) = \{\frac{\alpha_{ik}}{2}\}$ for all $\alpha_{ik} \in u$, then we would indeed see that:

$$T(u + v) = T\left(\begin{bmatrix} 3 \\ 2 \end{bmatrix} + \begin{bmatrix} 4 \\ 5 \end{bmatrix}\right) = T\left(\begin{bmatrix} 3 + 4 \\ 2 + 5 \end{bmatrix}\right) = T\left(\begin{bmatrix} 7 \\ 7 \end{bmatrix}\right) = \begin{bmatrix} \frac{7}{2} \\ \frac{7}{2} \end{bmatrix} = \begin{bmatrix} 3.5 \\ 3.5 \end{bmatrix} \quad (\text{E.19})$$

¹¹Perhaps totally bonkas!

While, we also see that

$$T(u)+T(v) = T\left(\begin{bmatrix} 3 \\ 2 \end{bmatrix}\right) + T\left(\begin{bmatrix} 4 \\ 5 \end{bmatrix}\right) = \begin{bmatrix} 3 * \frac{1}{2} \\ 2 * \frac{1}{2} \end{bmatrix} + \begin{bmatrix} 4 * \frac{1}{2} \\ 5 * \frac{1}{2} \end{bmatrix} = \begin{bmatrix} \frac{3+4}{2} \\ \frac{2+5}{2} \end{bmatrix} = \begin{bmatrix} \frac{7}{2} \\ \frac{7}{2} \end{bmatrix} = \begin{bmatrix} 3.5 \\ 3.5 \end{bmatrix} \quad (\text{E.20})$$

However, and keeping in mind that **linear transformations apply to the sequences item-wise**, we see that if instead we had an operation such as $S(u) = \{\alpha_{ik}^2\}$, then using the same u and v sequences above, we see that attempting to apply it separately to either sequence Vs applying it to their summation would result in different results as shown hereafter:

$$S(u+v) = S\left(\begin{bmatrix} 3 \\ 2 \end{bmatrix} + \begin{bmatrix} 4 \\ 5 \end{bmatrix}\right) = S\left(\begin{bmatrix} 3+4 \\ 2+5 \end{bmatrix}\right) = S\left(\begin{bmatrix} 7 \\ 7 \end{bmatrix}\right) = \begin{bmatrix} 7^2 \\ 7^2 \end{bmatrix} = \begin{bmatrix} 49 \\ 49 \end{bmatrix} \quad (\text{E.21})$$

$$S(u) + S(v) = S\left(\begin{bmatrix} 3 \\ 2 \end{bmatrix}\right) + S\left(\begin{bmatrix} 4 \\ 5 \end{bmatrix}\right) = \begin{bmatrix} 3^2 \\ 2^2 \end{bmatrix} + \begin{bmatrix} 4^2 \\ 5^2 \end{bmatrix} = \begin{bmatrix} 9+16 \\ 4+25 \end{bmatrix} = \begin{bmatrix} 25 \\ 29 \end{bmatrix} \quad (\text{E.22})$$

And so that for the case of a squaring transform applied linearly, we fail to meet the requirements that typical transformations are expected of in linear algebra. Or rather that...

$$(u+v \xrightarrow{S} z) \not\equiv ((u \xrightarrow{S} u^*) + (v \xrightarrow{S} v^*) = z^*) \quad (\text{E.23})$$

It should be worth noting that for transformatics — such as when we have the squaring transformer defined as such:

Transformer 20 (squaring transform, $S(\Theta)$). $\Theta^n \xrightarrow{S(\Theta)} \Theta^*$;
 $\mathcal{L}(\Theta^n) = \mathcal{L}(\Theta^*) = n$

$$\begin{aligned} \wedge \quad & \forall \theta_i \in \Theta \quad \exists \theta_i^* \in \Theta^* : \theta_i^* = \theta_i^2 \quad \forall i \in [1, n] \\ \wedge \quad & \theta, \theta^* \in \mathbb{R} \quad \forall \theta \in \Theta \quad \wedge \quad \theta^* \in \Theta^* \quad \square \end{aligned}$$

Transformer 20 would work on sequences of [real] numbers such as we see in **Equation E.21** and **Equation E.22**.

So that it starts to surely become clearer that though we might somewhat relate transformatics to linear algebra (perhaps by the fact that they both treat of sequences and transformations on them)... **AND YET, THEY REALLY ARE DIFFERENT!** First, since for transformatics we “care less”¹² about the actual types of the elements of a set or sequence¹³, and more on the structure [and overall composition] of the sequence *before and after the transformation*, and as for the elements themselves, perhaps mostly care about properties such as whether or not they belong to some symbol set or not — pre- and post-transform, etc. And that we can treat of sequences of elements whose types are not numbers (not just real numbers or integers as fields such as linear algebra and calculus especially focus on), so that, if we have transformativ operations such as the **Mirror** operation¹⁴, $M(\cdot)$, which, when given some sequence such the phrase “star”, returns it reflection as “rats”, so that we essentially have:

Transformation 43. $\langle s, t, a, r \rangle \xrightarrow{M} \langle r, a, t, s \rangle$

And that if we had two sequences, $u = \langle s, t, a, r \rangle$ and $v = \langle n, u, t, s \rangle$, then, applying this operation to them individually, and then to their summation or concatenation might not work as one might expect when dealing with linear algebraic transforms:

$$M(u) = M(\langle s, t, a, r \rangle) = \langle r, a, t, s \rangle = u_1^* \quad (\text{E.24})$$

and that

¹²Across a field such as Linear Algebra, it is typically expected or perhaps assumed that the structures being dealt with are at minimum, ones consisting of numbers — they might be integers, complex numbers or real numbers, but at least they are numbers — meaning, the transformations and operations being dealt with generally concern taking things like the sign and quantity of the number or symbol into consideration. However, Transformatics can also deal with sequences of unquantifiable symbols such as elements of the set ψ_{DNA} that we encountered in **Defition 1**.

¹³Especially because, where a transformation must rely upon or have the types of the sequence elements in consideration, then the necessary conditions constraining what the transformation can apply to or not, are typically specified as part of the transformation/-transformer definition or specification — such as the final specification line in **Transformer 20**.

¹⁴Also “lateral inversion” — refer to [5], [43], but also [30]

$$M(v) = M(\langle n, u, t, s \rangle) = \langle s, t, u, n \rangle = v_1^* \quad (\text{E.25})$$

and that

$$M(u) + M(v) = u_1^* + v_1^* = \langle r, a, t, s \rangle + \langle s, t, u, n \rangle = \langle r, a, t, s, s, t, u, n \rangle = u_1^* v_1^* \quad (\text{E.26})$$

But that

$$\begin{aligned} M(u + v) &= M(\langle s, t, a, r \rangle + \langle n, u, t, s \rangle) = \\ &M(\langle s, t, a, r, n, u, t, s \rangle) = \langle s, t, u, n, r, a, t, s \rangle = v_1^* u_1^* \end{aligned} \quad (\text{E.27})$$

So that then, unlike linear transformations, we see that for the case of transformations such as $M(\cdot)$, we have:

$$M(u + v) \neq M(u) + M(v) \quad (\text{E.28})$$

Or if we are to use an approach similiar to what we saw in **Equation E.18**:

$$(u + v \xrightarrow{M} v_1^* u_1^*) \not\equiv ((u \xrightarrow{M} u_1^*) + (v \xrightarrow{M} v_1^*) = u_1^* v_1^*) \quad (\text{E.29})$$

In fact, for the mirror operation, it is as if $M(u + v) = M(M(u) + M(v))$, which “totally” makes the kind of operations treated of in transformatics **so unlike what one would find in linear algebra**¹⁵ despite both fields treating of sequences.

¹⁵We say “so unlike” and not “totally unlike” because, as one might find, and this not to come off as a total surprise, linear transformations that are typical of linear algebra — such as what we saw in **Equation E.19** and **Equation E.20**, despite being “linear algebraic” since they conform to the requirements of vector spaces and linear transforms, are also tractable under the transformatics umbrella because nonetheless, they apply to sequences [and also result in sequences!] and that we can use the transformatics formalisms of transformers to specify and analyze them.

Further, and interestingly, the failure for the mirror operation to preserve the axioms of vector spaces would likewise remain the same even when we were to apply it to sequences of real numbers or numbers for that matter, and it is not the only kind of *peculiar* operation or transformation that we might come across when working with sequences in transformatics — others, such as the **LOMT: Multiplication by Lexical Order** transformation first introduced in [5] might likewise fail these tests!

Finally, before we leave linear algebra, it shall be worth noting that the transformer concept in transformatics might also be likened to the concept of **mappings** in linear algebra (and from there, to other fields including calculus for example¹⁶). So, how would this work?

First, note that, across many fields of mathematics where we encounter functions, the signature of a function — also technically equivalent to a mapping, is expressed using the notation: $f : D \rightarrow R$, which we can read as “the function f maps every element in the domain D to a corresponding value in the range R ”. Typically, the language used is that of sets, such as we see in the following excerpt from [78]:

General Mappings: Let E and F be two sets. A *mapping of E into F* is a rule which assigns to each element $x \in E$ a unique element $y \in F$.

The element $y \in F$ which is assigned to the element $x \in E$ by the mapping f is called the *image* of x under f and is denoted by $f(x)$. The set E is called the *domain* of f and F is called the *range* of f .

— Walter Nef, *Linear Algebra*, 1967[78]

And to help stress the possibility that we could actually express mappings in linear algebra using the new formalism of transformers, note how he goes on to clarify concerning the notation used to express mappings:

¹⁶[77] has an entire chapter dedicated to approximating nonlinear mappings by linear mappings for example.

Mappings will usually be denoted by small Roman letters

$f, g, \dots$

When an element $x \in E$ goes into the element $y \in F$ under the mapping f , this will be indicated by the notation

$x \rightarrow y = f(x)$.

— **Walter Nef**, *Linear Algebra*, 1967[78]

In transformatics, we would generally express such [mappings] concepts as such:

Transformer 21. $x \xrightarrow{f(x)} y;$
 $x \in E \quad \wedge \quad y \in F \quad \square$

So that, where we have been talking of a transformer (in transformatics), we might then talk of a mapping (or a function) in linear algebra and other related or derivative fields. Also, note that, when studying the structure of a [lean] transformer such as **Transformer 21** and comparing it to typical transformers in transformatics such as **Transformer 20** or perhaps a more involved case such as **Transformer 16**, that we typically prefer to flesh-out the transformer definition with some vital details about not just the types or character of the source and resultant (domain and range in this case), but also those essential details about the mapping/function itself — $f(x)$ in this case — so that, where it makes sense, we eliminate any ambiguity concerning how the elements in the source (domain) are transformed into the elements in the result (image/range). Thus, as see see in a case such as **Transformer 22** covered in the next section, we sometimes gain much by investing in a more rigorous specification of the specifics of the transformer that underlies the mapping being defined. However, it remains to be rigorously established if indeed, all *transformation mappings* are merely another kind of linear algebraic mapping.

E.2.3 Relation to Calculus

And then we encounter sequences and especially [linear] sequence transformations, when dealing with sequence-based mathematical structures known as series. Concerning this, note that Sequences and Series are especially commonplace in texts and mathematics dealing with Calculus such as [79], [77] and [84] to name a few. Moreover, and as laid out in [84], when sequences are treated of in calculus, it is

mostly, or especially in the context of “sequences of numbers”¹⁷ as we see when he says:

Definition 1 SEQUENCE: A sequence of real numbers is a function whose domain is the set of positive integers. The values taken by the function are called terms of the sequence.

NOTATION. We will often denote the terms of a sequence by a_n . Thus, if the function given by Definition 1 is f , then $a_n = f(n)$. With this notation, *we can denote the set of values taken by the sequence by $\{a_n\}$...*

The following are sequences of real numbers:

$$(a) \{a_n\} = \left\{\frac{1}{n}\right\}$$

$$(b) \{a_n\} = \{\sqrt{n}\}$$

$$\dots (e) \{a_n\} = \left\{\frac{e^n}{n!}\right\}$$

We sometimes denote a sequence by writing out the values $\{a_1, a_2, a_3, \dots\}$.

—**Stanley I. Grossman**, *Calculus of One Variable*, 1986[84]

And so, we see that first of all, the notation used to denote or represent sequences in transformatics [thus far] is somewhat significantly different from that used in calculus. But this is not all concerning how we might relate treatment of sequences in transformatics with sequences in calculus. It shall be worth noting that in our formal definitions of the subject (see **Definition 16**, but also **Definition 17**), an explicit focus is placed on “transformation” — particularly, we are specially concerned with how sequences get to be mapped either to other sequences or other mathematical structures and objects, such as reducing a sequence to some number, and this is where the next contrast between transformatics and calculus comes in.

¹⁷In fact, especially as “sequence of real numbers”, for which [77] says “A sequence of real numbers is a real-valued function whose domain is the set of natural numbers”, and which he generalizes as $f : \mathbb{N} \rightarrow \mathbb{R}$, but which we might also understand to be $f : \mathbb{N} \times \mathbb{R}$ when we wish to emphasize the fact that the sequence is otherwise reducible to set of pairs where for each natural number [position] index, there is a corresponding real number — this later notation is more common in our transformatics literature.

In **Section E.1**, we introduced the notion of the mathematical concept of series, and we noted that, it is conventionally the notion of taking a sequence (especially for calculus, a sequence of real numbers), and reducing it to some number via a finite or infinite summation of its terms starting from some index[79][84]. In our cross-domain explorations, we found that we might likewise think of series as a kind of application of linear combinations (a concept more common in Linear Algebra), however, this perhaps need be taken with a grain of salt¹⁸..

So, where in transformatics we might reduce a sequence to another sequence or to some scalar, a number or some symbolic expression¹⁹, in fields like calculus, the emphasis is on mapping sequences of numbers to some real number [only] as we see explained in these excerpts:

¹⁸For one, linear combinations might involve the summation of terms of one or more sequences, however, there is no guarantee that the combination always needs be via addition operation alone — yet for series, summation is the de facto signature. Also, with series, the sequences are typically of a kind where all the terms assume a general, similar form, so that the series is reducible to a formulation involving or based on just the general term and otherwise the start and end indices, whereas for linear combinations, the sequences involved might not necessarily have elements conforming to some particular structure, and thus, where summation needs be done, it might not be possible to merely derive the final/limit values without treating of each term individually, etc. But overall, and generally, it is agreed that the [infinite] series treated of in calculus are indeed linear[79].

¹⁹For example the kind of transforms treated of in this book in **Chapter 10**, involving transformation from DNA/na-Sequences into decimal-Sequences, then to ozin-expressions and finally into Lu-Genome Expressions that have entirely little to do with the original sequence forms!

Definition 1 INFINITE SERIES: Let $\{a_n\}$ be a sequence. Then the infinite sum

$$\sum_{k=1}^{\infty} a_k = a_1 + a_2 + a_3 + \cdots + a_n + \cdots \quad (\text{E.30})$$

is called an infinite series (or, simply, series). Each a_k in Equation E.30 is called a term of the series. The partial sums of the series are given by

$$S_n = \sum_{k=1}^n a_k \quad (\text{E.31})$$

The term S_n is called the n th partial sum of the series. If the sequence of partial sums $\{S_n\}$ converges to L , then we say that the infinite series $\sum_{k=1}^{\infty} a_k$ converges to L , and we write

$$\sum_{k=1}^{\infty} a_k = L \quad (\text{E.32})$$

Otherwise, we say that the series $\sum_{k=1}^{\infty} a_k$ diverges.

—**Stanley I. Grossman**, *Calculus of One Variable*, 1986[84]

And then [79] also has this to say, concerning how a sequence's indices might be utilized when dealing with these series:

Infinite series may begin with any index. For example,

$$\sum_{n=3}^{\infty} \frac{1}{n} = \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \cdots \quad (\text{E.33})$$

When it is not necessary to specify the starting point, we write simply $\sum a_n$. Any letter may be used for the index. Thus, we may write a_m , a_k , a_i , etc.

—Ron Larson and Bruce H. Edwards, *Calculus*, 2012[79]

And emphasizing the fact that infinite series can be reduced to finite series (the “partial sums”), perhaps to some limit when the underlying sequence converges, is illustrated in the result below taken from [77]:

$$\sum_{k=1}^{\infty} a_k = \lim_{n \rightarrow \infty} \left[\sum_{k=1}^n a_k \right] \quad (\text{E.34})$$

And so, overall, we find that sequence transformations in calculus are of this kind — as series, and which essentially consist in taking finite or infinite sequences of numbers and reducing them to some particular number (a summation of the terms in the sequence). Also, it is worth noting that, even though the notations might vary, and yet, the idea of taking a sequence and mapping it to some different value, using the transformatics notations and formalisms, but yet reflecting the formulations typical of such treatments in calculus can be exemplified by the following result (the **Geometric Sum Formula**) from [77], and which we then re-write as a transformatics transformer thereafter:

$$\sum_{k=0}^n r^k = \frac{1 - r^{n+1}}{1 - r} \quad \forall r \in \mathbb{R} \quad (\text{E.35})$$

Which, in transformatics we can express as the following [more generalized²⁰] transformer:

²⁰Especially for completeness' sake, and to capture all the peculiarities around this important result and series as captured in different calculus texts such as [79], [77] and [84] that we have reviewed.

Transformer 22 (The **Generic Geometric Series Sum Generator**, $gsum(\cdot)$).

$$\Theta = \langle \theta_i \rangle^n : \mathbb{N} \times \mathbb{R} \xrightarrow{gsum(\Theta)} \Theta^*;$$

$$\mathcal{L}(\Theta) = n \quad \wedge \quad \theta_i = cr^i \quad \forall \theta_i \in \Theta, \forall i \in [1, n] \quad \wedge \quad n \in \mathbb{N} \quad \wedge \quad r, c \in \mathbb{R}$$

$$\wedge \quad \Theta^* \in \mathbb{R} \quad \wedge \quad \Theta^* = \begin{cases} gsum(\Theta) = \sum_{\forall \theta_i \in \Theta}^{\mathcal{L}(\Theta)} \theta_i = \sum_{i=1}^n cr^i = \frac{c(1-r^{n+1})}{1-r} & \forall n \geq 1 \\ 1 & n = 0 \\ \emptyset & r = 0 \vee r = 1 \end{cases}$$

$\wedge \quad c \quad \text{is a constant, while} \quad r \quad \text{is the common ratio.}$

□.

And so, the attractiveness of transformatics when compared to calculus might be appreciated by the level of detail involved in rigorously specifying an important result such as the **geometric series summation** as depicted in **Transformer 22**²¹. The power of the transformer formalism is that, not just the core formulation of the series is specified, but all possible corner cases can likewise be catered for in a single formula, and that we can clarify on what [necessary] conditions and types all the various parameters and variables involved are or need to be so as to allow for a meaningful computation²² of the series when given some originating sequence. Also, concerning that transformer, note that we refer to it as a “generator” in the title, to emphasize the fact that in transformatics (as depicted in both **Foundational Result 9** and in [7]), often, we encounter *special* transformers that also happen to be a kind of **generator** as explained in the transformatics literature²³.

Finally, and revisiting the concept of mappings that we have encountered in **Section E.2.2** when dealing with linear algebra, note that these notions come up again and again in calculus texts too, and so, we might give them some attention here as well.

Larson Edwards et al. in [79] bring this concept up when discussing problems

²¹In fact, and no pun intended, but this is a great example of how Transformatics might be applied in Calculus!

²²And here is especially where transformatics shines — leaving no loopholes when a mathematical formalism or model needs to be applied or translated into a computable solution. It is as if, these transformatics specifications are a kind of computer program expressed using pure mathematics.

²³For example, where the calculus formulation depicted in **Equation E.35** fails to make sense for parameters such as when $r = 0$, or when the sequence being evaluated is empty, we find that our **Transformer 22** would otherwise still be able to return some meaningful result — including returning the empty set, $\emptyset$, when the formula can’t be meaningfully applied — a creative way to help deal with problematic scenarios in calculus such as having to resolve results like the **indeterminate form** $\frac{0}{0}$ concerning which, mathematical history[84] tells us the powerful Bishop George Berkeley attacked Isaac Newton’s methods such as his *fluxions*, and that it is another British mathematician, Collin Maclaurin (considered the finest after Newton’s generation, and after whom we have *Maclaurin Series*), that intervened to help save Newton’s work!

in calculus that require or call for *change of variables* — basically, the idea that many functions and analysis problems are simplified significantly when instead of working on them in their original formulations and reference frames, we instead develop some clever or useful substitution formulae that can help map the original expressions to alternative formulations that can then be tackled more directly or using more common methods. These authors introduce the concept as such:

A function $G : X \rightarrow Y$ from a set X (the domain) to another set Y is often called a map or a mapping. For $x \in X$, the element $G(x)$ belongs to Y and is called the image of x . The set of all images $G(x)$ is called the image or range of G . We denote the image by $G(X)$.

—Ron Larson and Bruce H. Edwards, *Calculus*, 2012[79]

Again, to relate this to how we would express mappings using the concept of transformers in transformatics, we would use a transformer definition such as:

Transformer 23. $x \xrightarrow{G(\cdot)} G(x);$
 $x \in X \quad \wedge \quad G(x) \in Y$
 $\wedge \quad G(X) = Y \quad \square$

It shall of course be interesting to compare **Transformer 23** based on the notion of mappings as expressed in a calculus text, to **Transformer 21** based on related notions in a linear algebra text. Also, note that not all calculus texts explicitly call out transformations nor mappings. For example, in [84], one comes as close to the above ideas of using mappings to effect expression simplifications as the use of the idea of **variable substitution** that they then apply to simplifying problems in differentiation such as in:

If u is a differentiable function of x , then

$$\frac{d}{dx} \cos(u) = -\sin(u) \frac{du}{dx} \quad (\text{E.36})$$

Let $u = \sqrt{x}$. Then $du/dx = 1/2\sqrt{x}$, and, using Equation E.36, we have

$$\frac{d}{dx} \cos(\sqrt{x}) = \frac{d}{dx} \cos(u) = -\sin(u) \frac{du}{dx} = (-\sin(\sqrt{x})) \left(\frac{1}{2\sqrt{x}} \right) = \frac{-\sin(\sqrt{x})}{2\sqrt{x}}$$

□

—**Stanley I. Grossman**, *Calculus of One Variable*, 1986[84]

And we also encounter this kind of usage when simplifying integrations such as in:

Compute $\int dx/(ax+b)$.

Solution. Let $u = ax+b$. Then $du = adx$ and

$$\int \frac{dx}{ax+b} = \frac{1}{a} \int \frac{adx}{ax+b} = \frac{1}{a} \int \frac{du}{u} = \frac{1}{a} \ln |u| + C$$

or

$$\int \frac{dx}{ax+b} = \frac{1}{a} \ln |ax+b| + C$$

□

—**Stanley I. Grossman**, *Calculus of One Variable*, 1986[84]

In these last two cases, we might [perhaps remotely] talk of “simplification of solutions via transformation”, and might express the ideas expressed using a transformativ notation for the last case involving integration as such:

Transformation 44. $\int \frac{dx}{ax+b} \xrightarrow{ax+b=u} \frac{1}{a} \int \frac{du}{u} = \frac{1}{a} \ln |u| + C \xrightarrow{u=ax+b} \frac{1}{a} \ln |ax+b| + C$

And then, last but not least, note that, when we still are dealing with the idea of mappings, we do encounter cases in calculus that somewhat relate to the solving of complex problems using the **chaining of transformers** or rather, the chaining of transformations. **Transformation 44** already exemplifies this idea for problems in calculus, however, refer to instances such as **Transformer 14** that we covered in GENETICS for the specification of a **Ribosome**, or perhaps refer to **Section D.2.9**) that formalizes the idea of approaching computer programs as compositions leveraging chaining of sequence transformers. Overall, one would find that, the more traditional concept that relates to all of these transformatics formalisms is that of **function composition** — or perhaps “compositions of mappings”.

Concerning the treatment of compositions in calculus, we have the following illuminating definition from Fitzpatrick’s book [77]:

Definition. For functions $f : D \rightarrow \mathbb{R}$ and $g : U \rightarrow \mathbb{R}$ such that $f(D)$ is contained in U , we define the composition of $f : D \rightarrow \mathbb{R}$ with $g : U \rightarrow \mathbb{R}$, denoted by $g \odot f : D \rightarrow \mathbb{R}$, by the formula

$$(g \odot f)(x) \equiv g(f(x)) \text{ for all } x \text{ in } D.$$

—**Patrick M. Fitzpatrick**, *Advanced Calculus: A Course in Mathematical Analysis*, 1995[77]

And so that, if we are to attempt to express the notion of composition as Fitzpatrick depicts above, but instead using the notation and methods of transformatics, we would process in a manner similar to what we did in **Transformation 44**, but more so, as we did in **Transformation 40** that indeed demonstrates the concept of composing **higher-order transformers** via the method of chaining basic transformees (in this case, functions or just mappings) back to back, and so that, we would rewrite his definition’s core concept as such:

Transformer 24 (Function Composition, $g(f(x))$).

$$\begin{array}{lcl} x \xrightarrow{g(f(x))} y^* & \equiv & x \xrightarrow{f(x)} y \xrightarrow{g(y)} y^*; \\ x \in D \quad \wedge \quad y \in U \quad \wedge \quad y^* \in \mathbb{R} \quad \wedge \quad f : D \rightarrow \mathbb{R} \quad \wedge \quad g : U \rightarrow \mathbb{R} & & \square \end{array}$$

The very exciting thing about this concept depicted in **Transformer 24**, and which as we have seen is essentially the traditional mathematics idea of function compositions, is that without any lying about it, is essentially what underlies the

method of using TEA programs to solve arbitrary problems. We for example can see this notion at work in **Listing D.2.9**, but also in more GENETICS-related cases such as shown in **Section 6.2** when we use a TEA program to compute the sequence characteristic of some DNA sequence held in a flat text file.

With those discussions then, we close our review of calculus vis-a-vis what is happening in transformatics.

E.2.4 Relation to Several Other Branches of [Applied] Mathematics

Next, we shall quickly look at a few other [sub-]fields that also somewhat relate to either the concepts or the nomenclature used in transformatics.

Signal Processing

In his book[85] that is critically important for modern practice in signal processing and analysis, **Professor Ruye Wang**²⁴ brings to surface many basic and nuanced applications of [especially linear] transformations on data originating from sensors and natural phenomena. We bring up Wang’s work here, not just because of the somewhat *shocking* absence of the term “transformer” throughout a book treating of transformations, and whose title — **“Introduction to Orthogonal Transforms: With Applications in Data Processing and Analysis”** — would make anyone already familiar with transformatics turn an eye; but especially because, as we shall briefly demonstrate, several [,if not most,] of the kind of transformation operations described in relation to data processing and analysis as presented in his work, could be reduced to or perhaps expressed using our notion of transformers — without loss of generality or accuracy.

First, we shall begin by clarifying how signals and transformatics are most likely to relate. For starters, we shall bring up Wang’s introduction concerning the formal concept of signals in the opening chapters of his book:

²⁴Professor of Engineering at Harvey Mudd College (Claremont, California), also past Principal Investigator at NASA’s Jet Propulsion Laboratory.

A physical signal can always be represented as a real- or complex-valued continuous function of time $x(t)$ (unless specified otherwise, such as a function of space). The continuous signal can be sampled to become a discrete signal $x[n]$. If the time interval between two consecutive samples is assumed to be Δ , then the n th sample is

$$x[n] = x(t)|_{t=n\Delta} = x(n\Delta) \quad (\text{E.37})$$

In either the continuous or discrete case, a signal can be assumed in theory to have infinite duration; i.e., $-\infty < t < \infty$ for $x(t)$ and $-\infty < n < \infty$ for $x[n]$. However, any signal in practice is finite and can be considered as the truncated version of a signal of infinite duration. We typically assume $0 \leq t \leq T$ for a finite continuous signal $x(t)$, and $1 \leq n \leq N$ (or sometimes $0 \leq n \leq N - 1$ for certain convenience) for a discrete signal $x[n]$. The value of such a finite signal $x(t)$ is not defined if $t < 0$ or $t > T$; similarly, $x[n]$ is not defined if $n < 0$ or $n > N$. However, for mathematical convenience we could sometimes assume a finite signal to be periodic; i.e., $x(t + T) = x(t)$ and $x[n + N] = x[n]$.

A discrete signal can also be represented as a vector $x = [\cdots, x[n - 1], x[n], x[n + 1], \cdots]^T$ of finite or infinite dimensions composed of all of its samples or components as the vector elements. We will always represent a discrete signal as a column vector (transpose of a row vector).

—**Ruye Wang**, *Introduction to Orthogonal Transforms: With Applications in Data Processing and Analysis*, 2012[85]

From that introduction alone, we can safely conclude that all signals being treated of then, can essentially be expressed as or handled as we might do with sequences in transformatics. But that's just about the data representation then. So, what of the kinds of operations we can apply to any such data? We learn from Wang, that...

Any of the arithmetic operations (addition/subtraction and multiplication/division) can be applied to two continuous signal $x(t)$ and $y(t)$, or two discrete signals $x[n]$ and $y[n]$, to produce a new signal $z(t)$ or $z[n]$:

- Scaling: $z(t) = ax(t)$ or $z[n] = ax[n]$.
- Addition/subtraction: $z(t) = x(t)y(t)$ or $z[n] = x[n]y[n]$.
- Multiplication: $z(t) = x(t)y(t)$ or $z[n] = x[n]y[n]$.
- Division: $z(t) = x(t)/y(t)$ or $z[n] = x[n]/y[n]$.

Note that these operations are actually applied to the amplitude values of the two signals $x(t)$ and $y(t)$ at each moment t , and the result becomes the value of $z(t)$ at the same moment; and the same is true for the operations on the discrete signals.

—**Ruye Wang**, *Introduction to Orthogonal Transforms: With Applications in Data Processing and Analysis*, 2012[85]

That theory brings to mind the realization that new/derived signals could be computed or modelled via modification or combination of other signals via basic arithmetic operations on their amplitudes [at specific moments or times]. For comparison with the formalisms and language of transformatics, we might want to note that we could express any such transformations using a single generic transformer such as:

Transformer 25 (General Signal Amplitude Transformer, $GSAT(x, y, op)$).

$$\begin{aligned}
 &\langle \Theta_x, \Theta_y \rangle \xrightarrow{GSAT(x, y, op)} \Theta_z; \\
 &\Theta_x : \mathbb{N} \times \mathbb{R} \quad \wedge \quad \Theta_y : \mathbb{N} \times \mathbb{R} \quad \wedge \quad \Theta_z : \mathbb{N} \times \mathbb{R} \\
 &\wedge \quad 0 \leq (\nu(\Theta_x) = \nu(\Theta_y) = \nu(\Theta_z)) \leq \infty \\
 &\wedge \quad \forall x_i = x(i) \in \Theta_x, y_i = y(i), z_i = z(i) \in \Theta_z \implies i = I(x_i, \Theta_x) = I(y_i, \Theta_y) = I(z_i, \Theta_z) \equiv t \quad (a \text{ moment in time}) \implies 0 \leq t \leq \nu(\Theta_x) \\
 &\wedge z_i = GSAT(x_i, y_i, op) = \begin{cases} 0 + ay_i & op \text{ is scaling and } x = 0 \quad \forall x \in \Theta_x \\ x_i \pm y_i & op \text{ is addition/subtraction} \\ x_i * y_i & op \text{ is multiplication} \\ \frac{x_i}{y_i} & op \text{ is division} \end{cases} \quad \square.
 \end{aligned}$$

And even though we could have gone further to also define or demonstrate other kinds of transformers that can not only generalize, but also combine multiple signal transformations into a single specification or transformatics operator, we shall only stop at **Transformer 25** for now, but otherwise note that, in light of the theory and discussions we have already presented when relating transformatics to Statistics (**Section E.2.1**), Linear Algebra (**Section E.2.2**) and Calculus (**Section E.2.3**), that, many, if not all of what is presented in [85] concerning transformations of one or more signals into other signals can without doubt be expressed or modeled using the formalisms of transformatics.

One finds that many of the signal transforms can in a way or another be based on the concept we have laid down in **Transformer 25**. These might for example involve more sophisticated formulations for how the resultant signal $z(t)$ is derived from one or more source signals — such as transforms involving both scaling and translation, or which instead of relying on transforms of the signal amplitude, rely on re-sampling, complex sampling and otherwise derivations of the final signal based on transformations of the time-dimension²⁵ and not anything concerning manipulating the signal amplitude itself, etc.

Also, one finds that, these observations shall likewise help make our transformatics relevant across many fields of science and the mathematical sciences especially, once one realizes that the theories and formalisms of signal transformation, processing and analysis likewise span and are relevant across many fields as we see Wang assure us when he says:

A generic system (electrical, mechanical, biological, economical, etc.) can be symbolically represented in terms of the relationship between its input $x(t)$ (stimulus, excitation) and output $y(t)$ (response, reaction):

$$\mathbb{O}[x(t)] = y(t) \tag{E.38}$$

where the symbol $\mathbb{O}[]$ represents the operation applied by the system to its input. A system is *linear* if its input-output relationship satisfies both *homogeneity* and *superposition*.

²⁵E.g. $z(t) = x(t + t_0)$, $z^*(t) = z(at)$, etc.

And further, clarifies that...

Homogeneity:

$$\mathbb{O}[ax(t)] = a\mathbb{O}[x(t)] = ay(t) \quad (\text{E.39})$$

Superposition: If $\mathbb{O}[x_n(t)] = y_n(t)$ ($n = 1, 2, \dots, N$), then

$$\mathbb{O} \left[\sum_{n=1}^N x_n(t) \right] = \sum_{n=1}^N \mathbb{O}[x_n(t)] = \sum_{n=1}^N y_n(t) \quad (\text{E.40})$$

or [for continuous signals]:

$$\mathbb{O} \left[\int_{-\infty}^{\infty} x(t) dt \right] = \int_{-\infty}^{\infty} \mathbb{O}[x(t)] dt = \int_{-\infty}^{\infty} y(t) dt \quad (\text{E.41})$$

[and combining which, we have]:

$$\mathbb{O} \left[\sum_{n=1}^N a_n x_n(t) \right] = \sum_{n=1}^N a_n \mathbb{O}[x_n(t)] = \sum_{n=1}^N a_n y_n(t) \quad (\text{E.42})$$

[and]:

$$\mathbb{O} \left[\int_{-\infty}^{\infty} a(\tau) x(t, \tau) d\tau \right] = \int_{-\infty}^{\infty} a(\tau) \mathbb{O}[x(t, \tau)] d\tau = \int_{-\infty}^{\infty} a(\tau) y(t, \tau) d\tau \quad (\text{E.43})$$

—**Ruye Wang**, *Introduction to Orthogonal Transforms: With Applications in Data Processing and Analysis*, 2012[85]

Thus, whether it is in economics, or in chemical engineering or perhaps any of the biological sciences or even bio-informatics to which our present major focus on computational genetics would lie, one finds that, when required, especially when we wish to model or formally specify the properties of some signal transformation²⁶ — such as for purposes of rendering it easy to simulate, process or ana-

²⁶Including *time-invariant* signals[85].

lyze using computer programs such as those expressed via a sequence-transformer paradigm such as with TEA, that then, transformatics and especially its sequence transformer notions, shall readily come in handy.

Artificial Intelligence and Machine Learning: Natural Language Processing via Neural Networks — so-called “Transformers”

Our final discussion is going to be centered around a very important blog-post[86] that I first came across while browsing the high-signal, ultra-busy technology and research news fire-hose known as **Hacker News**²⁷²⁸.

So, it happens that much of what we enjoy and love about contemporary high-tech that is AI-based or AI-driven — products such as Google AI search, Microsoft Co-Pilot, Apple’s Siri personal assistant, Bing Image Creator, ChatGPT-powered AI assistants, etc. etc. all have one major and fundamental thing in common — so-called “transformers”! In his article — see [86] — on the subject, **Brandon Rohrer**²⁹ introduced his detailed clarification on the critically important matter of modern neural networks that leverage the “transformer architecture” as such:

Transformers were introduced in this 2017 paper[87] as a tool for *sequence transduction*---converting one sequence of symbols to another. The most popular examples of this are translation, as in English to German. It has also been modified to perform sequence completion---given a starting prompt, carry on in the same vein and style. They have quickly become an indispensable tool for research and product development in natural language processing.

The stress on the term “sequence transduction” is ours, especially to emphasize the entry-point of our discussion on how transformatics and contemporary artificial intelligence (AI) and machine learning (ML) might be related. Rohrer is sharing insights into what goes on behind the scenes when a neural network is trained sample data (also technically known as “training data”), and which for example involves specially crafted mappings between queries and [labeled] outputs, and so that, when the network has finally learned (or inferred) the patterns (typ-

²⁷Very popular among elite technocrats all over the world, but especially in the Silicon Valley and mostly because of its **Y-Combinator**/Paul Graham-biased Internet Community regulars from across all kinds of [tech]-startups and firms big and small.

²⁸See <http://news.ycombinator.com/news>

²⁹Currently a Principal Machine Learning Engineer at Atlassian[32]

ically, neural-network parameters, or rather “weights”) what the training data encoded, it can then use that information to infer, predict or classify new [unlabeled] cases in a way that best resonates with what was [correctly] learned thus far. In this case too, focus isn’t on just any kind of neural network³⁰ but specifically the **Transformer**[87] — a kind neural network whose core architecture essentially relies on “Attention” — a special linear-algebraic sequence transformation function that maps a sequence consisting of a combination of queries, keys and value data as the sole input for computing the output that is essentially weighed values[87].

When one looks back at the theory we have developed and presented in our treatment of applying transformatics to genetics[8], especially looking at the work around the **Lu-Genome Expression System** (Section 10), one shall realize that what accomplished by mapping originally DNA (or rather “na-Sequence”) data to visual-spatial (pictographic) expressions — essentially, starting with text and ending up with images, based on careful and thoughtful chaining of our transformatics-oriented transformer functions is [conceptually³¹] very much akin to what sequence transduction is as originally presented in the 2012 foundational paper on the concept by **Alex Graves**[88]:

Many machine learning tasks can be expressed as the transformation---or *transduction*---of input sequences into output sequences: speech recognition, machine translation, protein secondary structure prediction and text-to-speech to name but a few.

Thus, we see that when we treat of scenarios in which data originates in some particular encoding format (e.g as a Statement of Intent (SOI) expressed in plain usual English language text), and then later [either manually or computationally] gets mapped to a corresponding or even only remotely similar pictograms (such as the sigils, hieroglyphs and even 3-dimensional constructs such as sculptures and pendants treated of in another work on applying transformatics — see [30]), that we in a way are applying the idea of transduction³². In Rohrer’s article[86], but also in the underlying paper[87] and also the much earlier paper[88], focus is

³⁰The core cited paper, Vaswani Ashish et al.[87], also mentions or mainly contrasts the new neural network architecture to other [earlier] models such as convolutional and recurrent neural networks.

³¹We stress “conceptually” here, because, unlike the works and presentations of AI researchers such as Graves[88] and Vaswani Ashish et al.[87], our use of transformers had little to do with conventional machine learning or even neural-network based processing of sequences, and more to do with a basic mathematical and conceptual processing in order to achieve what others might attempt using especially black-box machine learning models and computer programs.

³²The conceptual stretch might be more relevant here than not.

on especially transforms treating of especially data encoded as text — even where the problem involves processing either images or audio information³³.

And so, we learn from [86] when he talks about transformers as involving “matrix multiplications and touching on backpropagation (the algorithm for training the model)”, that even though these transformer models (but also much of what actually goes on in even earlier, perhaps less sophisticated neural network architectures[87]) might seem to most as “mysterious black boxes”, and yet, underlying these computational beasts that bring modern artificial intelligence to life is nothing other than an application of linear algebra (matrix operations and transforms), statistics (especially probability theory and statistical modeling), calculus (integration and lots of differentiation such as in “gradient descent”[88]) but also essential signal processing — all of which subjects or fields of mathematics we have somewhat touched upon in the discussions and illustrative contrasts between transformatics and other fields throughout **Chapter E**.

For example, concerning the [older] recurrent neural network (RNN) sequence transducers, we have the following enlightening excerpt from [88]:

Let $x = (x_1, x_2, \dots, x_T)$ be a length T *input sequence* of arbitrary length belonging to the set χ^* of all sequences over some input space χ . Let $y = (y_1, y_2, \dots, y_U)$ be a length U output sequence belonging to the set γ^* of all sequences over some output space γ . Both the inputs vectors x_t and the output vectors y_u are represented by fixed-length real-valued vectors; for example if the task is phonetic speech recognition, each x_t would typically be a vector of MFC coefficients and each y_t would be a one-hot vector encoding a particular phoneme. In this paper we will assume that the output space is discrete; however the method can be readily extended to continuous output spaces, provided a tractable, differentiable model can be found for γ .

³³Which, for our case, is also a fact that Lutalo’s UPLT[51] clearly cements.

Define the extended output space $\bar{\gamma}$ as $\gamma \cup \emptyset$, where $\emptyset$ denotes the null output. The intuitive meaning of $\emptyset$ is ‘output nothing’; the sequence $(y_1, \emptyset, \emptyset, y_2, \emptyset, y_3) \in \bar{\gamma}^*$ is therefore equivalent to $(y_1, y_2, y_3) \in \gamma^*$. We refer to the elements $a \in \bar{\gamma}^*$ as alignments, since the location of the null symbols determines an alignment between the input and output sequences. Given x , the RNN transducer defines a conditional distribution $Pr(a \in \bar{\gamma}^*|x)$. This distribution is then collapsed onto the following distribution over γ^* :

$$Pr(y \in \gamma^*|x) = \sum_{a \in \beta^{-1}(y)} Pr(a|x) \quad (\text{E.44})$$

where $\beta : \bar{\gamma}^* \rightarrow \gamma^*$ is a function that removes the null symbols from the alignments in $\bar{\gamma}^*$. Two recurrent neural networks are used to determine $Pr(a \in \bar{\gamma}^*|x)$. One network, referred to as the *transcription network* F , scans the input sequence x and outputs the sequence $\mathbf{f} = (f_1, \dots, f_T)$ of transcription vectors. The other network, referred to as the *prediction network* G , scans the output sequence y and outputs the prediction vector sequence $\mathbf{g} = (g_0, g_1, \dots, g_U)$.

—Alex Graves, *Sequence Transduction with Recurrent Neural Networks*, 2012[88]

Thus we see that some [costly] processing is involved in eventually generating the output of a network layer when working using RNNs, whereas, for the newer, simpler and also more performant transformer models[87], we learn from the Attention paper that the vital/newer sequence processing step — essentially, the “attention function”, is just:

We call our particular attention ‘“Scaled Dot-Product Attention”... The input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q . The keys and values are also packed together into matrices K and V . We compute the matrix of outputs as:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (E.45)$$

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

—Ashish Vaswani and Noam Shazeer et. al, *Attention Is All You Need*, 2017[87]

We shall not attempt to demonstrate if indeed any of the actual computations that go on inside of a [modern] neural network — convolutional, recurrent or transformer-based — are somewhat reducible to or expressible using **transformatic transformers** — instead leaving any such explorations as an exercise to the reader or reviewer, BUT — and this is a huge but — it is only to the blind that this wouldn't make sense; neural networks at core, are first of all a chaining of functions operating on ordered sets (**sequences transformers**) of especially real numbers³⁴ in order to produce/output other numbers (also **data** that is essentially also sequences or just text[51]), and which is then either then the final output, or which is otherwise fed back into the model (“back-propagation”) or is passed on to other/next layers/functions that are themselves also sequence processors or transformers, and so that ultimately, what we love about the “intelligence” in modern AI and ML is but the emergent properties of a [somewhat daunting] complex assembly line or factory whose entire machinery is but a care-

³⁴Without need to stress the case when “ancient” networks required special pre-training data alignments to ensure both consistence between types and structure/size across layers or networks meant for different kinds of data[88] — such as when a model can only accept binary inputs, etc.

ful chaining of sequence transformers — the stuff about which, our proposed field of transformatics is all about.

Finally, and before we close the diversion into AI and machine learning, it should be worth noting that not all kinds of automated intelligence need be approached via the methods of neural networks — NNs are well-known for being shockingly great at modeling all kinds of functions and transforms, however, just like not all statistical problems need be solved using gaussian models, or equivalently that not all probability distributions need be reduced to the normal distribution³⁵, we must realize that non-NN oriented transformers exist³⁶ — such as we demonstrate and model in **Scenario D** as well as other cases introduced in **Section 6.1** — that can effectively but also efficiently help both model and solve hard and simple problems that popular [and typically resource expensive] methods such as LLMs or NNs might otherwise only “magically” solve.

E.2.5 Transformatics as a new Algebra of Sequences

Finally, note that away from a generalization such as **Definition 16** — which is the latest and first formal definition of transformatics, in recent comparative studies on transformatics, such as the October 2025 review of **Professor Jeremy Gibbon’s** 1991 **tree algebras** thesis[71], we have the following telling signs that indeed, we might also approach the field of transformatics as a kind of algebra in the sense of the usual use of that term in both mathematics and say computer science:

...we might think of the mathematical field or theory of transformatics[23] as the algebra of sequences --- essentially, the treatment of sequences and operations on them. As laid out in the transformatics mini-thesis[23], we might for example think of mathematical objects such as ordered sets, symbol sets, numeric and alpha-numeric expressions, Lu-Numbers or Lu-Number Expressions[1], matrices, sequences of instructions in a TEA program, etc. And as for operations on them, we might call out operations such as concatenation, inversion, splitting, reduction, multiplication, etc. many of which have been well defined, introduced or laid down in Lutalo’s earlier works such as [8], [1] or [5].

³⁵Even though, and as a great fall-back-to heuristic, the law of large numbers dictates/guarantees this[83]

³⁶Here, the emphasis is on “transformers” as used in transformatics — as computable functions that can map a sequence to [another] sequence or scalar, and especially our argument is towards approaches such as more traditional *logical processors*, non-NN (sic) *statistical artificial intelligence*, *linear programming* etc.

E.3 A Final Definition and Justification of a Field

And thus, or rather, finally, we might advance a more concrete definition for the subject of transformatics as such:

Definition 17 (Transformatics). *The study of **sequence algebra**, **sequence transformations** and **sequence analysis** —- succinctly: the study of the algebra, transformations and analysis of sequences.*

So that then, transformatics is **based on** set theory, algebra and mathematical logic, but otherwise **can underlie** or power old and new ideas in linear algebra, calculus, statistics, computer science, software engineering and [computational or mathematical] genetics³⁷ to name but a few.

³⁷Or perhaps **bioinformatics**? computational biology?

Appendix F

Future Directions and Explorations of TRANSFORMATICS in both PHYSICS and GENETICS?

Definition 18 (The Transformatics of Teleportation¹).

Definition 19 (Teleportation). *is a natural, simulated or preternatural phenomena, by which a collection of particles existing in some definite space and time, are projected to an alternate collection of similar particles, with one or more of their properties altered — in the case of conventional concepts of teleportation, such might mean expressing a thing that is existing at the present time, as it were or shall be in either a past or future time frame respectively. In the language and calculus of transformatics, such might be a transformation of some given sequence of n -grams of dimension 1 or more — e.g particles of something, where each particle is described by a set of 4 or more numeric values such as their x, y, z coordinates in space, and a fourth coordinate, t , that captures the time at which that particular particle located at a definite location is expressed. A teleportation machine then, would be any abstract or actual machine, which, when presented with the thing — say, a matrix of particles expressing its [initial] location in some particular space-time, can then produce and return another matrix depicting how that space-expression shall be at a later or earlier (basically, different) time, as per the following **time machine** specification:*

Definition 20 (The Teleportation Machine, $M\{\Delta T \rightarrow \Delta P\}$). *is the machine, which, when given a sequence, $P^n\langle\rho_{i \in [1, \alpha]}|\mathcal{U}(\rho) = 3 + k \wedge \forall \rho_i \in \ddot{P} : k \in \mathbb{N}, i \in [1, 3 + k], \alpha = \frac{n}{3+k}\rangle$ such that $\ddot{P}$ is P^n expressed as a higher-order sequence of $(3 + k)$ -gram particles, and so that $\mathcal{U}(\ddot{P}) = \alpha$ or rather that $\mathcal{U}(\ddot{P}^\alpha) = \alpha$, and so $\ddot{P}^{2\alpha}$ is some derived resultant sequence of particles² of $3 + k$ dimension, and so that the machine produces that new sequence of similar dimensions and*

¹In this (mostly speculative) exploration, we shall simply treat **teleportation** and **time travel** as though they were the same or related phenomena

²Basically, coordinates in some $(3 + k)$ -space, such that $k = 0$ represents **3D** space for example. However, for the teleportation machine, we need to also cater for **the time property** of each particle in the sequence $\ddot{P}$, and so, perhaps we need $k \geq 1$ so as to also cater for space-time. $\ddot{P}^{2\alpha}$ then, is meant to be a sequence capable of containing the original sequence of particles as well as their new/alternate projections, losslessly.

protracted cardinality, $P^ = \ddot{P}^{2\alpha}$, such that for each original particle $\rho_i \in \ddot{P}^\alpha$, there is a corresponding complement or pairwise-transformed-partner particle, ρ_i^* , in the resultant sequence, P^* — the later half of $\ddot{P}^{2\alpha}$, as produced by the following sequence transformer specification:*

Transformer 26 (The **Teleportation Transform**). $P^n \rightarrow P^*$;

$$\mathfrak{L}(P^*) = 2 \times \mathfrak{L}(P^n) = 2 \times n$$

$$\wedge P^n \equiv \ddot{P}^\alpha \quad \wedge \quad P^* \equiv \ddot{P}^{2\alpha}$$

$$\wedge \forall \rho_i \in \ddot{P}^\alpha \quad \exists \rho_i^* \in \ddot{P}^{2\alpha} : \rho_i^* = \rho_i \quad \wedge \exists \rho_{i+\alpha}^* : \tilde{A}(\rho_i \rightarrow \rho_{i+\alpha}^*) > 0$$

$$\wedge \rho_{i+\alpha}^* = f(\rho_i, \Delta T) \quad \square$$

*With **Transformer 26**, and some time-sensitive translation function*

$$f(x, y, z, t_1, t_n) = \Gamma(\delta x, \delta y, \delta z, \Delta T) \quad (\text{F.1})$$

*We can predict/extrapolate any point or particle between two or more moments across space and time, relative to the absolute change one or more of the $(3+k)$ dimensions — in the simplest case, in the time-dimension (4^{th} dimension) as well as a particular resolution/reference space-time frame. Thus, and for example, we could take a plane and by translating it to another plane removed from the first by a reasonable significant distance, and such that the translation is orthogonal to some dimension $\text{dim} < 3 + k$, we might describe a solid — **a teleportation** of particles too, formed by rotating the plane about some axis such as orthogonal to it, and such that the space-time thus spanned is bound by the source and resultant plane sequences, but also defined by them. $\square$*

Because this volume is about how we might apply transformatrics in GENETICS, and yet the underlying *original* intent was to also explore how to further transformatrics as its own line of inquiry, not just in Genetics, but also in other sciences and perhaps even the arts(?), it shall make sense to add to what has already been discussed and shared in the rest of this book, with a sharing of a recent excerpt from the author's psychoanalytic notes — because, like we mentioned in the **Preface** of this work, Carl Jung's ideas[57] on leveraging **dream-analysis** to further not just our understanding of ourselves but also of the world, did have such a great impact on the author over the years, it wouldn't make sense closing this book without sharing an aspect of how dreaming and Jung's methods have helped in making much of this work possible, as well as how it might help define the future directions of our further research and studies in relation to the current subject (and transformatrics in general).

Talking of which, the following is a verbatim excerpt, and is accompanied by a painting — “Arrival of the Anunnaki”³, also by the author, and part of their **Zermelo Frankael Dream Collection**⁴ most of whose art pieces have since been either sold or given out as souvenirs to friends and family.

³Image credit from author's Art Portfolio: <https://www.deviantart.com/nemesisfixx/art/Arrival-of-the-Anunnaki-inverted-94288530>

⁴To get an idea what was in the other paintings in that collection from 2022, checkout <https://www.deviantart.com/nemesisfixx/art/The-Zermelo-Frankael-Dream-Collection-II-942887985> and <https://www.deviantart.com/nemesisfixx/art/Best-of-JWL-Art-from-2022-940119085>



Figure F.1: The surreal painting, “Arrival of the Anunnaki”, by Joseph W. Lutalo, as part of their 2022 **The Zermelo Frankael Dream Collection** that was first exhibited at Ndugu Art Gallery then in Entebbe Town.

In a dream, I have seen that I was somewhere in a laboratory with some very intelligent fellows and that we were exploring potential applications of TRANSFORMATICS in PHYSICS, by conducting simulation experiments using a large hologram machine that looked like a table around which we sat and held discussions.

I saw that among interesting analyses we conducted was the matter of trying to understand "The Bermuda Triangle Problem". In particular, we were simulating a strong magnetic field situated at the center of some location upon earth, and that if any flying object --- such as Flying Saucers (UFOs) attempted to fly close enough to this gravitating point, it would cause their trajectories to somewhat get warped towards the center, and so that, no matter how strong the tangential velocity of the craft was relative to the center of attraction at some moment with time $T=t_n$, at any other later moment such as $T=t(n+1)$, the centripetal force on the craft would be even stronger, and that the distance from the point of the center of attraction would be lesser at $t(n+1)$ than at t_n , and so that, eventually the craft would spiral down into the point and somewhat crash or (like for the case of the whirlpool-like Bermuda Triangle), get swallowed into some kind of black hole (in the ocean).

The simulations looked so realistic despite being generated by a super computer, and it felt like we were performing reconnaissance studies about aliens visiting Earth at say... Area 51! Then I woke up.

A Dream About The Bermuda Triangle Experiment & Applying TRANSFORMATICS in PHYSICS

NOTE: After waking up and thinking a bit about all this.. a stunning realization occurred to me; there's an acrylic painting I once did (titled "Arrival of the Annunaki", and which I had exhibited at the then Ndugu Art Gallery in Entebbe Town circa 2022, but which now was at home with me), in which I painted a vision that had occurred to me concerning a spaceship that visited Earth some day (in the future perhaps), and that the ship shone a very brilliant light upon some village, and that as people freaked out, a ladder descended from the ship onto the earth, and some few bold people ascended into the ship by climbing up that ladder, while others, too afraid, only scattered and ran away.

So, while thinking about this Bermuda experiment on TRANSFORMATICS and also this interesting painting, and trying to connect the dots... I feel like.. perhaps there is something about the DNA ladder-like structure that could get us to understanding the biology or natural powers of aliens? Life on other planets or worlds perhaps? Jacob's ladder from the Scriptures perhaps? Basically, it strongly feels like these two "dreams" might be telling us (or me in particular) or pointing us towards something both alarming and exhilarating! Also, it could indeed inspire future directions and explorations of TRANSFORMATICS in both PHYSICS and GENETICS!

#dream #diary #advanced #physics #transformatics #jwl #reflections #futuresdirections
#CREATED:Aug 22, 2025 03:02:32

Applying TRANSFORMATICS

“A woman is a string that is a replicating enclosure. You invest a substring of your string in her for a while, and then she transforms it into a higher string which she eventually kicks out into the world. The woman is a very special transformer, the man mostly a generator. That's most of natural biology expressed using TRANSFORMATICS.

Another way to look at it; Tukaruga ha musaana, twaija omunsi, yatuzaara.. kyakweta okutufoora. Baitu emara netubinga kugenda omumwanya kugarukayo... Kikuzooka niho KITARA.

So, it's perhaps merely humbling to see how a basic mathematics originally meant to explain artificial intelligence and abstract machines, automata, can likewise explain or express the creative, dynamic science of biological systems as well as the queer idea that life is an investment by the sun into planetary ecosystems, and which can later escape their closures to further express it in distant realms such as in remote galaxies and space-times!”

— a reflection about the new mathematical field of TRANSFORMATICS as applied to explaining various natural and artificial phenomenon.

Foundation Paper: <https://bit.ly/transformatics101>

fut. prof. J. Willrich
(currently at Nuchwezi Research)

Bibliography

- [1] Joseph Willrich Lutalo. **Philosophical and Mathematical Foundations of A Number Generating System: The Lu-Number System.** *PrePrints*, 2025. https://www.preprints.org/frontend/manuscript/65675cdcf92eaced19494771ba02fbd/download_pub.
- [2] Joseph Willrich Lutalo. **A general theory of number cardinality.** *Academia.edu*, Jan 2024. Accessible via https://www.academia.edu/43197243/A_General_Theory_of_Number_Cardinality.
- [3] Joseph Willrich Lutalo. **Introducing The Anagram Distance Statistic, $\tilde{A}$, A Quantifier of Lexical Proximity For Base-10 o-SSI And Any Arbitrary Length Ordered Sequences, Its Relevance And Proposed Applications in Computer Science, Engineering And Mathematical Statistics.** *Academia.edu*, 2025. https://www.academia.edu/download/123454710/arxiv_version_ADT_Anagram_Distance_Theory_paper_25JUNE2025_JWL.pdf.
- [4] Joseph Willrich Lutalo. **Concerning A Special Summation That Preserves The Base-10 Orthogonal Symbol Set Identity In Both Addends And The Sum.** *Academia*, 2025. Accessible via https://www.academia.edu/download/122499576/The_Symbol_Set_Identity_paper_Joseph_Willrich_Lutalo_25APR2025.pdf.
- [5] Joseph Willrich Lutalo. **The Theory of Sequence Transformers & their Statistics:** The 3 information sequence transformer families (anagrammatizers, protractors, compressors) and 4 new and relevant statistical measures applicable to them: Anagram distance, modal sequence statistic, transformation compression ratio and piecemeal compression ratio. *Academia*, 2025. https://www.academia.edu/download/123730923/TRANSFORMATICS_Theory_Of_Sequence_Transformers_10JUL2025_JWL_NuchweziResearch_v3.pdf.
- [6] Joseph Willrich Lutalo. **Transformatics for PSYCHOLOGY.** *PrePrints*, 9 2025. Presented at the 13th International East African Psychology Conference, a copy is accessible via: https://www.preprints.org/frontend/manuscript/bd2e5e192c083fae8ba6ed371c18f044/download_pub.

- [7] Joseph Willrich Lutalo. **Applying TRANSFORMATICS: Sequence Generators**. https://www.academia.edu/143116027/Applying_TRANSFORMATICS_Sequence_Generators, 2025.
- [8] Joseph Willrich Lutalo. **Applying TRANSFORMATICS in GENETICS**. *I*POW*, pages 1–206, Aug 2025. <https://www.academia.edu/143516392/>.
- [9] Joseph Willrich Lutalo. **TEA GitHub Project — The Reference Implementation of TEA (Transforming Executable Alphabet) computer programming language**. https://github.com/mcnemesis/cli_tttt/, 2024. *Single Source of Truth concerning the reference standards of TEA on the web and native operating systems*.
- [10] Wikipedia contributors. Base pair. https://en.wikipedia.org/wiki/Base_pair, 2025. Accessed August 2, 2025. Includes diagram of DNA double helix showing A–T and C–G base pairing.
- [11] GeeksforGeeks. Diagram of protein synthesis. <https://www.geeksforgeeks.org/biology/protein-synthesis-diagram/>, 2024. Accessed July 29, 2025. Shows transcription, translation, and ribosome-mediated protein synthesis.
- [12] Joseph Willrich Lutalo. **The OZIN Cipher**. https://www.academia.edu/145919940/The_OZIN_CIPHER, Jan 2026.
- [13] Joseph Willrich Lutalo. **The PLATO Cipher**. https://www.academia.edu/145920076/The_PLATO_CIPHER, Jan 2026.
- [14] Fir0002 / Flagstaffotos. Earthworm.jpg, 2007. Image taken in Swifts Creek, Victoria. Licensed under CC BY-NC and GFDL 1.2. Attribution required to Fir0002/Flagstaffotos.
- [15] Joseph Willrich Lutalo. *Shrines of the Free Men*. *I*POW*, 2023. No ISBN — The official, free-access electronic edition is accessible via: <https://t.me/ipowriters/26>.
- [16] Joseph Willrich Lutalo. **Statement of Purpose (University of Oxford)**. https://www.academia.edu/145809741/Statement_of_Purpose_University_of_Oxford_, Aug 2025.
- [17] Joseph Willrich Lutalo. **PROPOSED DOCTORAL THESIS: A Theory of Entropy Sources & Random Number Generators: DPhil Computer Science, Oxford University, Joseph Willrich Lutalo MSc. BSc. alumni Makerere University**, 6 2025. Available at: https://www.academia.edu/145809448/PROPOSED_DOCTORAL_THESIS_A_Theory_of_Entropy_Sources_and_Random_Number_Generators_DPhil_Computer_Science_Oxford_University_Joseph_Willrich_Lutalo_MSc_BSc_alumni_Makerere_University_1.

- [18] Carl Sagan. *Cosmos*. Book Club Associates, London, UK, 1981. Special edition for **Book Club Associates**.
- [19] Richard L. Gregory and Oliver L. Zangwill, editors. *The Oxford Companion to the Mind*. Oxford University Press, Oxford, UK, 1987. Available online at <https://archive.org/details/oxfordcompaniont00greg>.
- [20] BioExplorer.net. Do bacteria have nucleus? <https://www.bioexplorer.net/do-bacteria-have-nucleus.html/>, 2025.
- [21] Seungho Kang, Alexander K Tice, Frederick W Spiegel, Jeffrey D Silberman, Tomáš Pánek, Ivan Čepička, Martin Kostka, Anush Kosakyan, Daniel M C Alcântara, Andrew J Roger, et al. Between a pod and a hard test: The deep evolution of amoebae. *Molecular Biology and Evolution*, 34(9):2258–2270, 2017. <https://academic.oup.com/mbe/article/34/9/2258/3827454>.
- [22] OpenStax Biology. Viruses - biology libretexts. [https://bio.libretexts.org/Bookshelves/Introductory_and_General_Biology/General_Biology_1e_\(OpenStax\)/5:_Biological_Diversity/21:_Viruses](https://bio.libretexts.org/Bookshelves/Introductory_and_General_Biology/General_Biology_1e_(OpenStax)/5:_Biological_Diversity/21:_Viruses), 2025.
- [23] Joseph Willrich Lutalo. **TRANSFORMATICS 101 - explained**. *Thesis*, page 20, October 2025. <https://www.academia.edu/144344109/>.
- [24] Grady Venville and Jenny Donovan. Analogies for life: a subjective view of analogies and metaphors used to teach about genes and dna. *Teaching Science*, 52(1):18–22, 2006. Available at: <https://research-repository.uwa.edu.au/en/publications/analogies-for-life-a-subjective-view-of-analogies-and-metaphors-u>.
- [25] University of Nebraska–Lincoln. Complementary, antiparallel dna strands — dna and chromosome structure. <https://passel2.unl.edu/view/lesson/6f214d098527/4>, n.d. Accessed July 29, 2025.
- [26] Genomics Education Programme. Where does our genome come from? <https://www.genomicseducation.hee.nhs.uk/education/core-concepts/where-does-our-genome-come-from/>, 2025. Accessed July 29, 2025. Explains how sperm and egg each contribute half the genome, forming a unique combination in the zygote.
- [27] OpenStax Biology. Gametogenesis (spermatogenesis and oogenesis). [https://bio.libretexts.org/Bookshelves/Introductory_and_General_Biology/General_Biology_1e_\(OpenStax\)/43:_Animal_Reproduction_and_Development/43.3C:_Gametogenesis_\(Spermatogenesis_and_Oogenesis\)](https://bio.libretexts.org/Bookshelves/Introductory_and_General_Biology/General_Biology_1e_(OpenStax)/43:_Animal_Reproduction_and_Development/43.3C:_Gametogenesis_(Spermatogenesis_and_Oogenesis)), 2025. Accessed July 29, 2025.
- [28] University of Leicester. The cell cycle, mitosis and meiosis for higher education. <https://le.ac.uk/vgec/topics/cell-cycle/the-cell-cycle-higher-education>, n.d. Accessed July 29, 2025.

- [29] Joseph Willrich Lutalo. **TEA TAZ - Transforming Executable Alphabet A: to Z: COMMAND SPACE SPECIFICATION**. *Academia.edu*, 2024. <https://www.academia.edu/122871672/>.
- [30] Joseph Willrich Lutalo. *NOVUS MODERNUS GRIMOIRE LUMTAUTO MAGIA - A Modern Grimoire of 5 Eternal Magickal Languages*. I*POW, November 2025. <https://www.academia.edu/resource/work/145086513>.
- [31] Shinichi Mochizuki. The geometry of frobenioids i: The general theory. <https://www.kurims.kyoto-u.ac.jp/~motizuki/The-Geometry-of-Frobenioids-I.pdf>, 2008. Accessed August 2, 2025. Introduces Frobenioids as symbolic categorical structures encoding arithmetic transformations.
- [32] Microsoft Copilot. Clarifying discussions on dna sequence facts and genetics in general during drafting manuscript. AI-generated insights via Copilot discussion, 2026. Personal communication throughout preparation and revision of the manuscript.
- [33] Wikipedia contributors. Nucleic acid sequence. https://en.wikipedia.org/wiki/Nucleic_acid_sequence, 2025. Accessed July 2025.
- [34] Nature Education. The four bases – atcg. <https://www.nature.com/scitable/content/the-four-bases-atcg-6491969/>, n.d. Accessed July 2025.
- [35] Regina Bailey. Genetic code and rna codon table. <https://www.thoughtco.com/genetic-code-373449>, 2019. Accessed July 2025. Explains RNA nucleotide composition and codon structure.
- [36] National Center for Biotechnology Information (NCBI). Refseq: Ins homo sapiens insulin [nm_000207.2]. https://databases.lovd.nl/shared/refseq/INS_NM_000207.2_codingDNA.html, 2020. Accessed July 31, 2025.
- [37] J. Craig Venter et al. The sequence of the human genome. *Science*, 291(5507):1304–1351, 2001. Available at: <https://www.science.org/doi/pdf/10.1126/science.1058040>.
- [38] S. Anderson, A. T. Bankier, B. G. Barrell, et al. Sequence and organization of the human mitochondrial genome. *Nature*, 290:457–465, 1981. Available at: <https://www.nature.com/articles/290457a0.pdf>.
- [39] Joseph Willrich Lutalo. **Concerning Debugging in TEA and the TEA Software Operating Environment**. *Academia.edu*, 2025.
- [40] Wikipedia contributors. Dna and rna codon tables. https://en.wikipedia.org/wiki/DNA_and_RNA_codon_tables, 2025. Accessed August 2, 2025. Provides codon-to-amino acid mappings and genetic code/name tables.

- [41] Joseph Willrich Lutalo. **An Interview and Meta-Review of Transformatomics Introductory Lecture of 3rd FEB 2026.** *Preprints*, February 2026. Access Original version: <https://www.academia.edu/164509813/>.
- [42] Shusei Sato, Yasukazu Nakamura, Takakazu Kaneko, Erika Asamizu, and Satoshi Tabata. Complete structure of the chloroplast genome of arabidopsis thaliana. *DNA Research*, 6(5):283–290, 1999.
- [43] Joseph Willrich Lutalo. **TEA TAZ - Transforming Executable Alphabet A: to Z: COMMAND SPACE SPECIFICATION.** *Academia.edu*, 2024. Accessed on 14 May, 2025, via https://www.academia.edu/122871672/TEA_TAZ_Transforming_Executable_Alphabet_A_to_Z_COMMAND_SPACE_SPECIFICATION.
- [44] Valerie Illingworth, editor. *Oxford Dictionary of Computing*. Oxford University Press, Oxford, UK, 4 edition, 1996. Available at <https://openlibrary.org/books/OL412451M>.
- [45] Joseph Willrich Lutalo. **TEA RESEARCH: TEA ON THE WEB A High-Level Web Software Operating Environment Specification For The TEA Programming Language: Web TEA Architecture.** <https://www.academia.edu/142944443/>, 2025. Academia.
- [46] Kurt Gödel. Über formal unentscheidbare sätze der principia mathematica und verwandter systeme i. *Monatshefte für Mathematik und Physik*, 38:173–198, 1931.
- [47] Kurt Gödel. On formally undecidable propositions of principia mathematica and related systems i. In Solomon Feferman, John W. Dawson Jr., Stephen C. Kleene, Gregory H. Moore, Robert M. Solovay, and Jean van Heijenoort, editors, *Collected Works, Volume I: Publications 1929–1936*, pages 144–195. Oxford University Press, 1986. English translation of Gödel’s 1931 paper, accessible via: <https://archive.org/details/collectedworks0001gode>.
- [48] Centers for Disease Control and Prevention. What is genomic sequencing? <https://www.cdc.gov/amd/whatis-genomic-sequencing.html>, 2024.
- [49] Teresa Przytycka. Lecture 10: Whole genome sequencing and analysis. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6888888/>, 2025. Accessed August 2, 2025. Lecture slides from Introduction to Computational Biology.
- [50] Sussha Cheriyaedath. Start and stop codons. <https://www.news-medical.net/life-sciences/START-and-STOP-Codons.aspx>, 2019. Accessed July 29, 2025. Describes canonical and alternative start codons across organisms.
- [51] Joseph Willrich Lutalo. **Software Language Engineering-Text Processing Language Design, Implementation, Evaluation Methods.**

- Preprints*, 2024. Accessible via https://www.preprints.org/frontend/manuscript/3903e4cd075074a7005cb705a5ef26c5/download_pub.
- [52] Joseph Willrich Lutalo. **Pragmatic Computational Mysticism**. https://www.academia.edu/124536103/Pragmatic_Computational_Mysticism, 2024. Academia.
- [53] Joseph Willrich Lutalo. **Introducing ZHA, a Real q-AGI**. *Academia*, 2025. Available at: https://www.academia.edu/129344807/Introducing_ZHA_a_real_q_AGI.
- [54] Joseph Willrich Lutalo. **Unraveling Mysteries of The ZHA q-AGI Chatbot: an Interview by ICC, of Fut. Prof. JWJL and M*A*P Ade. Psymaz of Nuchwezi**. *Academia*, 2025. Available at: https://www.academia.edu/download/122791984/ZHA_Interview_paper_JWL_14MAY2025.pdf.
- [55] Joseph Willrich Lutalo. **Complete Genome Expression Example in the Lu-Genome Expression System**. <https://www.academia.edu/145920344/>, Jan 2026.
- [56] Henry Cornelius Agrippa. *Three Books of Occult Philosophy*. Llewellyn Publications, Woodbury, Minnesota, 2014. Annotated edition (Edited by Donald Tyson) available at <https://archive.org/details/three-books-of-occult-philosophy-henry-cornelius-agrippa-donald-tyson-edition>.
- [57] Carl G. Jung, Marie-Louise von Franz, Joseph L. Henderson, Aniela Jaffé, and Jolande Jacobi. *Man and His Symbols*. Aldus Books, London, 1964. Final editing by M.-L. von Franz after Jung's death.
- [58] Joseph Willrich Lutalo. **Rock 'N' Draw**. I*POW, 2025. *ISBN 978-9913-624-71-8 (First Edition)*. A physical copy might be accessed via the National Library of Uganda (NLU), but also, an education-purposes-only edition is accessible freely via Academia at <https://t.me/ipowriters/237>.
- [59] Joseph Lutalo. **Concerning a Transformative Power in Certain Symbols, Letters and Words**. *Academia*, 2025. The unstripped version accessible via: https://www.academia.edu/download/121530226/edtn_25FEB_Concerning_A_Transformative_Power_in_Certain_Symbols_Letters_and_Words.pdf.
- [60] Raymond S. Nickerson. Counting, computing, and the representation of numbers. *Human Factors*, 30(2):181–199, 1988. Available at: https://www.researchgate.net/publication/258138626_Counting_Computing_and_the_Representation_of_Numbers.
- [61] Joseph Willrich Lutalo. **Numbers from Arbitrary Text: Mapping Human Readable Text to Numbers in Base-36**. *Academia.edu*, 2024. Accessible via https://www.academia.edu/123296302/Numbers_from_

Arbitrary_Text_Mapping_Human_Readable_Text_to_Numbers_in_Base_36.

- [62] Michael J. Bossé and William J. Cook. Repeating decimal expansions in different bases. *Electronic Journal of Mathematics & Technology*, 2025. Accessible via https://ejmt.mathandtech.org/Contents/eJMT_v16n3p4.pdf.
- [63] Bruce Alberts, Alexander Johnson, Julian Lewis, Martin Raff, Keith Roberts, and Peter Walter. *Molecular Biology of the Cell*. Garland Science, 2002. Accessible via <https://archive.org/details/molecularbiology0004albe>.
- [64] Adrian Bird. Perceptions of epigenetics. *Nature*, 447(7143):396–398, 2007. Accessible via <https://www.nature.com/articles/nature05913.pdf>.
- [65] Errol C Friedberg, Graham C Walker, Wolfram Siede, Richard D Wood, Roger A Schultz, and Tom Ellenberger. *DNA Repair and Mutagenesis*. ASM Press, 2006. Accessible via <https://archive.org/details/dnarepairmutagen0000unse>.
- [66] Scott F Gilbert. *Developmental Biology*. Sinauer Associates, 2010. Accessible via https://archive.org/details/isbn_9780878935581.
- [67] Mary-Claire King and Allan C Wilson. Evolution at two levels in humans and chimpanzees. *Science*, 188(4184):107–116, 1975. Accessible via <https://www.jstor.org/stable/pdf/1739875.pdf>.
- [68] François Jacob. Evolution and tinkering. *Science*, 196(4295):1161–1166, 1977. Accessible via https://williams.chemistry.gatech.edu/course_Information/6572/papers/jacob_evolution_tinkering_1977.pdf.
- [69] Joseph Willrich Lutalo. **CODE: Computational Mysticism OGF**. <https://t.me/wwwrite/93>, Aug 2025.
- [70] Joseph Willrich Lutalo. **Transformatics 101 (1-Page Executive Summary)**, Oct 2025. *The 1-pager PDF is available at:* https://www.academia.edu/145185478/TRANSFORMATICS_101_On_1_Page_.
- [71] Joseph Willrich Lutalo. **A Comparative Review and Critique of Tree Algebras: Revisiting Gibbons (1991) Thesis in Light of Contemporary Modern Transformatics**. *Academia*, 10 2025. <https://www.academia.edu/144711639/>.
- [72] Joseph Willrich Lutalo. **A Mini-Review of RNG Theory**, 2025. https://www.academia.edu/143901839/A_Mini_Review_of_RNG_Theory.
- [73] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 623–656, 1948. *A copy of which is accessible via:* https://ia803209.us.archive.org/27/items/bstj27-3-379/bstj27-3-379_text.pdf.
- [74] Sheldon Axler. *Linear Algebra Done Right*. Springer, 3 edition, 2015.

- [75] Walter Rudin. *Principles of Mathematical Analysis*. McGraw-Hill, 3 edition, 1976.
- [76] Paul R. Halmos. *Naive Set Theory*. Van Nostrand, 1960.
- [77] Patrick M. Fitzpatrick. *Advanced Calculus: A Course in Mathematical Analysis*. PWS Publishing Company, Boston, 1995. Online edition available at <https://archive.org/details/advancedcalculus0000fitz>.
- [78] Walter Nef. *Linear Algebra*. McGraw-Hill, New York, 1967. Translated from the German: *Lehrbuch der linearen Algebra*. Digital edition available at <https://archive.org/details/linearalgebra0000nefw>.
- [79] Ron Larson and Bruce H. Edwards. *Calculus*. W. H. Freeman and Company, New York, 10th edition, 2012. Available online: <https://books.google.com/books/about/Calculus.html?id=AEuPzgEACAAJ>.
- [80] Konrad Knopp. *Theory and Application of Infinite Series*. Dover Publications, 1990.
- [81] Gilbert Strang. *Introduction to Linear Algebra*. Wellesley-Cambridge Press, 5 edition, 2016.
- [82] A. N. Kolmogorov and S. V. Fomin. *Introductory Real Analysis*. Dover Publications, 1970.
- [83] Geoffrey Mallin Clarke and Dennis Cooke. *A Basic Course in Statistics—3rd Ed.* Edward Arnold, 1992.
- [84] Stanley I. Grossman. *Calculus of One Variable*. Academic Press, Inc., Orlando, Florida, second edition, 1986. Available online: https://api.pageplace.de/preview/DT0400.9781483262468_A23867188/preview-9781483262468_A23867188.pdf.
- [85] Ruye Wang. *Introduction to Orthogonal Transforms: With Applications in Data Processing and Analysis*. Cambridge University Press, 2012. Electronic edition available at <https://doi.org/10.1017/CB09781139015158>.
- [86] Brandon Rohrer. Transformers from scratch. <https://e2eml.school/transformers.html>, 2020.
- [87] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS)*, pages 6000–6010. Curran Associates Inc., 2017. Electronic edition available at <https://arxiv.org/abs/1706.03762>.
- [88] Alex Graves. Sequence transduction with recurrent neural networks. *arXiv preprint arXiv:1211.3711*, 2012. Electronic edition available at <https://arxiv.org/abs/1211.3711>.

TO CITE:

Lutalo, Joseph Willrich (2025). **Applying TRANSFORMATICS in GENETICS** I*POW. Books. <https://www.academia.edu/143516392/>

Final Dedication

“ **Without** having to print their actual ***na-Sequences*** here... *This work is additionally dedicated to all ~~my~~ kids, especially **Arora J. Muntu**, currently only 2, a jolly, explorative fellow. If not especially for them, perhaps this **book would not have advanced past the front cover.***

About the Author⁵

Joseph Willrich Lutalo Cwa Mukama Rwemera Weira, currently **39**, is a Ugandan born scholar most active in the 21st Century.



As of this moment, Joseph has a vast spectrum of contributions to the theory and practice of computing, mathematics, philosophy, creative literature, the visual and aural arts as well as software engineering (his major focus), spanning

50+ academic works ([ORCID](#)),

18 scholarly articles ([Google Scholar](#)),

and **148 video lectures** voluntarily delivered thus far ([YouTube](#)).

Outside of academia and scientific writing, he has already penned and self-published **2 full-length novels** as well as **3 novellas** among other important known contributions to modern works of fiction ([I*POW](#)).

[**Fut. Prof.**] Joseph, apart from being the current **Editor in Chief** for I*POW – a role he fills purely voluntarily for the Internet Community, he likewise founded it and also serves there as a passionate mentor and supporter of several still unpublished but promising, young, Internet-era writers from around the world right now.

He is likewise founder and current **Internet President** of NUCHWEZI ([nuchwezi.com](#)), a somewhat embattled, though very relevant and globally impactful Research-oriented Innovations and Internet Culture company founded in and based in Garuga, UGANDA.

He is a father of 1 boy and no girl yet, is not yet married, was born into a Catholic family, and continues to voluntarily contribute towards furthering science, mathematics and philosophy, mostly based out of his private, home-based laboratory at Plot 266, 1st Cwa Road, in GARUGA.

Despite some challenges wearing multiple hats, **His Bytes J. Willrich Lutalo** is actively working towards, and hoping to secure his **PhD in Computer Science and Mathematics**, as well as working towards the undeniable possibility that he might have to serve as a leader of his country in the future.

⁵Refer to **ORCID**: <https://orcid.org/0000-0002-0002-4657>

Index

Index

- $\dot{\psi}_{DNA}$, 29
- ψ_{DNA} , 12
- ψ_{RNA} , 13
- ψ_{ascii} , 61
- $\psi_{na-START}$, 21
- $\psi_{na-STOP}$, 22
- ψ_{na} , 13
- ψ_{01} , 25
- ADM, 19, 45, 186
- algorithm
 - Ω -to-OZIN Genetic Code Transcription, 129
 - Ω -to-PLATONIC Form Genetic Code Expression, 134
 - Closest Sequence Search, 56
 - computing MCS, 66
 - First Lu-Shuffle (FLSA), 80
 - Second Lu-Shuffle (SLSA), 87
 - x-LGES Algorithm, 145
- amino acid, 113
- anagram, 30, 88
- Anagram Distance Measure, 45
- anagram distance measure
 - for non-anagrams, 54
- artificial intelligence, 108
- Base-Omega, 125
- characteristic, 50
 - population, 53
 - sequence, 49
- chromosomes, 6
- cipher, 124
 - ozin, 127
 - platonic, 133
- codon, 32, 113
 - complements, 32
- codons, 6, 17
- complement codon, 32
- complement digit, 30
- complement sequences, 27
- complement transform, 31, 33
- consensus overlapping sequence, 73
 - properties, 73
- data, 14
 - coded or raw, 217
- decision surface, 51, 54
- definition
 - ribosome, 121
- DNA, 5
 - as sequences, 11
 - base-pairing, 12
 - complement, 28
 - deoxyribonucleic acid, 5
 - examples, 36
 - library metaphor, 6
 - nucleotides, 12
 - random sequences, 108
 - sequence example, 15
 - symbol set, 12
 - test, 76
- entropy, 79, 96, 202
- eukaryotes, 5
- Euler virus, 125
- exons, 122
- first-order sequence, 18, 73
- flat sequence, 18
- Gödel number, 59

- Gödel number
 - sequence similarity test, 61
- Gaussian sequence, 79
- gene program
 - evil programs, 22
 - hacking, 23
 - pathologies, 23
- gene programs, 17
 - non-coding (introns), 22
- generator, 18
 - k- or n-gram generator, 18
 - natural symbol set, 20
 - protein generator, 122
 - RSG, 93
 - sequence characteristic, 49
 - Sequence Lookup System, 59
 - unspecific symbol set, 20
- genes, 6
- genetic code
 - origin problem, 111
- genetic disease, 23
- genetic programming, 120
- genetic sequence, 36
- genetic test, 76
- genetics, 5
- genome, 6
 - analysis, 19
 - sequencing, 63
- genome expression, 4
 - intermediate language, 126
- genome sequence, 72
 - defining problem, 72
 - limits on size, 73
 - of a population, 55
- genome sequencer, 74
- genome sequencing, 64, 76
 - a law, 76
 - underlying problems, 65
- GTNC, 61
- hereditary disease, 23
- Hi-Fi o-SSI, 30
- higher-order transformer, 94, 100
- IGS Law, 76
- introns, 22
- language transforms, 126
- LGES protocol, 127, 134
- Lu-Genome Expression System (LGES),
 - 125, 175
 - complete transformativ derivation, 138, 140
 - derivation (DNA sequences), 154
 - extension (x-LGES), 144
 - PFA resolution protocol, 136
- lu-number system, 25
- lu-shuffle algorithm, 80, 86
- machine learning, 54
- mappings, 11, 223
- matrices, 198
 - as linear transformations, 215
- matrix
 - definition, 214
- Maximum Common Subsequence (MCS), 66
- modal sequence statistic
 - for a population, 53
- mutations, 5
- na-Complement, 28
 - application in computing ORS, 72
- na-Sequence, 36
- na-sequence, 14
- nucleic acids, 12, 13
 - hypothetical, 125
 - universal symbol set, 13
- nucleobase, 7
- nucleotides, 7, 12, 13
- number expression, 125
- orthogonal symbol set identity, 30
- Overlap Resultant Sequence (ORS), 71

- ozin cipher, 127
- platonic cipher, 133
- population characteristic, 53
- program
 - FLSA, 86
 - Random Number Generator(RNG), 96
 - Random Sequence Generator (RSG), 100
 - SELECT, 99
 - Sequence Characteristic, 41
 - sequence protraction, 95
 - SHUFFLE: sequence anagrams, 98
 - SLSA, 91
 - unspecific symbol set, 94
- prokaryotes, 5
- random number generator(RNG), 92
- random sequence generator(RSG), 92
- random symbol generator(RSG), 92
- representative population modal sequence
 - statistic, 53
- ribosome, 122
 - flow-chart, 118
 - state machine, 119
- ribosome computer, 119
- RNA, 5
 - nucleotides, 13
 - ribonucleic acid, 5
 - symbol set, 13
- RPMSS, 53
- Second Lu-Shuffle Algorithm (SLSA), 86, 175
- sequence
 - algebra, 26, 244
 - anagram, 30, 88
 - as matrices, 198
 - cardinality under multiplication, 61
 - complement, 25, 27
 - definition, 209
 - flat-structure, 17
 - gene-program, 18
 - higher-order, 18, 59, 198, 217
 - shuffling, 83
 - string notation, 15
- sequence algebra, 244
- sequence analysis, 35
 - anagram distance, 19, 186
 - base membership, 19
 - genetic pathologies, 22
- sequence characteristic, 49
 - relation to cardinality, 50
- sequence entropy, 59
 - definition, 202
- sequences
 - decimal, 29, 125, 150
- sequencing, 12, 63, 64
- sex, 5
- START-codon, 116
- START-codons, 21
- statistical artificial intelligence, 54
- STOP-codon, 116
- STOP-codons, 22
- super-sequence, 197
- symbol set
 - codon, 113
 - natural, 20
 - orthogonal, 30
 - parent, 48
 - unspecific, 20, 93
 - unspecific (Python implementation), 94
- table
 - mathematical structures, 208
 - na-Sequence Analysis, 42
 - Platonic Form Encoding Keys via FLSA, 177
 - Platonic Form Encoding Keys via SLSA, 176
 - sequence transforms, 26

TEA

- as higher-order transformers, 199
- installation guide, 38

TEA program

- modal sequence statistic, 40
- sequence characteristic, 41, 51

transcription, 126

- LGES protocol, 127, 129

transformatics

- as an algebra, 244
- definition, 244
- genetics introduction, 5
- in medicine, 23
- mappings, 11, 223, 224
- relation to frobenoids, 11

transformer

- as mapping, 224
- complement, 28
- mRNA Encoder, 115
- RSG, 93, 100
- tGS, Genome Sequencer, 74
- tMCS, for maximum common subsequence, 66
- tORS, for Overlap Resultant Sequence, 71

transformer chain, 74, 201**virus, 5, 23**

- veuler (Euler virus), 125

Colophon

COVER-ART: **J. Willrich Lutalo**, as prompt engineer, using **Microsoft Bing Image Creator**, powered by DALL-E.

GRAPHICS: **Adobe Express** on Android.

PRIMARY-EDITOR: **TexMaker** v6.0.0 using **LaTeX** on Microsoft Windows v10.

REFERENCE-MANAGER: **BibTeX** v0.99d (package manager: **MiKTeX** v24.4).

SECONDARY-EDITOR: **VIM**, Vi IMproved v8.2 text editor, via **WSL**.

PUBLISHER: **I*POW** <https://t.me/ipowriters>

FONT: **Computer Modern Roman**

Applying TRANSFORMATICS In GENETICS

✓ 7 TABLES, 3 CORE PROBLEMS,
4 THEOREMS

✓ 16 DEFINITIONS, 3 LAWS

✓ 16 TRANSFORMERS, 30
TRANSFORMATIONS

✓ 8 ALGORITHMS, 2 CIPHER
KEYS & MORE!

✓ COMPUTING GENETIC
PROXIMITY VIA ADM

✓ GENETIC ANALYSIS WITH
N-GRAMS, MSS &
CHARACTERISTICS

✓ THE MATHEMATICS OF
GENOME SEQUENCING

✓ THE LU-GENOME EXPRESSION
SYSTEM & MORE!

I*POW
2025

t.me/ipowriters

JOSEPH WILLRICH LUTALO